
ADVANTEST®

ADVANTEST CORPORATION

ATL-51
Pattern Program
Reference Manual

MANUAL NUMBER 8289058-16

Applicable OS

ASX/U-51

Applicable Systems

T5377

T5376

T5375

T5371

T5336

T5311

Trademarks and Registered Trademarks

The following trademark and registered trademarks used in this manual.

Product names not listed here are generally trademarks or registered trademarks of their respective companies.

- ADVANTEST, ADVANsite, ATLworks and ATL are trademarks of Advantest Corporation.

Revision History

Rev.	Date	Notes
01	Oct 30/97	
02	Mar 10/98	
03	Jul 15/98	
04	Nov 5/98	
05	Jan 20/99	
06	Jun 30/99	
07	Oct 25/99	
08	Mar 3/00	
09	Dec 20/00	
10	Apr 10/01	
11	Aug 20/01	
12	Jan 10/02	
13	Aug 9/02	
14	Nov 12/02	
15	May 15/03	
16	Jul 25/03	

Table of Contents

Preface	Preface-1
Prerequisites to Read this Manual	Preface-1
Configuration of this Manual	Preface-1
Related Manuals	Preface-1
 1. Overview	 1-1
1. 1 General Information of Pattern Program Reference Manual	1-1
1. 2 ALPG Configuration	1-2
 2. Instructions for the Pattern Program	 2-1
2. 1 Pattern Program Configuration	2-1
2. 2 Compilation Control Instructions	2-5
2. 2. 1 MPAT Statement	2-5
2. 2. 2 RDX Statement	2-5
2. 2. 3 MODE Statement	2-5
2. 2. 4 REGISTER Statement	2-21
2. 2. 5 ARIRAM DEFINE Statement	2-29
2. 2. 6 ADDRESS DEFINE Statement	2-32
2. 2. 7 BURST BLOCK Statement	2-36
2. 2. 8 SDEF Statement	2-37
2. 2. 9 SCRAM Statement/PRESCRAM Statement	2-38
2. 2.10 START Statement	2-39
2. 2.11 DATA DEFINE Statement	2-40
2. 2.11. 1 PDS DATA Statement	2-40
2. 2.12 MODULE Statement	2-42
2. 2.13 AFM PATTERN Statement	2-42
2. 2.14 DBM PATTERN Statement	2-42
2. 2.15 BANK SAVE ADDRESS Statement	2-42
2. 2.16 BANK LOAD ADDRESS Statement	2-44
2. 2.17 END Statement	2-46
2. 2.18 Test Pattern Programming Format	2-46
2. 2.19 %REMARK Statement	2-46
2. 2.20 %IFE Statement, %IFN Statement, %ENDC Statement	2-47
2. 2.21 INSERT Statement	2-50

2. 3	Pattern Sequence Controller	2-53
2. 3. 1	Index Loop	2-57
2. 3. 2	Subroutine	2-64
2. 3. 3	Operation Register Setting Instruction (SET reg. data)	2-65
2. 3. 4	Condition Flag Reference Branch (JNCn)	2-68
2. 3. 5	Data Flag Reference Branch (JZD)	2-69
2. 3. 6	Timer Flag Reference Branch (JET)	2-70
2. 3. 7	Match Flag Sense Instruction	2-71
2. 3. 8	Burst Data Transfer Instruction (OUT)	2-74
2. 3. 9	Pattern Sequence Control Register Setting Instruction (ST reg)	2-74
2. 3.10	WAIT	2-75
2. 3.11	STPS, STFL	2-76
2. 3.12	Control Bits	2-76
2. 3.13	Fail Flag Reference Branch (JAF)	2-77
2. 4	Address Pattern Generation Instructions	2-79
2. 4. 1	Hardware Configuration of Address Generator Section	2-79
2. 4. 2	Configuration of Address Generation Instruction	2-85
2. 4. 3	Outline of Address Generation Instruction	2-85
2. 4. 4	Base Address Field	2-90
2. 4. 5	Current Address Field	2-100
2. 4. 6	Source Address Field	2-121
2. 4. 7	N Register Field	2-124
2. 4. 8	B Register Field	2-125
2. 4. 9	Address Inversion Field	2-126
2. 4.10	Refresh Counter Field	2-127
2. 4.11	D3, D4 Register Field	2-128
2. 4.12	Address Swap Field	2-130
2. 4.13	Address Shift Field	2-133
2. 4.14	Instructions and Execution Cycle	2-135
2. 4.15	Notes on Program Generation	2-136
2. 5	Data Pattern Generation Instructions	2-138
2. 5. 1	Hardware Configuration for Data Pattern Generator Section	2-138
2. 5. 2	Configuration of Data Generation Instruction	2-142
2. 5. 3	Outline of the Data Generation Instruction	2-143
2. 5. 4	TP Register Field	2-144
2. 5. 5	FP Function Field	2-146
2. 5. 6	DSD, RCD, CCD Field	2-155
2. 5. 7	Temporary Data Inversion(/D, /D2)(Data Inversion Field)	2-156
2. 5. 8	Inversion with Data Flag (DFLG)	2-156
2. 5. 9	Area Inversion	2-157
2. 5.10	Saving Output Data (PD) (Data Hold Field)	2-157
2. 5.11	Data Swapping (DS) (Data Swap Field)	2-157
2. 5.12	Data Shift Field	2-157

2. 5.13	Source Data Field	2-158
2. 5.14	Expected Data Inversion Field (DINV)	2-159
2. 5.15	Instructions and Execution Cycle	2-159
2. 5.16	Notes on Program Generation	2-160
2. 6	Control Bits	2-161
2. 6. 1	Configuration of Control Instruction	2-161
2. 6. 2	TS Field	2-162
2. 6. 3	MUT Field	2-163
2. 6. 4	Data Pattern on Control Terminals of DUT	2-163
2. 6. 5	Specifying the Cycles for Performing Logical Comparison	2-164
2. 6. 6	Specifying to Turn Driver ON or OFF in I/O Operation	2-166
2. 6. 7	Specifying Real Time Inversion of the Driver	2-167
2. 6. 8	Timing Lot Switching and Address Multiplex Control	2-167
2. 6. 9	Specifying Real Time Swap Cycle	2-169
2. 6.10	Specifying Real Time Clock Prohibit Cycle	2-169
2. 6.11	DBMAP Field	2-171
2. 6.12	Address Control Field	2-171
2. 6.13	SCRAM Control Field	2-172
2. 6.14	ARIRAM Control Field	2-172
2. 6.15	PM Control Field	2-172
2. 6.16	PRESCRAM Control Field	2-173
2. 6.17	Cycle Palette Field	2-174
2. 6.18	X Address Save Field	2-174
2. 6.19	Y Address Save Field	2-177
2. 6.20	Burst Length Field	2-181
2. 6.21	Wrap Type Field	2-183
2. 7	ROM Test Pattern Data Program Section	2-185
2. 7. 1	Programming Format	2-185
2. 7. 1. 1	AFM PATTERN Statement	2-185
2. 7. 1. 2	BITSIZE Statement	2-186
2. 7. 1. 3	AMAX Statement	2-187
2. 7. 1. 4	AMIN Statement	2-188
2. 7. 1. 5	FMA Statement	2-188
2. 7. 1. 6	ROM Test Pattern Data	2-189
2. 7. 2	Example of Programming	2-192
2. 8	DBM Pattern	2-194
2. 8. 1	Programming Format	2-194
2. 8. 2	DBM CHANNEL Statement	2-195
2. 8. 3	DBMAP Statement	2-197
2. 8. 4	DBM Data Field	2-197
2. 8. 4. 1	REPEAT Statement	2-200
2. 8. 5	Control of Driver Enable/Comparator Enable	2-201
2. 8. 6	ALPG Pattern and DBM Pattern	2-202

3. Test Plan Program Instructions Related to Pattern Generation

3-1

- 3. 1 Instruction List 3-2
- 3. 2 Functional Test Execution 3-5
- 3. 3 Register Setting and Modification 3-10
- 3. 4 Failure Analysis Memory (not Available on the T5336 and T5311) 3-11
- 3. 5 Shmoo Test 3-13
- 3. 6 Scramble Program Transfer 3-15
- 3. 7 Window Address Specification 3-18

4. Programming I 4-1

- 4. 1 Power Supply Current Measurement 4-1
- 4. 2 Output Voltage Measurement (When Output Voltage is Maintained) 4-5
- 4. 3 DRAM Test 4-11
 - 4. 3. 1 Address Multiplex 4-11
 - 4. 3. 2 Pause Test 4-16
 - 4. 3. 3 Refresh Test 4-18
 - 4. 3. 4 Area Inversion Test (ARIRAM) 4-43
- 4. 4 Bump Test (Burst block) 4-45
 - 4. 4. 1 How to Specify Conditions in the Test Plan Program 4-45
 - 4. 4. 2 How to Specify Conditions in the Pattern Program 4-48
- 4. 5 Partial Test (Z address) 4-53
 - 4. 5. 1 Partial Test Using Z Address 4-53
 - 4. 5. 2 Programming 4-53

5. Programming II 5-1

- 5. 1 Page Mode Test 5-1
 - 5. 1. 1 Page Mode Test 5-1
 - 5. 1. 2 How to Control the Waveform by Control Bit 5-3
 - 5. 1. 3 Auto Page Mode Test 5-7
 - 5. 1. 4 Auto Page Mode Refresh Test 5-13
 - 5. 1. 5 Auto Page Mode RAS Only Refresh Test 5-14

5. 1. 6	Auto Page Mode TRAS Test	5-18
5. 2	Nibble Mode Test	5-21
5. 2. 1	Nibble Mode Test	5-21
5. 2. 2	Nibble Mode Test Program	5-26
5. 2. 3	Nibble Mode Refresh Test Program	5-32
5. 3	Test for Multiple-Bit Memory Devices	5-34
5. 3. 1	Data Generation	5-34
5. 3. 2	I/O Control	5-36
5. 3. 3	Generation of Parity Data	5-38
5. 4	Read Modify Write Test	5-41
5. 4. 1	Read/Write Data Inversion	5-41
5. 4. 2	Programming	5-41
5. 5	EPROM Test	5-44
5. 5. 1	Using the Match Flag Sense Instruction	5-44
5. 5. 2	Programming	5-46

Appendix 1. Pattern Program Instruction List	A1-1
---	-------------

List of Figures	F-1
------------------------------	------------

List of Tables	T-1
-----------------------------	------------

Index	I-1
--------------------	------------

Preface

Prerequisites to Read this Manual

- ALPG configuration
Address/data generation unit within Algorithmic Pattern Generator (ALPG) of T5377, T5376, T5375, T5371, T5336 and T5311 has one way.
Dual PG function (main PG, sub PG) and interleave mode test is not enabled like memory test systems having two ways of address/data generation unit.
- Compatibility of pattern program
Pattern program has three kind of types depending on test systems.
 1. For T5500 series (ASX/U-51)
 2. For T5377, T5376, T5375, T5371, T5336 and T5311 series (ASX/U-51)
 3. For T5300 series (MOST-50, ASX/U-50)

There is no compatibility between these pattern programs.
However, there are many functions commonly used in these programs, pattern programs for other test systems can be used by partly modification.
Pattern object file not compatible to a test system cannot be executed.

Configuration of this Manual

Chapter 1: Overview

Explains the configuration of the whole ALPG.

Chapter 2: Instructions for the Pattern Program

Explains instructions, grouped according to their purposes. It also contains block diagrams of hardware for sequence control and for address and data generation.

Chapter 3: Test Plan Program Instructions Related to Pattern Generation

Explains test plan program statements related to pattern generation and statements described in the test plan program for controlling pattern program execution.

Chapter 4, Chapter 5: Programing I, II

Chapters 4 and 5 explain programming methods for testing memory devices. They also show the correspondence between the methods and test plan programs, as required.

Related Manuals

ATL-51 Test Plan Program Reference Manual(No. : 8431498)

ATL-51 Pattern Program Reference Manual (T5500)(No. : 8256292)

ATL-51 Socket Program Reference Manual(No. : 8256293)

ATL-51 Scramble Program Reference Manual(No. : 8256294)

T5500 Series Tester Utility Program Reference Manual(No. : 8256295)

T5500 Series Controller Command Reference Manual(No. : 8256297)

ATLworks Tool User's Manual(No. : 8311174)

ASX/U-51 Hybrid Controller User's Manual(No. : 8409598)

ASX/U-51 Hybrid Controller Reference Manual(No. : 8409600)

Error Code Table Reference Manual(No. : 8141091)

XATL/U-51 Cross Compiler User's Manual(No. : 8256237)

ADVANsite Test System Software Installation/System Start User's Manual VOL1 Basic OS Installation(No. : 8227728)

ADVANsite Test System Software Installation/System Start User's Manual VOL2 Tester Environment Software Installation(No. : 8227729)

T5500 Series Memory Test System Control Box/Trigger Panel User's Manual(No. : 8256298)

T5300 Series Flash Memory Parallel Measurement Function Description(No. : 8239605)

T5500 Series Memory Test System Calibration Program Reference Manual(No. : 8256296)

T5500 Series Memory Test System Precision Calibration Program Reference Manual(No. : 8311137)

T5500 Series Memory Test System Precision Calibration2 Program Reference Manual(No. : 8350393)

T5500 Series MPATsim User's Manual(No. : 8311103)

1. Overview

This chapter provides an overview of the pattern program manual and explains the ALPG configuration.

1. 1 General Information of Pattern Program Reference Manual

This manual explains how to create a test pattern program on the T5377, T5376, T5375, T5371, T5336 and T5311 memory test system and outlines the algorithmic pattern generator (ALPG), which generates test patterns for each function.

1. 2 ALPG Configuration

The ALPG mainly consists of the three sections described below. [Figure 1-1](#) and [Figure 1-2](#) show a block diagram of the ALPG function.

Address generation section

The address generator section, which consists of the X, Y, and Z address generators, generates address pattern data entered in the DUT at testing. The address generator section calculates addresses in realtime with the address operation instruction from the control section at testing, and enters the address patterns into the DUT.

➔ For information on a detailed block diagram, refer to [2. 4. 1 "Hardware Configuration of Address Generator Section" on page 2-79](#).

Data generator section

The data generator section generates data patterns to be entered in the DUT when writing, and expected patterns used for determining values read from the DUT during reading.

➔ For information on a detailed block diagram, refer to [2. 5. 1 "Hardware Configuration for Data Pattern Generator Section" on page 2-138](#).

Control section

The control section controls the flow of the address patterns and data patterns entered into the DUT. It consists of the instruction memory, Writable Control Storage (WCS), in which operation or selection instructions to be entered in the address or data generator section are stored, the sequence control section, and the DUT control signal generation circuit which generates the DUT control signals.

➔ For information on a function block diagram, refer to [2. 3 "Pattern Sequence Controller" on page 2-53](#).

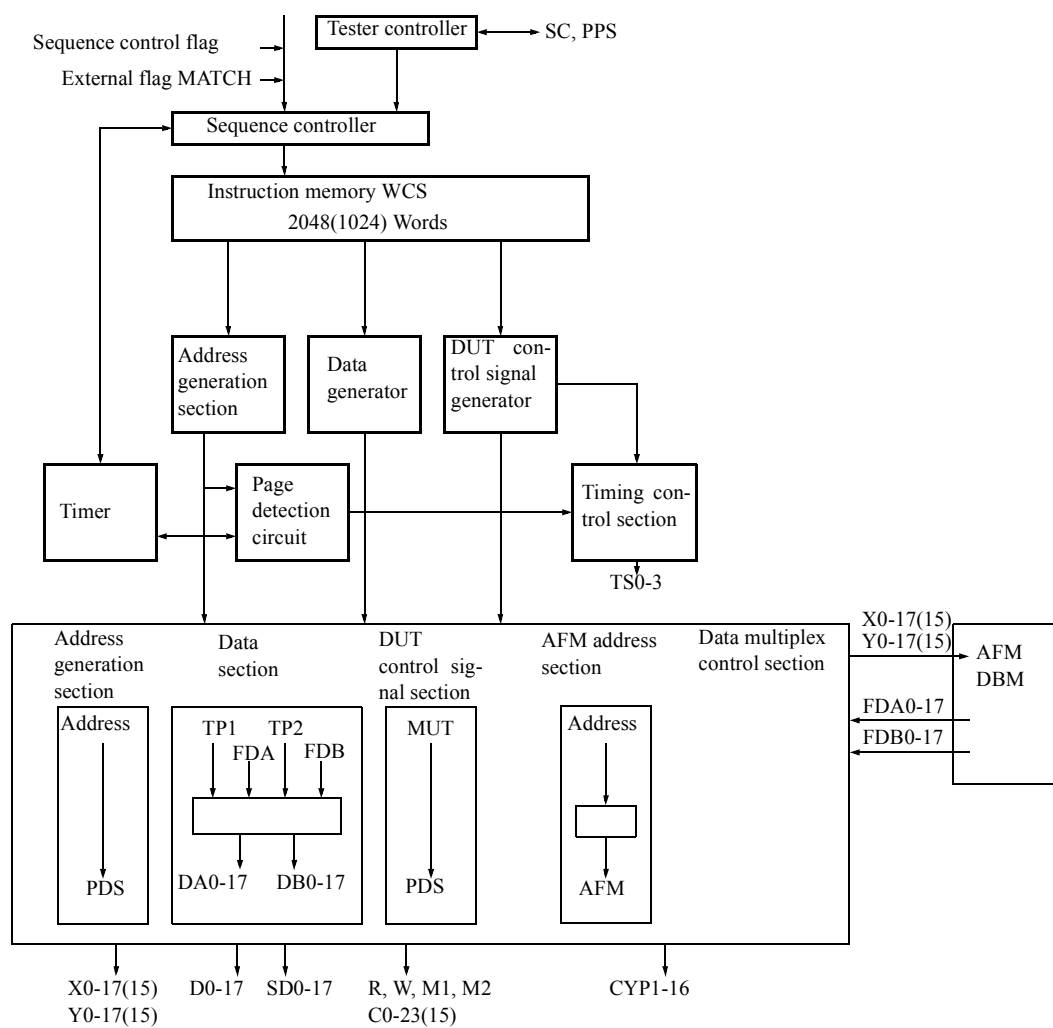
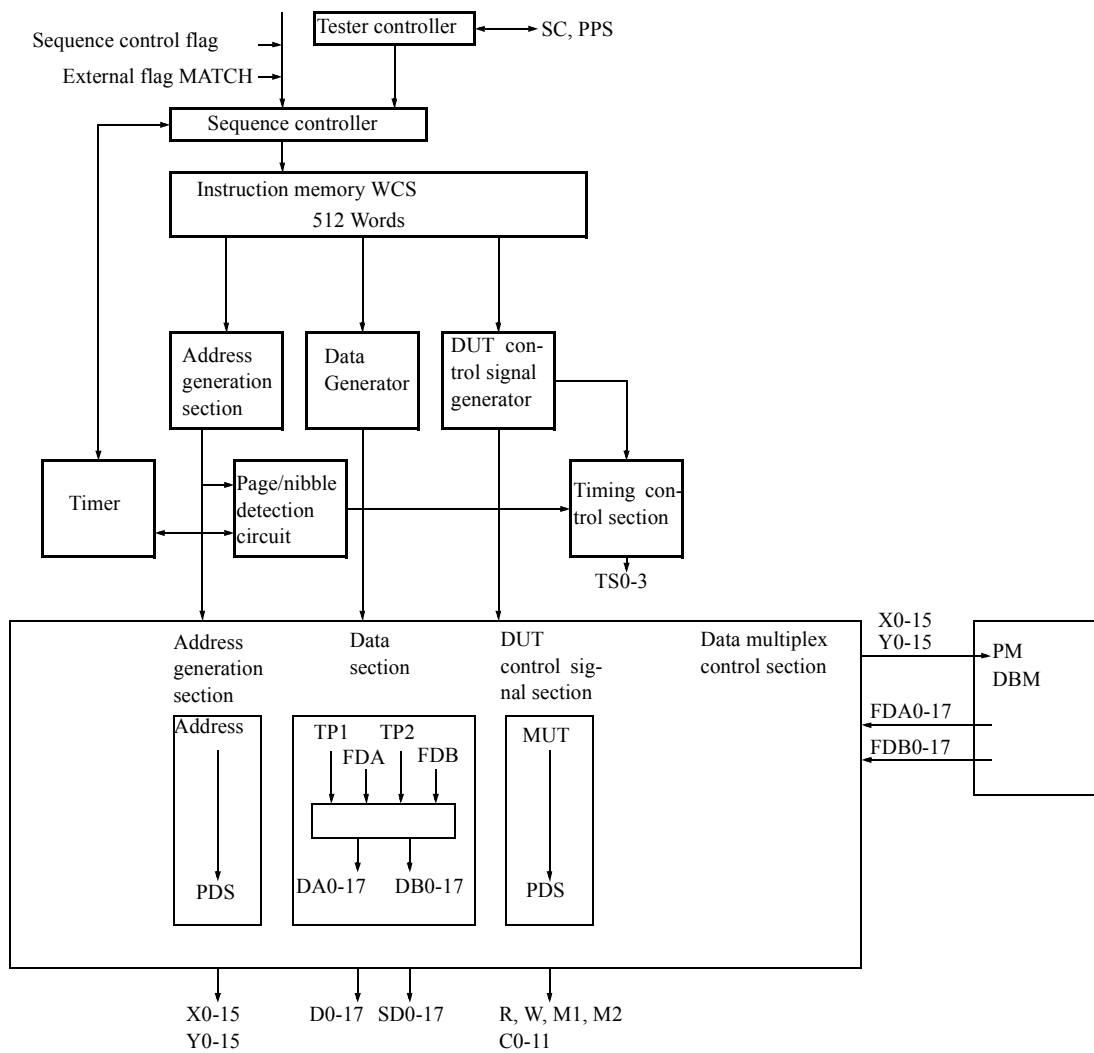
Figure 1-1 ALPG Function Block Diagram 1 (T5377, T5376, T5375 and T5371)

Figure 1-2 ALPG Function Block Diagram 2 (T5336 and T5311)

2. *Instructions for the Pattern Program*

This chapter explains the configuration of and commands used in the pattern program. It provides an overview of hardware used in pattern generation.

2. 1 Pattern Program Configuration

The pattern program begins with an MPAT statement and ends with an END statement. It consists of the common section and the module section as shown in ["Common Section and Module Section of Pattern Program" on page 2-2](#).

Each pattern program is described in a module section. Setting of data common to modules, for example, setting of data and registers such as XMAX and YMAX defining the memory capacity, is described in the common section.

Up to 29 modules can be coded in the module section.

The instructions shown in ["Module Description Example" on page 2-3](#) can be entered in a module.

Data common to modules is set in the common section, and the instructions shown in ["Example of Common Section Description" on page 2-4](#) can be described.

Conditions for setting a pattern program can be changed by the test plan program.

➔ Refer to [3. "Test Plan Program Instructions Related to Pattern Generation" on page 3-1](#) and [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Selection of a pattern program to be executed at testing is performed with the following instruction in the test plan program:

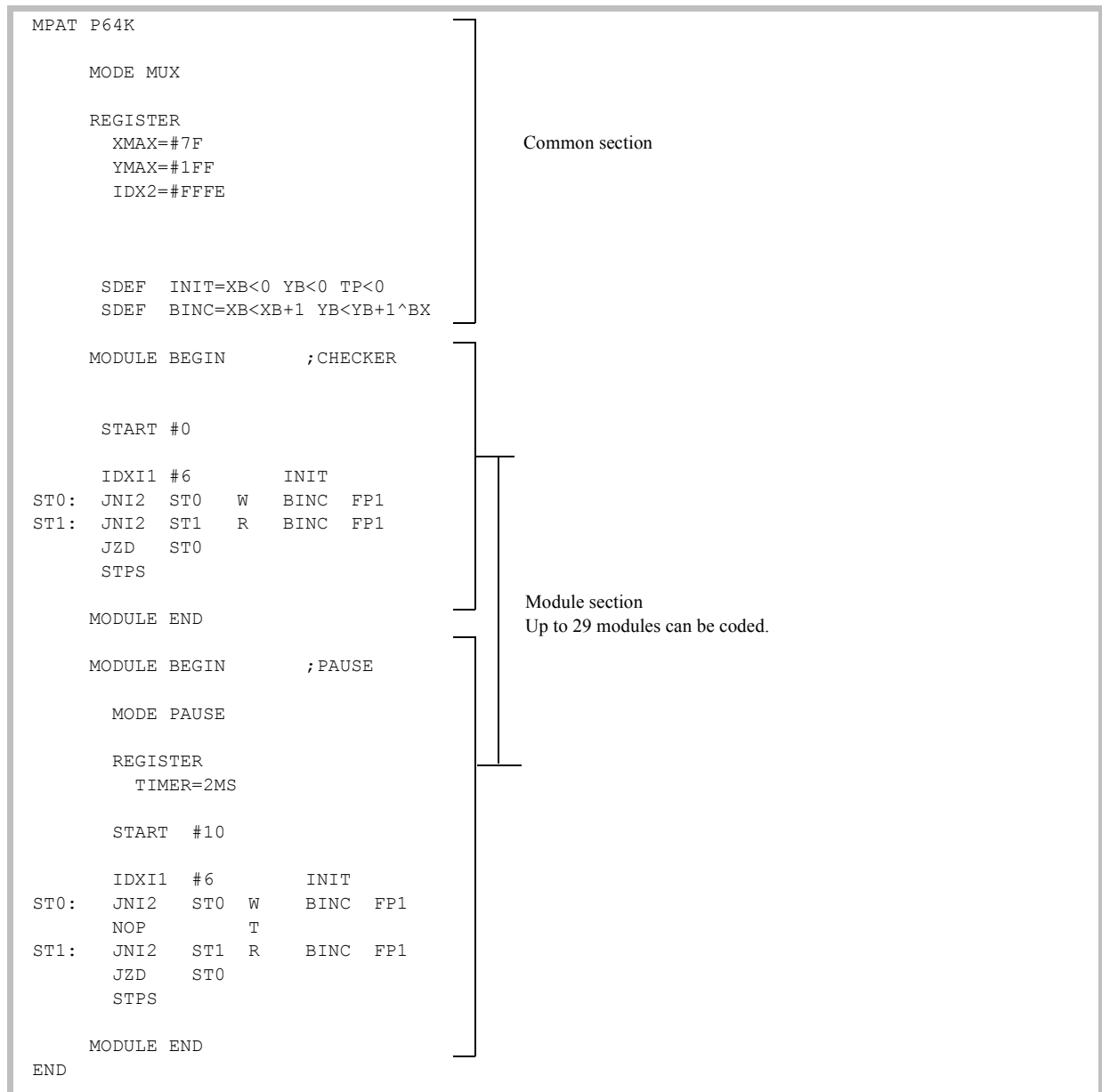
```
REG MPAT PC = n
```

The above n is an address specified by the START statement in the module section.

If data set for the same item is coded by the module section, the common section, and the test plan program, the data priority follows the order shown below:

Test plan program > Module section > Common section

◆ Common Section and Module Section of Pattern Program



◆ Module Description Example

```

MODULE BEGIN

    MODE MUX

    RDX 16

    REGISTER
    XMAX=#FF
    YMAX=#3FF

    ARIRAM DEFINE
    X=X7
    INV=X

    ADDRESS DEFINE
    PAGE=X0-7

    BURST BLOCK BEGIN(A)
    VS1=5.5V

    BURST BLOCK END

    SDEF INIT=XB<0 YB<0 TP<0
    SDEF INC=XB<XB+1 YB<YB+1^BX

    SCRAM S256K

    START #0

    IDX11 #6      INIT
ST0:  JN12  ST0  W  INC  FP1
ST1:  JN12  ST1  R  INC  FP1
      JZD   ST0
      STPS

MODULE END

```

◆ Example of Common Section Description

```
MPAT filename

MODE    MUX

RDX     16

REGISTER
  XMAX=#FF
  YMAX=#3FF

ARIRAM DEFINE
  X=X7
  INV=X

ADDRESS DEFINE
  PAGE=X0-7

BURST BLOCK BEGIN (A)
  VS1=5.5V

BURST BLOCK END

SDEF INIT=XB<0 YB<0 TP<0
SDEF INC=XB<XB+1 YB<YB+1^BX

SCRAM S256K

MODULE BEGIN
    :

MODULE END

MODULE BEGIN
}
    :

MODULE END

END
```

2. 2 Compilation Control Instructions

2. 2. 1 MPAT Statement

Function: MPAT statement indicates the title statement of a pattern program.

Syntax

```
MPAT program-name
```

Argument: program-name

Name of pattern program

This title is the name of an object file after compilation. It is also an identifier when the test plan program specifies a pattern file.

A pattern program name must be specified by using up to 32 characters which include alphanumeric characters, periods (.), and dollar signs (\$).

2. 2. 2 RDX Statement

Function: RDX statement defines the radix of a constant entered in the pattern program.

Syntax

```
RDX n
```

Argument: n

A decimal number $2 \leq n \leq 16$ can be specified. Omitting this instruction results in $n = 16$.

When describing a constant as a hexadecimal, however, and the lead character is A to F, you must add a 0 or "#" to the beginning of the constant.

2. 2. 3 MODE Statement

Function: The MODE statement specifies the ALPG operation mode.

Syntax

```
MODE mode1 [ mode2 .... ]
```

Argument: mode

[Figure 2-1](#), [Figure 2-2](#), [Figure 2-3](#), and [Figure 2-4](#) show the operation modes that can be specified.

Tip

When setting the operation mode for the both the module and the common sections:

The specified mode is valid for both. When different operation modes are specified in both the module section and the common section in each of the following groups, the operation mode specified in the module section becomes valid.

- *MUX, PAGE, NIBBLE*
- *REFRESHA, REFRESHB*
- *LSBOUT, MSBOUT*
- *SRDOUT, DINV*

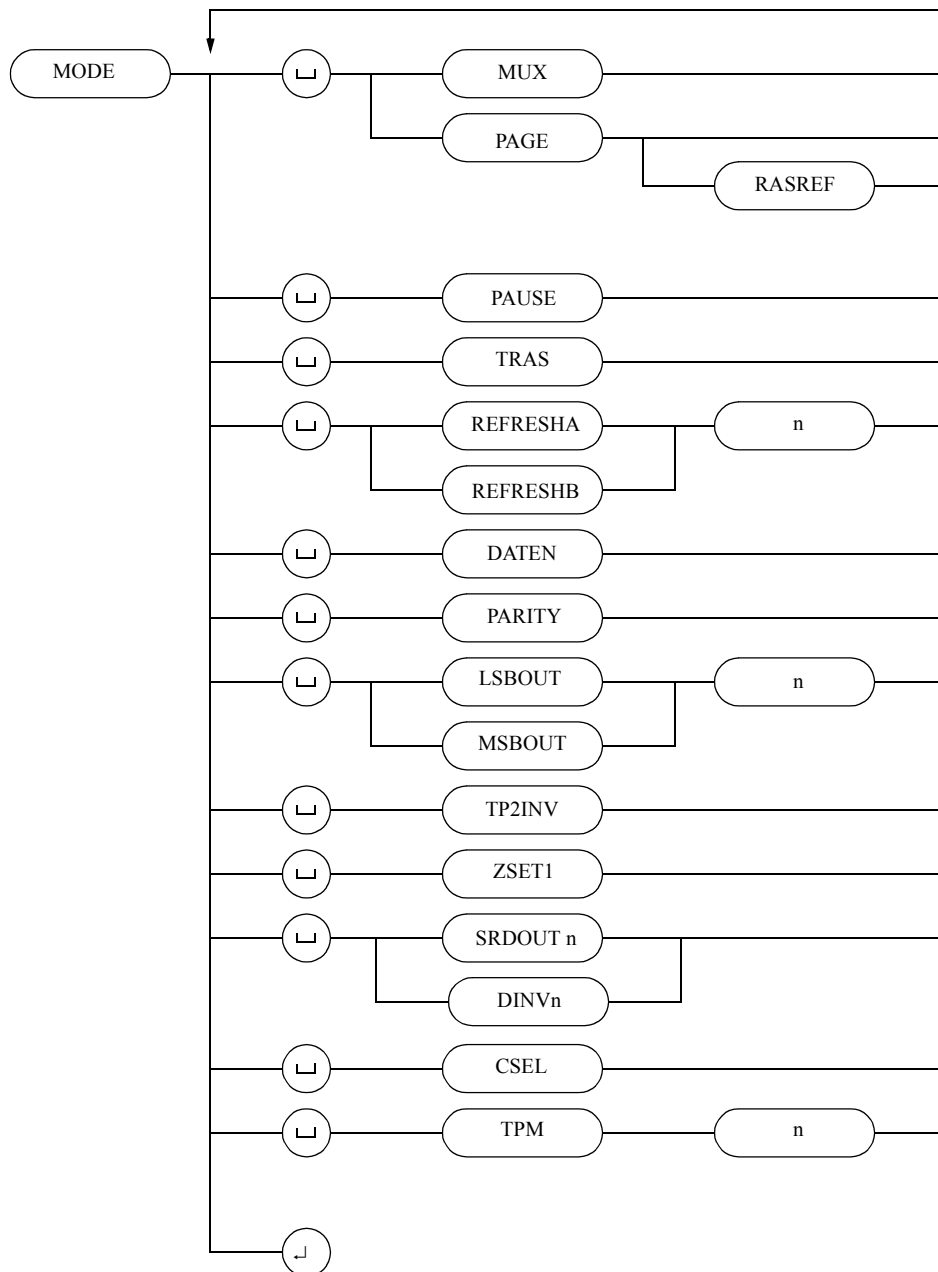
Figure 2-1 Specifiable Operating Modes (on the T5377)

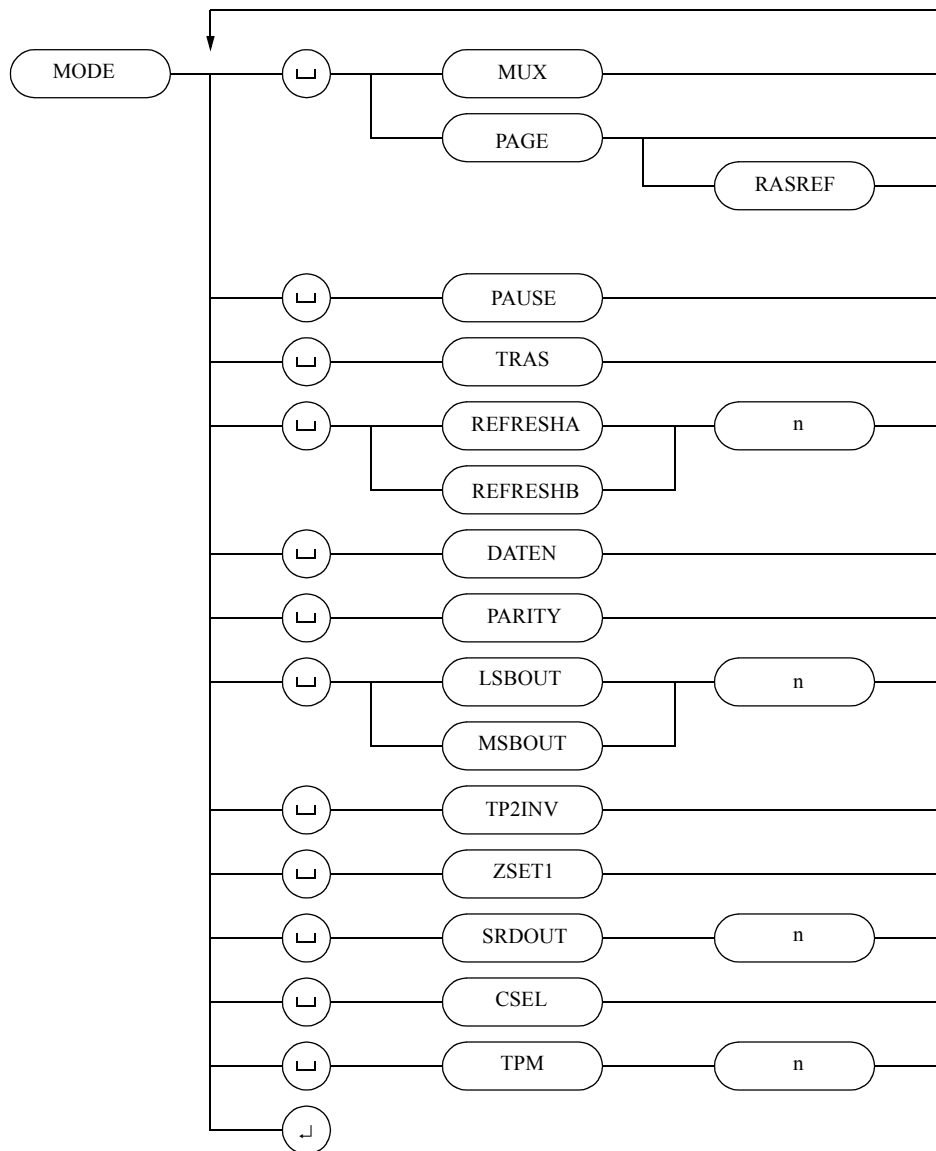
Figure 2-2 Specifiable Operating Modes (on the T5376 and T5375)

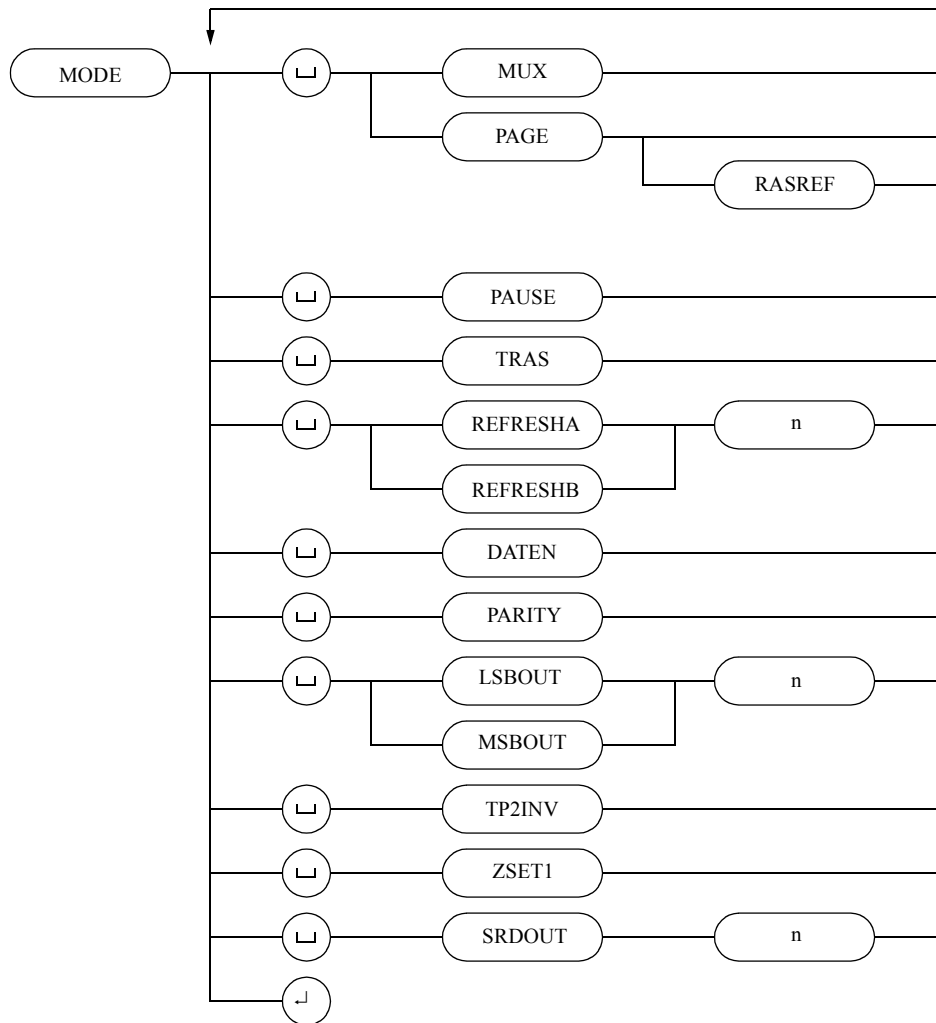
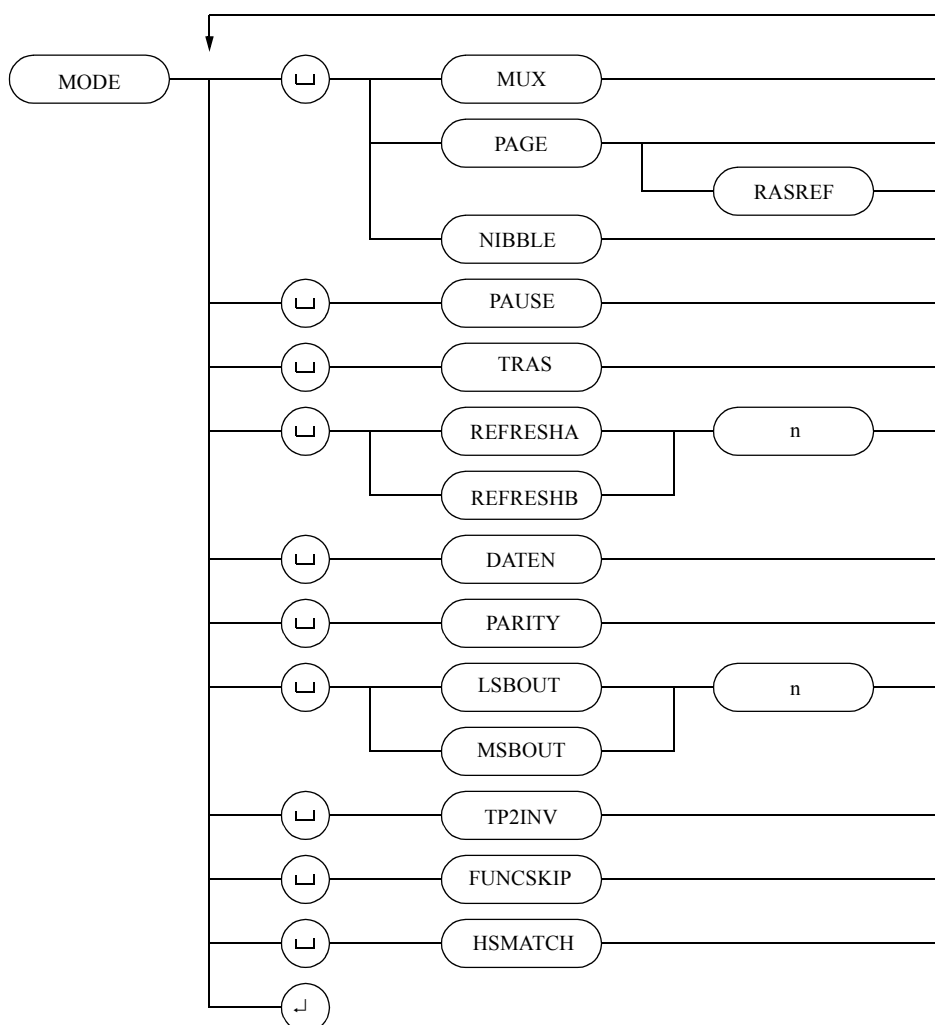
Figure 2-3 Specifiable Operating Modes (on the T5371)

Figure 2-4 Specifiable Operating Modes (on the T5336 and T5311)**MUX**

MUX specifies an address multiplex mode. When specifying this mode, specify SDM and pin data A, pin data B of data selector using the pin statement in a test plan program. See the examples below.

◆ Example

```
PD1 = IN1, XOR, ACLK1, BCLK1, SDM, <X0, Y0>
```


Specification of this mode on the T5377, T5376, T5375 and T5371 is invalid because address-multiplex mode is set by the SDM specification in a pin statement.

PAGE

PAGE specifies the auto page mode test. If the PAGE mode is specified, MUX need not be specified.

RASREF

RASREF specifies the RAS only refresh test. If this mode is specified, the multiplex mode is inhibited in the refresh cycle and only pin data A is output for pins.

NIBBLE (available on the T5336 and T5311)

NIBBLE specifies the nibble mode test.

PAUSE

PAUSE declares the pause test. In the PAUSE mode, pattern generation stops for the time defined in the PAUSE timer (TPAUSE) during the cycle for which control bit T or TM2 is written in the pattern program.

To use this operation mode, do not set the PAUSE timer (TPAUSE) to 0.

TRAS

In TRAS mode, the TRAS timer setting value is defined as the RAS maximum clock pulse width in the auto page mode.

Setting this mode starts TRAS timer operation in the page-in (MP) cycle.

REFRESHA n, REFRESHB n

The modes are used for the auto refresh test with the universal timer (TREFRESH) and the interrupt function. The choice of mode A or B depends on the relationship between the TREFRESH setting time and the interrupt timing.

➔ For details, refer to [4. 3. 3 "Refresh Test" on page 4-18](#).

! Important

- To use the REFRESHB mode and burst by the OUT instruction at the same time, the timer restart instruction (NOP T) is required after the OUT instruction.
- To use the REFRESHB and PAUSE modes at the same time, the REFRESH timer restart instruction (NOP TM1) is required after the PAUSE timer is started.
- To use the REFRESHA or REFRESHB mode on the T5371, set the operation period (RATE) to 21 ns or more in every cycle.

Limitation

- The restriction is applied when the refresh mode (REFRESHA or REFRESHB) is specified and the index loop command is executed with the T5376, T5375, or T5371 test system.

If a program satisfies conditions 1 to 3 and operation 4 is performed, restrictions are applied.

- The refresh mode is specified in either the MODE statement in the pattern program or the SELECT REFRESH statement in the test plan program.
- The JNIn, JNInI, or JNInD instruction and its branch destination specify the same address or the IDXIn instruction is specified without describing the interrupt disable control bit "I" in the step.
- One of the IDXIn, JNIn, JNInI, JNInD, and JSIn index loop instructions is described immediately after 2.
- An interrupt by a refresh timer occurs at the cycle that is immediately before the first loop of the loop count, which was specified by using the instruction described in 3, is executed.

If conditions 1 through 4 are satisfied at the same time, the number of loops specified in the index loop instruction described in 3 is assumed to be #100000000(4294967296) instead of the setting.

(Example 1) A program to which the restriction is applied -1

In this example, after the pattern program is executed, loop A in line 2 (JN11) is complete. When an interrupt by a refresh timer occurs immediately before executing line 3 (JN12), the restriction is applied.

However, if an interrupt by a refresh timer occurs immediately before executing the second loop or later in line 3 (JN12) in the loop B, no restriction is applied.

```

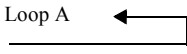
MODE REFRESHA

REGISTER
  IDX1=#E
  IDX2=#1E

  TIMER=1MS
  ISP=ST1

START #00
1      NOP T
2      ST0: JN11 .           Loop A
3      JN12 ST0             Loop B
4
5      STPS
6
7      ST1: RTN

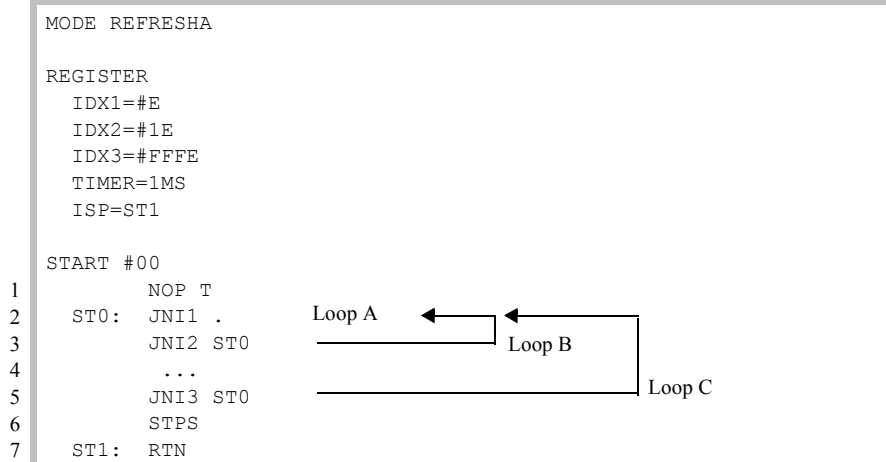
```



(Example 2) A program to which restriction is applied -2

Line 5 (JN13) is added to the pattern program used in Example 1.

In this example, loop C increases cycles that execute the first loop of the loop count in line 3 (JN12). Therefore, there is a high probability that an interrupt by a refresh timer will occur immediately before the first loop.



(Example 3) A program to which no restriction is applied -1

Adding the underlined line avoids the restriction because the program does not satisfy the condition 3.

```

MODE REFRESHA

REGISTER
  IDX1=#E
  IDX2=#1E
  IDX3=#FFFE
  TIMER=1MS
  ISP=ST1

START #00
  NOP T
  ST0: JN11 .
      NOP          ←Added
      JN12 ST0
      JN13 ST0
      STPS

  ST1: RTN
  
```

(Example 4) A program to which no restriction is applied -2

Adding the underlined interrupt disable control bit "I" avoids the restriction because the program does not satisfy condition 2.

```

MODE REFRESHA

REGISTER
  IDX1=#E
  IDX2=#1E
  IDX3=#FFFE
  TIMER=1MS
  ISP=ST1

START #00
  NOP T
  ST0: JN11 . I      ←-Added

        JN12 ST0
        JN13 ST0
        STPS

ST1: RTN

```

- To detect pattern programs to which the restriction is applied, execute compilation by specifying the -m switch of the compilation command.

```
trans -m source(source:Specification of a source file name)
```

When the compiler detects the restriction, the following processes are performed depending on the loop count that is set to 2 or 3 by using the index loop instruction.

When the loop count is set to 2

Displays the "WARNING !" message and generates the object program.

When the loop count is set to 3

A compilation error (CODE = 23614) occurs.

However, if the refresh mode is specified in the SELECT REFRESH statement in the test plan program, pattern programs to which the restriction is applied cannot be detected even though compilation is performed by specifying the -m switch.

➔ For more information on the format of the compilation execution command, refer to [XATL/U-51 Cross Compiler User's Manual\(No. : 8256237\)](#).

DATEN

Declares the realtime swap mode. Data can be output from the driver from which addresses are normally output only in the cycle in which C0 has been specified in the pattern program.

This mode is effective only on pins for which RDSM has been set as a pin condition.

DATEN specification is invalid on the T5377, T5376, T5375 and T5371, because the mode is set by RDSM in a pin statement.

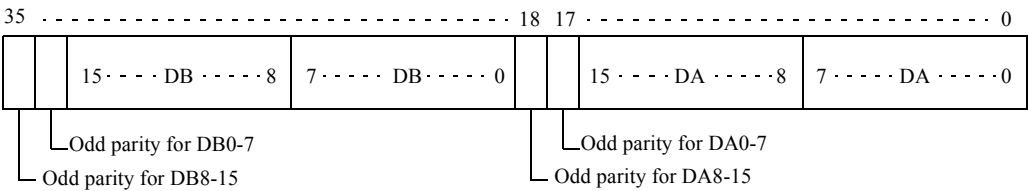
PARITY

Declares the mode in which an odd parity is automatically generated.

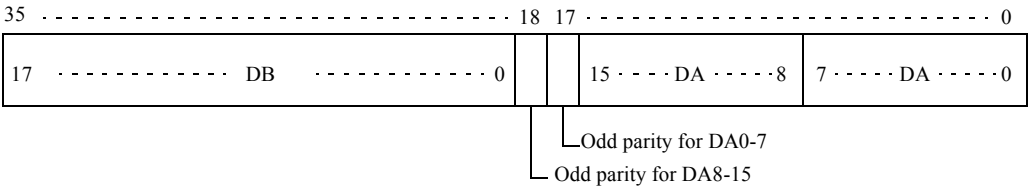
The parity for DA0-7 is output to DA16, the parity for DA8-15 is output to DA17, the parity for DB0-7 is output to DB16, and the parity for DB8-15 is output to DB17.

If TPM36 is specified in the MODE statement, the parity for DA0-7 is output to DA16, the parity for DA8-15 is output to DA17, and no parities are output to DB16 and DB17.

- In MODE TPM18



- In MODE TPM36



LSBOUTn

Declares address shift mode which connects X, Y addresses generated in ALPG to LSB, and performs serial output of the LSB to X0 by shift control instruction, (XYSL, XYSR, XYLOAD).

n is used to specify the number of bits for serial output, and can take on a value of 1, 2, 4, or 8.

If n is not specified, it defaults to a value of 1.

Output addresses in each bit configuration are as follows:

- LSBOUT1 : X0
- LSBOUT2 : X0, X1
- LSBOUT4 : X0, X1, X2, X3

LSBOUT8 : X0, X1, X2, X3, X4, X5, X6, X7

MSBOUTn

Declares address shift mode which connects X, Y addresses generated in ALPG to MSB, and performs serial output of the MSB to X0 by shift control instruction, (XYSL, XYSR, XYLOAD).

n is used to specify the number of bits for serial output, and can take on a value of 1, 2, 4, or 8.

If n is not specified, it defaults to a value of 1.

Output addresses in each bit configuration are as follows:

MSBOUT1 : X0

MSBOUT2 : X0, X1

MSBOUT4 : X0, X1, X2, X3

MSBOUT8 : X0, X1, X2, X3, X4, X5, X6, X7

TP2INV

Declares a reverse control mode by ARIRAM (area inversion), FP function and DFLG for the data of TP2 register.

If this operation mode is not specified, the data of TP2 register is not the target.

FUNCSKIP (available on the T5336 and T5311)

This is the mode for executing the flash memory test in high speed.

The high speed test can be executed by skipping some functions of ALPG.

Limitation

The following restrictions exist. Be careful, as neither the compiler nor the tester driver controller checks for them.

- XB and YB base operation can be used for address generation instructions.
- The address descrambler is not executed.
- Address function is not executed.
- ARIRAM is not executed.
- The match rate limitation is 1.15 μs.

This mode cannot be specified with NIBBLE mode or PAGE mode.

HSMATCH (available on the T5336 and T5311)

This is the mode for executing the match test of flash memory in high speed.

➔ For details, refer to [T5300 Series Flash Memory Parallel Measurement Function Description\(No. : 8239605\)](#).

ZSET1 (available on the T5377, T5376, T5375 and T5371)

ZSET1 sets event time 1 to operate the DSET instruction for the Z register.

SET Data for the Z register is valid on the step where the instruction is written.
(Z<0 is not recognized as a DSET instruction.)

If this mode is omitted, event time 2 is set the same as the other registers and the result is output on the next step where the instruction is written.

◆ Event Time 1

	output
NOP Z<#1	; Z=1
NOP Z<#2	; Z=2
NOP Z<#3	; Z=3
NOP	; Z=3

◆ Event Time 2

	output
NOP Z<#1	; Z=indefinite
NOP Z<#2	; Z=1
NOP Z<#3	; Z=2
NOP	; Z=3

! Important

When ZSET1 has not specified, the event time of the DSET instruction for the Z register is different between T5377, T5376, T5375 or T5371 and T5336 or T5311.

T5377, T5376, T5375 or T5371

Event time 2 (It can be changed to event time 1 by specifying ZSET1.)

T5336 or T5311

Event time 1 (ZSET1 cannot be specified. Event time fixed.)

To specify ZSET1, event time of the T5377, T5376, T5375 or T5371 can be the same as the one set for the T5336 or T5311.

SRDOUTn (available on the T5377, T5376, T5375 and T5371)

SRDOUTn declares a data shift mode. This mode performs serial outputs of the data bits (only for TP1) which ALPG generates by executing the control instruction (DLOAD and DSR).

n is used to specify the number of bits for serial output, and can take on a value of 1, 2, 4, or 8.

If n is not specified, it defaults to a value of 1.

Output data in each bit configuration are as follows:

SRDOUT1 : D0

SRDOUT2 : D0, D1

SRDOUT4 : D0, D1, D2, D3

SRDOUT8 : D0, D1, D2, D3, D4, D5, D6, D7

Limitation

The DINV n mode cannot be specified in the common sections and modules in which SRDOUT n mode is specified.

TPM n (available on the T5377, T5376 and T5375)

TPM n declares the bit width of the TP1 and TP2 registers.

TPM18

Declares the 18-bit mode.

The TP1 and TP2 registers are each defined as having a bit width of 18 bits.

In each of the TP1 and TP2 registers, data in the 18 low-order bits is used and data in the 18 high-order bits is not used.

TPM36

Declares the 36-bit mode.

The TP1 and TP2 registers are each defined as having a bit width of 36 bits.

If this mode is omitted, the 18-bit mode is used.

When the register is 36-bit mode, the method used to output data in the TP1 and TP2 registers to DA0-17 and DB0-17 depends on the Source data field instruction.

➔ For details, refer to [2. 5. 13 "Source Data Field" on page 2-158](#).

CSEL (available on the T5377, T5376 and T5375)

CSEL declares that a carry or borrow occurs at a value other than 2^n during the XB or YB address operation.

In this mode, a value other than 2^n-1 (n : number of significant bits) can be specified in the LMAX and HMAX registers.

- If MODE CSEL is not specified

A carry or borrow is generated at the most significant bit among the significant bits specified in the LMAX and HMAX registers during the XB or YB address operation.

- If MODE CSEL is specified

During the XB or YB address operation, a carry or borrow is generated under the following conditions.

XB operation result = value specified in LMAX register

YB operation result = value specified in HMAX register

Limitation

If this mode is set, the following address generation instructions cannot be specified.

XB<XB-1, XB<XB+2, XB<XB-2, XB<XB+Dn, XB<XB-Dn

YB<YB-1, YB<YB-1^BX, YB<YB-1+BX, YB<YB-1-BX

YB<YB+Dn, YB<YB-Dn, YB<YB+Dn^BX, YB<YB-Dn^BX,
YB<YB+Dn+BX, YB<YB-Dn-BX

Z<Z+1^CY, Z<Z-1^CY

XC, XS, XK, YC, YS, YK address operation instruction
(Current address field operation instruction)

D3<D3*2L, D4<D4*2L

LMAX<>HMAX

XYSL, XYSR, XYLOAD

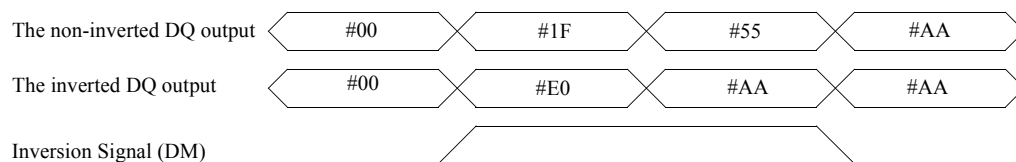
(Dn=D1A to D1H, D2A to D2D)

DINVn (available for the T5377)

Declares the data inversion mode in which the expected value is inverted by the data transition.

Some of the multi-bit memory devices can reduce the output data transition to reduce the simultaneous switching noise of the output data. Specifically, if there are a lot of output data transitions, the output data is inverted and the signal (DM), which shows that the output data is inverted, is set to "1". (The DM signal is included in the data transitions.)

A timing chart is shown below.

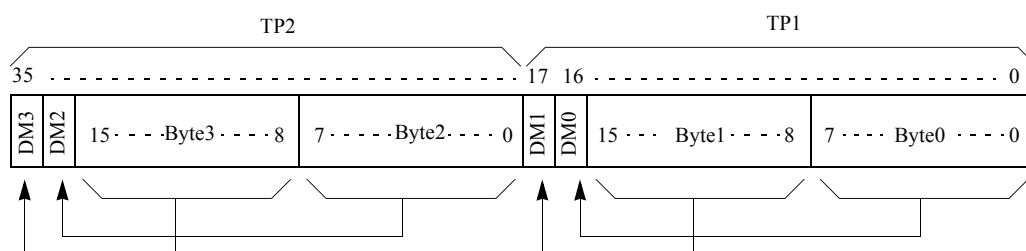


The data inversion mode is used to generate the expected values of memory devices. Following are the data inversion modes selected by specifying the DINVn:

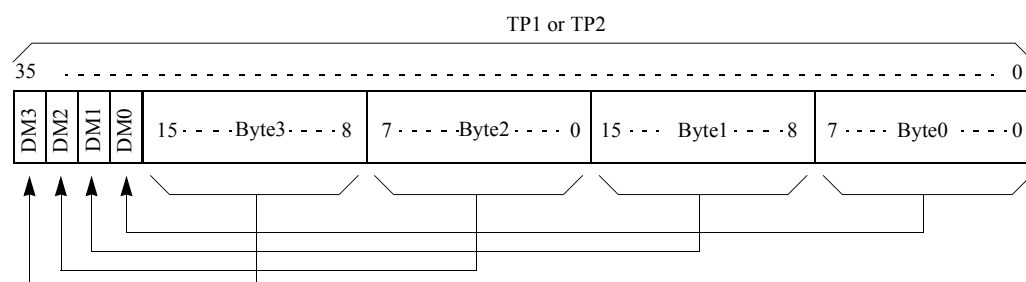
Mode	Operation
DINV1	Outputs the original data before the burst operation, and after that inverts the data by the data transition.
DINV2	Sets the data before the burst operation as "0", and inverts the data by the data transition after that.
DINV3	Sets the data before the burst operation as "1", and inverts the data according to the data transition which occurs after that.

In the data inversion mode, the inversion of the expected data is controlled according to a byte-by-byte data transition. The expected data is inverted within the five-bit transition of the data and the inversion signal of one byte. The output bit (DM0-3) for each byte and inversion signal are as follows according to the TPMn specification of the MODE statement.

- In the MODE TPM18



- In the MODE TPM36



If the data inversion mode is specified in the MODE DINV1 to 3, this function is valid in the cycle in which the control bit DINV is specified.

Limitation

The SRDOUT n mode cannot be specified in the common sections and modules in which DINVn mode is specified.

2. 2. 4 REGISTER Statement

The REGISTER statement is used to initialize each register in ALPG. [Table 2-1](#) shows the initial values of the registers in ALPG.

When data is set in the same register in the common and module sections, the data which was set in the module section is valid.

Table 2-1 Register List

Register	Maximum value				Default value	Remarks
	T5377	T5376 T5375	T5371	T5336 T5311		
TIMER	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
TREFRESH	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
TRAS	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
TPAUSE	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
PC	#7FF	#7FF	#3FF	#1FF	#0	--
ISP	#7FF	#7FF	#3FF	#1FF	#0	--
BAR	#7FF	#7FF	#3FF	#1FF	#0	--
IDX1 to 8	#FFFFFFFF	#FFFFFFFF	#FFFFFFFF	#FFFFFFFF	#0	--
IDX9 to 16	#FFFFFFFF	#FFFFFFFF	--	--	#0	--
CFLG	#FFFF	#FFFF	#FF	#FF	#0	--
RC	--	--	--	#FFF	#0	#1 to #FFF
CPE1 to 4	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	R	--
DRE(DRE1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	W	Can be described as DRE1.
DRE2	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	W	--

2. 2 Compilation Control Instructions

Register	Maximum value				Default value	Remarks
	T5377	T5376 T5375	T5371	T5336 T5311		
RINV	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	R	--
LMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	XMAX	--
HMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	YMAX	--
XMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFF	--
YMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFF	--
XH	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
YH	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
ZH	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
NH	#FF	#FF	#FF	#F	#0	--
D1(D1A)	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	Can be described as D1A.
D1B to D1H	#3FFFF	--	--	--	#0	--
D2(D2A)	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	Can be described as D2A.
D2B to D2D	#3FFFF	--	--	--	#0	--
D3B, D4B	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
XMASK	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFD	--
YMASK	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFD	--
RCD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
CCD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
DSD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
XOS	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
YOS	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
XT	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
YT	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
TPH(TPH1, TPH1A)	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	Can be described as TPH1 or TPH1A.
TPH1B to 1D	#FFFFFFFF	--	--	--	#0	--
TPH2 (TPH2A)	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	Can be described as TPH2A.

Register	Maximum value				Default value	Remarks
	T5377	T5376 T5375	T5371	T5336 T5311		
TPH2B to 2D	#FFFFFFFF	--	--	--	#0	--
DCMR (DCMR1)	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	Can be described as DCMR1.
DCMR2	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	--
D5	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	--
D6	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	--
MR1	#FF	#FF	#FF	#FF	#0	--
RLMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	#FF	#3F, #7F, #FF, #1FF, #3FF, #7FF, #FFF, #1FFF, #3FFF, #7FFF, #FFFF, #1FFFF or #3FFFF can be specified.
ZD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
ND(ND1)	#FF	#FF	--	--	#0	Can be described as ND1.
ND2 to 7	#FF	--	--	--	#0	--
BD(BD1)	#FF	#FF	--	--	#0	Can be described as BD1.
BD2 to 7	#FF	--	--	--	#0	--
BH	#FF	#FF	--	--	#0	--
FPSEL	Address function name ^(*2)	Address function name ^(*2)	--	--	DISABLE	--

-- means the register cannot be specified.

^{*1} Specify the following signal names in the CPE1 to 4, DRE, DRE2, and RINV registers.

T5377, T5376 and T5375

FIXL(FL), FIXH(FH), R(RD), W(WT), M1 to 2, C0 to 23, PREF

T5371

FIXL(FL), FIXH(FH), R(RD), W(WT), M1 to 2, C0 to 15, PREF

T5336 and T5311

FIXL(FL), FIXH(FH), R(RD), W(WT), M1 to 2, C0 to 11, PREF

Note that PREF can only be specified for the CPE1 to 4 registers.

^{*2} Specify the following address function names or DISABLE in the FPSEL register.

FP0, FP1, FP3, FP4, FP5, FP6, FP7, FP8, FP9 or FP10

TIMER

Three timers can be set, one each for the pause test (Timer2), refresh test (Timer1), and TRAS test (Timer3). Settings can be made in units of cycles or time. When PAUSE mode is specified by the MODE statement, however, time cannot be set in cycles.

TREFRESH

To perform the refresh test (Timer1), set the time for Timer1. Settings can be made in units of cycles or time.

TPAUSE

To perform the pause test (Timer2), set the time for Timer2. Settings can also be made in units of time.

TRAS

To perform the TRAS test (Timer3), set the time for Timer3. Settings can be made in units of cycles or time.

- How to set time in cycles
1 to 4095 or #1 to #FFF
- How to set time in time
100 ns to 409.5 s

PC(Program Counter)

Set the ALPG start address.

When described dividedly by a MODULE statement, data is stored into ALPG from the address indicated by the START statement. In practice, however, the start address of a pattern is specified by the REG MPAT PC statement of the test plan program.

ISP(Interrupt Save Pointer)

ISP specifies the address to which control branches if an interrupt is caused by the internal timer in the auto refresh test. For the specification, an address can be set directly or a label can be set.

Limitation

The following conditional branch instructions cannot be used as the first executable instruction for the branch address specified by ISP:

FLGLIm (m = 1 to 4)

BAR(Branch Address Register)

The BAR defines the branch address of the flag sense instruction FLGLIn.

IDX1 to 8

IDX1 to 8 sets the number of loops in a loop instruction (JNIn, etc.) of MAIN ALPG.

➔ For details, refer to [2. 3. 1 "Index Loop" on page 2-57](#).

IDX9 to 16

IDX9 to 16 stores the number of loops. It is used by the LDIn instruction to change the number of loops in registers IDX1 to IDX8.

➔ For details, refer to [2. 3. 1 "Index Loop" on page 2-57](#).

CFLG

CFLG sets the branch condition of the JNCn instruction. The bit of the CFLG register correspond to JNC1 to JNC16 (JNC1 to JNC8 on the T5371, T5336 and T5311) as shown below. If the corresponding CFLG bit is set to 1, the JNCn instruction branches to the specified address (label).

➔ For more information on JNCn instructions, refer to [2. 3. 4 "Condition Flag Reference Branch \(JNCn\)" on page 2-68](#).

CFLG register

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
JNC16	JNC15	JNC14	JNC13	JNC12	JNC11	JNC10	JNC9	JNC8	JNC7	JNC6	JNC5	JNC4	JNC3	JNC2	JNC1

If CFLG is used to change the execution sequence in a pattern program, data is usually set with the REG MPAT statement in a test plan program.

RC(Repeat Counter)

RC sets the number of repetitions of a nibble bit in the nibble mode test. If control bit N is entered in the pattern program, in the step, only the operation of register N is repeated $n + 1$ times, where n is the number set in RC.

➔ For a program example, refer to [5. 2. 2 "Nibble Mode Test Program" on page 5-26](#).

CPE1 to 4

CPE1 to 4 sets a control bit (MUT signal) for specifying a cycle for logical comparison. The following signals are used as the control bit. R, W, FIXL, FIXH, M1, M2, C0 to C23 (C12 to C23 cannot be specified on the T5336 and T5311, and C16 to C23 cannot be specified on the T5371), and PREF.

➔ For a program example, refer to [2. 6. 5 "Specifying the Cycles for Performing Logical Comparison" on page 2-164](#).

DRE(DRE1), DRE2

DRE specifies a control bit (MUT signal) which turns a driver for the I/O pin of the DUT on or off. The available signals are from R, W, FL, M1, M2 and C0 to C23 (C12 to C23 cannot be specified on the T5336 and T5311, and C16 to C23 cannot be specified on the T5371). DRE and DRE1 are the same.

➔ For a program example, refer to [2. 6. 6 "Specifying to Turn Driver ON or OFF in I/O Operation" on page 2-166](#).

RINV

Sets a control bit (MUT signal) for inverting a data pattern in realtime for the pin, set as $P_n = \dots$, RIVN, ... in the test plan program. It can be selected from R, W, M1, M2, and C0 to C23 (C12 to C23 cannot be specified on the T5336 and T5311, and C16 to C23 cannot be specified on the T5371).

LMAX, HMAX

LMAX and HMAX specify valid bits of registers when register groups (X, Y, and Z groups) for address generation are to be concatenated.

➔ For details, refer to [2. 4 "Address Pattern Generation Instructions" on page 2-79](#).

XMAX, YMAX

XMAX and YMAX specify valid bits of addresses X and Y to be entered in the DUT.

➔ For details, refer to [2. 4 "Address Pattern Generation Instructions" on page 2-79](#).

XH, YH, ZH, NH

These registers contain initial values for the XB, YB, Z, and N registers. The initial value is loaded into each register by the following operation:

$$XB < XH \quad YB < YH \quad Z < ZH \quad N < NH$$
D1(D1A to D1H), D2(D2A to D2D), D3B, D4B

D1, D2, D3B, and D4B are used for the operation of registers XC, XS, XK, YC, YS, and YK.

D1 (D1A to D1H) and D2 (D2A to D2D) are also used to calculate the XB and YB registers.

TPH(TPH1, TPH1A to TPH1D), TPH2(TPH2A to TPH2D)

The TPH (TPH1, TPH1A to TPH1D) initializes register TP (TP1).

It is stored in register TP (TP1) when the operation is $TP < TPH1A$ to TPH1D.

TPH, TPH1, and TPH1A have the same meaning. Similarly, TP and TP1 are also the same.

TPH2 (TPH2A to TPH2D) initializes register TP2.

It is stored in register TP2 when the operation is $TP2 < TPH2A$ to TPH2D.

TPH2 and TPH2A have the same meaning.

DCMR, DCMR2(Data Complement Mask Register)

DCMR sets the mask bit which determines whether to invert data in the TP or TP2 register with the /D or /D2 instruction. D0 to D35 (D0 to D17 for the T5371, T5336 and T5311) corresponding to a 0 bit is inverted.

➔ For details, refer to [2. 5. 7 "Temporary Data Inversion\(/D, /D2\)\(Data Inversion Field\)" on page 2-156](#).

XMASK, YMASK

XMASK and YMASK are used for address functions FP7 (mask parity), FP8 (row bar), FP9 (column bar), and FP10 (row/column bar).

➔ For details, refer to [2. 5. 5 "FP Function Field" on page 2-146](#).

RCD, CCD(Row Address Complement Data, Column Address Complement Data)

RCD and CCD are used for address functions FP8 (row bar), FP9 (column bar), and FP10 (row/column bar).

➔ For details, refer to [2. 5. 5 "FP Function Field" on page 2-146](#).

DSD(Diagonal Shift Data)

DSD is the register used as shift data for address functions FP3 (diagonal), FP4 (inverted diagonal), FP5 (diagonal2), and FP6 (inverted diagonal2).

➔ For details, refer to [2. 5. 5 "FP Function Field" on page 2-146](#).

MR1

MR1 is a mode register to control the operation mode of the PG and PG timer. This register controls DISABLE and ENABLE status of the TRAS, REFRESHB, PAUSE, PAGE, RASREF or ARIRAM mode.

When the MR1 register is not specified

Each mode is set by the MODE statement and ARIRAM DEFINE statement.

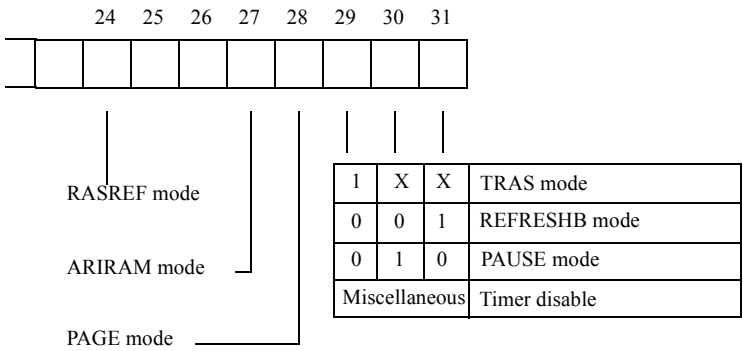
When the MR1 register is specified

The settings of each mode are controlled by the MR1 register setting.

Specifying 1 to a MR1 bit corresponding to a mode sets the mode ENABLE, and specifying 0 sets DISABLE.

Note that two out of the TRAS, REFRESHB and PAUSE modes cannot be set to ENABLE at the same time. Be careful of the bit configuration for MR1.

Meaning of MR1 bit



! Important

- REFRESHA mode cannot be enabled by the MR1 register.
- In case of the REFRESHA mode is specified in the MODE statement, when MR1 register is specified then the REFRESHA mode is released.
This release is out of all relation to the value of the MR1 register and the assignment point (common section, module section or main program).

- When ENABLE/DISABLE of ARIRAM is controlled, SELECT ARIRAM statement has priority over MR1 register.
When the MR1 register is used, NORMAL should be set in the SELECT ARIRAM statement.
- The SELECT REFRESH statement takes priority over the M1 register in the REFRESHB mode control.
To control the M1 register, specify NORMAL in the SELECT REFRESH statement.
- The SELECT PAUSE statement takes priority over the MR1 register in the PAUSE mode control.
To control the MR1 register, specify NORMAL in the SELECT PAUSE statement.
- The MR1 register is provided to make MOST-50 and ASX/U-50 compatible with each other.
To control the REFRESHB/PAUSE/ARIRAM operation mode, use the SELECT REFRESH, SELECT PAUSE or SELECT ARIRAM statement.

Order of priority for modes specified in the MR1 register

Pattern program specification	<	MR1 register specification	<	SELECT statement specification
RASREF, PAGE, TRAS	<	REG MPAT MR1	<	----
REFRESHA	<	REG MPAT MR1 (*1)	<	SELECT REFRESH
REFRESHB	<	REG MPAT MR1	<	SELECT REFRESH
PAUSE	<	REG MPAT MR1	<	SELECT PAUSE
ARIRAM DEFINE statement	<	REG MPAT MR1	<	SELECT ARIRAM

**1 It is impossible to control the REFRESHA mode using the MR1 register. When the MR1 register is set, the REFRESHA mode is canceled.*

RLMAX

RLMAX is the register used to set the counter to the refresh mode. If the value is not specified with this register, the default is the value of n in REFRESHA n (or REFRESHB n) in the MODE statement.

ND(ND1 to ND7)

ND is the register used with N register-related operations.

BD(BD1 to BD7)

BD is the register used with B register-related operations.

BH

BH is the register used to store the initial value of the B register. The initial value is loaded into the B register by the following operation.

$B < BH$

FPSEL

FPSEL is the register used to select an address function during the cycle in which the FPX instruction is written.

➔ For details, refer to [2. 5. 5 "FP Function Field" on page 2-146](#).

2. 2. 5 ARIRAM DEFINE Statement

ARIRAM DEFINE statement is set for inversion of the data pattern with the area inversion function. Set by ARIRAM DEFINE statement.

In case of the common section is not assigned, ARIRAM in the common section is set in DISABLE. When the module section is not assigned, ARIRAM in the common section is set as the default.

ARIRAM DEFINE statement can be defined by the scramble program. When this happens, priorities are assigned as follows: (The lower the number, the higher the priority.)

- 1. SCRAM specified in test plan program
- 2. ARIRAM DEFINE statement specified in pattern program module
- 3. SCRAM specified in pattern program module
- 4. ARIRAM DEFINE statement specified by common section of pattern program
- 5. SCRAM specified by common section of pattern program

➔ For details on ARIRAM DEFINE statement and regional inversion, refer to [4. 3. 4 "Area Inversion Test \(ARIRAM\)" on page 4-43](#).

There are three types of area inversion.

- Area inversion using the INV statement
According to X and Y address conditions and inverting condition given by the INV statement, pattern data D0 to 35 (D0 to 17 for the T5371, T5336 and T5311) is inverted.

ARIRAM DEFINE

X = Xk₁ O₁ Xk₂.....
Y = Yl₁ O₂ Yl₂.....
INV = X O₃ Y.....

} Specification of address range for inversion

The data pattern is inverted when inversion conditions are specified as INV = 1.

0 ≤ kn, 1m ≤ 11
On &= . AND.
"= . XOR.
'= . OR.
%= . NOT.

↑

↑

Meaning
Operator

Up to 50 operators and operational elements in total can be used.

◆ **When Using INV Statements with ARIRAM DEFINE Statement**

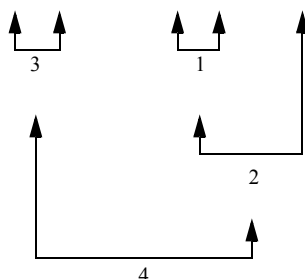
```
ARIRAM DEFINE
X = X7
INV = X
```

If the X7 to be entered in the DUT is set to X7 to 1, the data pattern to be entered in the DUT is inverted.

Also, the hierarchy of operations can be specified in the parentheses.

◆ **Priority of Logical Operations According to ()**

```
X = .NOT.X0.AND.(.NOT.X1.OR.X2)
```



In this case, operations are executed in the hierarchy of 1 to 4.

The maximum usable parentheses number is 127.



Limitation

Number types of X and Y elements described in the right sides is a total of 12 in conditional equations.

• **Area inversion on the data bit basis (1)**

Specifying address conditions inverts desired data bits.

```
ARIRAM DEFINE
Dn      = Xh1 O1 Yi1 ...
Dn, m   = Xh2 O1 Yi2 ... (a)
Dn-m    = Xh3 O1 Yi3 ... (b)
```

(a) and (b) can be specified at the same time.

n and m specify data bits. n and m cannot be omitted.

$0 \leq n, m \leq 17$

h and i must be specified within the ranges shown below.

For T5371, T5336 and T5311

$0 \leq h \leq 15$

$0 \leq i \leq 15$

For the T5377, T5376 and T5375

$$0 \leq h \leq 17$$

$$0 \leq i \leq 17$$

For operators, operational elements and "()", refer to area inversion using the INV statement on the preceding pages.

Limitation

- Number of types of X and Y element in the right sides should be 12 or less in total.

◆ Example of Describing X and Y

```
D0,1=X1.XOR.(Y0.AND..NOT.X7)
D2  =X1.XOR.X7.XOR.(Y0.AND.X6)
D3  =X1.XOR.X7.XOR.(Y0.AND.X6)
```

In the above example, 4 types of X and Y variables are used in the right sides.

- In the same module and to the same ALPG, this function cannot be used with the area inversion function using the INV or ARIn statement at the same time.
- If a data bit is given inverting conditions twice or more, the conditions given last are effective.

- Area inversion on each data bit (2)

```
ARIRAM DEFINE
  ARIl = Xh1 O1 Yi1 ...
  Dn = ARIl
  Dn, m = ARIl --- (a)
  Dn-m = ARIl --- (b)
```

(a) and (b) can be specified at the same time.

l must be specified within the ranges shown below.

$$0 \leq l \leq 7$$

h and i must be specified according to the following ranges:

For T5371, T5336 and T5311

$$0 \leq h \leq 15$$

$$0 \leq i \leq 15$$

For the T5377, T5376 and T5375

$$0 \leq h \leq 17$$

$$0 \leq i \leq 17$$

For operators, operational elements and "()", refer to area inversion using the INV statement on the preceding pages.

n and m specify data bits.

$0 \leq n, m \leq 17$

Limitation

- Twelve types of X and Y elements can be written on the right side of ARIn.
- In the same module and to the same ALPG, this function cannot be used with the area inversion function using the INV statement or $Dn =$ equations at the same time.
- If the ARIn is defined twice or more to the same data bit, the last defined value are effective.
- For $Dn = \text{ARIn}$, if ARIn is not defined in the upper line, a compiler error is occurred.

2. 2. 6 ADDRESS DEFINE Statement

Data specified by ADDRESS DEFINE statement is used to generate the test patterns below.

Set using the ADDRESS DEFINE statement.

In case of the common section is not assigned, ADDRESS data in the common section is set in DISABLE.

In case of the module section is not assigned, the data should be in accordance with ADDRESS data in the common section.

◆ N, Z and B

The address data in the N and Z registers as specified by ADDRESS DEFINE statement is output to the bits of addresses X4 to X17 and Y4 to Y17 (X4 to X15 and Y4 to Y15 for the T5371, T5336 and T5311) that are applied to the DUT.

On the T5377, T5376 and T5375, the address data in the B register as specified by ADDRESS DEFINE statement is output to the bits of addresses X0 to X17 and Y0 to Y17 that are applied to the DUT.

```
ADDRESS DEFINE
Xl-m = Zp-q, Nr-s
Yl-m = Zp-q, Nr-s
Xn-o = Bt-u
Yn-o = Bt-u
XY = XY.OR.ZN or ZN
      (ZN can be replaced by ZNB)
```

Specification Range	T5377, T5376 and T5375	T5371	T5336 and T5311
l, m	4 to 17	4 to 15	4 to 15
p, q	0 to 7	0 to 7	0 to 7
r, s	0 to 7	0 to 7	0 to 3
n, o	0 to 17	Unavailable	Unavailable
t, u	0 to 7	Unavailable	Unavailable

Specify address bits sequentially for p-q, r-s, and t-u.

The following instructions can be used to specify the output of either the bits defined by Xl-m = Zp-q, Nr-s, Yl-m = Zp-q, Nr-s, or the bits defined by Xn-o = Bt-u, Yn-o = Bt-u. This can be accomplished either by using the X or Y addresses of the bits in an OR operation or by giving priority to the specified Z, N and B addresses for output.

XY .OR. ZN (ZNB)

The X or Y address is ORed with the N, Z and B addresses, and then the result is output.

ZN (ZNB)

The X or Y address is inhibited, and only the N, Z and B addresses are output.

If XY = XY .OR. ZN (ZNB) or ZN (ZNB) is not specified, the X or Y address is ORed with the N, Z and B addresses, and then the result is output.

Important

- When the Z address is defined to the X or Y address, make specifications so as to achieve the following combination:

X (same for Y)													
17	16	15	14	13	12	11	10	9	8	7	6	5	4
Z5	Z4	Z3	Z2	Z1	Z0	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0
Z4	Z3	Z2	Z1	Z0	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	Z7
Z3	Z2	Z1	Z0	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	Z7	Z6
Z2	Z1	Z0	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	Z7	Z6	Z5
Z1	Z0	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	Z7	Z6	Z5	Z4
Z0	Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	Z7	Z6	Z5	Z4	Z3
Z7	Z6	Z5	Z4	Z3	Z2	Z1	Z0	Z7	Z6	Z5	Z4	Z3	Z2
Z6	Z5	Z4	Z3	Z2	Z1	Z0	Z7	Z6	Z5	Z4	Z3	Z2	Z1

- When the N address is defined to the X or Y address, make specifications so as to achieve the following combination:

For the T5377, T5376, T5375 and T5371

X (same for Y)													
17	16	15	14	13	12	11	10	9	8	7	6	5	4
N5	N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0
N4	N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N3
N3	N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N3	N2
N2	N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	N3	N2	N1
N1	N0	N7	N6	N5	N4	N3	N2	N1	N0	--	--	--	--
N0	N7	N6	N5	N4	N3	N2	N1	N0	--	--	--	--	--
N7	N6	N5	N4	N3	N2	N1	N0	--	--	--	--	--	--
N6	N5	N4	N3	N2	N1	N0	--	--	--	--	--	--	--

For the T5336 and T5311

X (same for Y)											
15	14	13	12	11	10	9	8	7	6	5	4
N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0
N2	N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3
N1	N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2
N0	N3	N2	N1	N0	N3	N2	N1	N0	N3	N2	N1

- When the B address is defined for the X or Y address, specify values so as to achieve the combinations below:

X (same for Y)																	
17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0
B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7
B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6
B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5
B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4
B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3
B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2
B2	B1	B0	B7	B6	B5	B4	B3	B2	B1	B0	B7	B6	B5	B4	B3	B2	B1

◆ Example of Using the ADDRESS DEFINE Statement

```
ADDRESS DEFINE
X8=N0          ; NO is ORed with X8, and the result is output to X8.
Y8=N1          ; N0 is ORed with Y8, and the result is output to Y8.
```

◆ Example of XY=ZN being Specified in an ADDRESS DEFINE Statement

```
ADDRESS DEFINE
X4-6 =Z0-2      ; Z0 to Z2 are output to X4 to X6.
Y12-15=Z0-3     ; Z0 to Z3 are output to Y12 to Y15.
XY=ZN
```

◆ Example of XY=XY.OR.ZNB being Specified in an ADDRESS DEFINE Statement

```
ADDRESS DEFINE
X0-3 =B1-4      ; B1 to B4 are ORed with X0 to X3, and the result is output to
                  X0 to X3.
Y14-17=B0-3     ; B0 to B3 are ORed with Y14 to Y17, and the result is output to
                  Y14 to Y17.
XY=XY.OR.ZNB
```

◆ PAGE

This instruction selects the address bit used to detect a page cycle during the auto page mode test.

```
ADDRESS DEFINE
PAGE = Xl - m, Yn - o
```

l, m, n, o = 0 to 17 (for T5377, T5376 and T5375)

l, m, n, o = 0 to 15 (for T5371, T5336 and T5311)

◆ **Example of PAGE being Specified in an ADDRESS DEFINE Statement**

```
ADDRESS DEFINE
PAGE = X0-7
```

◆ **X or Y Address is Interrupted by Z Address**

X or Y address is interrupted by Z address.

```
ADDRESS DEFINE
X = Z0
Y = Z0
or
X = Z0-n
Y = Z0-n
```

n is a bit number of the Z address. Bit numbers can be specified sequentially in ascending order.

n = 0 to 17 (for T5377, T5376 and T5375)

n = 0 to 15 (for T5371, T5336 and T5311)

The specified Z address bits are assigned to bits 0 to n of X (or Y).

◆ **Example of Causing the Z Address to Interrupt the X or Y Address in an ADDRESS DEFINE Statement**

```
ADDRESS DEFINE
X = Z0-7      : X0 to 7 is interrupted by Z0 to 7.
Y = Z0        : Z0 is interrupted by Y0.
```

2. 2. 7 BURST BLOCK Statement

Function: BURST BLOCK statement sets, in the on-the-fly mode, the condition for the setting alteration test, for example, a bump test of the measurement section specified by the OUT instruction.

Syntax

```
BURST BLOCK BEGIN(Block-name)
           : Setting test conditions
BURST BLOCK END
```

Argument: Block-name

Names referenced by OUT instruction.

Specify characters including alphabetic characters, numbers, and _ (underscore) of up to 32 characters beginning with an alphabetic character.

Description

Sixteen sets can be entered in a pattern program.

Data in [Table 2-2](#) can be set. Test conditions can be set in each block in [Table 2-2](#), up to a total of 31 words.

Table 2-2 Burst Data Word Count

Mnemonic	Number of words
VSn = vs, R (vr) (refer to Limitation) VSn = vs	2
WAIT TIME t	1

 Limitation

- Specify the n for VSn as between 1 to12.
- VS5 and VS9, VS6 and VS10, VS7 and VS11, and VS8 and VS12 cannot be specified simultaneously.
- Do not specify other than VS described in the test plan program.

2. 2. 8 SDEF Statement

Function: SDEF statement defines the address generation instruction and the data generation instruction with an arbitrary mnemonic. A pattern program can be described using the mnemonics specified by this instruction.

Syntax

SDEF name = instructions

Argument: name

Specify characters including alphabetic characters, numbers, and _ (underscore) of up to 32 characters beginning with an alphabetic character.

Up to 50 codes can be specified.

instructions

The data generation instruction and packet instruction



Limitation

Definitions for control bits N, I, T, TM1, and TM2 cannot be defined using SDEF statement.

Example

(1) Example of Using the SDEF Statement

```
SDEF INIT = XB<0  YB<0  TP<0
SDEF INC = XB<XB+1  YB<YB+1^BX
:
START      #0

      IDXI1 #6  INIT
ST0 :JNI2 ST0 W INC FP1
ST1 :JNI2 ST1 R INC FP1
      JZD  ST0
      STPS
      :
```

2. 2. 9 SCRAM Statement/PRESCRAM Statement

SCRAM statement

Specifies the name of a scramble program containing data of the address descrambler.

```
SCRAM name
SCRAM name1 name2
```

name

Specifies the name of a scramble program when NORMAL, DIVIDE1, DIVIDE2, XDIVIDE, YDIVIDE1, or YDIVIDE2 mode is specified in the SELECT SCRAM MODE statement.

name1

Specifies the name of a scramble program, which is used for DUT address conversion, when the SEPARATE mode is specified in the SELECT SCRAM MODE statement.

name2

Specifies the name of a scramble program, which is used for FM address conversion, when the SEPARATE mode is specified in the SELECT SCRAM MODE statement.

PRESCRAM statement

Specifies the name of a scramble program containing data of the prescrambler. The program name cannot be specified in the T5336 and T5311.

```
PRESCRAM name
```

2. 2.10 START Statement

Function: The START statement is used to define the start address for storing test pattern generating instructions.

Syntax

```
START n
```

Argument: n

The start address can be defined within the following range.

For the T5377, T5376 and T5375

$$\#0 \leq n \leq \#7FF$$

For the T5371

$$0 \leq n \leq \#3FF$$

For the T5336 and T5311

$$0 \leq n \leq \#1FF$$

Description

Up to one hundred START statements can be defined in a program.

Example**(1) Example of Storing from Address 0 of ALPG WCS**

```
START #0
IDX11 #6 INIT
ST0 : JN12 ST0 W INC FP1
ST1 : JN12 ST1 R INC FP1
JZD ST0
STPS
```

2. 2.11 DATA DEFINE Statement

Specify DATA DEFINE statement in the PDS DATA statement to select the data to be output to the programmable data selector (PDS).

2. 2.11. 1 PDS DATA Statement

Function: The PDS DATA statement selects the data to be output as pin data (<Dn> or <SDn> in the PIN statement).
The DATA DEFINE statement must be specified prior to the PDS DATA statement.

Syntax

```
PDS DATA = d/sd
```

Argument: d

Test data output as pin data D0 to 17

sd

Test data output as pin data SD0 to SD17

Description

Specify d/sd using the following combinations.

d	sd
D.XOR.FDA	D2
D.XOR.FDA	FDB
D	D2
FDA	D2
D	FDB
FDA	FDB
D.XOR.FDA	D2.XOR.FDB (Can be specified on the T5377, T5376 and T5375.)
D	D2.XOR.FDB (Can be specified on the T5377, T5376 and T5375.)
FDA	D2.XOR.FDB (Can be specified on the T5377, T5376 and T5375.)

If this statement is not specified in the common section, D.XOR.FDA/D2 is used for the common section.
If it is not specified in the module section, the specification in the common section is used for the module section.

Test data D, D2, FDA and FDB indicate the following.

D	ALPG TP1 (refer to Tip)
D2	ALPG TP2 (refer to Tip)
FDA	Output from PM in the pattern mode (EXP or DBM) of PM expected value.
FDB	Output from PM in the pattern mode (EXP or DBM) of PM expected value. (However, only when bit size is 36-bit long in EXP mode.)



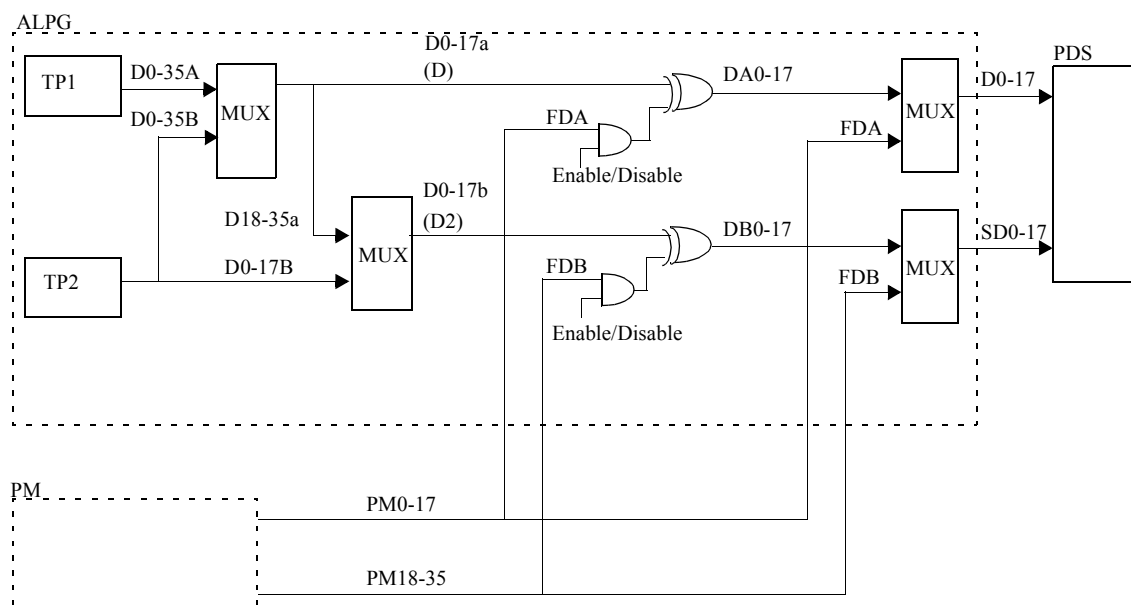
Tip

On the T5377, T5376 and T5375, the output of data D and D2 varies depending on the Source data field instruction and MODE TPMn specification.

➔ For details, refer to [2. 5. 13 "Source Data Field" on page 2-158](#).

Data flow with D, D2, FDA and FDB is shown on the T5377, T5376 and T5375.

Figure 2-5 D, D2, FDA and FDB Data Flow for the T5377, T5376 and T5375



Important

The definitions described in this section can be changed using the SELECT PDS DATA statement in the test plan program. (Test plan program specifications receive priority.)

2. 2.12 MODULE Statement

MODULE statement is a separator when two or more patterns are entered in one pattern program. The format always begins with MODULE BEGIN and terminates with MODULE END.

Normally, the START statement explicitly indicates from which address of WCS the module is allocated, and the REG MPAT PC statement in the test plan program specifies the module whose pattern is used for the test.

The section prior to the first module section in the pattern program is called the common section. A specification by the module section has priority over one by the common section, if the two sections specify different data.

In case of the common section is assigned and the module section is not assigned then the assignment in the common section is reflected to the module section.

➔ For examples of MODULE description, refer to ["Common Section and Module Section of Pattern Program" on page 2-2](#) and ["Module Description Example" on page 2-3](#).

2. 2.13 AFM PATTERN Statement

This statement declares the start of a ROM test pattern.

This statement is used to write an ROM test pattern in a pattern program. It can be declared for the common section and each module in the program.

➔ For more information on the ROM test pattern data description format, refer to [2. 7 "ROM Test Pattern Data Program Section" on page 2-185](#).

2. 2.14 DBM PATTERN Statement

This statement declares the start of a DBM test pattern.

The DBM PATTERN statement must be declared in the pattern program before specifying the random logic pattern to be stored into the DBM for the DUT test.

It can be declared for the common section and each module in the program.

➔ For more information on the DBM description format, refer to [2. 8 "DBM Pattern" on page 2-194](#).

2. 2.15 BANK SAVE ADDRESS Statement

The T5377 can save and load the row addresses on a bank basis by the XASV and XALD instructions in order to test memory devices which contain two or more banks.

➔ For more information, refer to [2. 6. 18 "X Address Save Field" on page 2-174](#).

By specifying each row bank address in the BANK SAVE ADDRESS statement, the row addresses (the X addresses of the ALPG) of each cycle, in which the XASV instruction is specified, can be saved for each bank address. (Up to 16 row addresses can be saved.)

The format of the BANK SAVE ADDRESS statement is as follows:

```
BANK SAVE ADDRESS a0 [,a1, a2, a3]
```

a0

Selects the address which corresponds to row bank address bit 0.

a1

Selects the address which corresponds to row bank address bit 1.

a2

Selects the address which corresponds to row bank address bit 2.

a3

Selects the address which corresponds to row bank address bit 3.

From a0 to a3 selects the address bits of the X and Y addresses (the addresses in the ALPG, that have not been through the PRESCRAM), which are used in the row bank addresses. The address bits that can be specified are as follows:

X0 to X17 and Y0 to Y17, 0

"0" indicates the Fix Low bit regardless of the bank address. (Indicates that the address bit is not selected.)

a1 to a3 can be omitted. In this case, the bank addresses for the omitted bits are not selected. (It is the same as selecting "0".)

◆ Example

```
BANK SAVE ADDRESS X13,X14
```

a0 to a3 can be specified as follows:

Stand-alone specification

Specify address bit one-by-one and separate by using commas.

◆ Example

```
BANK SAVE ADDRESS X13,X14,0,0
```

Consecutive specification

Specifies an address bit range by specifying the first and last address bits of the range and connecting with a hyphen.

Address bits can be specified consecutively in both ascending and descending order.

X_{n-m} $0 \leq n, m \leq 17$

Y_{n-m}

X_n-X_m

Yn-Ym

◆ **Example**

```
BANK SAVE ADDRESS X12-X15
```

Combining both stand-alone specification and consecutive specification

The stand-alone specification can be specified in combination with the consecutive specification.

◆ **Example**

```
BANK SAVE ADDRESS X12-X14, 0
```

The BANK SAVE ADDRESS statement can be specified in the common sections and the modules. If the BANK SAVE ADDRESS statement is not specified when transferring the common section, the bank address is not selected. (It is the same as selecting "0" for a0 to a3.) If the BANK SAVE ADDRESS statement is specified in the common section, the bank address is selected by the BANK SAVE ADDRESS statement.

If the BANK SAVE ADDRESS statement is not specified in modules when transferring the modules, the bank address is selected by the BANK SAVE ADDRESS statement in the common section. If the BANK SAVE ADDRESS statement is specified in the modules, the bank address is selected by the BANK SAVE ADDRESS statement in the modules.

 Tip

- The BANK SAVE ADDRESS statement is available for the T5377.
- The addresses selected for the X and the Y addresses specified by the BANK SAVE ADDRESS statement have not been through the PRESCRAM section.

2. 2.16 BANK LOAD ADDRESS Statement

The T5377 can save and load the row addresses on a bank basis by the XASV and XALD instructions in order to test memory devices which contain two or more banks.

➔ For more information, refer to [2. 6. 18 "X Address Save Field" on page 2-174](#).

By specifying the column bank addresses in the BANK LOAD ADDRESS statement, the row addresses which were saved by the XASV instruction can be loaded on a bank address basis in the cycle in which the XALD instruction is specified.

The format of the BANK LOAD ADDRESS statement is as follows:

```
BANK LOAD ADDRESS a0 [, a1, a2, a3]
```

a0

Selects the address which corresponds to column bank address bit 0.

a1

Selects the address which corresponds to column bank address bit 1.

a2

Selects the address which corresponds to column bank address bit 2.

a3

Selects the address which corresponds to column bank address bit 3.

From a0 to a3 selects the address bits of the X and Y addresses, which are used in the column bank addresses. The address bits that can be specified are as follows:

X0 to X17 and Y0 to Y17, 0

"0" indicates the Fix Low bit regardless of the bank address. (Indicates that the address bit is not selected.)

a1 to a3 can be omitted. In this case, the bank addresses for the omitted bits are not selected. (It is the same as selecting "0".)

◆ Example

```
BANK LOAD ADDRESS X13,X14
```

The specification methods of a0 to a3, the common sections, and the modules in the BANK LOAD ADDRESS statement are the same as those in the BANK SAVE ADDRESS statement.

➔ For more information, refer to [2. 2. 15 "BANK SAVE ADDRESS Statement" on page 2-42.](#)

Tip

- The BANK LOAD ADDRESS statement is available for the T5377.
- The address selected for the X address specified by the BANK LOAD ADDRESS statement has not been through the PRESCRAM section, and the address selected for the Y address has been through the PRESCRAM section. The address that has been through the PRESCRAM section is the address after being controlled by the following statement.
 - SEND PRESCRAM statement
 - SELECT PRESCRAM statement
 - SELECT PRESCRAM MODE statement
 - SELECT PRESCRAM ADDRESS statement
 - SET WRAP ADDRESS statement
 - YASL instruction

2. 2.17 END Statement

This statement terminates the compilation of a program.
Be sure to write this statement at the end of programs.

2. 2.18 Test Pattern Programming Format

The description format for the pattern program is as follows:

field A

PC operation code, operand & N, I, T, TM1, TM2

field B

ALPG instruction

---A--- ---B---

Tip

When description of Field B is omitted, default value is inserted.

The default value is the following instructions.

TS1 XB<XB YB<YB XC<XC XS<XS XK<XK YC<YC YS<YS YK<YK D3<D3

D4<D4 Z<Z N<N RF<RF X<XB Y<YB TP<TP DSD<DSD

RCD<RCD CCD<CCD B<B

2. 2.19 %REMARK Statement

By this statement, a character string following %REMARK statement and ended with the carriage return is output at the end of an object as a revision data.

Alphanumeric characters, asterisks (*), underlines (_), periods (.) and dollar marks (\$) are all legal.

The maximum number of characters is 80.

This statement may only be described once in a program.

Important

- This statement can be described in both the common section and module section. It cannot follow the START statement.
- If more than 80 characters are given, 81st character and followings are ignored.
- If this statement is used twice or more in a program, the second and subsequent occurrences are ignored.

2. 2.20 %IFE Statement, %IFN Statement, %ENDC Statement

Function: The %IFE, %IFN, and %ENDC statements are used for conditional compilation.

Syntax

(1) Specifying Alphanumeric Characters for a Condition Switch

```
%IFE X
    Group of statements to be compiled conditionally.
%ENDC

%IFN X
    Group of statements to be compiled conditionally.
%ENDC
```

(2) Specifying an Operation for a Conditional Switch

```
%IFE operation
    Group of statements to be compiled conditionally.
%ENDC

%IFN operation
    Group of statements to be compiled conditionally.
%ENDC
```

Argument: x

Specifies a global switch used as a conditional switch. Specify as a single alphanumeric character.

Operation

Global switch for logical operation. The maximum number of characters is 384.

Types of operations (logical operations)

A logical operation can be described in specifying a global switch used as a conditional switch. The following operators may be used.

Also, parentheses can be used to prioritize the operators.

.OR.	Logical sum
.XOR.	Exclusive OR
.AND.	Logical product
.NOT.	Negation

Priority of the logical operation

() → .NOT. → .AND. → .OR.

.XOR.

Priority between .OR. and .XOR. is written order.

Description

A single global switch

Since it is true that the global switch corresponding to a conditional switch of a conditional compilation statement is specified in a compilation command and it is false that the switch is not specified in the command, conditional compilation is performed as follows:

%IFE x	}	If true, compilation takes place. If false, no compilation.
:		
%ENDC		
%IFN x	}	If true, no compilation. If false, compilation takes place.
:		
%ENDC		

Logical operation

When the logical operation for the global switch is true, and the result is 1, and false when the result is 0.

Specify the global switch in the compilation command.

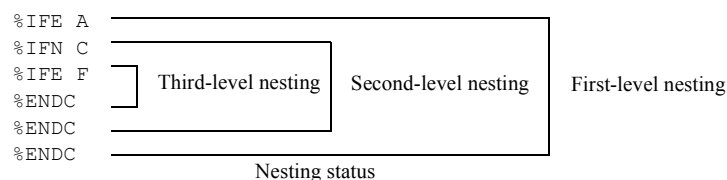
%IFE <i>operation expression</i>	}	If true, compilation takes place. If false, no compilation.
:		
%ENDC		
%IFN <i>operation expression</i>	}	If true, no compilation. If false, compilation takes place.
:		
%ENDC		

◆ Example

trans -C AF		
%IFE F.AND. (A.OR.H)	}	Compilation takes place.
:		
%ENDC		
%IFN F.AND. (A.OR.H)	}	No compilation
:		
% END C		

! Important

- Up to ten levels of nesting are allowed for conditional compilation statements.



- The total number of %IFE statements and %IFN statements need not be the same as the number of %ENDC statements, but if the number of %ENDC statements is larger, a compilation error occurs.
- Statements that would not be compiled according to a conditional compilation statement can be compiled by the following instructions:
%IFE, %IFN, %ENDC, %INSERT
→ For more information on the %INSERT statement, refer to [2. 2. 21 "INSERT Statement" on page 2-50](#).
- Conditional compilation statements must be at the left side of the lines.

Argument: File name

Specifies the name of the source file to be inserted. Specify as follows:

◆ Example

```
INSERT  ABC001
```

ABC001 is the name of the source file.

Description

The specified source file is inserted into the line and then compiled.

There are two types of INSERT statement : %INSERT and INSERT. The %INSERT statement is not affected by conditional compilation statements (%IFE, %IFN, and %ENDC). This means that the %INSERT file inserts the source file even though it will not be compiled by a conditional compilation statement.

☞ Limitation

- Up to five levels of nesting is allowed for INSERT statements.

◆ When Compiling ASC AO

ASC AO

```

MPAT  A0
REGISTER
  IDX1=#1
  IDX2=#2
INSERT BO
  IDX7=#7

START #0
NOP
STPS
END

```

ASC BO

```

IDX3=#3
INSERT CO
nmIDX6=#6

```

ASC CO

```

IDX4=#4
IDX5=#5

```

Compilation results

```

1st line      MPAT  A0
2nd line      REGISTER
3rd line      IDX1=#1
4th line      IDX2=#2
5th line
6th line      IDX3=#3
7th line
8th line      IDX4=#4
9th line      IDX5=#5
10th line     IDX6=#6
11th line     IDX7=#7
12th line
13th line     START #0
14th line     NOP
15th line     STPS
16th line     END

```

- Up to 49 INSERT statements can be used in one program.

2. 3 Pattern Sequence Controller

The pattern sequence controller is configured as shown in [Figure 2-6](#).

The test pattern generating instructions described in the pattern program are stored in the instruction memory (WCS).

As shown in [Figure 2-7](#), the pattern sequence controller is comprised of registers, memory, and a timer. The controller applies each operation instruction stored in WCS (instruction described in Field B) to the address generator, data generator, and MUT signal generator in ALPG respectively by decoding and executing the operation code (instruction described in Field A) stored in WCS.

Figure 2-6 Function Block Diagram of Sequence Control Section-1

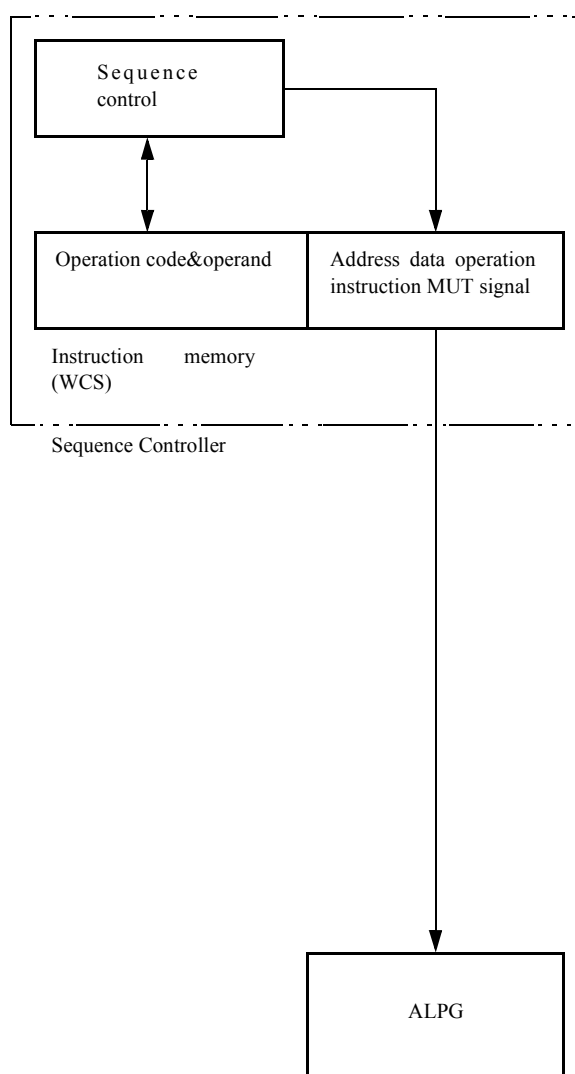
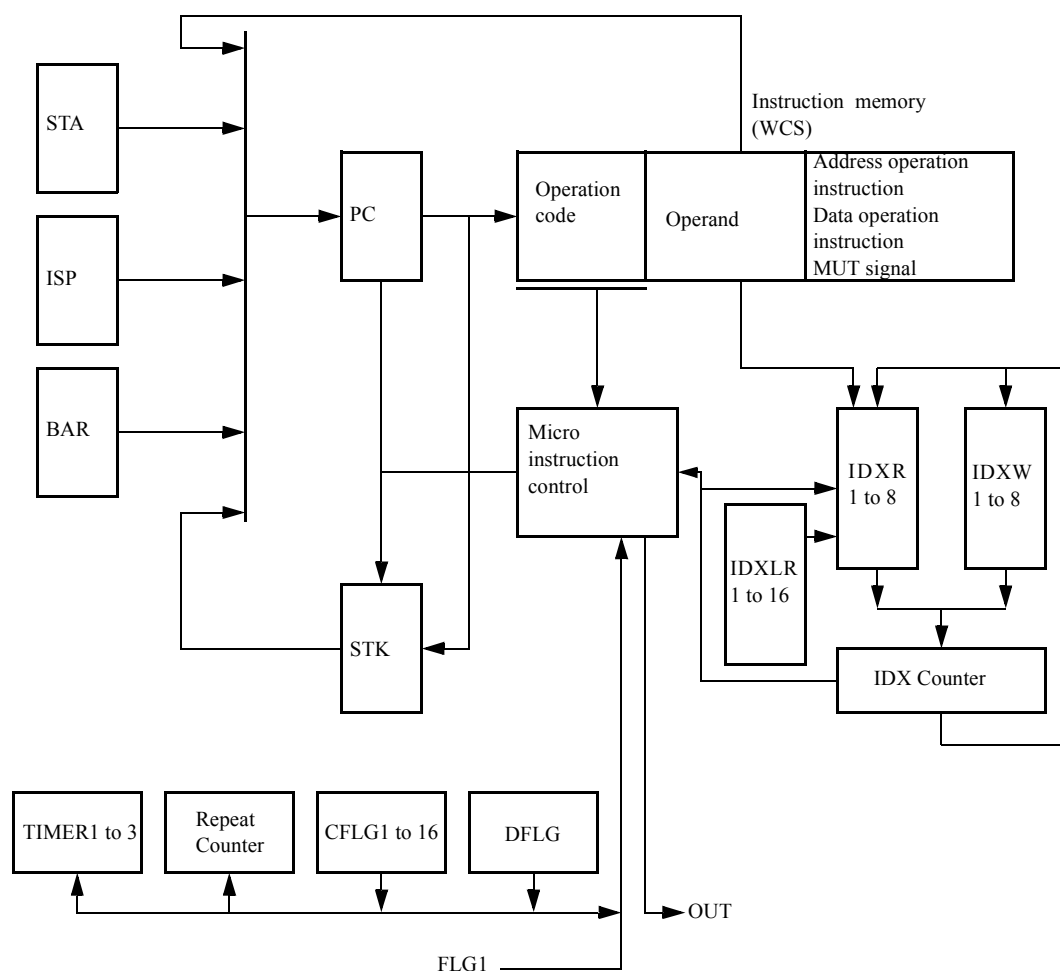


Figure 2-7 Function Block Diagram of Sequence Control Section-2

Tip

A Repeat Counter is present on the T5336 and T5311.

IDXLRL1 to 16 and CFLG9 to 16 are present on the T5377, T5376 and T5375.

◆ List of Pattern Sequence Control Instructions

The sequence control instruction (field A) has the following commands:

◆ **Field A**

NOP			N		I		T
INCn							TM1
FLGW1							TM2
WAIT							
RTN							
STPS							
STFL							
JMP	Branch address						
JSR							
JNIn							
JNInI							
JNInD							
JNCm							
FLGLI1							
FLGLIZ							
FLGL1							
FLGS1							
FLGJ1							
JZD							
JET							
HFLGLI							
FJNI8							
JAF							
STIn	Set value						
STTM							
STTM1							
STTM2							
STRC							
STBAR							
STISP							
IDXIn							
LDIn							
OUT	Block-name						

!

Important

- FLGLIZ can be executed on the T5377, T5376, T5375 and T5371.
- HFLGLI, FJNI8, STRC and N can be executed on the T5336 and T5311.
- "n" uses the value from 1 to 8.
- "m" has the following ranges:

T5377, T5376 and T5375

m = 1 to 16

T5371, T5336 and T5311

m = 1 to 8
- LDIn can be executed on the T5377, T5376 and T5375.

SET		Set Value
	XH	
	YH	
	ZH	
	NH	
	Z	
	ZD	
	XOS	
	YOS	
	XT	
	YT	
	D1	
	D2	
	D3B	
	D4B	
	XMASK	
	YMASK	
	DSD	
	CCD	
	RCD	
	TPH(TPH1)	
	TPH2	
	TP(TP1)	
	TP2	
	DCMR(DCMR1)	
	DCMR2	
	D5	
	D6	
	DBMAP	
	ND	
	BD	
	BH	

As shown below, there are three methods for specifying the destination of a branch instruction of the sequence control instructions.

- Specification with label

```
ST0: JN11 ST0
```

Specify characters including alphabetic characters, numbers, and _ (underscore) of up to 32 characters beginning with an alphabetic character.

The same label can be used for different modules.

- Specification of absolute address in WCS

```
START #0
NOP
JN11 #11
```

- Specification of relative address

```
JNI1 .
JNI1 .±n
```

Branches to an address before the current one (.-n), or an address after the current one (.+n).

If only a period is entered, the current address is specified.

[Table 2-3](#) and [Table 2-4](#) show operation code instructions and operations.

Test pattern sequences are generated by combining the various instructions shown in the table.

Since test pattern generation is stopped by the STPS or STFL instruction, enter STPS or STFL at the end of the test pattern.

PASS : Stopped with STPS

FAIL : Stopped with STFL

2. 3. 1 Index Loop

Use the index reference instruction for pattern program loops.

[Table 2-3](#) shows the index reference instructions of the sequence controller.

Table 2-3 Index Reference Instructions for the Sequence Controller

Mnemonic	Action
INCn (n = 1 to 8)	(PC) + 1 → PC (IDXRn) + 1 → IDXRn
STIn m (n = 1 to 8)	(PC) + 1 → PC m → IDXRn
JNIn m (n = 1 to 8)	1st m → PC (IDXRn) → IDX (IDX) → IDXWn 2nd if (IDXWn) = 0 then (PC) + 1 → PC else m → PC (IDXWn) → IDX (IDX) - 1 → IDXWn
JNInI m (n = 1 to 8)	1st m → PC (IDXRn) → IDX (IDX) → IDXWn 2nd if (IDXWn) = 0 then (PC) + 1 → PC (IDXRn) + 1 → IDXRn else m → PC (IDXWn) → IDX (IDX) - 1 → IDXWn

Mnemonic	Action
JNInD m (n = 1 to 8)	1st $m \rightarrow PC$ $(IDXR_n) \rightarrow IDX$ $(IDX) \rightarrow IDXW_n$ 2nd if $(IDXW_n) = 0$ then $(PC) + 1 \rightarrow PC$ $(IDXR_n) - 1 \rightarrow IDXR_n$ else $m \rightarrow PC$ $(IDXW_n) \rightarrow IDX$ $(IDX) - 1 \rightarrow IDXW_n$
IDXIn m	1st $(PC) \rightarrow PC$ $m \rightarrow IDX$ 2nd if $(IDX) = 0$ then $(PC) + 1 \rightarrow PC$ else $(PC) \rightarrow PC$ $(IDX) - 1 \rightarrow IDX$
LDIn m (n=1 to 8, m=1 to 16 or m = n) Can be executed on the T5377, T5376 and T5375.	$(PC) + 1 \rightarrow PC$ $IDXLR_m \rightarrow IDXR_n$

Index reference instructions perform loops using the IDXR1 to 8 registers, IDXW1 to 8 registers, and IDX counter, which are shown in [Figure 2-7](#).

Registers IDXR1 to 8 store the number of loops; the number of loops can be set using the following instructions:

- STIn instruction
- LDIn instruction
- Set by the REGISTER statement in the pattern program

```
REGISTER
IDXn=#FFFFFF n=1 to 8
```

- Set by the REG statement in the test plan program

```
REG MPAT IDXn=#FFFFFF n=1 to 8
```

The number of loops stored in the IDXRn register can be changed by using the following instructions.

- INCn instruction
- JNInI instruction
- JNInD instruction

Registers IDXW1 to 8 is used to save IDX counter data when performing nested loops. Each index reference instruction is explained below.

INCn (n=1 to 8)

Use this instruction when adding 1 to the number of loops stored in IDXRn.

STIn m (n=1 to 8, m=#0 to #FFFFFFF)

Use this instruction when setting the number of loops (m) in IDXRn.

JNIn m (n=1 to 8)

This instruction jumps to the PC indicated by operand the number of setting loops + 1 time shown by IDXRn, and executes PC+1→PC at the number of setting loops n + 2 times. Use it when creating pattern program loops.

JNInI m (n=1 to 8)

This instruction jumps to the PC indicated by operand the number of setting loops n + 1 time shown by IDXRn, and executes PC+1→PC and IDXRn+1→IDXRn at the number of setting loops n + 2 times. Use this instruction for the pattern program loop in which the number of loops is increases one by one at the same loop position.

JNInD m (n=1 to 8)

This instruction jumps to the PC indicated by operand the number of setting loops n + 1 time shown by IDXRn, and executes PC+1→PC and IDXRn-1→IDXRn at the number of setting loops n + 2 times. Use this instruction for the pattern program loop in which the number of loops is decreased one by one at the same loop position.

IDXIn m (n=1 to 8)

This is a one pattern loop instruction that holds PC (m+2) times indicated by the operand, and proceeds to the next step.

LDIn m (n=1 to 8, m=9 to 16 or m=n)

This instruction sets the number of loops stored in registers IDXLm to IDXRn.

If m=9 to 16, use the following instructions to store the number of loops in IDXLm.

- Set by the REGISTER statement in the pattern program

```
REGISTER
  IDXm=#0 to #FFFFFFF (m=9 to 16)
```

- Set by the REG statement in the test plan program

```
REG MPAT IDXm=#0 to #FFFFFFF (m=9 to 16)
```

If m = n (1 to 8), IDXLm contains the number of loops that has been stored in IDXRn by the following instructions.

- Set by the REGISTER statement in the pattern program

```
REGISTER
  IDXn=#0 to #FFFFFFF (n=1 to 8)
```

- Set by the REG statement in the test plan program

```
REG MPAT IDXn=#0 to #FFFFFFFF (n=1 to 8)
```

As a result, if the number of loops in IDXRn is changed by the LDIn, STIn, INCn, JNInI, or JNInD instruction, the original number of loops can be restored by executing the LDIn instruction with m = n.

◆ Example of a Program Using the LDIn Instruction

```

MPAT mpat_file_name
REGISTER
    IDX1 =#6      ; Setting of IDX1 and IDXL1
    IDX2 =#FD     ; Setting of IDX2 and IDXL2
    IDX3 =#3D     ; Setting of IDX3 and IDXL3
    IDX4 =#2      ; Setting of IDX4 and IDXL4
    IDX5 =#6      ; Setting of IDX5 and IDXL5
    IDX9 =#D      ; Setting of IDXL9
    IDX10=#3D     ; Setting of IDXL10
    IDX11=#FD     ; Setting of IDXL11
    IDX12=#6      ; Setting of IDXL12
    IDX13=#2      ; Setting of IDXL13

START #0
ST0: NOP
    JN1 ST0      ; #6+2 loops
    NOP
    JN1 ST0      ; #FD+2 loops
    NOP
    JN1 ST0      ; #3D+2 loops
    NOP
    JN1 ST0      ; #2+2 loops
    NOP
    JN1 ST0      ; #6+2 loops

    LDI1 9       ; IDXR1 ← IDXL9
    LDI2 10      ; IDXR2 ← IDXL10
    LDI3 11      ; IDXR3 ← IDXL11
    LDI4 12      ; IDXR4 ← IDXL12
    LDI5 13      ; IDXR5 ← IDXL13

    NOP
ST1: JN1 ST1     ; #D+2 loops
    NOP
    JN1 ST1     ; #3D+2 loops
    NOP
    JN1 ST1     ; #FD+2 loops
    NOP
    JN1 ST1     ; #6+2 loops
    NOP
    JN1 ST1     ; #2+2 loops

    LDI1 1       ; IDXR1 ← IDXL1
    LDI2 2       ; IDXR2 ← IDXL2
    LDI3 3       ; IDXR3 ← IDXL3
    LDI4 4       ; IDXR4 ← IDXL4
    LDI5 5       ; IDXR5 ← IDXL5
    JZD ST0
    STPS

```

} Return to number of loops set by the REGISTER statement

! Important

Follow the points below when creating programs using the index reference instructions.

- When STIm instruction is described in the step immediately after JNInI, JNInD, or INCn instruction.

```

      :
ST0 : JNInI ST0
      STIm
      :

```

```

      :
ST0 : JNInD ST0
      STIm
      :

```

```

      :
      INCn
      STIm
      :

```

When $n=m$, STIm instruction has been executed.

- When INC instruction is described in consecutive steps normally.

```

      :
      INCn
      INCm
      :

```

Normal operation does not occur when $n=m$.

- The following instruction combinations do not operate:

```

:
JNIn ST0
:
ST0 : JNIm ST1
:

```

```

:
JNInI ST0
:
ST0 : JNImI ST1
:

```

```

:
JNInD ST0
:
ST0 : JNImD ST1
:

```

When JNIn, JNInI, JNInD are issued successively for n=m, and when PC=ST0, the instructions operate normally.

- When a JNIm, JNImI, or JNImD instruction is issued in the step immediately after JNInI or JNInD.

```

:
ST0 : JNInI (or JNInD) ST0
      JNIm (or JNInI and JNInD)
:

```

When n=m, JNImI, JNImD, and JNIm instructions operate by referring IDX Rm before being incremented or decremented by JNInI, D instruction.

- When IDXIn instruction is used. (For the T5336 and T5311)

```

:
START #10 ; Address(PC)
      NOP ; #10
      IDXI1 #11 ; #11
      NOP ; #12
:

```

When the address describing instruction (PC) is the same as the number of loop m in IDXIn m, it is not operated normally.

2. 3. 2 Subroutine

Subroutine call instructions are shown in [Table 2-4](#).

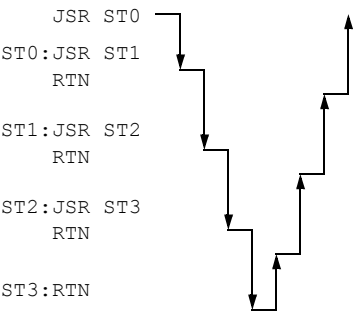
Table 2-4 Subroutine Call Instructions

Mnemonic	Action
JSR m	(PC) + 1 → STK ↓ (Push) m → PC
JSIn m (n = 1 to 8)	1st (PC) → STK ↓ (Push) m → PC (IDXRn) → IDX (IDX) → IDXWn 2nd if (IDXWn) = 0 then (PC) + 1 → PC else (PC) → STK ↓ (Push) m → PC (IDXWn) → IDX (IDX) - 1 → IDXWn
RTN	STK ↑ → PC (pop)

Up to eight subroutines can be nested in the T5377 and up to four subroutines can be nested in the T5376, T5375, T5371, T5336, and T5311.

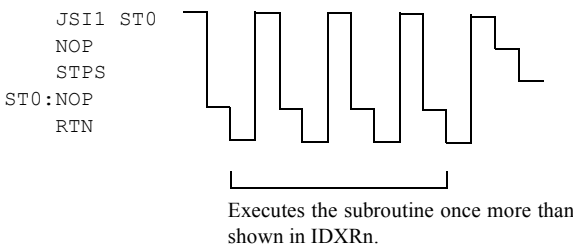
JSR m

This instruction pushes (PC)+1 to the subroutine stack and jumps to the PC shown by operand m.



JSIn m (n=1 to 8)

This instruction pushes (PC) to the subroutine stack and jumps to the PC shown by operand m. The instruction executes, then proceeds to the next step after executing this procedure once more than shown in IDXRn.



RTN

The instruction used to return from a subroutine-called subroutine. The instruction jumps to the PC shown in STK and pops the subroutine stack.

2. 3. 3 Operation Register Setting Instruction (SET reg. data)

SET register name m (m: setting value)

The contents of the registers used for address operation or data operation in ALPG can be set with the SET instruction.

Data, set in a register with the SET instruction, is valid in the next and subsequent steps. (Only the data set for the Z register is valid from a current step on the T5336 and T5311.)

```
D1 = #1
:
SET D1 #2      YC < YC + D1 ← D1=#1 used in operation
NOP           YC < YC + D1 ← D1=#2 used in operation
```

Table 2-5 shows the registers that can be set by the SET instruction.

Table 2-5 List of Registers that can be Set by the SET Instruction

Register	Bit size		
	T5377, T5376 and T5375	T5371	T5336 and T5311
XH	18	16	16
YH	18	16	16
ZH	18	16	16
NH	8	8	4

Register	Bit size		
	T5377, T5376 and T5375	T5371	T5336 and T5311
Z	18	16	16
ZD	18	16	16
XOS	18	16	16
YOS	18	16	16
XT	18	16	16
YT	18	16	16
D1(D1A)	18	16	16
D2(D2A)	18	16	16
D3B	18	16	16
D4B	18	16	16
XMASK	18	16	16
YMASK	18	16	16
DSD	18	16	16
CCD	18	16	16
RCD	18	16	16
TPH(TPH1, TPH1A)	18(36)	18	18
TPH2(TPH2A)	18(36)	18	18
TP(TP1)	18(36)	18	18
TP2	18(36)	18	18
DCMR(DCMR1)	18(36)	18	18
DCMR2	18(36)	18	18
D5	18(36)	18	18
D6	18(36)	18	18
DBMAP	24	16	16
ND(ND1)	8	--	--
BD(BD1)	8	--	--
BH	8	--	--

The bit widths of TPH1, TPH2, TP1, TP2, DCMR1, DCMR2, D5, and D6 vary as follows depending on MODE TPMn.

MODE TPM18:18 bits

MODE TPM36:36 bits

! Important

- The previous cycle data is held in the SET instruction execution.

```

      :
REGISTER
      TPH =#1
      :
      :          Output data
NOP      TP<0
NOP      0
NOP      TP<TPH  0
SET XH #0      0 ← TPH data is not output but the data in the
                  previous cycle is output.
NOP      1 ← TPH data is output in this cycle.
NOP      1
      :

```

The registers indicated with underlines are the same as any in [Table 2-5](#).

- The data set for a Z register is valid in different cycles between the T5377, T5376, T5375 or T5371 and the T5336 or T5311.

T5377, T5376, T5375, and T5371

Effective from the next step (Event time 2)

T5336 and T5311

Effective from a current step (Event time 1)

If ZSET1 mode is specified in the MODE statement, event time can be changed 2 to 1 on the T5377, T5376, T5375 or T5311.

Register name<m (DSET instruction, m: setting value)

The register which can be described by the SET instruction can easily specify the register setting without the SET instruction. The following registers can be set. However, the setting range is the same as the SET instruction.

Group A	Group B	Group C	
XH	YH	TP1	TP2
D1	D2	TPH1	TPH2
D3B	D4B	D5	D6
NH	ZH	DCMR1	DCMR2
ZD	Z	XMASK	YMASK
XOS	YOS	RCD	CCD
XT	YT	DSD	DBMAP
ND	BH		
	BD		

The different group registers can be described in the same cycles.

```
START #0
NOP   XH<#1FF  YH<#10  TP1<#1FF ...
NOP   D1<#100  D2<#0   DSD<#0 ...
```

The data which is set the same as the SET instruction is effective from the next step.
(Only the data set for the Z register is valid from a current step on the T5336 and T5311.)

```
REGISTER
D1=#1

START #0
NOP   D1<#2  YC<YC+D1 ← D1 = #1 used in operation
NOP   YC<YC+D1 ← D1 = #2 used in operation
```

!

Important

- This instruction can not describe in the cycle of the SET instruction.
- When this instruction is described in the same cycle as the operated instruction, the data setting in this instruction is prior to others.
- The control bit BLn instruction cannot be specified in the cycle in which the SET instruction and the DSET instruction which specifies the group A registers are specified. In addition, the control bit WPTSQ instruction and the WPTIL instruction cannot be specified in the cycle in which the SET instruction and the DSET instruction which specifies the group B registers are specified.
- The data set for a Z register is valid in different cycles between the T5377, T5376, T5375 or T5371 and the T5336 or T5311.

T5377, T5376, T5375, and T5371

Effective from the next step (Event time 2)

T5336 and T5311

Effective from a current step (Event time 1)

If ZSET1 mode is specified in the MODE statement, event time can be changed 2 to 1 on the T5377, T5376, T5375 or T5311.

2. 3. 4 Condition Flag Reference Branch (JNCn)

Table 2-6 shows the condition flag reference branch instruction.

Table 2-6 Condition Flag Reference Branch Instruction

Mnemonic	Action
JNCn m T5377, T5376 and T5375 n = 1 to 16 T5371, T5336 and T5311 n = 1 to 8	if CFLGn = 0 then (PC) + 1 → PC else m → PC

The JNCn instruction references CFLG (condition flags) when the sequence of the pattern program is changed in the test plan program.

```

:                                MPAT M16K
REG MPAT CFLG = #1              :
MEAS MPAT M16K                  JNC1 ST0
:                                NOP
                                ST0: NOP
:                                :
                                END

```

Branches to ST0 with CFLG1 = 1.

2. 3. 5 Data Flag Reference Branch (JZD)

Table 2-7 shows the data flag reference branch instruction.

Table 2-7 Data Flag Reference Branch Instruction

Mnemonic	Action
JZD m	if DFLG = 0 then m → PC 1 → DFLG else (PC) + 1 → PC 0 → DFLG

The JZD instruction references DFLG (Data Flag) when a data pattern to be entered in the DUT is inverted for a test. When DFLG=1 the contents of the TP register are inverted. TP2INV must be specified to invert the contents of the TP2 register.

```

NOP          XB < 0   YB < 0   TP < 0
ST0 : JN11 ST0 W XB < XB+1 YB < YB+1^BX
ST1 : JN11 ST1 R XB < XB+1 YB < YB+1^BX
      JZD ST0
      STPS

```

Executed 2 times by JZD instruction.
Outputs of DA0-17 and DB0-17 are inverted at the second execution.

!

Important

When a TP (TP2) operation instruction is specified in the loop of the JZD instruction, the inverted patterns, which are opposite to the first patterns, may not be generated in the second loop, depending on the operation result.

The TP (TP2) register must be initialized at the beginning of the loop before the TP (TP2) operation instruction is specified.

- This Example Shows the Program that cannot Generate the Inverted Patterns**

Patterns are inverted twice by the JZD and TP</TP instructions in the second loop. That is, patterns end up being equal to the original ones used in the first loop.

				1st time TP value	2nd time TP value
	NOP	XB<0	YB<0	TP<0	;
ST0:	JNI1	. W	XB<XB+1	YB<YB+1^BX	; 0
	JNI1	. R	XB<XB+1	YB<YB+1^BX	; 0
	NOP			TP</TP	; 0
	JNI1	. W	XB<XB+1	YB<YB+1^BX	/X /Y
	JNI1	. R	XB<XB+1	YB<YB+1^BX	/X /Y
	JZD	ST0			; 1
	STPS				

2. 3. 6 Timer Flag Reference Branch (JET)

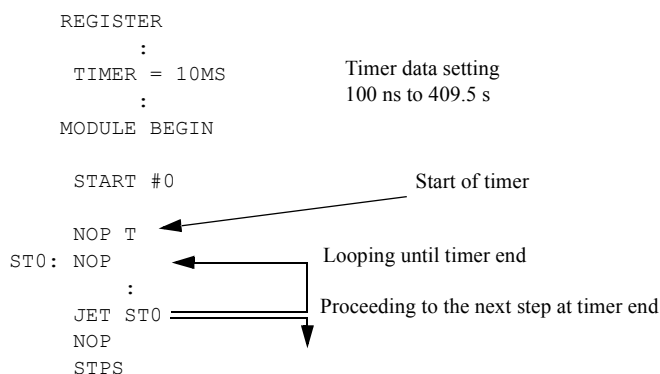
Table 2-8 shows the timer flag reference branch instruction.

Table 2-8 Timer Flag Reference Branch Instruction

Mnemonic	Action
JET m	if TFLG = 0 then m → PC else (PC) + 1 → PC 0 → TFLG

The JET instruction references TFLG (Timer flag) when the time program set in the internal timer is executed repeatedly.

TFLG is set when the internal timer ends and is cleared when it is referenced by this instruction.



The loop execution time of JET instruction is as follows.

Table 2-9 Loop Execution Time of the JET Instruction

Test system	AFM ACTION statement	The loop execution time of the JET instruction
T5377	No specification of the mask mode	Set value of a timer + Rate inside the loop of JET * 59
	With the specification of the mask mode	Set value of a timer + Rate inside the loop of JET * 135
T5376 and T5375	---	Set value of a timer + Rate inside the loop of JET * 54
T5371, T5336 and T5311	---	Set value of a timer + Rate inside the loop of JET * 25

--- in the table shows that the relevant test systems are independent of the specification by the AFM ACTION statement.

! Important

When using the JET instruction on the T5371, set the operation period (RATE) to 21 ns or more in all cycles.

2. 3. 7 Match Flag Sense Instruction

The match flag sense instruction is a conditional branch instruction based on a flag (FLG1) that indicates the result of a logical comparison. [Table 2-10](#) shows the match flag sense instructions.

Table 2-10 Match Flag Sense Instruction

Mnemonic	Action
FLGW1	If FLG1 = 0 then (PC) → PC else (PC) + 1 → PC

Mnemonic	Action
FLGJ1 m	If FLG1 = 0 then (PC) → PC else m → PC
FLGS1 m	If FLG1 = 0 then (PC) + 1 → PC else m → PC
FLGL1 m	If FLG1 = 0 then m → PC else (PC) + 1 → PC
FLGLI1 m	1st (IDXR8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC 2nd If (IDXW8) = 0 then (BAR) → PC else (IDXW8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC
FLGLIZ m (Available on the T5377, T5376, T5375 and T5371)	1st (IDXR8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC 2nd If FLG1 = 0 then If (IDXW8) = 0 then (BAR) → PC else (IDX) - 1 → IDXW8 m → PC else (PC) + 1 → PC

The following are control instructions for the flash memory simultaneous measurement function:
(Available on the T5336 and T5311)

➔ For details, refer to [T5300 Series Flash Memory Parallel Measurement Function Description\(No. : 8239605\)](#).

Mnemonic	Action
HFLGLI m	IF FLG1 = 1 then (PC) + 1 → PC else 1st (IDXR8) → IDX (IDX) → IDW8 m → PC 2nd If (IDW8) = 0 then (BAR) → PC else (IDW8) → IDX (IDX) - 1 → IDW8 m → PC
FJNI8 m	IF FLG1 = 1 then (ISP) → PC else 1st (IDXR8) → IDX (IDX) → IDW8 m → PC 2nd If (IDW8) = 0 then (ISP) → PC else (IDW8) → IDX (IDX) - 1 → IDW8 m → PC

FLG1 becomes "1" when it matches the expected value and "0" when it does not.

In the cycles in which match flag sense instructions are described, set the RATE so that the following condition is satisfied.

Table 2-11 RATE Setting Conditions for the Cycle in which the Match Flag Sense Instruction is Written

Test system	AFM ACTION statement	AFM RATE statement	RATE setting condition
T5377	No specification of the mask mode	HS	RATE ≥ STRB1 setting value + 1.0 μs
		MS	RATE ≥ STRB1 setting value + 1.4 μs
		LS	RATE ≥ STRB1 setting value + 1.7 μs
	With the specification of the mask mode	HS	RATE ≥ STRB1 setting value + 1.6 μs
		MS	RATE ≥ STRB1 setting value + 2.3 μs
		LS	RATE ≥ STRB1 setting value + 3.8 μs

Test system	AFM ACTION statement	AFM RATE statement	RATE setting condition
T5376 and T5375	---	HS, MS	RATE ≥ STRB1 setting value + 1.0 μs
		LS	RATE ≥ STRB1 setting value + 1.4 μs
T5371	---	---	RATE ≥ STRB1 setting value + 1.0 μs
T5336 and T5311	---	---	RATE ≥ STRB1 setting value + 1.4 μs

--- in the table shows that the relevant test systems are independent of the specification by the AFM ACTION statement and the AFM RATE statement.

➔ For details, refer to [5. 5. 1 "Using the Match Flag Sense Instruction" on page 5-44](#).

Limitation

The index instruction which can be used in the FJN18 instruction loop at the MODE HSMATCH setting is reserved for IDX17 and JN17.

2. 3. 8 Burst Data Transfer Instruction (OUT)

```
OUT m (m: burst block name)
```

The burst data transfer instruction changes the unit setting condition by using the realtime trigger at pattern declaration, for the BURST BLOCK data entered in the pattern program.

It is used for a bump test for DRAM.

2. 3. 9 Pattern Sequence Control Register Setting Instruction (ST reg)

STBAR

Mnemonic	Action
STBAR m	(PC) + 1 → PC m → BAR

STBAR sets a value (m) in the Branch Address Register (BAR). The BAR defines a conditional branch address for the FLGLI1, FLGLIZ or HFLGLI instruction.

STISP

Mnemonic	Action
STISP m	(PC) + 1 → PC m → ISP

STISP sets a value (m) in the Interrupt Save Pointer (ISP) register. The ISP register defines the branch address set by the PC if an interrupt from the internal timer occurs.

This instruction is used for the refresh test.

STRC (available on the T5336 and T5311)

Mnemonic	Action
STRC m	(PC) + 1 → PC m → RC

STRC sets a value (m) in the Repeat Counter (RC) register. The RC register defines the number of times that a nibble bit is repeated in the nibble mode test.

STTM, STTM1 and STTM2

Mnemonic	Action	Set value
STTM m	(PC) + 1 → PC m → Timer1, 2	m: 100ns to 409.5s #1 to #FFF
STTM1 m	(PC) + 1 → PC m → Timer1	
STTM2 m	(PC) + 1 → PC m → Timer2	m: 100ns to 409.5s

STTM, STTM1 and STTM2 instructions set the data of the internal timer 1 to 2.

Data set by the STTM instruction is valid when this step is entered.

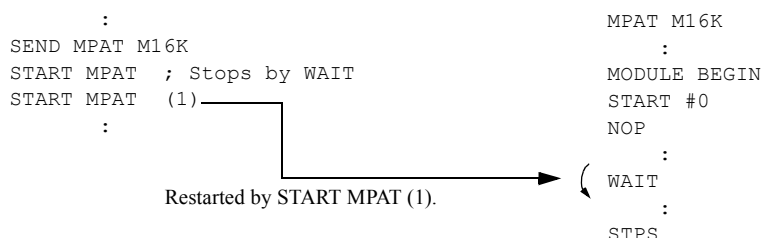
```

:
STTM 2MS
NOP T
:
Timer start
Data 2 ms set by the STTM instruction is valid.
```

2. 3.10 WAIT

The WAIT instruction stops pattern generation; it can be restarted by an instruction from the test plan program.

Control bits T, TM1, and TM2 cannot be specified for a cycle in which a WAIT instruction is used.



2. 3.11 STPS, STFL

STPS (Set pass) sets pass or STFL (Set fail) sets fail as the result when a test terminates. Then control of the program is returned to the test plan program. Be sure to write this statement at the end of each module.

STPS is used for a normal functional test.

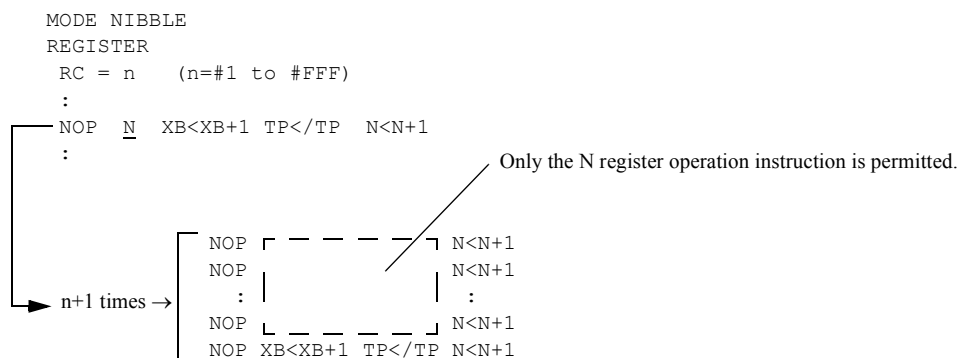
N, I, T, TM1 and TM2 control bits, address/data operation instructions, and MUT signals cannot be specified for a cycle in which STPS or STFL is used.

2. 3.12 Control Bits

The control bits in the main sequence controller are "N", "I", "T", "TM1" and "TM2".

N (available on the T5336 and T5311)

When nibble mode is specified by a MODE statement, a step describing control bit "N" is executed by the number of times set in the repeat counter (RC) + 1.



If N is coded in steps, the calculation instruction is repeated $n + 1$ times only for register N, and other operation instructions are executed in the last cycle.

I

In a cycle in which control bit "I" is described, an interruption into the interruption by the internal timer is inhibited.

Use control bit "I" to access only the nibble address repeatedly in combination with control bit "N" mainly in nibble mode.

```
RC = #3
:
IDXI1  #F  N I  R  N<N+1  ← The interruption occurred in an IDXI1
                                instruction is held until releasing the
                                IDXI1 instruction.
:
```

T, TM1, TM2

These control bits start the internal timer at a described cycle.
The timers started for T, TM1, and TM2 are as follows:

T
Simultaneously starts timers (Timer 1,^{*1} Timer 2^{*2}) specified by the MODE statement.

TM1
Starts Timer 1 only.

TM2
Starts Timer 2 only.

**1 A timer used for MODE REFRESHA, B*
**2 A timer used for MODE PAUSE*

 Important

For TM2 description, specify PAUSE mode in the MODE statement.

2. 3.13 Fail Flag Reference Branch (JAF)

Table 2-12 shows the fail flag reference branch instruction.

Table 2-12 Fail Flag Reference Branch Instruction

Mnemonic	Action
JAF m	if FAIL = 0 then (PC) + 1 → PC else m → PC

When all DUT of the test target are failed, the program branches to m address by JAF instruction.
➔ For details, refer to [T5300 Series Flash Memory Parallel Measurement Function Description\(No. : 8239605\)](#).

! Important

When the JAF instruction is used on the T5377, T5376 and T5375, set a test rate (RATE) 14 ns or longer for all cycles.

2. 4 Address Pattern Generation Instructions

2. 4. 1 Hardware Configuration of Address Generator Section

[Figure 2-8](#), [Figure 2-9](#) and [Figure 2-10](#) show function block diagrams for the address generator section.

Figure 2-8 Function Block Diagram of Address Generator Section (T5377)

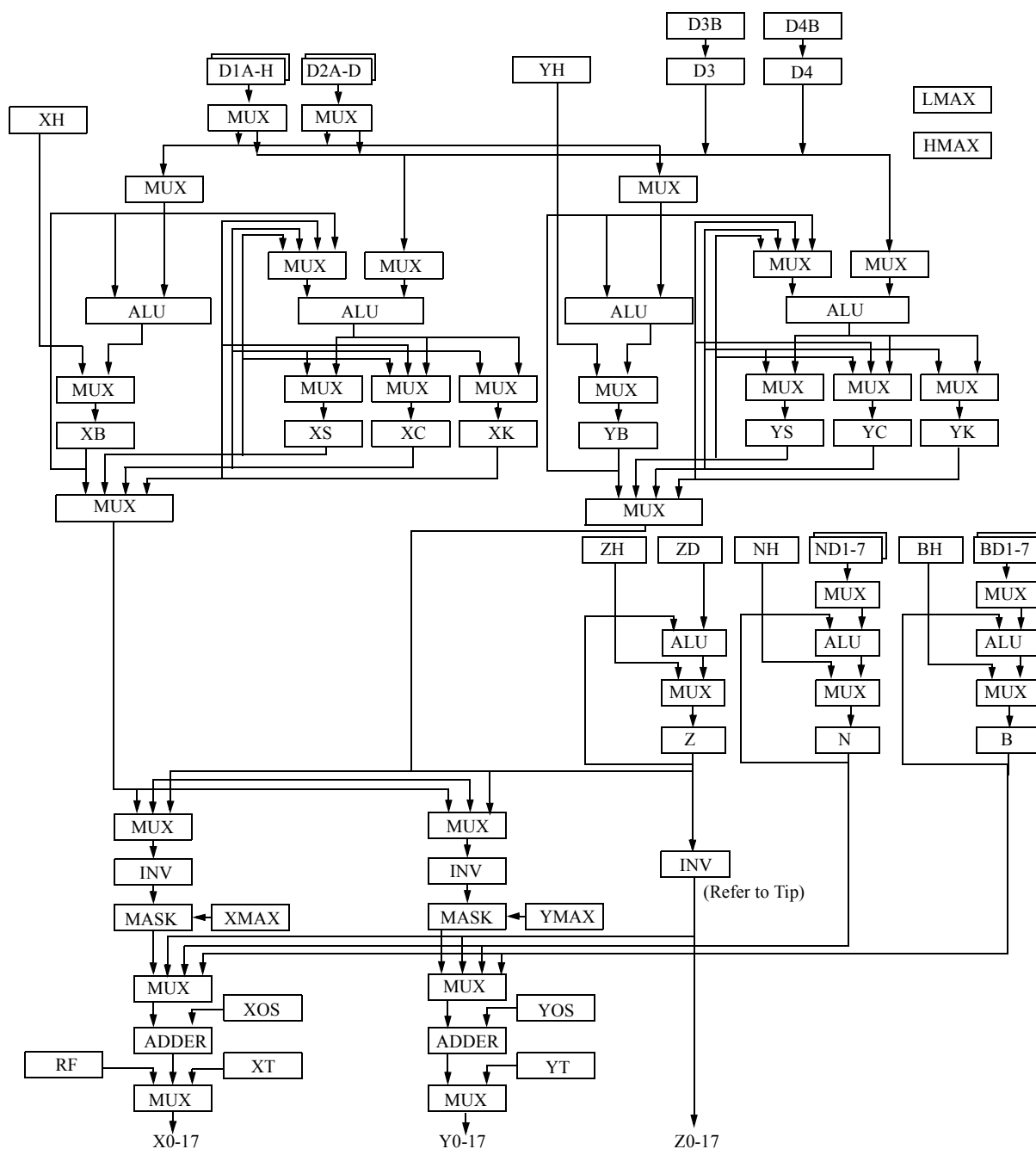
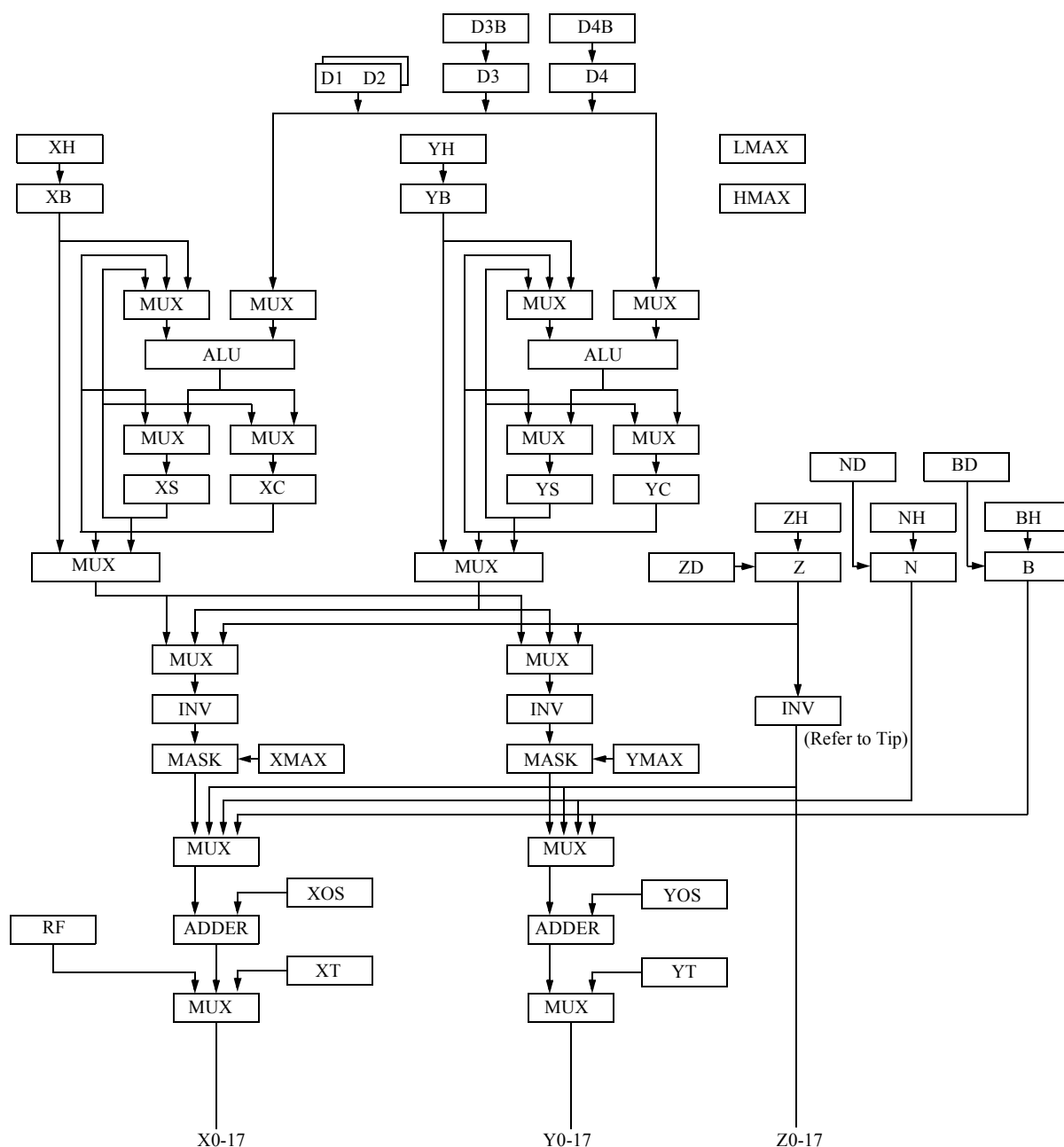


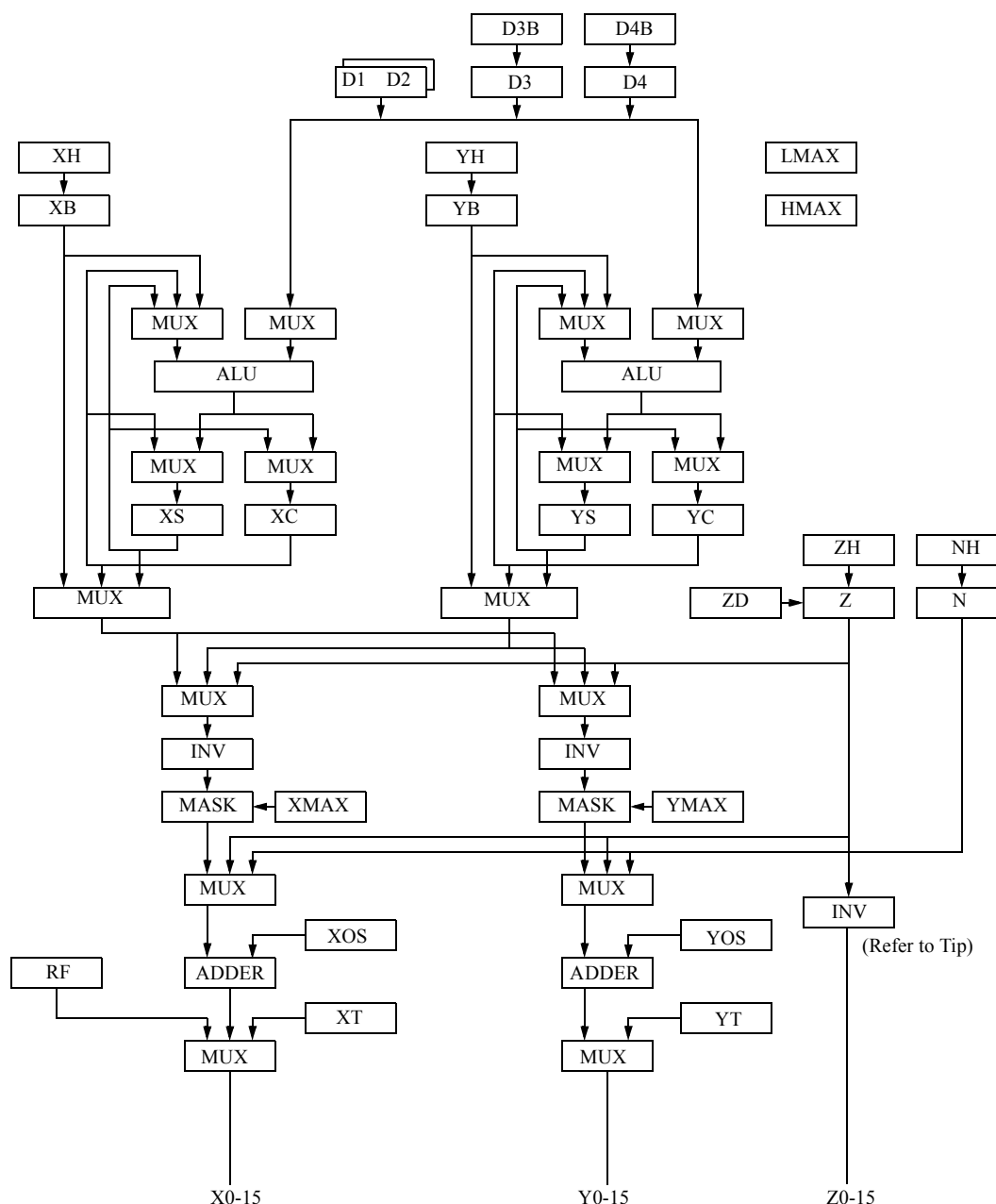
Figure 2-9 Function Block Diagram of Address Generator Section (T5376 and T5375)



Tip

When using the T5377, T5376 and T5375, the output from the Z-address inversion circuit (INV) is enabled before the Z address interrupts the X and Y addresses. (On the T5371, (INV) is enabled after the Z address interrupts the X and Y addresses.)

Figure 2-10 Function Block Diagram of Address Generator Section (T5371, T5336 and T5311)



 Tip

On the T5371, the output from the Z-address inversion circuit (INV) is enabled after the Z address interrupts the X and Y addresses. (On the T5377, T5376 and T5375, INV is enabled before the Z address interrupts the X and Y addresses.)

XB, YB, Z

The base register for each address. These registers are primarily used to generate target cell addresses.

XH, YH, ZH

Registers to store the initial values to be loaded into the base register of each address. These can be set by the SET instruction, DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test pattern program.

XC, YC

These are used to generate addresses (disturb addresses) for testing the relation to the cells to be tested, and to generate test pattern addresses that cannot be generated by base registers.

XS, XK, YS, YK

These are used mainly as save registers to save each address generated by XC and YC. Since XS, XK, YS and YK registers have functions of the same level as XC and YC registers, more complicated addresses can be generated by combining the XS, XK, YS and YK with XC and YC registers.

D1(D1A to D1H), D2(D2A to D2D)

Used as offset data when calculating the XB, YB, XC, YC, XS, YS, XK, and YK addresses.

D1A (D1) and D2A (D2) can be set by the SET and DSET instructions in the pattern program.

D1A to D1H and D2A to D2D can be set by the REGISTER statement in the pattern program and by the REG statement in the test plan program.

D3, D4

Used as offset data when calculating the XC, YC, XS, YS, XK, and YK addresses.

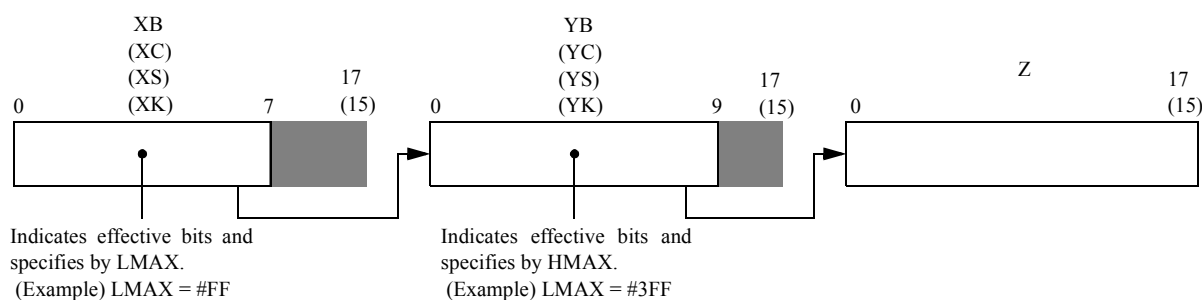
D3B, D4B

These are registers to store the initial values to be loaded into D3, and D4 registers. These can be set by the SET instruction, DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test pattern program.

LMAX, HMAX

LMAX and HMAX are used to specify the effective bits of the X and Y registers when using a conditional operation. These registers can set 2^n-1 data (n: the number of effective bits) by the REGISTER statement in the pattern program and the REG statement in the test plan program.

When MODE CSEL is specified, any value other than 2^n-1 can be set.



N, NH, ND(ND1 to ND7)

These registers are primarily used to generate bank addresses.

NH is used to store the initial value loaded into the N register.

ND1 to ND7 is used to operate the N register.

NH and ND1 can be set by the SET and DSET instructions in the pattern program. NH and ND1 to ND7 can be set by the REGISTER statement in the pattern program and by the REG statement in the test plan program.

B, BH, BD(BD1 to BD7)

These registers are primarily used to generate bank addresses.

BH is used to store the initial value loaded into the B register.

BD1 to BD7 is used to operate the B register.

BH and BD1 can be set by the SET and DSET instructions in the pattern program. BH and BD1 to BD7 can be set by the REGISTER statement in the pattern program and by the REG statement in the test plan program.

RF

The register used to generate the refresh address. Outputs to the refresh cycle as an X address

XMAX, YMAX

Registers to indicate the effective bits of each address to be output to X0-17, Y0-17 (X0-15, Y0-15 for the T5371, T5336 and T5311).

These registers can set 2^n-1 data (n: the number of effective bits) by the REGISTER statement in the pattern program and the REG statement in the test plan program.

XOS, YOS

Register to store the offset data to be added to the addresses. These are used to move the programmed pattern whose origin are addresses (0, 0) in the small area to the other area.

XOS and YOS can be set by the SET instruction, DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test plan program.

XT, YT

Registers to store the logical data when temporarily outputting the logical data to the address line. These are used to measure the device to be executed the mode setting from address line.

XT and YT can be set by the SET instruction, DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test plan program.

ZD

Register for Z address operation.

Z address cannot be applied to the DUT. But Z address can be generated inside of ALPG and interrupts to X/Y addresses.

ZD can be set by the SET instruction, DSET instruction and the REGISTER statement in the pattern program and by the REG statement in the test plan program.

2. 4. 2 Configuration of Address Generation Instruction

The address generator is configured with the following instructions:

- Base address field
- Current address field
- Source address field
- N register field
- Address inversion field
- Refresh counter field
- D3, D4 register field
- Address swap field
- Address shift field
- B register field

2. 4. 3 Outline of Address Generation Instruction

Shown below are the mnemonics of instructions in the address generation instruction part.

Only one instruction can be selected for a single cycle from a group of instructions enclosed by a pair of square brackets. If an instruction is enclosed in a rectangle, it indicates that that instruction has been selected from the group.

Base address field

- X base address field

XB<XB	A<D1A
XB<XB+1	A<D1B
XB<XB-1	A<D1C
XB<XH	A<D1D
XB<0	A<D1E
XB<XB+2	A<D1F
XB<XB-2	A<D1G
XB<A	A<D1H
XB<XB+A	A<D2A
XB<XB-A	A<D2B
	A<D2C
	A<D2D

- Y base address field

YB<YB	A<D1A *1
YB<YB+1	A<D1B *1
YB<YB-1	A<D1C *1
YB<YH	A<D1D *1
YB<YB+1^BX	A<D1E *1
YB<YB-1^BX	A<D1F *1
YB<YB+1+BX	A<D1G *1
YB<YB-1-BX	A<D1H *1
YB<0	A<D2A *1
YB<A *1	A<D2B *1
YB<YB+A *1	A<D2C *1
YB<YB-A *1	A<D2D *1
YB<YB+A^BX *1	
YB<YB-A^BX *1	
YB<YB+A+BX *1	
YB<YB-A-BX *1	

- Z base address field

Z<Z
Z<Z+1
Z<Z-1
Z<ZH
Z<Z*2
Z</Z
Z<0
Z<Z+1^BX
Z<Z-1^BX
Z<Z+1^CY
Z<Z-1^CY
Z<H
Z<ZD
Z<Z' ZD (OR)
Z<Z&ZD (AND)
Z<Z"ZD (XOR)
Z<Z+ZD
Z<Z-ZD
Z</ZD

*1 Can be executed on the T5377.

 Tip

- D1A can be described as D1.
- D2A can be described as D2.
- Registers D1A to D1H do not allow two or more descriptions in one step.
- Registers D2A to D2D do not allow two or more descriptions in one step.

2. 4 Address Pattern Generation Instructions

Current address field

• X current address field

XC<XC	XS<XS	XK<XK ^{*2}	F<A	A<XB	B<D1A
XC<F	XS<XS	XK<XK ^{*2}	F<A+1	A<XC	B<D1B ^{*2}
XC<XS	XS<XS	XK<XK ^{*2}	F<A-1	A<XS	B<D1C ^{*2}
XC<XK ^{*2}	XS<XS	XK<XK ^{*2}	F<A*2	A<XK ^{*2}	B<D1D ^{*2}
XC<XC	XS<F	XK<XK ^{*2}	F</A		B<D1E ^{*2}
XC<XC	XS<XC	XK<XK ^{*2}	F<B		B<D1F ^{*2}
XC<XC	XS<XS	XK<F ^{*2}	F<A+B		B<D1G ^{*2}
XC<XC	XS<XS	XK<XC ^{*2}	F<A-B		B<D1H ^{*2}
XCS<F ^{*1}		XK<XK ^{*2}	F<A&B		B<D2A
XCK<F ^{*1} ^{*2}	XS<XS		F<A"B		B<D2B ^{*2}
XCSK<F ^{*1} ^{*2}			F<A'B		B<D2C ^{*2}
XC<XS<F		XK<XK ^{*2}	F<0		B<D2D ^{*2}
XS<XC<F		XK<XK ^{*2}			B<D3
XC<XK<F ^{*2}	XS<XS				B<D4
XK<XC<F ^{*2}	XS<XS				
XC<>XS		XK<XK ^{*2}			
XC<>XK ^{*2}	XS<XS				

• Y current address field

YC<YC	YS<YS	YK<YK ^{*2}	F<A	A<YB	B<D1A
YC<F	YS<YS	YK<YK ^{*2}	F<A+1	A<YC	B<D1B ^{*2}
YC<YS	YS<YS	YK<YK ^{*2}	F<A-1	A<YS	B<D1C ^{*2}
YC<YK ^{*2}	YS<YS	YK<YK ^{*2}	F<A*2	A<YK ^{*2}	B<D1D ^{*2}
YC<YC	YS<F	YK<YK ^{*2}	F</A		B<D1E ^{*2}
YC<YC	YS<YC	YK<YK ^{*2}	F<B		B<D1F ^{*2}
YC<YC	YS<YS	YK<F ^{*2}	F<A+B		B<D1G ^{*2}
YC<YC	YS<YS	YK<YC ^{*2}	F<A-B		B<D1H ^{*2}
YCS<F ^{*1}		YK<YK ^{*2}	F<A&B		B<D2A
YCK<F ^{*1} ^{*2}	YS<YS		F<A"B		B<D2B ^{*2}
YCSK<F ^{*1} ^{*2}			F<A'B		B<D2C ^{*2}
YC<YS<F		YK<YK ^{*2}	F<0		B<D2D ^{*2}
YS<YC<F		YK<YK ^{*2}	F<A+1+CY		B<D3
YC<YK<F ^{*2}	YS<YS		F<A-1-CY		B<D4
YK<YC<F ^{*2}	YS<YS		F<A+B+CY		
YC<>YS		YK<YK ^{*2}	F<A-B-CY		
YC<>YK ^{*2}	YS<YS		F<A+1+BX		
			F<A-1-BX		
			F<A+B+BX		
			F<A-B-BX		
			F<A+1^CY		
			F<A-1^CY		
			F<A+B^CY		
			F<A-B^CY		
			F<A+1^BX		
			F<A-1^BX		
			F<A+B^BX		
			F<A-B^BX		

*1 The operation result, F, is loaded into the XC, XS and XK registers in the same operation cycle.

$XCS<F$; Loads F in the XC and XS registers

$XCK<F$; Loads F in the XC and XK registers

$XCSK<F$; Loads F in the XC, XS, and XK registers

The above applies also to the YC, YS, and YK registers.

A, B, and F in the mnemonics are formal registers for explanation. In programming, describe mnemonics by replacing the selected A, B, and F with the mnemonics on their right side.

(Example) When $XCS<F$ $F<A+B$ $A<XB$ $B<D1$ are selected



$XCS<XB+D1$; Mnemonic in programming

*2 Can be executed on the T5377.

Tip

- D1A can be described as D1.
- D2A can be described as D2.
- Registers D1A to D1H do not allow two or more descriptions in one step.
- Registers D2A to D2D do not allow two or more descriptions in one step.

Source address field

$X<XB$	$Y<YB$
$X<XC$	$Y<YC$
$X<XS$	$Y<YS$
$X<XK$ *1	$Y<YK$ *1
$X<YB$	$Y<XB$
$X<YC$	$Y<XC$
$X<YS$	$Y<XS$
$X<YK$ *1	$Y<XK$ *1
$X<XB'Z$	$Y<XB'Z$
$X<XC'Z$	$Y<XC'Z$
$X<XS'Z$	$Y<XS'Z$
$X<XK'Z$ *1	$Y<XK'Z$ *1
$X<Z$	$Y<Z$
$X<RF$	

*1 Can be executed on the T5377.

Tip

In a group of X address generation registers (XB, XC, XS, and XK) and a group of Y address generation registers (YB, YC, YS, and YK), different registers in the same group cannot be selected as a combination of the output selection of X and Y.

$X<XB$ $Y<XC$ (not allowed)

$X<YB$ $Y<YC$ (not allowed)

$X<XB$ $Y<XB$ (allowed)

$X<XB$ $Y<YB$ (allowed)

N register field

N<N		A<ND1	*1
N<NH		A<ND2	*2
N<N+1		A<ND3	*2
N<N-1	*1	A<ND4	*2
N<N+A	*1	A<ND5	*2
N<N-A	*1	A<ND6	*2
N<A	*2	A<ND7	*2

*1 Can be executed on the T5377, T5376, and T5375.

*2 Can be executed on the T5377.



Tip

ND1 can be described as ND.

B register field

B<B	*1	A<BD1	*1
B<BH	*1	A<BD2	*2
B<B+1	*1	A<BD3	*2
B<B-1	*1	A<BD4	*2
B<B+A	*1	A<BD5	*2
B<B-A	*1	A<BD6	*2
B<A	*2	A<BD7	*2

*1 Can be executed on the T5377, T5376, and T5375.

*2 Can be executed on the T5377.



Tip

BD1 can be described as BD.

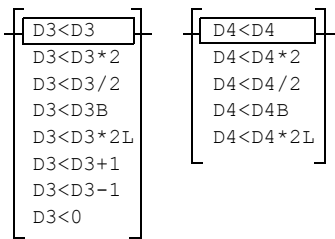
Address inversion field

[/X] [/Y] [/Z]

Refresh counter field

RF<RF
RF<RF+1

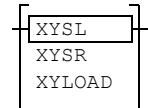
D3, D4 register field



Address swap field

[LMAX<>HMAX]

Address shift field



2. 4. 4 Base Address Field

The Base address field comprises the operation instructions, as described in the following, used to generate test target cell addresses for the generation of N^2 , $N^{3/2}$ type test patterns.

HOLD (XB<XB YB<YB Z<Z)

Used to execute nothing.
This instruction is omissible.

```
NOP XB<0 YB<0
ST0:JNi1 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
NOP XC<XB+1 YC<YB+1^BX X<XB Y<YB
|| Equivalent (Specifications can be omitted.)
NOP XB<XB YB<YB XC<XB+1 YC<YB+1^BX X<XB Y<YB
```


CLEAR (XB<0 YB<0 Z<0)

Used to initialize XB, YB and Z.

MPAT CHECK

REGISTER

XMAX=#7F

YMAX=#7F

IDX1=#3FFF

PC=#0

START #00

When using XB, YB and Z, initialize them
always at the first step.

NOP XB<0 YB<0

ST0:JNi1 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB

LOAD (XB<XH YB<YH Z<ZH XB<Dn YB<Dn)

Used to initialize XB, YB and Z. When this instruction is executed, the contents of the XH, YH, ZH, and Dn registers are loaded into the XB, YB and Z registers.

XB<XH XH → XB

YB<YH YH → YB

Z<ZH ZH → Z

XB<Dn Dn → XB

YB<Dn Dn → YB

Dn indicates registers D1A to D1H and D2A to D2D.

INC (XB<XB+1 YB<YB+1 Z<Z+1)

Used to increment the XB, YB and Z addresses. This instruction operates as follows:

XB<XB+1 XB+1 → XB

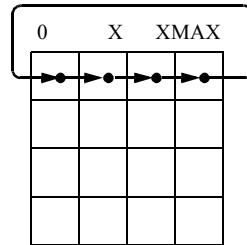
YB<YB+1 YB+1 → YB

Z<Z+1 Z+1 → Z

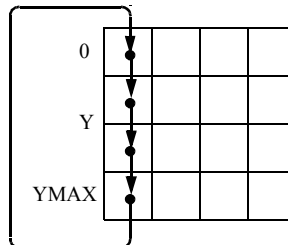
When the values of XB and YB registers are output on the X and Y addresses, the effective bits are the values specified as XMAX and YMAX.

The following address sequences can be generated by using this instruction.

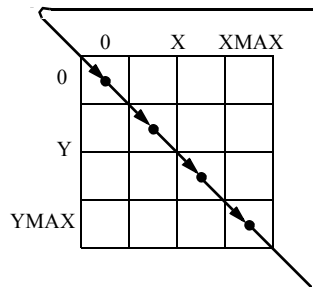
- ST0: JINn ST0 XB<XB+1 X<XB



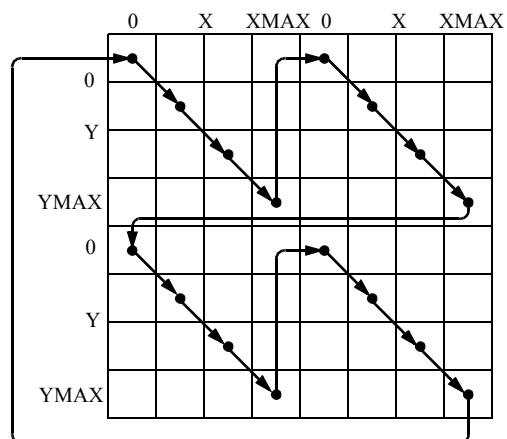
- ST0: JINn ST0 YB<YB+1 Y<YB



- ST0: JINn ST0 XB<XB+1 YB<YB+1 X<XB Y<YB



- ST0: JINn ST0 XB<XB+1 YB<YB+1 X<XB Y<YB
JINm ST0 XB<XB+1 YB<YB+1 Z<Z+1 X<XB Y<YB



Z=0	Z=1
Z=2	Z=3

DEC (XB<XB-1 YB<YB-1 Z<Z-1)

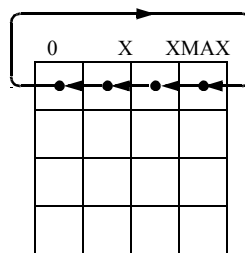
These are used to decrement each address of XB, YB, and Z. The following operations are executed.

XB<XB-1 (XB)-1 → XB

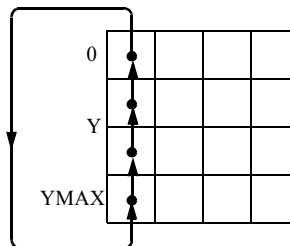
YB<YB-1 (YB)-1 → YB

Z<Z-1 (Z)-1 → Z

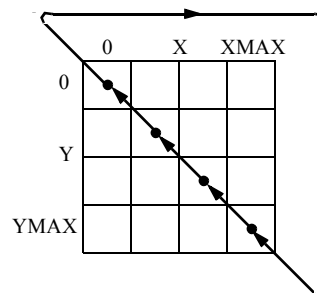
- ST0: JNIn ST0 XB<XB-1 X<XB



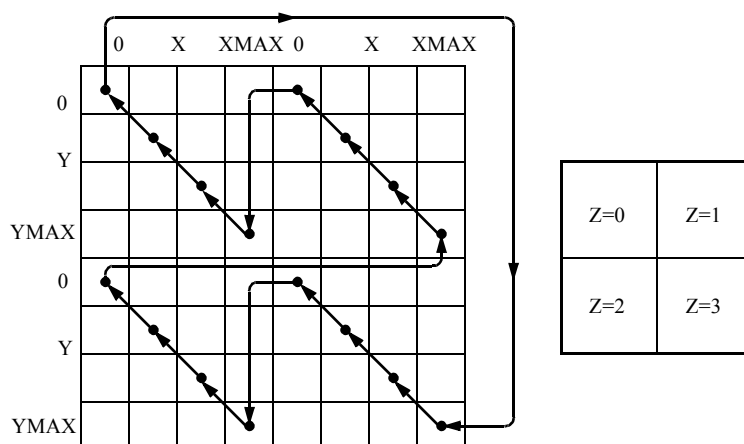
- ST0: JNIn ST0 YB<YB-1 Y<YB



- ST0: JNIn ST0 XB<XB-1 YB<YB-1 X<XB Y<YB



- ST0: JNIn ST0 XB<XB-1 YB<YB-1 X<XB Y<YB
JNIm ST0 XB<XB-1 YB<YB-1 Z<Z-1 X<XB Y<YB



Addition-subtraction instruction (XB<XB±Dn YB<YB±Dn Z<Z±ZD)

This instruction is used to add or subtract addresses in the XB, YB, and Z registers. The following operations are performed.

XB<XB±Dn	XB±Dn → XB
YB<YB±Dn	YB±Dn → YB
Z<Z±ZD	Z±ZD → Z

Dn indicates registers D1A to D1H and D2A to D2D.

Tip

When the values of XB and YB registers are output to the X and Y addresses, the effective bits are the values specified as XMAX and YMAX.

Conditional operation instruction

A conditional operation instruction is used to generate an address by connecting the X, Y, and Z addresses.

Conditional operation instruction YB or Z is performed based on the carry or borrow caused by the operation results of XB or YB.

The carry or borrow occurrence conditions vary depending on whether the CSEL mode is specified in the MODE statement.

- If MODE CSEL is not specified

A carry or borrow occurs according to the most significant bit of the significant bits stored in LMAX and HMAX during the XB or YB operation.

Data of 2^n-1 (n: number of significant bits) can be set in the LMAX and HMAX registers.

2. 4 Address Pattern Generation Instructions

The conditions and operation of the conditional operation instructions are shown below.

Conditional operation instruction	Condition	Decision	Operation
YB<YB+1^BX (Refer to Tip)	Carry or borrow caused by XB operation	then	YB←YB+1
		else	YB←YB
YB<YB-1^BX (Refer to Tip)	Carry or borrow caused by XB operation	then	YB←YB-1
		else	YB←YB
YB<YB+1+BX (Refer to Tip)	Carry or borrow caused by XB operation	then	YB←YB+2
		else	YB←YB+1
YB<YB-1-BX (Refer to Tip)	Carry or borrow caused by XB operation	then	YB←YB-2
		else	YB←YB-1
YB<YB+Dn^BX	Carry or borrow caused by XB operation	then	YB+Dn → YB
		else	YB → YB
YB<YB-Dn^BX	Carry or borrow caused by XB operation	then	YB-Dn → YB
		else	YB → YB
YB<YB+Dn+BX	Carry or borrow caused by XB operation	then	YB+Dn+1 → YB
		else	YB+Dn → YB
YB<YB-Dn-BX	Carry or borrow caused by XB operation	then	YB-Dn-1 → YB
		else	YB-Dn → YB
Z<Z+1^BX (Refer to Tip)	Carry or borrow caused by XB or YB operation	then	Z←Z+1
		else	Z←Z
Z<Z-1^BX (Refer to Tip)	Carry or borrow caused by XB or YB operation	then	Z←Z-1
		else	Z←Z
Z<Z+1^CY	Carry or borrow caused by XB or YB operation	then	Z←Z+1
		else	Z←Z
Z<Z-1^CY	Carry or borrow caused by XB or YB operation	then	Z←Z-1
		else	Z←Z

Dn indicates registers D1A to D1H and D2A to D2D.

Tip

On the T5371, this instruction cannot be written with the instruction (XB<XB-1) when it generates a borrow because of an XB operation.

- If MODE CSEL is specified

A carry occurs if $XB = LMAX$ or $YB = HMAX$ in the XB or YB operation. No conditional operations are enabled with a borrow.

Data other than $2^n - 1$ (n : number of effective bits) can be set in the $LMAX$ and $HMAX$ registers.

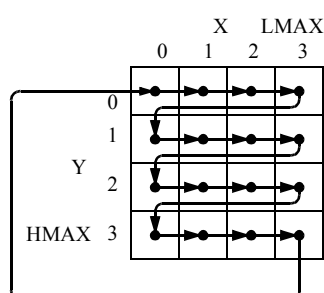
The conditions and operation of the conditional operation instructions are shown below.

Conditional operation instruction	Condition	Decision	Operation
$YB < YB + 1 \wedge BX$	$XB = LMAX$	then	$YB \leftarrow YB + 1$ ($XB \leftarrow 0$)
		else	$YB \leftarrow YB$
$Z < Z + 1 \wedge BX$	$XB = LMAX$ and $YB = HMAX$	then	$Z \leftarrow Z + 1$ ($XB, YB \leftarrow 0$)
		else	$Z \leftarrow Z$
$Z < Z - 1 \wedge BX$	$XB = LMAX$ and $YB = HMAX$	then	$Z \leftarrow Z - 1$ ($XB, YB \leftarrow 0$)
		else	$Z \leftarrow Z$

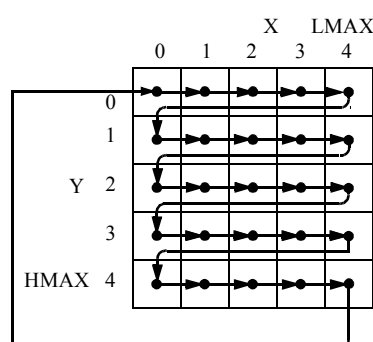
Examples of using conditional operation instructions to generate address sequences are shown below.

- ST0: JNIn ST0 $XB < XB + 1$ $YB < YB + 1 \wedge BX$ $X < XB$ $Y < YB$

When $LMAX, HMAX = 2^n - 1$



When $LMAX, HMAX \neq 2^n - 1$
(MODE CSEL is specified)



- ST0: JNIn ST0 $XB < XB - 1$ $YB < YB - 1 \wedge BX$ $X < XB$ $Y < YB$

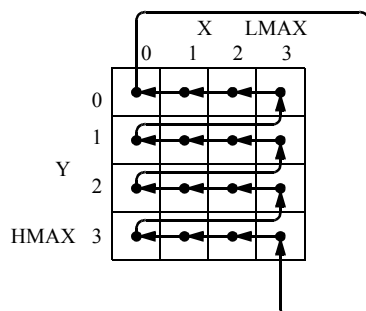
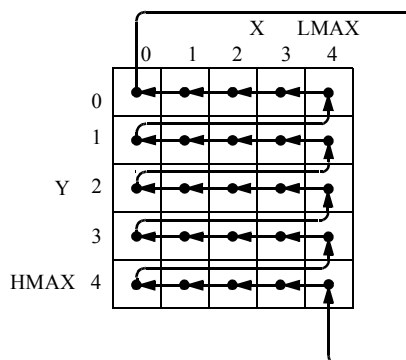
When MODE CSEL is specified, no operation such that $YB < YB - 1 \wedge BX$ can be performed. When using the T5336 or T5311, the address sequence (B) must be used if the operation $YB < YB - 1 \wedge BX$ is used.

If the address sequence (A) is generated, the operation must be as follows:

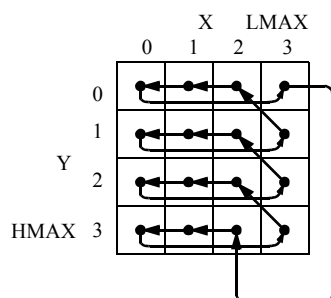
ST0: JNIn ST0 $XB < XB + 1$ $YB < YB + 1 \wedge BX$ $X < XB$ $Y < YB / X / Y$

2. 4 Address Pattern Generation Instructions

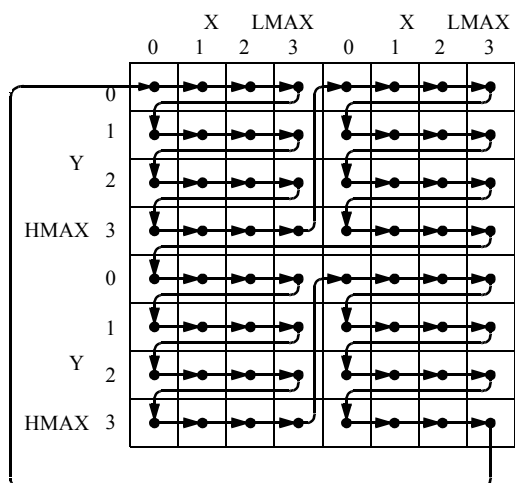
(A)

When LMAX, HMAX = $2^n - 1$ When LMAX, HMAX $\neq 2^n - 1$
(MODE CSEL is specified)

(B)

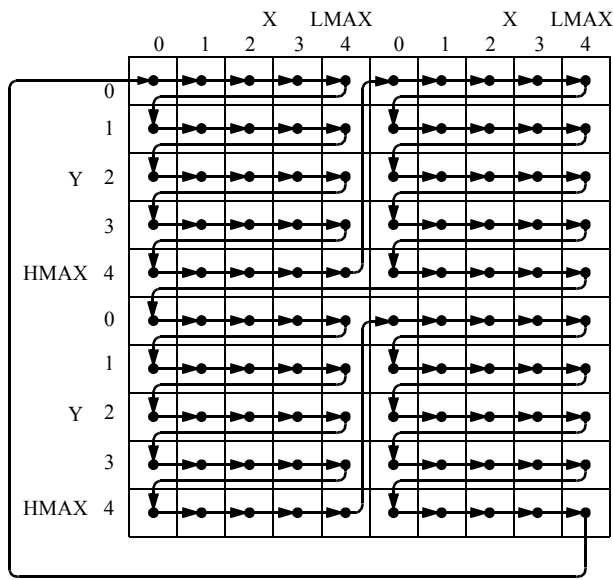


- ST0: JNIn ST0 XB<XB+1 YB<YB+1^BX Z<Z+1^BX X<XB Y<YB
When LMAX, HMAX = $2^n - 1$

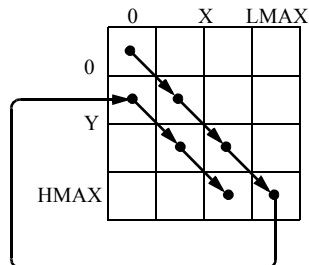


Z=0	Z=1
Z=2	Z=3

When LMAX, HMAX $\neq 2^n - 1$
(MODE CSEL is specified)



- ST0: JNIn ST0 XB<XB+1 YB<YB+1+BX X<XB Y<YB



Shift ($Z < Z*2$, $Z < Z/2$)

This instruction is used to control the block selection of memory modules, etc., that can be block selected.

$Z < Z*2$ $Z*2 \rightarrow Z$ Shift left (LSB=0)

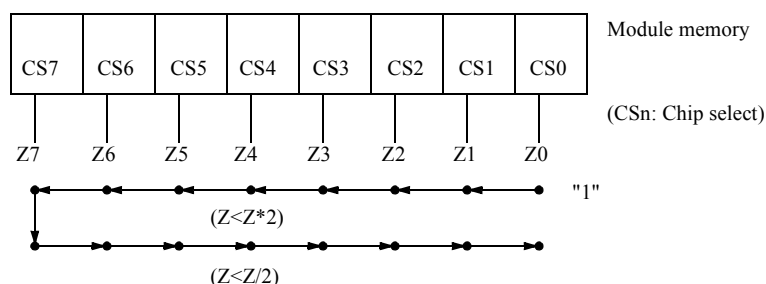
$Z < Z/2$ $Z/2 \rightarrow Z$ Shift right (MSB=0)

◆ Example of Using $Z<Z*2$, $Z<Z/2$

```

REGISTER
  ZH = #1
  IDX1=#6
  :
  NOP      Z<ZH
ST0 :  JSR   ST2
      JN11  ST0  Z<Z*2
ST1 :  JSR   ST2  Z<Z/2
      JN11  ST0
      :
ST2 :
      :
      Test for 1 block
      :
      RTN

```



As shown above, the generation of test patterns of modules, etc., that are executing output select by CS with common output can be tested for each block at one place.

Complement ($Z</Z$)

This instruction reverses the internal window of Z register.

$Z</Z$.NOT. (Z)→ Z

Other ($Z<H$)

This instruction turns all the output of the Z register to "1" in only the registered next cycle without changing the contents of the register.

$Z<H$

T5377, T5376 and T5375

Z0 to 17=#3FFFF

T5371, T5336 and T5311

Z0 to 15=#FFFF

2. 4. 5 Current Address Field

The current address field is used to generate test cells and relation-testing addresses (disturb addresses) in N^2 , $N^{3/2}$ type test pattern, and to generate the addresses of test cells in N type test patterns that cannot be generated by a Base field operation.

The operation instruction of the current address field comprises the following three types of instruction groups.

- $C[S][K] < f(B)$
- $C[S][K] < f(C[S][K])$
- $C[S][K] < f(B, C[S][K], t)$

B shows XB or YB.

$C[S][K]$ shows XC, YC, XS, YS or XK, YK.

◆ $C[S][K] < f(B)$

$C[S][K] < f(B)$ is a group of instructions used to obtain the addresses of disturb cells to be stored in XC, YC, in XS, YS or in XK, YK through the operation from the address of the test cells to be stored in XB, YB. $C[S][K] < f(B)$ comprises the following instructions.

$C[S][K] < B$	
$C[S][K] < B \pm 1$	
$C[S][K] < B \pm Dn$	
$C[S][K] < B * 2$	Shift left
$C[S][K] < B / B$	Complement
$C[S][K] < B \& Dn$	Logical product
$C[S][K] < B \text{ " } Dn$	Exclusive OR
$C[S][K] < B \text{ ' } Dn$	Logical sum
$C[S][K] < B \pm 1 \wedge BX$ (Refer to Tip)	} Only YC, YS or YK is valid
$C[S][K] < B \pm Dn \wedge BX$ (Refer to Tip)	
$C[S][K] < B \pm 1 \pm BX$ (Refer to Tip)	
$C[S][K] < B \pm Dn \pm BX$ (Refer to Tip)	
$C[S][K] < B \pm 1 \wedge CY$	
$C[S][K] < B \pm Dn \wedge CY$	
$C[S][K] < B \pm 1 \pm CY$	
$C[S][K] < B \pm Dn \pm CY$	



Tip

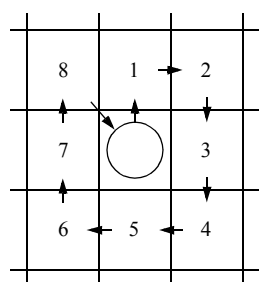
On the T5371, this instruction cannot be written with the instruction ($XB < XB-1$) when it generates a borrow because of an XB operation.

- $C[S][K] < B$ (XC(XS)(XK) < XB, YC(YS)(YK) < XB)
These are used to initialize the registers of XC(XS)(XK) and YC(YS)(YK). The contents of the registers XB and YB are loaded into the registers of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] < B \pm 1$
Use this instruction to generate the addresses of before and after (± 1) the address of the test cells stored in XB and YB.

The values of $XB \pm 1$ and $YB \pm 1$ are loaded into $XC(XS)(XK)$ and $YC(YS)(YK)$. Use this instruction to initialize the generation of the Surround-disturb pattern used for accessing only the periphery of test cells, and the generation of the address of the disturb cell in N^2 type test patterns such as Galloping and Walking.

See the examples below.

◆ **Surround disturb**



:

```

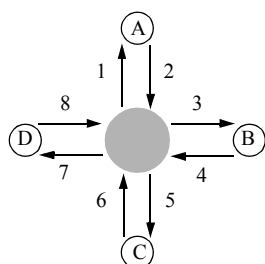
NOP XC<XB    YC<YB-1
NOP XC<XB+1  YC<YB-1
NOP XC<XB+1  YC<YB
NOP XC<XB+1  YC<YB+1
NOP XC<XB    YC<YB+1
NOP XC<XB-1  YC<YB+1
NOP XC<XB-1  YC<YB
NOP XC<XB-1  YC<YB-1

```

:

2. 4 Address Pattern Generation Instructions

This instruction is used in combination with the operation function of the D3 register for Butterfly pattern generation. A Butterfly pattern generation example is shown in the following.



After accessing sequentially, from 1 to 8, the distance from the target cell is incremented, and the operation repeated.

● are target cells. XB, YB

○ are disturb cells. XC, YC

```

REGISTER
D3B=#1
:
START #00

NOP XB<0 YB<0 TP<TPH D3<D3B
ST0: JN11 ST0 W XB<XB+1 YB<YB+1 ^BX X<XB Y<YB
ST1: NOP W YC<YB-D3 X<XB Y<YB /D
ST2: NOP R X<XB Y<YC
NOP R XC<XB+D3 X<XB Y<YB /D
NOP R X<XC Y<YB
NOP R YC<YB+D3 X<XB Y<YB /D
NOP R X<XB Y<YC
NOP R XC<XB-D3 X<XB Y<YB /D
NOP R X<XC Y<YB D3<D3+1
JN12 ST2 R YC<YB-D3 X<XB Y<YB /D
JN13 ST1 W XB<XB+1 YB<YB+1 X<XB Y<YB D3<D3B
:

```

- $C[S][K] < B \pm 1, Dn^{\wedge}BX \ (YC(YS)(YK) < YB \pm 1^{\wedge}BX, YC(YS)(YK) < YB \pm Dn^{\wedge}BX)$

This instruction is used to link XB to YC, YS or YK. It operates as follows.

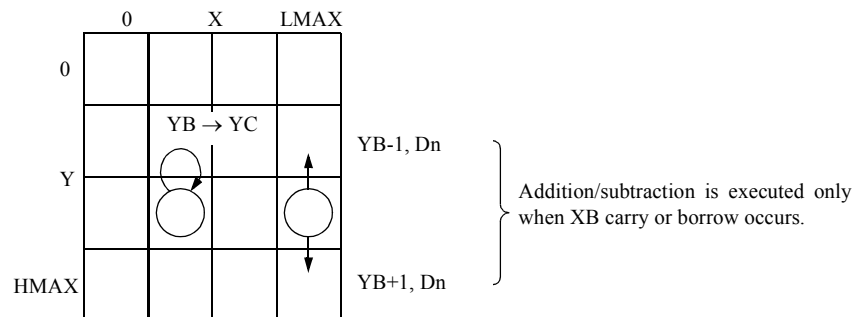
$YC(YS)(YK) < YB \pm 1, Dn^{\wedge}BX$

When no XB carry or borrow occurs YB $\rightarrow$ YC, YS or YK

When XB carry or borrow occurs $YB \pm 1, Dn \rightarrow$ YC, YS or YK

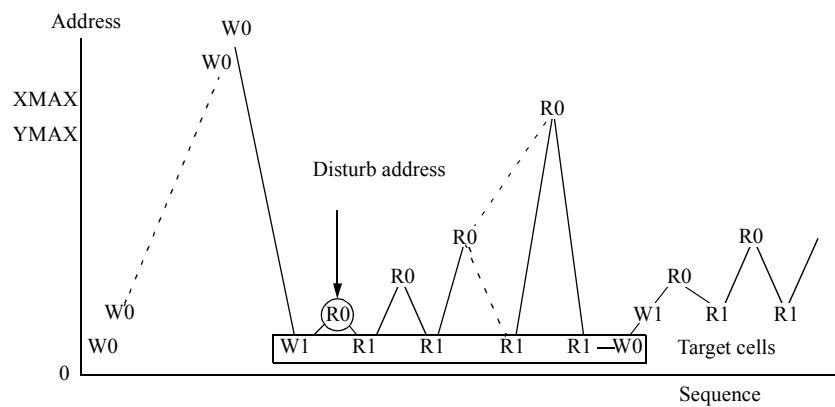
Dn indicates registers D1A to D1H, D2A to D2D, D3 and D4.

2. 4 Address Pattern Generation Instructions



This instruction is used when and N² type test pattern like, for example, a Galloping, Walking or Galwrec pattern is to be generated by linking XB to YC, YS or YK.

An example of Ping pong pattern generation by use of this instruction is shown next.



2. 4 Address Pattern Generation Instructions

The use of the instruction becomes necessary because, whereas the disturb cell starting Y address is the same as that of the target cell as long as the target cell X address has not reached the LMAX, the Y address is incremented after the target cell X address reaches the LMAX.

MPAT PING

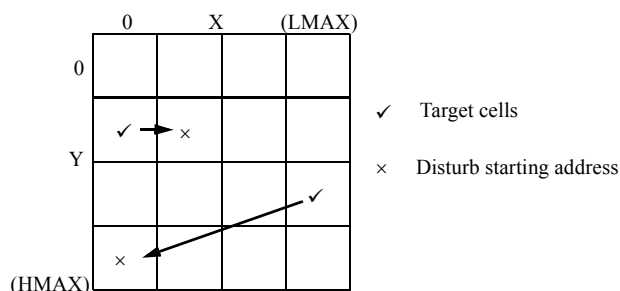
```
REGISTER
XMAX=#7F
YMAX=#7F
TPH=#0
DCMR=#0
IDX1=#3FFF
IDX2=#3FFD
```

PC=#00

START #00

```

NOP          XB<0 YB<0 TP<TPH
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP     W XC<XB+1 YC<YB+1^BX X<XB Y<YB /D
ST2: NOP     R X<XC Y<YC
JN12 ST2 R XC<XC+1 YC<YC+1^CY X<XB Y<YB /D
JN11 ST1 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
NOP
STPS
END
```



The disturb cell starting Y address is incremented only after the target cell X address reaches the LMAX.

- $C[S][K]<B\pm 1, Dn\pm BX$ ($YC(YS)(YK)<YB\pm 1\pm BX, YC(YS)(YK)<YB\pm Dn\pm BX$)

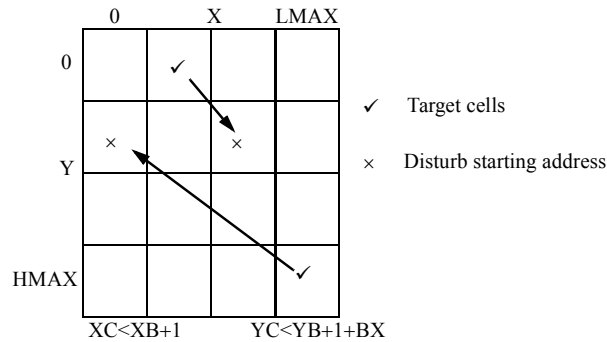
This instruction is used to add or subtract the carry or borrow caused by the YC, YS or YK operation instruction during the XB operation. The instruction operates as follows.

$YC(YS)(YK)<YB\pm 1, Dn\pm BX$ ($Dn=D1A$ to $D1H, D2A$ to $D2D, D3, D4$)

When no XB carry or borrow occurs $YB\pm 1, Dn \rightarrow YC, YS$ or YK

When XB carry or borrow occurs $YB\pm 1, Dn\pm 1 \rightarrow YC, YS$ or YK

This instruction is required mainly for setting the following disturb-start address from target addresses.



- $C[S][K] < B \pm 1, Dn \wedge CY$ ($YC(YS)(YK) < YB \pm 1 \wedge CY, YC(YS)(YK) < YB \pm Dn \wedge CY$)

The conditional operation instructions of YC, YS and YK control the operation of YC, YS and YK by a carry and borrow generated in the operation of XC, XS or XK. Such carry and borrow are generated at the most significant effective bit stored in LMAX in the operation of XC, XS and XK.

$YC(YS)(YK) < YB \pm 1, Dn \wedge CY$ ($Dn = D1A$ to $D1H, D2A$ to $D2D, D3, D4$)

When no XC(XS)(XK) carry or borrow occurs YB $\rightarrow YC, YS$ or YK

When XC(XS)(XK) carry or borrow occurs $YB \pm 1, Dn$ $\rightarrow YC, YS$ or YK

Use this instruction, in the same way as $C[S][K] < B \pm 1 \wedge BX$, to specify the disturb-start address beginning from the test target address.

Although $C[S][K] < B \pm 1, Dn \wedge BX$ executes $YB \pm 1, Dn$ only if a carry or borrow occurs during the XB operation of the test target address, this instruction executes $YB \pm 1, Dn$ if an XC, XS or XK carry or borrow occurs, regardless of whether carry or borrow occurs during the XB operation of the test target address.

- $C[S][K] < B \pm 1, Dn \pm CY$ ($YC(YS)(YK) < YB \pm 1 \pm CY, YC(YS)(YK) < YB \pm Dn \pm CY$)

This instruction adds and subtracts the carry and borrow generated by the operation of XC, XS or XK in the operation instruction of YC, YS and YK.

Use this instruction, in the same way as $C[S][K] < B \pm 1 \pm BX$, to specify the disturb-start address beginning from the test target address.

$YC(YS)(YK) < YB \pm 1, Dn \pm CY$ ($Dn = D1A$ to $D1H, D2A$ to $D2D, D3, D4$)

When no XC(XS)(XK) carry or borrow occurs $YB \pm 1, Dn$ $\rightarrow YC, YS$ or YK

When XC(XS)(XK) carry or borrow occurs $YB \pm 1, Dn \pm 1$ $\rightarrow YC, YS$ or YK

- $C[S][K] < B * 2$ ($XC(XS)(XK) < XB * 2, YC(YS)(YK) < YB * 2$)

Use this instruction to store in XC(XS)(XK), YC(YS)(YK) the addresses stored in XB and YB and left-shifted addresses ($XB * 2, YB * 2, LSB = 0$).

- $C[S][K] \leftarrow B(XC(XS)(XK) \leftarrow XB, YC(YS)(YK) \leftarrow YB)$
Use this instruction to store in $XC(XS)(XK)$ and $YC(YS)(YK)$ the addresses obtained by reversing the addresses stored in XB and YB .
- $C[S][K] \leftarrow B \& D_n(XC(XS)(XK) \leftarrow XB \& D_n, YC(YS)(YK) \leftarrow YB \& D_n)$
($D_n = D1A$ to $D1H$, $D2A$ to $D2D$, $D3$, $D4$)
Use this instruction to store in $XC(XS)(XK)$ and $YC(YS)(YK)$ the addresses stored in XB and YB , and the addresses of the logical product of data indicated by D_n .
- $C[S][K] \leftarrow B' D_n(XC(XS)(XK) \leftarrow XB' D_n, YC(YS)(YK) \leftarrow YB' D_n)$
($D_n = D1A$ to $D1H$, $D2A$ to $D2D$, $D3$, $D4$)
Use this instruction to store in $XC(XS)(XK)$ and $YC(YS)(YK)$ the addresses stored in XB and YB , and the addresses of the logical product of data indicated by D_n .
- $C[S][K] \leftarrow B'' D_n(XC(XS)(XK) \leftarrow XB'' D_n, YC(YS)(YK) \leftarrow YB'' D_n)$
($D_n = D1A$ to $D1H$, $D2A$ to $D2D$, $D3$, $D4$)

Use this instruction to generate patterns such as the bit-changing patterns to generate addresses obtained by reversing the bits indicated by D_n of the addresses stored in XB and YB . It executes the following operations:

$XC \leftarrow XB'' D_n$	$XB \oplus D_n \rightarrow XC$	
$XS \leftarrow XB'' D_n$	$XB \oplus D_n \rightarrow XS$	
$XK \leftarrow XB'' D_n$	$XB \oplus D_n \rightarrow XK$	
$YC \leftarrow YB'' D_n$	$YB \oplus D_n \rightarrow YC$	(Dn=D1A to D1H, D2A to D2D, D3, D4)
$YS \leftarrow YB'' D_n$	$YB \oplus D_n \rightarrow YS$	
$YK \leftarrow YB'' D_n$	$YB \oplus D_n \rightarrow YK$	

⊕: Exclusive OR

An example of bit changing pattern generation performed by use of this instruction is shown in the following.

Bit changing pattern

Address selection errors and multiple address selection faults occurring in the decoder circuit can be efficiently detected by using bit changing patterns. An example of bit changing pattern for testing the row address decoder is described in the following.

- Pattern sequence
 1. "0" is written on every cell.
 2. "0" is written on the target cell.
 3. "1" is written on the address generated by inverting a bit of the target cell address.
 4. "0" is read from the target cell.
 5. Steps 2 to 4 are repeated after changing the bit to be inverted.
 6. Steps 2 to 5 are repeated after changing the target cell.

◆ Test Program

```

MPAT BIT      ;ROW
REGISTER
XMAX=#7F
YMAX=#7F
TPH=#0
DCMR=#0
PC=#0
D3B=#1
IDX1=#3FFE
IDX2=#3
IDX3=#7E

START      #0
NOP        XB<0 YB<0 TP<TPH D3<D3B
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP      W XC<XB"D3          X<XB Y<YB
      NOP      W                  X<XC Y<YB /D
      JN12 ST1 R                  X<XB Y<YB D3<D3*2
      NOP      W XC<XB"D3          X<XB Y<YB
      NOP      W                  X<XC Y<YB /D
      JN13 ST1 R XB<XB+1          X<XB Y<YB D3<D3B
      NOP
      STOP
END

```

◆ $C[S][K] < f(C[S][K])$

$C[S][K] < f(C[S][K])$ is an instruction group for obtaining by operation the address generation of disturb cell using $XC(XS)(XK)$ and $YC(YS)(YK)$, comprising the following instructions.

$C < C$ or $S < S$	
$C[S][K] < C[S][K] \pm 1$	
$C[S][K] < Dn$	
$C[S][K] < 0$	
$C[S][K] < C[S][K] * 2$	Shift left
$C[S][K] < \sim C[S][K]$	Complement
$C[S][K] < C[S][K] \& Dn$	Logical product
$C[S][K] < C[S][K] \oplus Dn$	Exclusive OR
$C[S][K] < C[S][K] \vee Dn$	Logical sum
$C[S][K] < C[S][K] \pm 1 \wedge BX$ (Refer to Tip)	} Only YC, YS or YK is valid
$C[S][K] < C[S][K] \pm Dn \wedge BX$ (Refer to Tip)	
$C[S][K] < C[S][K] \pm 1 \pm BX$ (Refer to Tip)	
$C[S][K] < C[S][K] \pm Dn \pm BX$ (Refer to Tip)	
$C[S][K] < C[S][K] \pm 1 \wedge CY$	
$C[S][K] < C[S][K] \pm Dn \wedge CY$	
$C[S][K] < C[S][K] \pm 1 \pm CY$	
$C[S][K] < C[S][K] \pm Dn \pm CY$	

 Tip

On the T5371, this instruction cannot be written with the instruction (XB<XB-1) when it generates a borrow because of an XB operation.

C means XC or YC.

S means XS or YS.

K means XK or YK.

C[S][K] means XC, XS or XK, and YC, YS or YK.

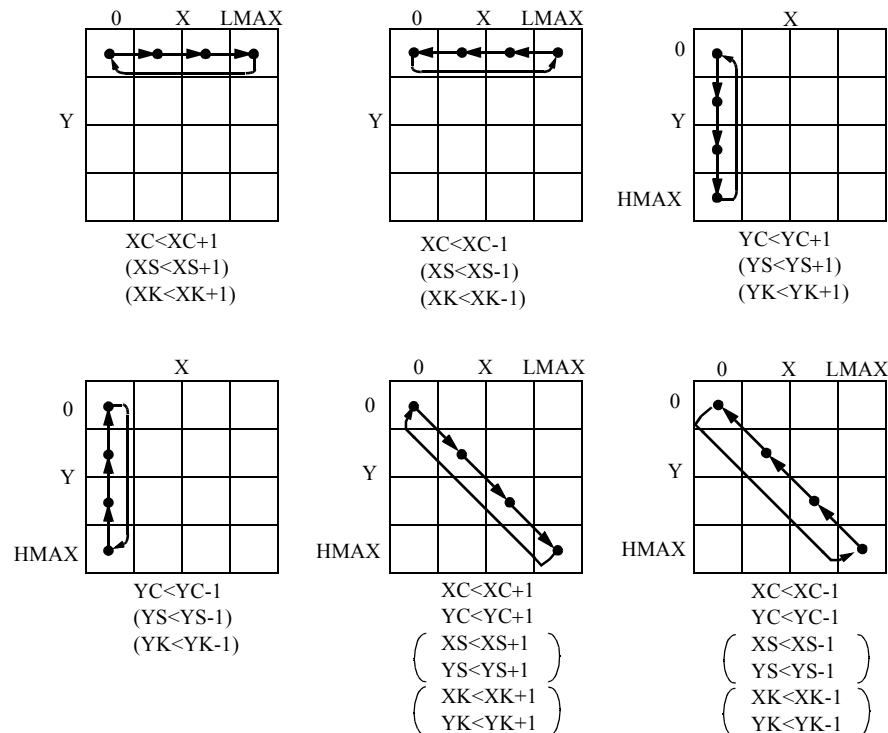
Dn indicates registers D1A to D1H, D2A to D2D, D3 and D4.

- C<C, S<S or K<K (XC<XC, XS<XS, XK<XK, YC<YC, YS<YS, YK<YK)

Use this instruction when nothing is executed by the hold instruction. This instruction can be omitted.

- C[S][K]<C[S][K]±1 (XC(XS)(XK)<XC(XS)(XK)±1, YC(YS)(YK)<YC(YS)(YK)±1)

Use this instruction to increment or decrement addresses. This instruction can generate the following sequences.



- C[S][K]<C[S][K]±Dn (XC(XS)(XK)<XC(XS)(XK)±Dn, YC(YS)(YK)<YC(YS)(YK)±Dn)

This instruction is used to increase or decrease the disturb cell addresses stored in XC(XS)(XK) and YC(YS)(YK).

It executes the following operations:

$C < C + D_n \quad (XC(XS)(XK) < XC(XS)(XK) + D_n, YC(YS)(YK) < YC(YS)(YK) + D_n)$

For $C + D_n < MAX + 1 \quad (C + D_n) \rightarrow C$

For $C + D_n > MAX \quad (C + D_n - MAX) \rightarrow C$

MAX represents LMAX and HMAX.

D_n indicates registers D1A to D1H, D2A to D2D, D3 and D4.

$C < C - D_n \quad (XC(XS)(XK) < XC(XS)(XK) - D_n, YC(YS)(YK) < YC(YS)(YK) - D_n)$

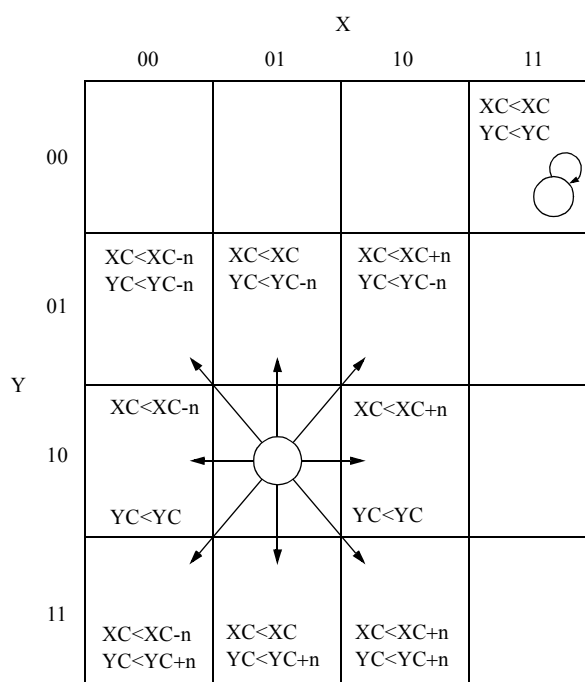
For $C - D_n \geq 0 \quad (C - D_n) \rightarrow C$

For $C - D_n < 0 \quad XMAX + 1 - (C - D_n) \rightarrow C$

MAX represents LMAX and HMAX.

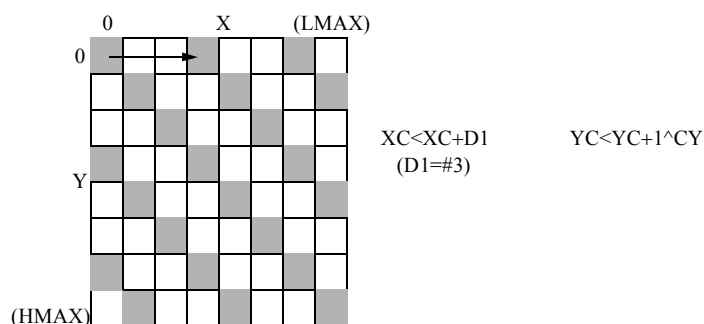
D_n indicates registers D1A to D1H, D2A to D2D, D3 and D4.

This instruction is used when generating addresses as follows:



D_n indicates registers D1A to D1H, D2A to D2D, D3 and D4.

This instruction is used, for example, when sequentially accessing the addresses shown by hatching below.



```

MPAT SAMPLE
REGISTER
XMAX=#7
YMAX=#7

D1=#3          Number of operations

IDX1=#14
PC=#0

START      #0
NOP XB<0   YB<0
NOP XC<XB  YC<YB ; initialization of XC, YC

ST0: JN11 ST0  XC<XC+D1  YC<YC+1^CY  X<XC Y<YC
:
END

```

- $C[S][K] < C[S][K] \pm 1, Dn \wedge BX$ ($YC(YS)(YK) < YC(YS)(YK) \pm 1 \wedge BX$)
 $(YC(YS)(YK) < YC(YS)(YK) \pm Dn \wedge BX)$

This is a conditional operation instruction used in the event of a carry or borrow during the XB operation. The instruction operates as follows.

$YS(YS)(YK) < YC(YS)(YK) \pm 1 \wedge BX$

When no XB carry or borrow occurs

$YC(YS)(YK) < YC(YS)(YK)$

When XB carry or borrow occurs

$YC(YS)(YK) < YC(YS)(YK) \pm 1$

$$YS(YS)(YK) < YC(YS)(YK) \pm Dn \wedge BX$$

When no XB carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK)$$

When XB carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn$$

- $C[S][K] < C[S][K] \pm 1, Dn \pm BX$ $(YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm BX)$
 $(YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm BX)$

This instruction adds or subtracts a carry or borrow caused by the YC, YS or YK operation instruction during the XB operation. The instruction operates as follows.

$$YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm BX$$

When no XB carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm 1$$

When XB carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm 2$$

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm BX$$

When no XB carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn$$

When XB carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm 1$$

- $C[S][K] < C[S][K] \pm 1, Dn \wedge CY$ $(YC(YS)(YK) < YC(YS)(YK) \pm 1 \wedge CY)$
 $(YC(YS)(YK) < YC(YS)(YK) \pm Dn \wedge CY)$

This instruction is a concatenation instruction for each address of XC(XS)(XK) and YC(YS)(YK), executing the following operations. This instruction is valid only when combined as the add/subtract instruction for $(XC(XS)(XK) < XC(XS)(XK) \pm 1, Dn)$.

$$YC(YS)(YK) < YC(YS)(YK) \pm 1 \wedge CY$$

When no XC(XS)(XK) carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK)$$

When XC(XS)(XK) carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm 1$$

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \wedge CY$$

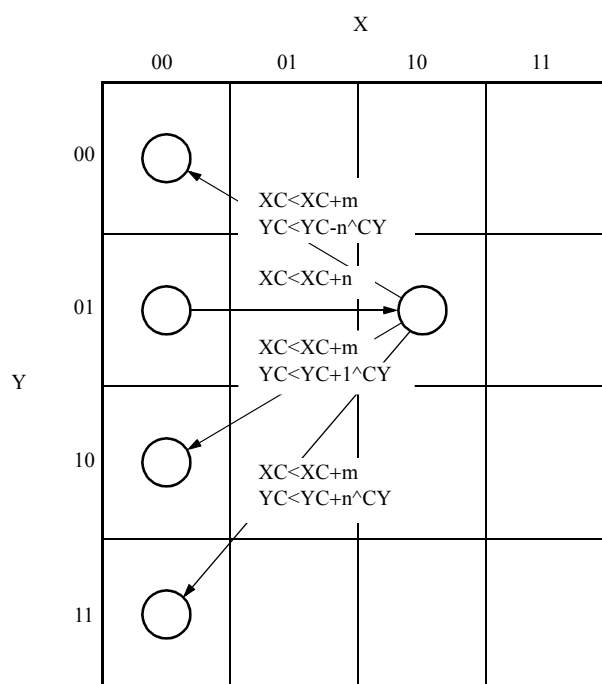
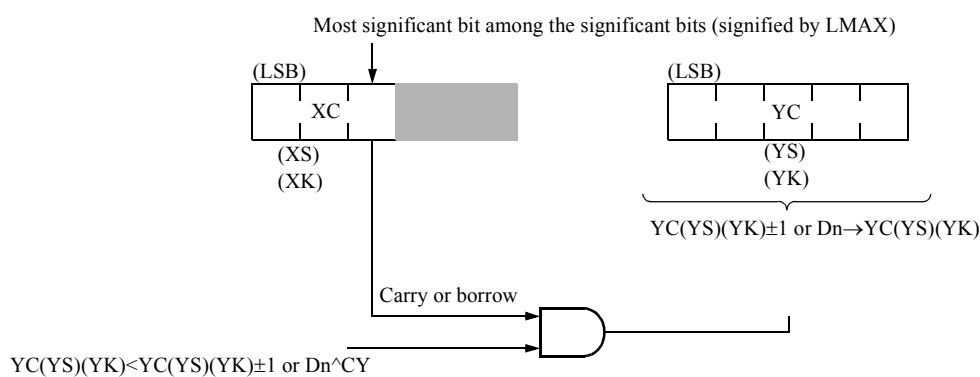
When no XC(XS)(XK) carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK)$$

When XC(XS)(XK) carry or borrow occurs

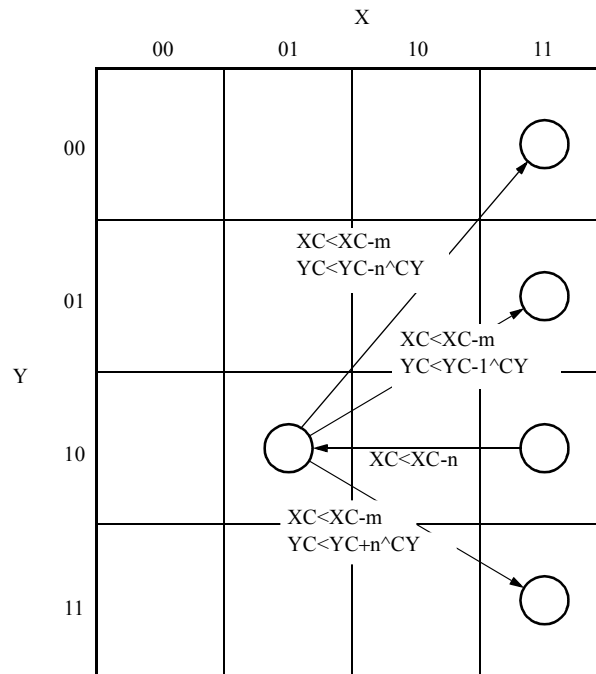
$$YC(YS)(YK) < YC(YS)(YK) \pm Dn$$

This instruction executes $YC(YS)(YK) \pm 1$ or $D_n \rightarrow YC(YS)(YK)$ only when carry or borrow is generated in the operation of $XC(XS)(XK)$ described in the same cycle. When the above condition is not satisfied, this instruction $YC(YS)(YK) \rightarrow YC(YS)(YK)$ is executed.



n and m indicates registers D1A to D1H, D2A to D2D, D3 and D4.

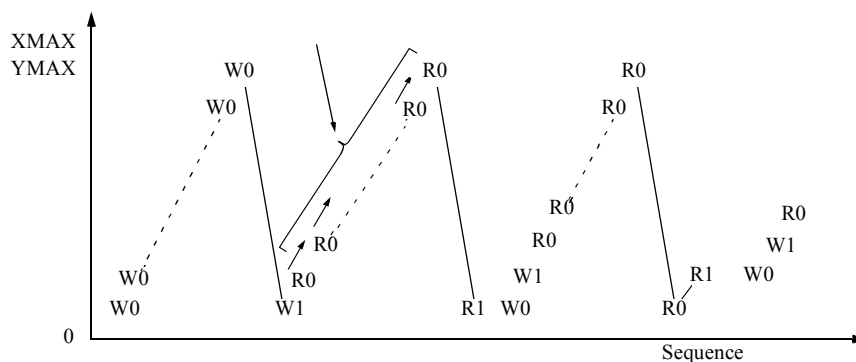
m signifies a value such that $XC + m$ satisfies the condition that $LMAX < XC + m$.



n and m indicates registers D1A to D1H, D2A to D2D, D3 and D4.
m signifies a value such that $XC-m$ satisfies the condition that $0 > XC-m$.

2. 4 Address Pattern Generation Instructions

$YC(YS)(YK) < YC(YS)(YK) \pm 1^{CY}$ is used when incrementing (+1) or decrementing (-1) the disturb cell address in generating a test pattern like a Galloping pattern or a Walking pattern. In this case, $XC(XS)(XK) < XC(XS)(XK) + 1$ or $XC(XS)(XK) < XC(XS)(XK) - 1$ etc. are used.



MPAT WALK

REGISTER

XMAX=#7F

YMAX=#7F

TPH=#0

DCMR=#0

IDX1=#3FFE

IDX2=#3FFD

PC=#0

START #0

NOP XB<0 YB<0 TP<TPH

ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB

ST1: NOP W XC<XB+1 YC<YB+1^BX X<XB Y<YB /D

ST2: JN12 ST2 R XC<XC+1 YC<YC+1^CY X<XC Y<YC

NOP R X<XB Y<YB /D

JN11 ST1 W XB<XB+1 YB<YB+1^BX X<XB Y<YB

NOP

STPS

END

2. 4 Address Pattern Generation Instructions

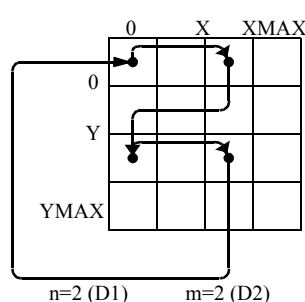
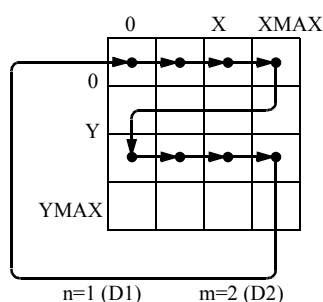
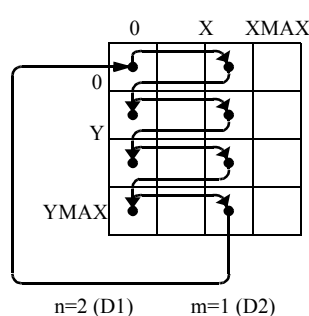
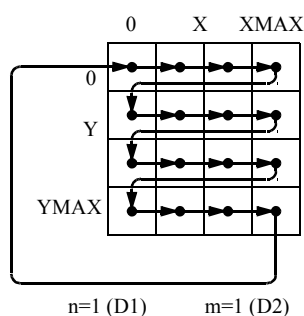
Use $YC(YS)(YK) < YC(YS)(YK) \pm Dn^CY$ when the operand used in the address operation is other than "1" in an N type test pattern such as Waltzing. See the examples below.

```

MPAT SAMPLE
REGISTER
:
D1=n
D2=m
:
START #00

ST0: JN11 ST0 XC<XC+D1 YC<YC+D2^CY X<XC Y<YC
:
END

```



- $C[S][K] < C[S][K] \pm 1, Dn \pm CY$ ($YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm CY$)
 $(YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm CY)$

This instruction adds and subtracts the carry and borrow generated by the operation of XC, XS and XK in the operation instruction of YC, YS and YK. This instruction is valid only in the combination of the add/subtract instruction of XC(XS)(XK) $(XC(XS)(XK) < XC(XS)(XK) \pm 1$, or Dn).

$$YC(YS)(YK) < YC(YS)(YK) \pm 1 \pm CY$$

When no XC(XS)(XK) carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm 1$$

When XC(XS)(XK) carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm 2$$

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm CY$$

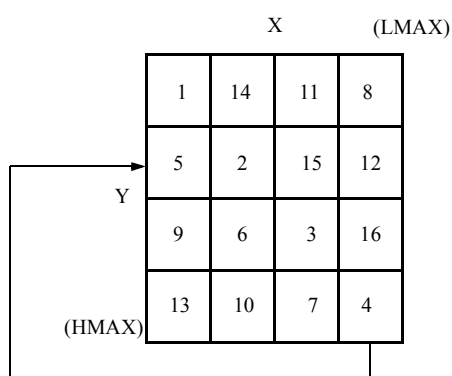
When no XC(XS)(XK) carry or borrow occurs

$$YC(YS)(YK) < YC(YS)(YK) \pm Dn$$

When XC(XS)(XK) carry or borrow occurs

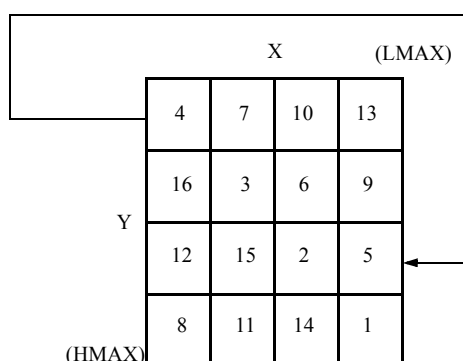
$$YC(YS)(YK) < YC(YS)(YK) \pm Dn \pm 1$$

Use this instruction to generate disturb addresses such as Shift diagonal.



ST0 : JN11 ST0 XC<XC+1 YC<YC+1+CY X<XC Y<YC

YC<YC+2 is executed only when "XC carry = 1" (4, 8, 12, 16 in the figure to the left); in other steps, YC<YC+1 is executed.



ST0 : JN11 ST0 XC<XC-1 YC<YC-1-CY X<XC Y<YC

YC<YC-2 is executed only when "XC borrow=1" (4, 8, 12, 16 in the figure to the left); in other steps, YC<YC-1 is executed.

- $C[S][K] < Dn$ (XC(XS)(XK) < Dn, YC(YS)(YK) < Dn)
This instruction initializes each register of XC(XS)(XK) and YC(YS)(YK), loading the contents of the registers Dn (D1A to D1H, D2A to D2D, D3, D4).
- $C[S][K] < 0$ (XC(XS)(XK) < 0, YC(YS)(YK) < 0)
This instruction zeros (#0) each register of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] < C[S][K] * 2$ (XC(XS)(XK) < XC(XS)(XK) * 2, YC(YS)(YK) < YC(YS)(YK) * 2)
This instruction left-shifts (LSB=0) the contents of the specified register and loads them into each register of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] < /C[S][K]$ (XC(XS)(XK) < /XC(XS)(XK), YC(YS)(YK) < /YC(YS)(YK))

This instruction reverses the contents of the specified register and loads them into each register of XC(XS)(XK) and YC(YS)(YK).

- $C[S][K] < C[S][K] \& D_n$ (XC(XS)(XK) < XC(XS)(XK) & D_n, YC(YS)(YK) < YC(YS)(YK) & D_n)
This instruction obtains the logical product of the contents of the specified registers of XC(XS)(XK) and YC(YS)(YK) and the data indicated by D_n, and loads it into each register of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] < C[S][K] \text{ " } D_n$ (XC(XS)(XK) < XC(XS)(XK) " D_n, YC(YS)(YK) < YC(YS)(YK) " D_n)
This instruction obtains the exclusive logical sum of the contents of the specified registers of XC(XS)(XK) and YC(YS)(YK) and the data indicated by D_n, and loads it into each register of XC(XS)(XK) and YC(YS)(YK).
- $C[S][K] \text{ ' } D_n$ (XC(XS)(XK) < XC(XS)(XK) ' D_n, YC(YS)(YK) < YC(YS)(YK) ' D_n)
This instruction obtains the logical sum of the contents of the specified registers of XC(XS)(XK) and YC(YS)(YK) and the data indicated by D_n, and loads it into each register of XC(XS)(XK) and YC(YS)(YK).

◆ $C[S][K] < f(B, C[S][K], t)$

ALPG has the save registers of XS, XK, YS and YK to save each address of XC and YC by instruction because the address sequence generates complex test patterns.

The following instructions are provided to control these save registers.

$C < S$

$S < C$

$C \diamond S$

$CS < OPE$

$C < S < OPE$

$S < C < OPE$

$C < K$

$K < C$

$C \diamond K$

$CK < OPE$

$C < K < OPE$

$K < C < OPE$

$SK < OPE$

$CSK < OPE$

C means XC or YC.

S means XS or YS.

CS means XCS or YCS.

OPE means each operation of $f(B)$ and $f(C[S][K])$.

K means XK or YK.

CK means XCK or YCK.

SK means XSK or YSK.

CSK means XCSK or YCSK.

OPE indicates the operation instruction that can use the operation instructions of $f(B)$ and $f(C[S][K])$ of "[C\[S\]\[K\]<f\(B\)](#)" on page 2-100 and "[C\[S\]\[K\]<f\(C\[S\]\[K\]\)](#)" on page 2-108.

- $C<S(XC<XS, YC<YS)$

This instruction loads into XC and YC registers the addresses saved in the save registers.

- $S<C(XS<XC, YS<YC)$

This instruction saves the addresses of XC and YC registers into save registers.

- $C\Diamond S(XC\Diamond XS, YC\Diamond YS)$

This instruction exchanges the addresses stored in the save registers of XS and YS with the addresses stored in the registers of XC and YC.

- $CS<OPE(XCS<OPE, YCS<OPE)$

This instruction stores an operation result in XS and YS simultaneously in XC and YC operation instructions. OPE means the operation instruction. This instruction can use the operation instructions $f(B)$ or $f(C[S][K])$.

- $C<S<OPE(XC<XS<OPE, YC<YS<OPE)$

This instruction loads the addresses stored in each save register of XS and YS into each register of XC and YC, and stores the operation result only in each save register of XS and YS.

Before-loading, addresses of XC(XS) and YC(YS) are used for operation.

OPE means the operation instruction. This instruction can use the operation instructions $f(B)$ or $f(C[S][K])$.

- $S<C<OPE(XS<XC<OPE, YS<YC<OPE)$

This instruction loads the addresses stored in each register of XC and YC into each save register of XS and YS, and stores the operation result in each register of XC and YC.

Before-loading the addresses of XC(XS) and YC(YS) are used for operation.

OPE means the operation instruction. This instruction can use the operation instructions $f(B)$ or $f(C[S][K])$.

- $C<K(XC<XK, YC<YK)$

This instruction loads the addresses saved in the XK and YK save registers into the XC and YC registers.

- $K<C(XK<XC, YK<YC)$

This instruction saves the addresses of the XC and YC registers into the XK and YK save registers.

- $C\Diamond K(XC\Diamond XK, YC\Diamond YK)$

This instruction exchanges the addresses stored in the XK and YK save registers with the addresses stored in the XC and YC registers.

- $CK<OPE(XCK<OPE, YCK<OPE)$

This instruction stores operation results in XK and YK simultaneously in XC and YC operation instructions. OPE indicates the operation instruction that can use operation instructions $f(B)$ and $f(C[S][K])$.

- C<K<OPE(XC<XK<OPE, YC<YK<OPE)

This instruction loads the addresses stored in each XK and YK save register into each XC and YC register, and stores only the operation result in each XK and YK save register.

The XC(XK) and YC(YK) addresses for operation are used before being loaded.

OPE indicates the operation instruction that can use the operation instructions f(B) and f(C[S][K]).

- K<C<OPE(XK<XC<OPE, YK<YC<OPE)

This instruction loads the addresses stored in each XC and YC register into each XK and YK save register, and stores the operation result in each XC and YC register.

The addresses of XC(XK) and YC(YK) for operation are used before being loaded.

OPE indicates the operation instruction that can use the operation instructions f(B) and f(C[S][K]).

- SK<OPE(XSK<OPE, YSK<OPE)

This instruction stores operation results in XS, XK, YS, and YK simultaneously in XC and YC operation instructions.

OPE indicates the operation instruction that can use the operation instructions f(B) and f(C[S][K]).

- CSK<OPE(XCSK<OPE, YCSK<OPE)

This instruction stores operation results in XC, XS, XK, YC, YS, and YK simultaneously in XC and YC operation instructions.

OPE indicates the operation instruction that can use the operation instructions f(B) and f(C[S][K]).

2. 4. 6 Source Address Field

This instruction selects the addresses to be applied to DUT. The following addresses can be selected and output as each address of X and Y.

X<XB	Y<YB
X<XC	Y<YC
X<XS	Y<YS
X<XK	Y<YK
X<YB	Y<XB
X<YC	Y<XC
X<YS	Y<XS
X<YK	Y<XK
X<XB'Z	Y<XB'Z
X<XC'Z	Y<XC'Z
X<XS'Z	Y<XS'Z
X<XK'Z	Y<XK'Z
X<Z	Y<Z
X<RF	

RF means REFRESH counter.

The description of the following instructions can be omitted. (when such instructions are not described, the following instructions are executed.)

X<XB

Y<YB

This instruction cannot select different registers in the same group as a combination of the output selection of X and Y in the groups of X address generation registers (XB, XC, XS, XK) and Y address generation registers (YB, YC, YS, YK).

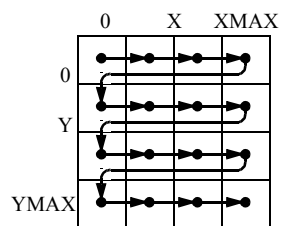
Specify the effective bits of X and Y addresses to be applied to DUT by XMAX register and YMAX register respectively. Specify XMAX and YMAX by 2^n-1 when the number of effective bits is n.

An example of using the address shift mode follows.

2. 4 Address Pattern Generation Instructions

$X < X_B \quad Y < Y_B$
 $X < X_C \quad Y < Y_C$
 $X < X_S \quad Y < Y_S$
 $X < X_K \quad Y < Y_K$

Use this instruction mainly to scan the address generation sequence in X direction as shown below.

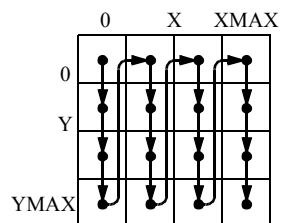


```

NOP      :
          XB<0      YB<0
ST0: JN11 ST0  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB
          :
```

$X < Y_B \quad Y < X_B$
 $X < Y_C \quad Y < X_C$
 $X < Y_S \quad Y < X_S$
 $X < Y_K \quad Y < X_K$

Use this instruction mainly to scan the address generation sequence in the Y direction as shown below.



```

NOP      :
          XB<0      YB<0
ST0: JN11 ST0  XB<XB+1  YB<YB+1^BX  X<YB  Y<X_B
          :
```


$X < X_B'Z$ $Y < X_B'Z$

$X < X_C'Z$ $Y < X_C'Z$

$X < X_S'Z$ $Y < X_S'Z$

$X < X_K'Z$ $Y < X_K'Z$

Use this instruction mainly when X-carry controls the address generation by Y or when Y-carry controls the address generation by X, just like the moving inversions.

```

MPAT      MOVI      ; MOVING INVERSIONS TEST PATTERN

REGISTER
XMAX=#1FF      ; 2n-1
YMAX=#1FF      ; 2m-1
IDX1=#3FFFE    ; 2(n+m) -2
IDX2=#7        ; n-2
IDX3=#7        ; m-2
D3B =#1
D4B =#0

START #0
NOP        XC<0   YC<0   Z<0   D3<D3B   D4<D4B   TP<0
ST0: JN11 ST0 W X<XC   Y<YC   XC<XC+1   YC<YC+1^CY

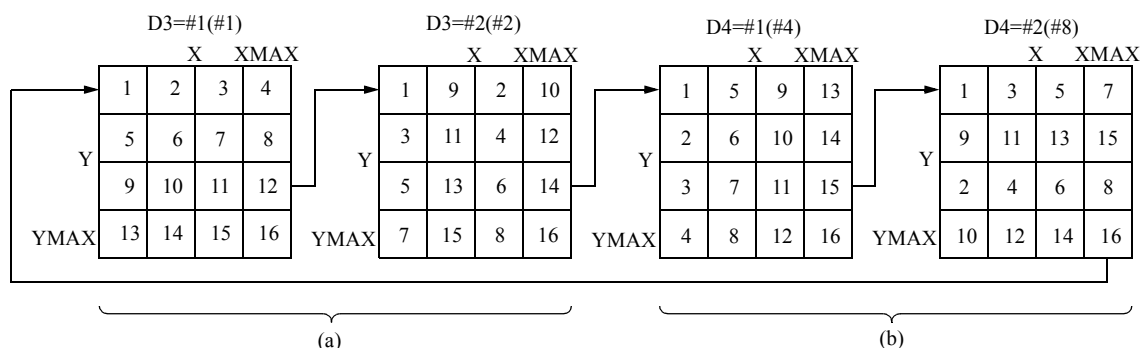
ST1: NOP      R X<XC'Z   Y<YC
      NOP      W X<XC'Z   Y<YC /D
      JN11 ST1 R X<XC'Z   Y<YC /D XC<XC+D3   YC<YC+1^CY   Z<Z+1^CY
      JN12 ST1 Z<0   D3<D3*2L   D4<D4*2L   TP</TP
      } (a)

ST2: NOP      R X<YC   Y<XC'Z
      NOP      W X<YC   Y<XC'Z /D
      JN11 ST2 R X<YC   Y<XC'Z /D XC<XC+D4   YC<YC+1^CY   Z<Z+1^CY
      JN13 ST2 Z<0   D4<D4*2   TP</TP
      } (b)

JZD ST0 D3<D3B D4<D4B
STPS
END

```

The address sequence for MOVING INVERSIONS is as follows: (when n=2 and m=2)



Parenthesized figures are addends to MOVING INVERSIONS.

$X < Z$ $Y < Z$

Use this instruction to set data other than addresses in DUT by mainly using the address line.

X<RF

This instruction outputs the contents of the refresh counter to the X address of DUT. Y address of the cycle describing this instruction is output with the address of the preceding cycle held.

2. 4. 7 N Register Field

The N register field is a set of operation instructions for the N register used to generate a bank address. The N register field includes the following instructions.

N<N

N<NH

N<N+1

N<N-1

N<N+NDn

N<N-NDn

N<NDn

N<N

This is the HOLD instruction for the N register. Use this instruction if the N register operation is not executed.

Omitting an N register operation instruction containing this instruction is the same as specifying this instruction.

N<NH

This instruction loads the contents of the NH register into the N register. This instruction is used to initialize the N register.

N<N+1

This instruction increments the N register.

N<N-1

This instruction decrements the N register.

N<N+NDn

This instruction adds the contents of the NDn register to the N register.

N<N-NDn

This instruction subtracts the contents of the NDn register from the N register.

N<NDn

This instruction loads the contents of the NDn register into the N register. This instruction is used to initialize the N register.

NDn indicates ND1 to ND7.

 **Tip**

ND1 can be described as ND.

2. 4. 8 B Register Field

The B register field is a set of operation instructions for the B register used to generate a bank address. The B register field includes the following instructions.

B<B

B<BH

B<B+1

B<B-1

B<B+BDn

B<B-BDn

B<BDn

The operation instruction for the B register can be specified on the T5377, T5376 and T5375.

B<B

This is the HOLD instruction for the B register. Use this instruction if the B register operation is not executed.

Omitting an B register operation instruction containing this instruction is the same as specifying this instruction.

B<BH

This instruction loads the contents of the BH register into the B register. This instruction is used to initialize the B register.

B<B+1

This instruction increments the B register.

B<B-1

This instruction decrements the B register.

B<B+BDn

This instruction adds the contents of the BDn register to the B register.

B<B-BDn

This instruction subtracts the contents of the BDn register from the B register.

B<BDn

This instruction loads the contents of the BDn register into the B register. This instruction is used to initialize the B register.

BDn indicates BD1 to BD7.



Tip

BD1 can be described as BD.

2. 4. 9 Address Inversion Field

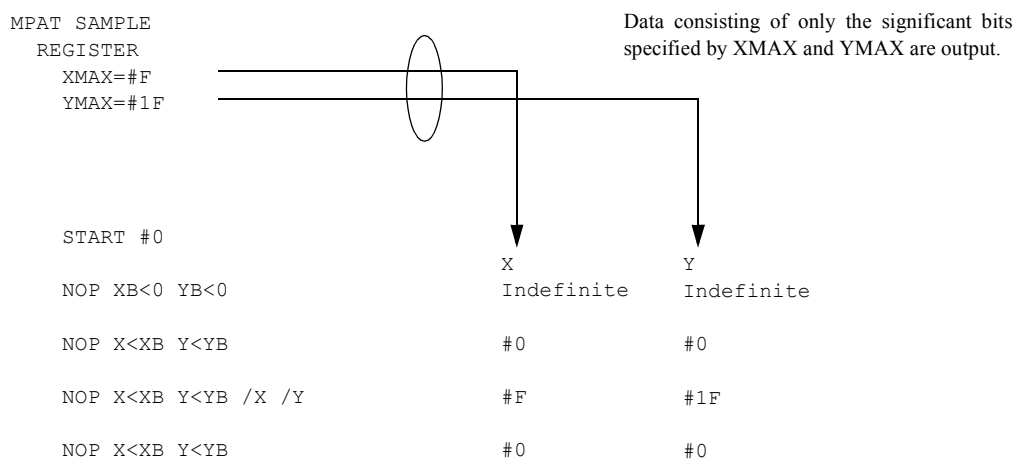
The address inversion field instructions invert addresses to be applied to the DUT. When an inversion field instruction is entered, the corresponding address, X or Y or Z, is inverted only in the current cycle.

/X XMAXn $\overline{oX_n}$ → Xn

/Y YMAXn $\overline{oY_n}$ → Yn

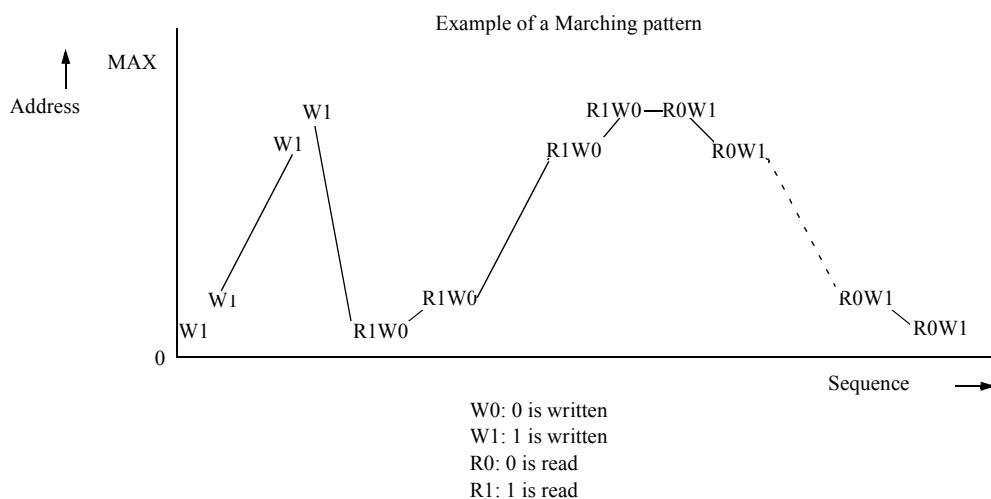
/Z $\overline{Z_n}$ → Zn

n = 0 to 17 (n = 0 to 15 on the T5371, T5336 and T5311)



The address inversion field instructions are used to generate inverted addresses for the generation of a Marching or Complement Marching pattern.

See the examples below.



```
MPAT MARCH
REGISTER
XMAX=#7F
YMAX=#7F
TPH=#0
DCMR=#0
IDX1=#3FFE
PC=#0

START #00
NOP XB<0 YB<0 TP<TPH
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB /D
ST1: NOP R X<XB Y<YB /D
JN11 ST1 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST2: NOP R X<XB Y<YB /X /Y
JN11 ST2 W XB<XB+1 YB<YB+1^BX X<XB Y<YB /X /Y /D
NOP
STPS
END
```

2. 4.10 Refresh Counter Field

The ALPG incorporates two REFRESH counters, RL and RH, to generate addresses to be refreshed when conducting a refresh test.

RF<RF+1 is used to control the REFRESH counters. This instruction increments the counter.

Further, RF<RF is an instruction to hold the REFRESH counter; its description may be omitted and, when omitted, it functions in the same manner as RF<RF.

An example of use of the refresh counter field instruction is described next.

```

MPAT REF      ;DISTRIBUTED REFRESH
MODE REFRESHB 128
REGISTER
  XMAX=#7F
  YMAX=#7F
  TPH=#0
  DCMR=#0
  TIMER=16US
  ISP=#10
  IDX1=#3FFE
  IDX2=#3FFD
  PC=#00

  START #00

  NOP T XB<0 YB<0 TP<TPH
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP      W XC<XB+1 YC<YB+1^BX X<XB Y<YB /D
ST2: NOP      R                      X<XC Y<YC
      NOP      R                      X<XB Y<YB /D
      JN12 ST2 R XC<XC+1 YC<YC+1^CY X<XC Y<YC
      JN11 ST1 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
      NOP
      STPS

      START      #10

      RTN      RF<RF+1      X<RF
      END
      ↑

```

This instruction is accessed every 16 μ s.

It increments the refresh counter and outputs the incremented count as the address to be applied to the DUT.

2. 4.11 D3, D4 Register Field

The D3, D4 field instructions comprise the following operation instructions to be executed against the D3, D4 register that stores the operand and its initial value needed when generating disturb cell addresses which are stored into XC and YC.

D3<D3	D4<D4
D3<D3*2	D4<D4*2
D3<D3/2	D4<D4/2
D3<D3B	D4<D4B
D3<D3*2L	D4<D4*2L
D3<D3+1	
D3<D3-1	

D3<0

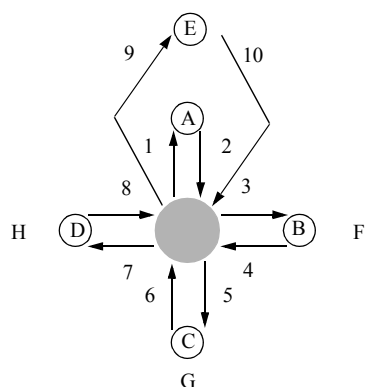
The operation of each instruction is described in the following.

D3<D3B, D4<D4B

The D3, D4 register consists of a buffer register and a COUNTER/SHIFTER. This instruction loads the contents of the buffer register into the COUNTER/SHIFTER.

D3<D3+1

This instruction increments the contents of the D3 register, for example, during Butterfly pattern generation. An example of use of this instruction for Butterfly pattern generation is shown in the following.



After accessing sequentially, from 1 to 8, the (E) cell is accessed by incrementing the D3 register.

```

MPAT BUTTER
REGISTER
:
D3B=#1      Initialization of the D3B register
:
START      #0

NOP        XB<0 YB<0 TP<TPH D3<D3B
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1: NOP W YC<YB-D3 X<XB Y<YB /D
ST2: NOP R        X<XB Y<YC
NOP R XC<XB+D3 X<XB Y<YB /D
NOP R        X<XC Y<YB
NOP R YC<YB+D3 X<XB Y<YB /D
NOP R        X<XB Y<YC
NOP R XC<XB-D3 X<XB Y<YB /D
NOP R        X<XC Y<YB D3<D3+1
JN12 ST2 R YC<YB-D3 X<YB Y<YB /D
JN13 ST1 W XB<XB+1 YB<YB+1 X<XB Y<YB D3<D3B
NOP
STPS

END

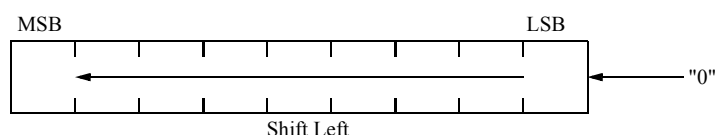
```

D3<D3-1

Decrements the contents of the D3 register.

D3<D3*2, D4<D4*2

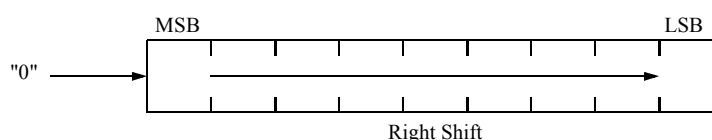
Shifts the contents of the D3, D4 register as illustrated below.



The LSB becomes "0".

D3<D3/2, D4<D4/2

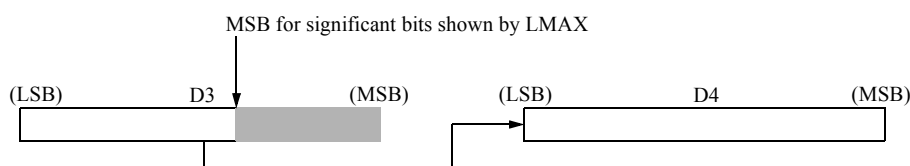
Shifts the contents of the D3, D4 register as illustrated below.



The MSB becomes "0".

D3<D3*2L D4<D4*2L

This instruction concatenates and shifts the D3 register and D4 register. The data to be shifted out from MSB of the effective bits in the D3 register shown by LMAX are loaded as LSB data in D4 register.

**D3<0**

This instruction initializes the D3 register, zeroing the D3 register.

2. 4.12 Address Swap Field

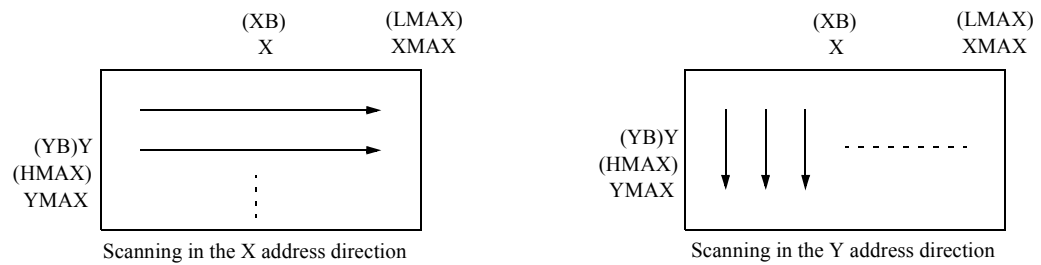
Generally, LMAX controls the effective bits of the operation of XB, XC, XS, XK and D3 register, and HMAX controls the effective bits of YB, YC, YS and YK. The values of the registers controlled by LMAX and HMAX are exchanged in the cycle immediately after any of the following instructions.

LMAX<>HMAX

This instruction is a toggle instruction; an odd number of execution of the instruction exchanges LMAX with HMAX, and an even number of executions restores the original state of LMAX and HMAX.

2. 4 Address Pattern Generation Instructions

Use this instruction to switch in real time from the scanning in the X address direction to scanning in the Y address direction in a DUT test with different bit lengths between X address and Y address.



◆ Example

```

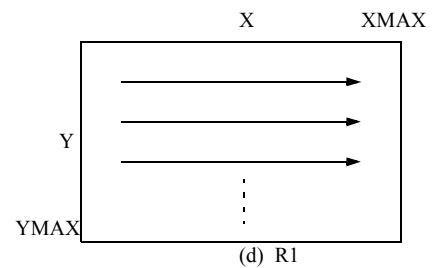
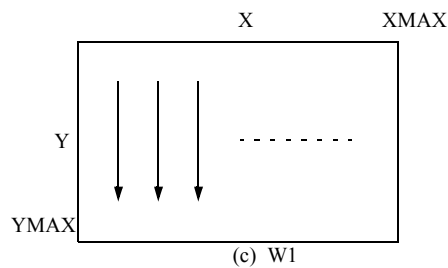
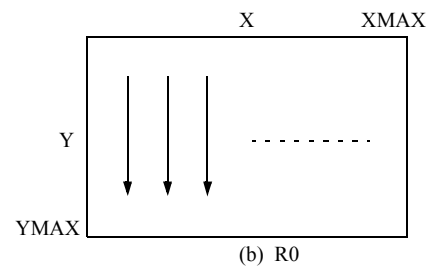
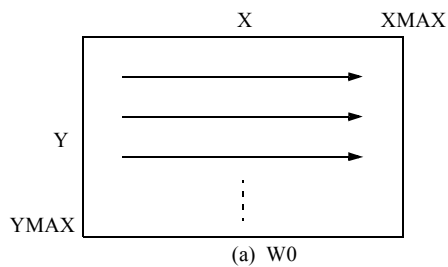
MPAT SWAP

REGISTER
  XMAX=#3FF
  YMAX=#FF
  IDX1=#3FFFE
  PC=#0

START      #0
  NOP      XC<0      YC<0      TP<0
(a) ST0:   JN11 ST0   XC<XC+1  YC<YC+1^CY  X<XC  Y<YC  W
  NOP      LMAX<>HMAX
(b) ST1:   JN11 ST1   XC<XC+1  YC<YC+1^CY  X<YC Y<XC  R
(c) ST2:   JN11 ST2   XC<XC+1  YC<YC+1^CY  X<YC Y<XC  W /D
  NOP      LMAX<>HMAX
(d) ST3:   JN11 ST3   XC<XC+1  YC<YC+1^CY  X<XC  Y<YC  R /D
  STPS

END

```



2. 4.13 Address Shift Field

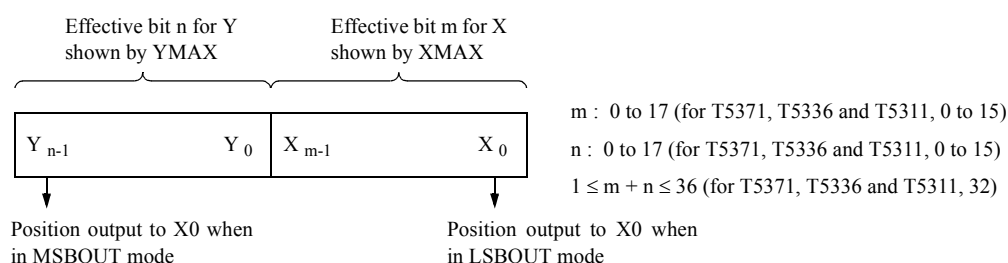
Address shift field is valid only when the address shift mode of LSBOUT or MSBOUT is specified by MODE statement. This command controls the shift direction of addresses and cycle, and initializes the shift data. This command is used for tests to input and output DUT addresses or data in serial sequence. The shift operation is executed for data after scrambled by the address scrambler.

XYSL (available with MSBOUT)

XYSR (available with LSBOUT)

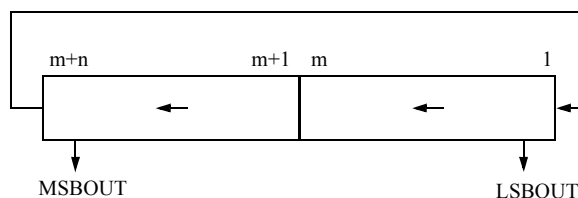
XYLOAD

These commands are valid on the next cycle on description.



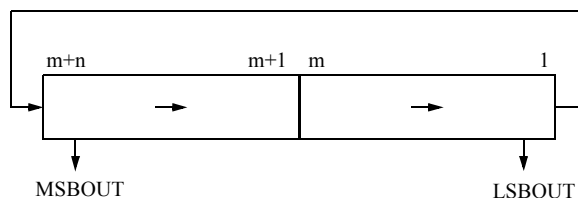
XYSL (Shift Left)

Shifts data output to X0 by 1-bit to the left.



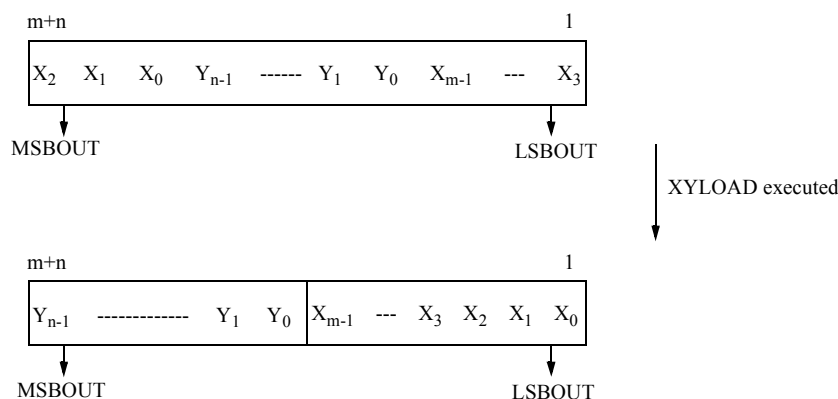
XYSR (Shift Right)

Shifts data output to X0 by 1 bit to the right.



XYLOAD

XYLOAD initializes the shifted data. The operation is made as follows.



Tip

The shift data used in the address shift mode changes by executing address operation for X and Y.

```

MODE  LSBOUT
REGISTER
  XMAX=#3F
  YMAX=#FF
START  #0
      :
NOP XB<0 YB<0      Shift data  Output data for X0
NOP                #0000          #0 (X0)
NOP XYSR /X         #003F          #1 (X0)
                        (The inversed data is shifted by /X.)
NOP                #0000          #0 (X1)

```

An example for using the address shift mode follows.

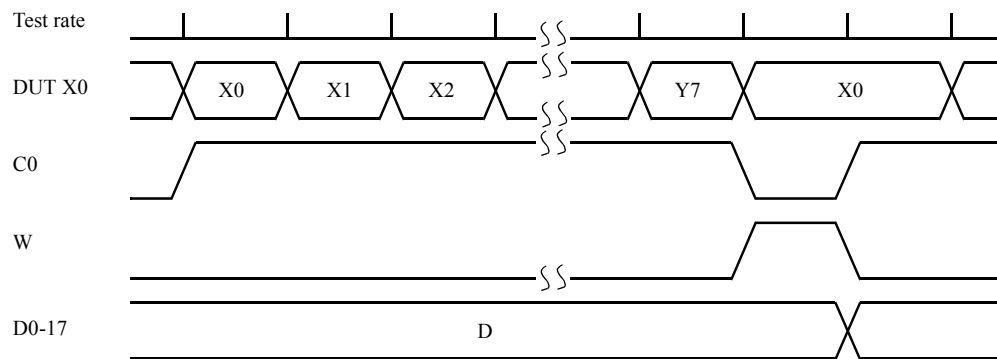
```

MPAT SHIFT
MODE  LSBOUT      ..... Adress shift mode and LSBOUT are specified.
REGISTER
  XMAX=#3F        } ..... The number of effective bits for X, Y is specified.
  YMAX=#FF        } ..... The number of shifts is specified.
  IDX1=#C
  IDX2=#3FFE
START  #0
NOP   XB<0 YB<0
ST0:  JN11 .      XB<XB+1 YB<YB+1^BX
      JN12 ST0   TP</TP
      :

```

Address operation for X and Y is not made in order to change the shift data.

An example appears below:



C0 : Used to control the clock that shift-loads addresses to the DUT

W: Used to control the write clock when data is written to the DUT

Tip

- The shift operation mentioned above is effective for X0 output only, and the outputs of X1-17, Y0-17 (X1-15, Y0-15 for T5371, T5336 and T5311) are not changed by this shift instructions (XYSR, XYSL).
- The bit configuration of serial output can be specified.
➔ Refer to the description of MODE LSBOUT and MSBOUT.

2. 4.14 Instructions and Execution Cycle

There are two types of address operation instructions, depending on the cycle in which the result of the operation is output to X, Y address as shown below.

Execution in the step in which instructions are described (Event time 1)

The instructions below are valid as soon as they are entered.

- Source field

X<n

Y<n

NOP XB<XB+1 YB<YB+1^BX X<XB Y<YB

└───────────> XB and YB are output to X and Y before the operation in the same step is done.

LMAX, HMAX

LMAX and HMAX are specified to use conditional operation instructions in the YB, YC, YS, YK and Z operations. If LMAX and HMAX are not specified, the following are the default values:

LMAX=XMAX

HMAX=YMAX

Default values of address operation

The default values of the address operation instructions are as follows:

XB<XB YB<YB XC<XC XS<XS XK<XK YC<YC YS<YS YK<YK

D3<D3 D4<D4 RF<RF N<N Z<Z X<XB Y<YB B<B

2. 5 Data Pattern Generation Instructions

2. 5. 1 Hardware Configuration for Data Pattern Generator Section

Figure 2-11 Function Block Diagram of Data Generator Section (T5377)

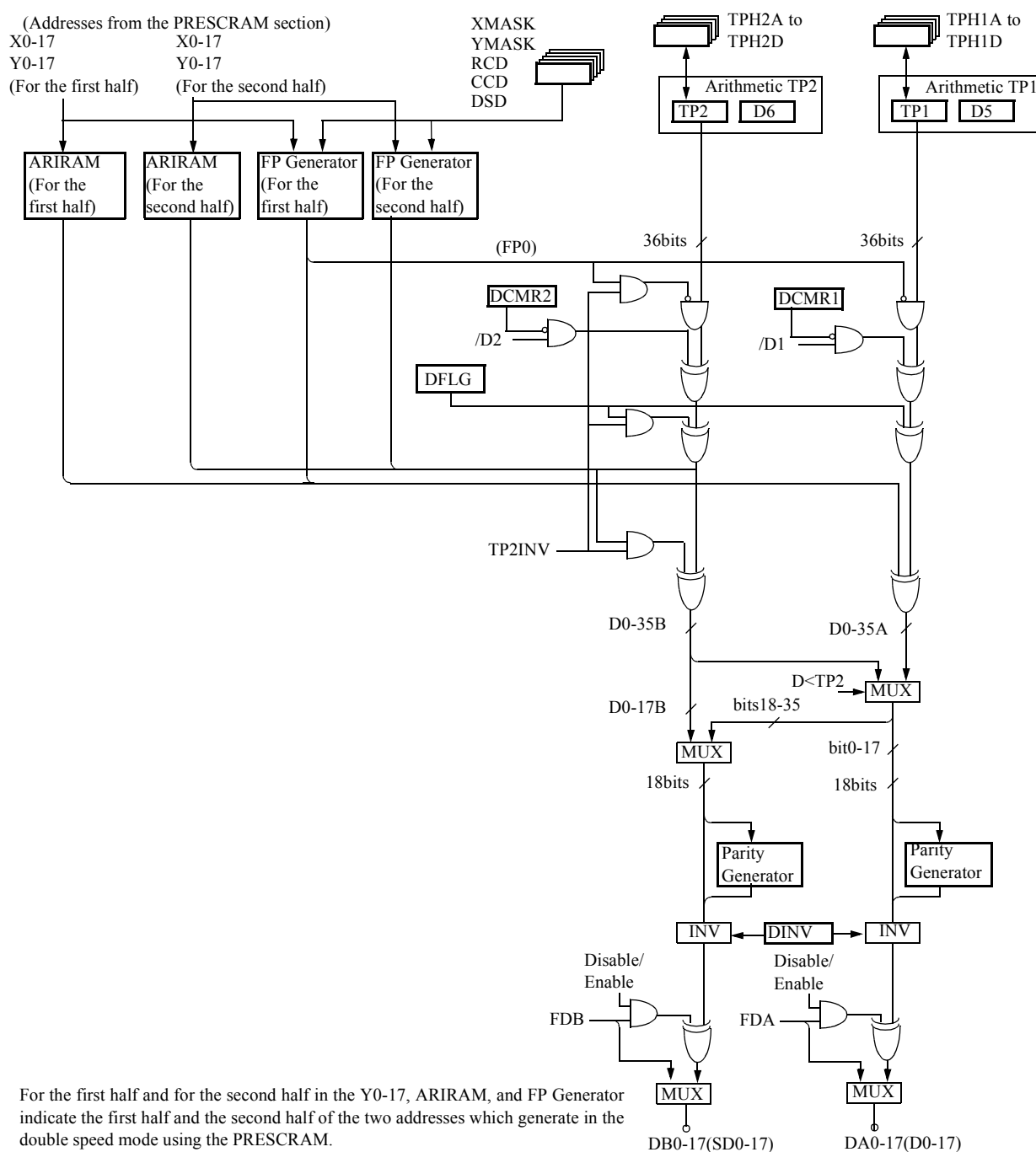


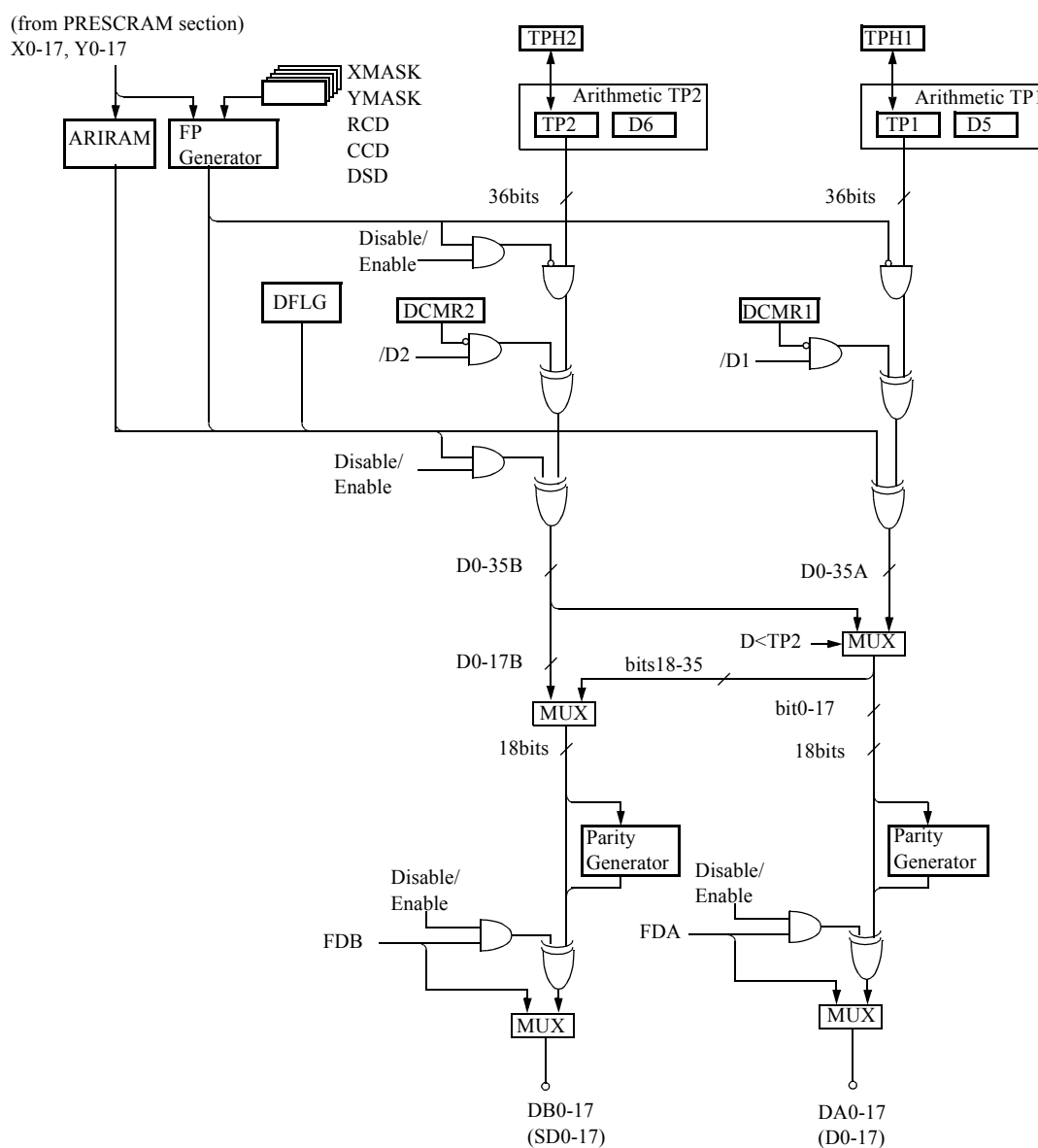
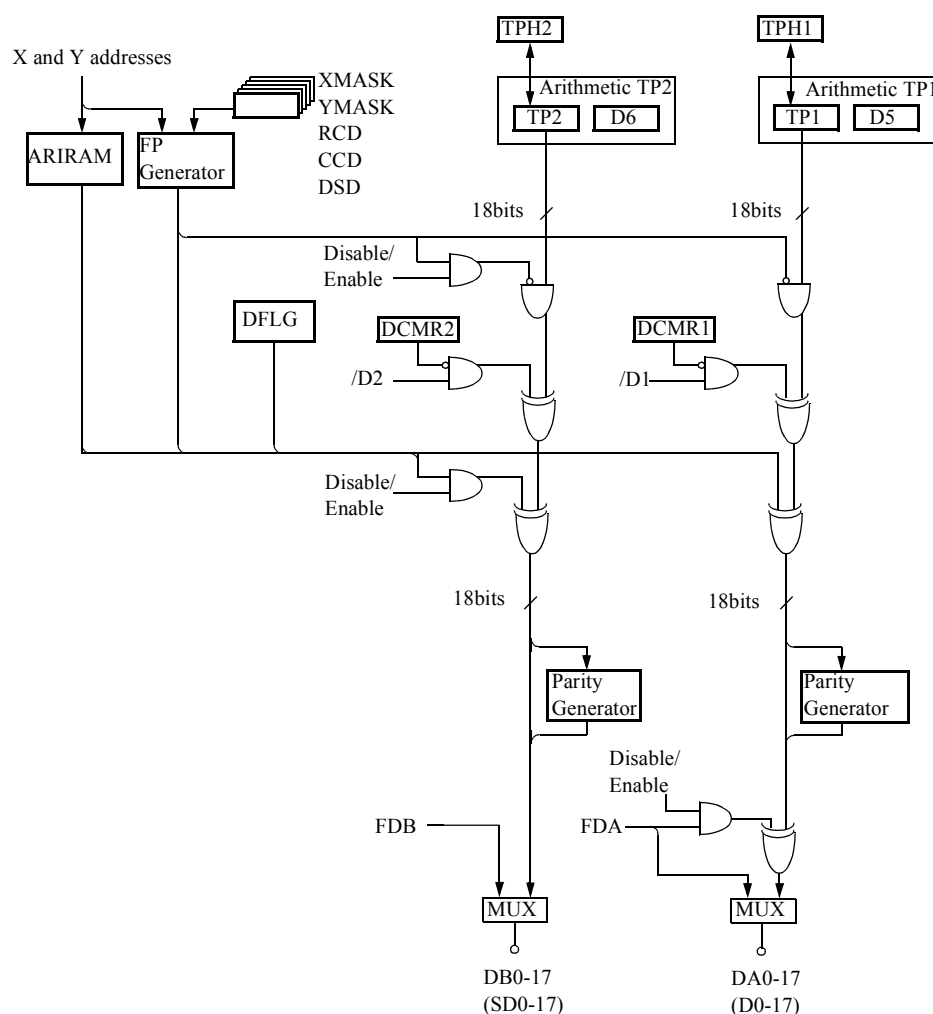
Figure 2-12 Function Block Diagram of Data Generator Section (T5376 and T5375)

Figure 2-13 Function Block Diagram of Data Generator Section (T5371,T5336 and T5311)



Tip

X and Y addresses are the addresses generated from an address generator in the ALPG on the T5336 and T5311 which have not been descrambled by using address descramble memory. On the T5371, X and Y addresses are generated from PRESCRAM.

TP, TP2

The TPs are used to generate comparison data (expected value) between the data to be applied to DUT and data to be output from DUT. Also, the TP2 is used to generate a mask data in the DRAM test including the write per bit function.

TPH(TPH1, TPH1A to TPH1D), TPH2(TPH2A to TPH2D)

The TPH is a register which stores the initial value to be loaded into TP and TP2 registers.

TPH and TPH2 can be set by the SET instruction and the DSET instruction in the pattern program. TPH(TPH1 and TPH1A to TPH1D) and TPH2(TPH2A to TPH2D) can be set by the REGISTER statement in the pattern program and by the REG statement in the test plan program.

DCMR, DCMR2

The DCMR is a register used to specify the mask of the bits to be reversed by the /D and /D2 of a data pattern generation instruction. Data will be logically reversed at a cycle describing the /D and /D2 of "0" bits in this register. This register can be set by SET instruction, DSET instruction and the REGISTER statement in a pattern program and the REG statement in a test plan program. When this register is not set, DCMR=#0 and DCMR2=#0.

DFLG

When the DFLG register is "1", the value inverted in the TP register output to DA0-17.

The DFLG register is reversed by JZD instruction.

To use the DFLG register to invert the value in the TP2 register, specify TP2INV in the MODE statement.

FP Generator

The FP Generator is an address function using the X and Y addresses. If the output of this address function is "1", the value in the TP register is inverted.

To use the FP Generator to invert the value in the TP2 register, specify TP2INV in the MODE statement.

If the double speed mode which uses the pre-scrambler is specified, each Y addresses for the first half and for the second half of the double speed mode can be used as the inversion conditions for the TP1 and TP2 data.

Y address used for the TP1 data inversion condition : The Y addresses for the first half of the double speed mode

Y address used for the TP2 data inversion condition : The Y addresses for the second half of the double speed mode

➔ For more information on the double speed mode, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

XMASK, YMASK

This register is used to specify the effective bits of X and Y to be used as an address function. This can be set by SET instruction, the DSET instruction and the REGISTER statement in a pattern program and the REG statement in a test plan program.

DSD, RCD, CCD

This register is used as offset values or comparison values of the address function. The register can be set by the SET instruction, the DSET instruction and the REGISTER statement in a pattern program, and the REG statement in a test plan program.

D5, D6

D5 for TP and D6 for TP2 are used when data operation are executed with them. This can be set by SET instruction, the DSET instruction and the REGISTER statement in a pattern program and the REG statement in a test plan program.

ARIRAM (Area inversion)

This is a specific address written in ARIRAM. It is used to invert the value in the TP register.

To use ARIRAM to invert the value in the TP2 register, specify TP2INV in the MODE statement.

If the double speed mode which uses the pre-scrambler is specified, each Y addresses for the first half and for the second half of the double speed mode can be used as the inversion conditions for the TP1 and TP2 data.

Y address used for the TP1 data inversion condition : The Y addresses for the first half of the double speed mode

Y address used for the TP2 data inversion condition : The Y addresses for the second half of the double speed mode

➔ For more information on the double speed mode, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

Parity Generator

This is used to output DA0 to 7 odd parity to DA16 and DA8 to 15 odd parity to DA17 when PARITY is specified by the MODE statement.

The odd parity refers to the parity generation in which "1" is output to the parity bit (DA16 or DA17) when an even number of bits are flagged with "1" and "0" is output when an odd number of bits are flagged with "1".

Parity generation for data on the DB0 to 17 side is performed in the same way as described above.

However, if TPM36 is specified in the MODE statement, no parity is generated for data on the DB0 to 17 side.

When using DA16, DB16 or DA17, DB17 as even parity for the input data or the expected data in DUT, specify </D16> or </D17> by the pin statement.

PD1 = IN1</D16>

PC33 = OUT1</D16>

2. 5. 2 Configuration of Data Generation Instruction

The data generation part comprises the following instructions.

- TP register field
- FP function field
- DSD, RCD or CCD field
- Data inversion field
- Data hold field
- Data swap field
- Data shift field
- Source data field
- Expected data inversion field

2. 5. 3 Outline of the Data Generation Instruction

The instruction mnemonics of the data generation part are shown below.

Only one instruction can be selected for a single cycle from a group of instructions enclosed by a pair of square brackets. If an instruction is enclosed in a rectangle, it indicates that that instruction has been selected from the group.

TP register field

TP<TP		TP2<TP2	
TP<TP*2		TP2<TP2*2	
TP<TP/2		TP2<TP2/2	
TP</TP		TP2</TP2	
TP<0		TP2<0	
TP<TP+1		TP2<TP2+1	
TP<TP-1		TP2<TP2-1	
TP<TPH		TP2<TPH2	
TP1<TPH1B	*1	TP2<TPH2B	*1
TP1<TPH1C	*1	TP2<TPH2C	*1
TP1<TPH1D	*1	TP2<TPH2D	*1
TP<>TPH		TP2<>TPH2	
TP1<>TPH1B	*1	TP2<>TPH2B	*1
TP1<>TPH1C	*1	TP2<>TPH2C	*1
TP1<>TPH1D	*1	TP2<>TPH2D	*1
TP<D5		TP2<D6	
TP<TP'D5 (OR)		TP2<TP2'D6 (OR)	
TP<TP&D5 (AND)		TP2<TP2&D6 (AND)	
TP<TP"D5 (XOR)		TP2<TP2"D6 (XOR)	
TP<TP+D5		TP2<TP2+D6	
TP<TP-D5		TP2<TP2-D6	
TP</D5		TP2</D6	

*1 Can be executed on the T5377.

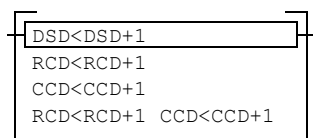
TP can be described as TP1, TPH can be described as TPH1 or TPH1A, and TPH2 can be described as TPH2A.

FP function field

FP0	(Fix 0)
FP1	(Checkerboard)
FP3	(Diagonal)
FP4	(Inversion diagonal)
FP5 *1	(Diagonal 2)
FP6 *1	(Inversion diagonal 2)
FP7	(Masked parity)
FP8	(Row bar)
FP9	(Column bar)
FP10	(Row/column bar)
FPX *1	(Universal FP)

*1 Can be executed on the T5377, T5376 and T5375.

DSD, RCD or CCD field



Data inversion field

```

[ /D ]
[ /D2 ]

```

Data hold field

```

[ PD ]

```

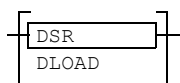
Data swap field

```

[ DS ]

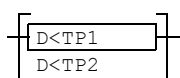
```

Data shift field



DSR and DLOAD can be specified on the T5377, T5376, T5375 and T5371 if the data shift mode (MODE SRDOUTn) is set.

Source data field



D<TP1 and D<TP2 can be specified on the T5377, T5376 and T5375 if the 36-bit mode (MODE TPM36) is set.

Expected data inversion field

```

[ DINV ]

```

The DINV can be specified in the data inversion mode (MODE DINVn) for the T5377.

2. 5. 4 TP Register Field

The TP register field is a set of instructions that generates pattern data and expected data to be applied to the DUT by operating the TP and TP2 registers.

The TP and TP2 registers are each 36 bits long. However, if TPM18 (default) is specified in the MODE statement, only data of the 18 low-order bits is output. To output data from all 36 bits, specify TPM36 in the MODE statement.

TP<TP TP2<TP2 (HOLD)

This instruction is used when the operation of TP is not carried out, and may not be described.
When not described, this functions in the same manner as TP<TP.

TP<TPH(TPH1A to TPH1D), TP2<TPH2(TPH2A to TPH2D) (LOAD)

This instruction initializes the TP, TP2 registers, loading the contents of the TPH1A to TPH1D, TPH2A to TPH2D registers into the TP, TP2 registers.

When using the TP register for data generation, initialize this instruction as follows:

```
MPAT SAMPLE

REGISTER
:

TPH=#0

:
START #0

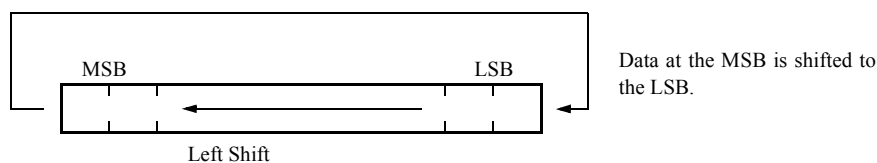
NOP XB<0 YB<0   TP<TPH   .... Loads TPH and initializes.

ST0: JNl1 ST0 W XB<XB+1 YB<YB+1^BX
:

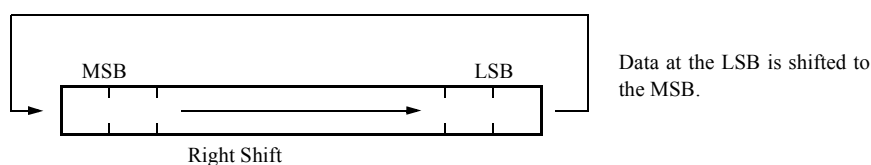
END
```

TP<TP*2, TP2<TP2*2 (Shift left)

Shifts the contents of the TP register leftward as shown below.

**TP<TP/2, TP2<TP2/2 (Shift Right)**

Shifts the contents of the TP register rightward as shown below.

**TP<TP+1, TP2<TP2+1 (Increment)**

This instruction increments (+1) the contents of the TP, TP2 registers.

TP<TP-1, TP2<TP2-1 (Decrement)

This instruction decrements (-1) the contents of the TP, TP2 registers.

TP<0, TP2<0 (Clear)

This instruction zeros (#0) the contents of the TP, TP2 registers. Use this instruction to initialize the TP, TP2 registers when generating data by the register.

TP<>TPH(TPH1A to TPH1D), TP2<>TPH2(TPH2A to TPH2D) (SWAP)

This instruction is used to exchange the contents of the TP, TP2 registers with those of the TPH(TPH1A to TPH1D), TPH2(TPH2A to TPH2D) registers. Use this instruction to save the contents of the TP, TP2 registers in TPH(TPH1A to TPH1D), TPH2(TPH2A to TPH2D).

TP<D5, TP2<D6

This instruction loads each contents of the D5 and D6 registers to the TP, TP2 registers.

TP<TP'D5, TP2<TP2'D6

This instruction loads the logical sum of the TP(TP2) register and the D5(D6) register into the TP(TP2) register.

TP<TP&D5, TP2<TP2&D6

This instruction loads the AND of the TP(TP2) register and the D5(D6) register into the TP(TP2) register.

TP<TP"D5, TP2<TP2"D6

This instruction loads the exclusive OR of the TP(TP2) register and the D5(D6) register into the TP(TP2) register.

TP<TP+D5 TP2<TP2+D6

This instruction loads the arithmetic addition of the TP(TP2) register and the D5(D6) register into the TP(TP2) register.

TP<TP-D5, TP2<TP2-D6

This instruction loads the arithmetic subtraction of the TP(TP2) register and the D5(D6) register into the TP(TP2) register.

TP</D5, TP2</D6

This instruction loads each reverse data of the D5 register and D6 register to the TP, TP2 registers.

 Tip

TPs and TP1s are interchangeable in the various TP register field instructions. Similarly, TPH can be described as TPH1 or TPH1A, and TPH2 can be described as TPH2A. This exchange causes no change in operation.

2. 5. 5 FP Function Field

The FP function field (address function) is a set of instructions that generates pattern data and the expected data to be applied to the DUT using logical operation of the X and Y addresses to be applied to the DUT. If the output of the address function is "1", the value in the TP register is

inverted. To use the address function to invert the value in the TP2 register, TP2INV must be specified in the MODE statement. The functions and the instructions that can be selected are explained in the following.

FP0 (FIX 0)

When this instruction is selected, "0" is always output regardless of the contents of the TP (TP2) register or the values of X and Y addresses.

This instruction is used to generate background data for a test pattern like Galloping or Walking.

FP1 (Checker board)

When this instruction is selected, the contents of TP (TP2) register is inverted according to the exclusive-OR condition between the LSBs of the X and Y addresses to be applied to the DUT.

		X				
		00	01	10	11	
Y	00	0	1	0	1	$X0 \oplus Y0$
	01	1	0	1	0	
	10	0	1	0	1	
	11	1	0	1	0	

FP3 (Diagonal)

The contents of TP (TP2) register is inverted when the algebraic sum of the Y address to be applied to the DUT and the contents of the DSD register equals the X address.

The algebraic sum and X address are compared within the range of the significant bits specified by the XMAX register.

		DSD → X				
		00	01	10	11	
Y	00	0	1	0	0	$(Y + \text{DSD}) \text{ .AND. } X_{\text{MAX}} = (X \text{ .AND. } X_{\text{MAX}})$ (Illustration at left is for when DSD = 1.)
	01	0	0	1	0	
	10	0	0	0	1	
	11	1	0	0	0	

The data patterns output depending on the contents of the DSD register are illustrated in the following.

		X			
		00	01	10	11
Y	00	1			
	01		1		
	10			1	
	11				1

DSD=0

		X			
		00	01	10	11
Y	00		1		
	01			1	
	10				1
	11	1			

DSD=1

		X			
		00	01	10	11
Y	00			1	
	01				1
	10	1			
	11		1		

DSD=2

		X			
		00	01	10	11
Y	00				1
	01	1			
	10		1		
	11			1	

DSD=3

FP4 (Inversion diagonal)

This instruction inverts the value of the TP (TP2) register if the 1s complement of the algebraic sum of the Y address applied to the DUT and the DSD register equals the X address in the inversion diagonal pattern data of FP3 (Diagonal). The algebraic sum and X address are compared within the range of significant bits specified by the conjunction of the XMAX register.

		X			
		00	01	10	11
Y	00	0	0	1	0
	01	0	1	0	0
	10	1	0	0	0
	11	0	0	0	1

$$\overline{(Y + DSD) \text{ .AND. XMAX}} = (X \text{ .AND. XMAX})$$

(Illustration at left is for when DSD = 1.)

The data patterns output depending on the contents of the DSD register are illustrated in the following.

		X			
		00	01	10	11
Y	00				1
	01			1	
	10		1		
	11	1			

DSD=0

		X			
		00	01	10	11
Y	00			1	
	01		1		
	10	1			
	11				1

DSD=1

		X			
		00	01	10	11
Y	00		1		
	01	1			
	10				1
	11			1	

DSD=2

		X			
		00	01	10	11
Y	00	1			
	01				1
	10			1	
	11		1		

DSD=3

FP5 (Diagonal 2)

The contents of TP (TP2) register is inverted when the algebraic sum of the Y address to be applied to the DUT and the contents of the DSD register equals the X address. The algebraic sum and X address are compared within the range of the significant bits specified by the conjunction of the XMAX and YMAX registers.

FP5 is the same instruction as FP3 (Diagonal) except for the significant bits.

The FP5 instruction is available on the T5377, T5376 and T5375.

		DSD →			X			XMAX	
		000	001	010	011	100	101	110	111
Y	000	0	1	0	0	0	1	0	0
	001	0	0	1	0	0	0	1	0
	010	0	0	0	1	0	0	0	1
YMAX	011	1	0	0	0	1	0	0	0

$(Y + DSD).AND.(XMAX.AND.YMAX)$
 $= X.AND.(XMAX.AND.YMAX)$
 (Illustration at left is for when DSD = 1.)

The data patterns output depending on the contents of the DSD register are illustrated in the following.

		X				XMAX			
		000	001	010	011	100	101	110	111
Y	000	1				1			
	001		1				1		
	010			1				1	
YMAX	011				1				1

DSD=0

		X				XMAX			
		000	001	010	011	100	101	110	111
Y	000			1				1	
	001				1				1
	010	1				1			
YMAX	011		1				1		

DSD=2

FP6 (Inversion diagonal 2)

This instruction inverts the value of the TP (TP2) register if the 1s complement of the algebraic sum of the Y address applied to the DUT and the DSD register equals the X address in the inversion diagonal pattern data of FP5 (Diagonal 2). The algebraic sum and X address are compared within the range of significant bits specified by the conjunction of the XMAX and YMAX registers.

FP6 is the same instruction as FP4 (Inversion diagonal) except for the significant bits.

The FP6 instruction is available on the T5377, T5376 and T5375.

		X									
		000	001	010	011	100	101	110	111		
										DSD	
										←	
										XMAX	
Y	000	0	0	1	0	0	0	1	0		
	001	0	1	0	0	0	1	0	0		
	010	1	0	0	0	1	0	0	0		
	011	0	0	0	1	0	0	0	1		
YMAX											

$$\overline{(Y + DSD)}.AND.(XMAX.AND.YMAX)$$

$$= X.AND.(XMAX.AND.YMAX)$$

(Illustration at left is for when DSD = 1.)

The data patterns output depending on the contents of the DSD register are illustrated in the following.

		X				XMAX			
		000	001	010	011	100	101	110	111
Y	000				1				1
	001			1				1	
	010		1				1		
	YMAX 011	1				1			

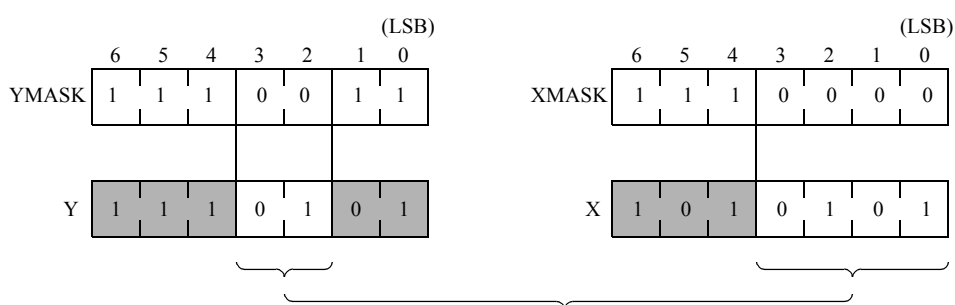
DSD=0

		X				XMAX			
		000	001	010	011	100	101	110	111
Y	000		1				1		
	001	1				1			
	010				1				1
	YMAX 011			1				1	

DSD=2

FP7 (Mask parity)

The contents of TP (TP2) register is inverted when the number of "1" bits, which are specified by the XMASK and YMASK registers, of the X and Y addresses to be applied to the DUT is odd.



When the 1 bits in X0 to 3 and Y2 to 3 are odd, the contents of the TP (TP2) are inverted.
The diagonal-line portion is masked by XMASK and YMASK.

See the examples below.

XMASK=#3FE YMASK=#3FE

$X0 \oplus Y0$ generates a checker board.

$XMASK = \#3FD$ $YMASK = \#3FD$

$X1 \oplus Y1$ generates a double checker board.

$XMASK = \#0$ $YMASK = \#3FF$

X-parity results.

$XMASK = \#3FF$ $YMASK = \#0$

Y-parity results.

FP8 (Row bar)

When this instruction is selected, the XMASK register compares "0" bits with the contents of RCD at the address to be applied to DUT. When the two are equal, the contents of the TP (TP2) register are inverted.

If $(X.AND.XMASK) = RCD$, the contents of TP (TP2) register are inverted.

See the examples below.

$XMASK = \#3FE$ $RCD = \#1$

		X				
		00	01	10	11	
Y	00	0	1	0	1	
	01	0	1	0	1	
	10	0	1	0	1	
	11	0	1	0	1	

Stripe pattern

$XMASK = \#3FD$ $RCD = \#2$

		X				
		00	01	10	11	
Y	00	0	0	1	1	
	01	0	0	1	1	
	10	0	0	1	1	
	11	0	0	1	1	

When X1 of the X address is "1", the TP (TP2) register is inverted.

FP9 (Column bar)

When this instruction is selected, the YMASK register compares "0" bits with the contents of CCD at the address to be applied to DUT. When the two are equal, the contents of the TP (TP2) register are inverted.

When $(Y.AND.YMASK)$, the contents of TP (TP2) register is inverted.

See the examples below.

YMASK = #3FE CCD = #1

		X					
		00	01	10	11		
Y	00	0	0	0	0		Stripe pattern
	01	1	1	1	1		
	10	0	0	0	0		
	11	1	1	1	1		

FP10 (Row/column bar)

The AND of row bar and column bar inverts the contents of TP (TP2) register.

XMASK = #3FE RCD = #1

YMASK = #3FE CCD = #1

		X					
		00	01	10	11		
Y	00	0	0	0	0		
	01	0	1	0	1		
	10	0	0	0	0		
	11	0	1	0	1		

FPX (Universal FP)

For the cycle in which this instruction is specified, select the address function (FP0, FP1, FP3, FP4, FP5, FP6, FP7, FP8, FP9 or FP10) specified in the FPSEL register. If DISABLE is specified with the value in the FPSEL register, no address function is selected, which is the same as the condition in which the FPX instruction is not specified.

The FPSEL register can be specified by the REGISTER statement in the pattern program or the REG MPAT statement in the test plan program.

Address functions can be selected from the test plan program by specifying the FPSEL register in the REG MAPT statement.

The FPX instruction is available on the T5377, T5376 and T5375.

Sample programs are shown below.

```

PRO pro_file_name
:
TEST 100
REG MPAT PC = #0
REG MPAT FPSEL = FP3      ; Select FP3 for FPX.
MEAS MPAT mpat_file_name
:
TEST 200
REG MPAT PC = #0
REG MPAT FPSEL = FP4      ; Select FP4 for FPX.
MEAS MPAT mpat_file_name
:
END

```

```

MPAT mpat_file_name
:
START #0
:
NOP  FPX      ; FP3 is selected in TEST100, and FP4 is selected in TEST200.
:
STPS
END

```

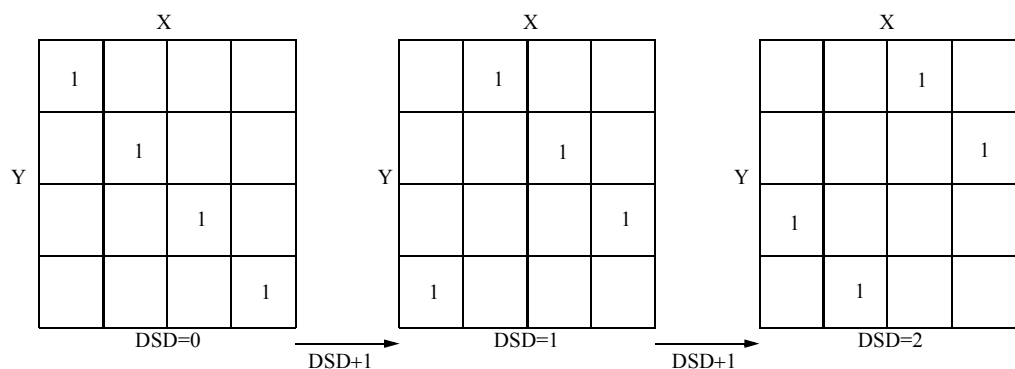
2. 5. 6 DSD, RCD, CCD Field

The DSD, RCD, or CCD field comprises the following instructions that are used to modify the DSD, RCD or CCD register needed when pattern data are generated by means of address functions.

DSD<DSD+1

Increments the contents of the DSD register for pattern generation by FP3, FP4, FP5 or FP6.

This instruction is used for Shifted-diagonal pattern generation.



RCD<RCD+1

This instruction increments the RCD register used for generating FP8 and FP10.

CCD<CCD+1

This instruction increments the CCD register used for generating FP9 and FP10.

RCD<RCD+1 CCD<CCD+1

This instruction increments the RCD register and the CCD register in the same cycle.

2. 5. 7 Temporary Data Inversion(/D, /D2)(Data Inversion Field)

The /D instruction inverts data in the TP register.

The bits inverted by the /D instruction can be specified by the DCMR register.

The bits in the TP register corresponding to the "0" bits in the DCMR register are inverted.

If DCMR is not specified, the DCMR is set to #0.

The /D2 instruction inverts data in the TP2 register.

The bits inverted by the /D2 instruction can be specified by the DCMR2 register.

If DCMR2 is not specified, the DCMR2 is set to #0.

```

REGISTER
DCMR = #3FF00
:
START #0
; DA0-17
NOP TP<0 ; X
NOP ; #0
NOP /D ; #FF DA8 to 17 are not inverted.
```

2. 5. 8 Inversion with Data Flag (DFLG)

When DFLG = 1, the contents of the TP register are inverted.

The DFLG register can be reversed by JZD instruction.

```

START #0
NOP XB<0 YB<0 TP<0

[
ST0:JNI1 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB
ST1:JNI1 ST1 R XB<XB+1 YB<YB+1^BX X<XB Y<YB
]
JZD ST0
STPS
Processing in this step is executed twice.
The data in TP register are inverted when the DFLG is set to 0 for the first time,
then the DFLG is set to 1 for the second time.
```

Important

TP2INV should be specified in the MODE statement for being inverted the data in TP2 register with the DFLG.

2. 5. 9 Area Inversion

➔ For more information on area inversion, refer to [4. 3. 4 "Area Inversion Test \(ARIRAM\)" on page 4-43](#).

2. 5.10 Saving Output Data (PD) (Data Hold Field)

Data in DA0 to DA17 and DB0 to DB17 in the previous cycle output to DA0 to DA17 and DB0 to DB17 unchanged in the cycle in which PD is entered.

◆ Example

		; TP0-17	DA0-17
NOP	TP<0	; X	X
NOP	TP</TP	; #0	#0
NOP	TP<0	; #3FFFF	#3FFFF
NOP	<u>PD</u>	; #0	#3FFFF
NOP	<u>PD</u>	; #0	#3FFFF
NOP		; #0	#0

2. 5.11 Data Swapping (DS) (Data Swap Field)

Changes the output data to the data output from PM (expected data in expected pattern mode or DBM data).

Only for the cycle with DS description, FDA/FDB is output as a test data. It is not affected by selected test data by DATA DEFINE statement or SELECT PDS DATA statement in the main program. The DS instruction cannot be specified concurrently with the Source data field instruction.

2. 5.12 Data Shift Field

DSR and DLOAD are effective when SRDOUT is specified in the MODE statement and can output TP1 data serially according to the specified bit configuration.

DSR (Data Shift Right)

When SRDOUTn is specified, DSR shifts output data n bits to the right.

DLOAD (Data Load)

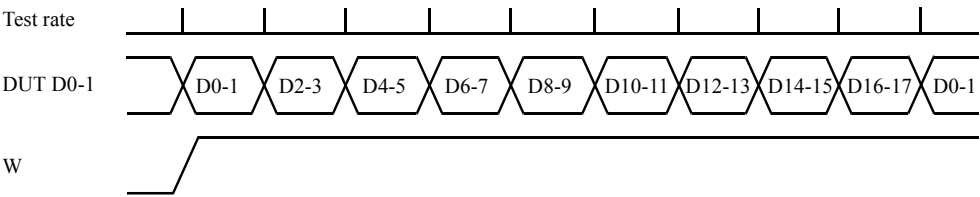
DLOAD initializes the shifted data.

◆ Example of Using DATA SHIFT

```
MPAT SRDOUT
MODE SRDOUT2
REGISTER
    XMAX=#3F
    YMAX=#FF
    TPH=#0
    IDX1=#6
    IDX2=3FFE

START #0
    NOP XB<0 YB<0 TP<TPH
    NOP DLOAD
ST0: JN11 . DSR W
    JN12 ST0 XB<XB+1 YB<YB+1^BX
```

An example appears below:



!

Important

- Before data is shifted using DSR, DLOAD must already be executed.
- If the 36-bit mode (MODE TPM36) is specified, bits 18 to 36 of TP1 cannot be output serially.

2. 5.13 Source Data Field

The Source data field instruction selects data to be output to the programmable data selector (PDS) in real time. It can be specified in the 36-bit mode (MODE TPM36).

If D and D2 are specified for test data in the DATA DEFINE statement in the pattern program or in the SELECT PDS DATA statement in the test plan program, data DA0 to DA17 is output to D0 to D17 of the PIN statement, and data DB0 to DB17 is output to SD0 to 17.

The Source data field instruction and MODE TPMn can be used to select the data sent to DA0 to DA17 and DB0 to DB17, as shown in [Table 2-13](#).

If FDA and FDB are specified for test data, data output from PM is output to DA0 to DA17 and DB0 to DB17 independently of the Source data field instruction.

If D.XOR.FDA and D2.XOR.FDB are specified for test data, the data output by D and D2 and the data output by FDA and FDB are exclusive-ORed and output to DA0 to DA17 and DB0 to DB17, respectively.

Omitting the Source data field is the same as specifying D<TP1 in the 36-bit mode.

Table 2-13 Data Output to DA0 to DA17 and DB0 to DB17 when D and D2 are Specified for the Test Data

MODE TPM18	Source data field instruction	Output data	
		17.....DB.....0	17.....DA.....0
MODE TPM18	Unavailable	17.....TP1.....0	17.....TP1.....0
MODE TPM36	D<TP1 (TP1 data)	35.....TP1.....0	
	D<TP2 (TP2 data)	35.....TP2.....0	

2. 5.14 Expected Data Inversion Field (DINV)

The DINV instruction specifies the expected data of TP1 and TP2 in the cycles which are inverted and controlled by data transitions of one byte.

This instruction is valid when DINVn is specified in the MODE instruction.

➔ For more information, refer to [2. 2. 3 "MODE Statement" on page 2-5](#).

2. 5.15 Instructions and Execution Cycle

There are two types of data operation instructions depending on the cycle in which the results of the operation output to DA0 to DA17 and DB0 to DB17, as follows:

Execution in the step in which instructions are described (Event time 1)

The following instructions are valid as soon as they are entered:

- FP function field instruction
- Data inversion field instruction
- Data hold field instruction
- Source data field instruction
- Expected data inversion field instruction

◆ Example

```
ST0: JN11 ST0 W XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
```

The contents of the TP and TP2 registers are inverted if the addresses, which are selected using X<XB and Y<YB (described at the same step as FP1), give a result of X0.XOR.Y0=1.

Execution in the step next to the one in which instructions are described (Event time 2)

The results of the calculation for TP1, TP2, DSD, RCD, and CCD are valid in the next step.

◆ **Example**

```

                                TP
NOP  TP < 0                      ; undefined
NOP  TP < TP+1                  ; #0 ← Operation result for TP < 0
NOP                                ; #1 ← Operation result for TP<TP+1

```

2. 5.16 Notes on Program Generation

Duplicate conditions for inverting data bits

When the following conditions are executed an odd number of times, data bits (DA0 to DA17 and DB0 to DB17) are inverted. When the conditions are executed an even number of times, the data bits are not inverted.

The inversion condition for data is as follows:

/D, /D2, DFLG, address function, ARIRAM (area inversion), DINV

Default value for data operation

The default values of the data operations are as follows:

TP1<TP1, TP2<TP2

2. 6 Control Bits

2. 6. 1 Configuration of Control Instruction

The control part comprised the following instructions.

TS field

TS1 to TS16

MUT field

T5377, T5376 and T5375

R, W, M1 to 2, C0 to 23, MMA, MMB

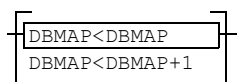
T5371

R, W, M1 to 2, C0 to 15, MMA, MMB

T5336 and T5311

R, W, M1 to 2, C0 to 11

DBMAP field



Address control field

XT, YT

SCRAM control field

SCROFF

ARIRAM control field

ARIOFF

PM control field

PM/2, PM/4

PRESCRAM control field (Can be specified on the T5377, T5376, T5375 and T5371.)

PSCROFF

Cycle Palette field (Can be specified on the T5377, T5376, T5375 and T5371.)

CYP1 to CYP16

X Address save field (Can be specified on the T5377.)

XASV, XALD

Y Address save field (Can be specified on the T5377.)

YASV, YALD, YASL

Burst Length field (Can be specified on the T5377.)

BLn (n=2,4,8,16)

Wrap Type field (Can be specified on the T5377.)

WPTSQ

WPTIL

2. 6. 2 TS Field

The period and phase of the clock, used to generate the address, data, and clock to be applied to DUT, can be changed at each cycle by the following statement in the pattern program.

```

:
NOP TSn (or /Tn)   n=1 to 16
:

```

When nothing is described, TS1 will be allocated.

```

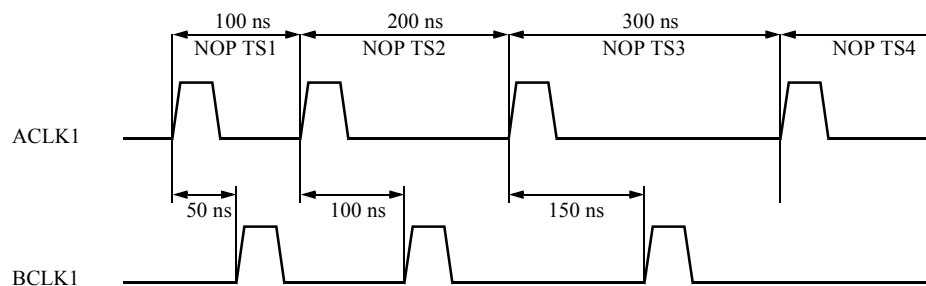
PRO file name
:
RATE  = 100NS, 200NS, 300NS, 400NS
ACLK1 = 0NS:4
BCLK1 = 50NS, 100NS, 150NS, 200NS
:
END

```

```

MPAT file name
:
NOP TS1
NOP TS2
NOP TS3
NOP TS4
:
END

```



2. 6. 3 MUT Field

In the MUT field (MUT signal), the following control bits can be described.

T5377, T5376 and T5375

R, W, M1 to 2, C0 to 23, MMA, MMB

T5371

R, W, M1 to 2, C0 to 15, MMA, MMB

T5336 and T5311

R, W, M1 to 2, C0 to 11

R, W, M1, M2, C0 to 23, MMA, MMB is used for the purposes shown below.

- Data patterns on the control terminals of $\overline{WE}$, $\overline{CS}$, and $\overline{OE}$ of DUT. (CPE1 to 4)
- Control of Enable/Disable of logical comparison. (CPE1 to 4)
- Control of Driver Enable/Disable of I/O pins. (DRE1 to 2)
- Real time inversion of driver time. (RINV)
- Timing set switching and inhibition of address multiplex in page mode test and nibble mode test. (R, W, M1, M2)
- Specifying the cycles of real time swap (RDSM) when MODE DATEN is specified. (C0)
- Specifying a cycle in which clocks are inhibited when the real time clock inhibitions (RINHA, RINHB and RINHC) are specified in the pin statement. (R, W, M1, M2, C0 to 11, PGI, PG, PGC, PM1, PGCC11)

However, the real time clock inhibitions is not available on the T5377, T5376, T5375 and T5371. MMA and MMB are control bits to specify the fail data fetch cycle when AF and BF are specified by using AFM ACTION.

2. 6. 4 Data Pattern on Control Terminals of DUT

Describing the control bits (R, W, M1 to 2 and C0 to 23) in pattern programs generates the data patterns on control terminals of $\overline{WE}$, $\overline{CS}$, and $\overline{OE}$ of DUT.

Specify the data to be applied to each pin by the pin statement in a test plan program.

```

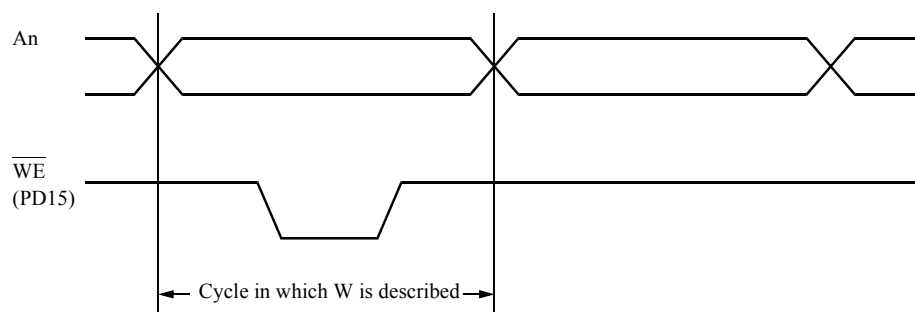
:
:
PD15=IN1,/RZO,BCLK2,(CCLK2),<WT>;/WE
:
:           ↓
:           Indicates "W".

```

```

MPAT M16K
:
:
START #0
NOP      XB<0  YB<0  TP<0
ST0: JN11 ST0  W  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
ST1: JN11 ST1  R  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
JZD ST0
STPS
END

```



2. 6. 5 Specifying the Cycles for Performing Logical Comparison

The data output from DUT is logically compared with the expected value in the pattern program at a cycle describing the control bits (R, W, M1, M2, C0 to 23, PREF) specified by CPE1 to 4.

PREF is output according to the following conditions:

$R_t .OR. (R_{t-1} .AND. M1_t)$

STRB to control logical comparison by CPE is shown below.

	STRB	WSTRB	MWS ^{*1} mode
CPE1	STRB1, 3	--	WSTRB1, 3
CPE2	STRB2, 4	WSTRB1, 2	WSTRB2, 4
CPE3	STRB1, 3	--	WSTRB1, 3
CPE4	STRB2, 4	WSTRB1, 2	WSTRB2, 4

```

MPAT M16K
REGISTER
XMAX = #7F
YMAX = #7F
CPE1 = R ; FIXL, FIXH, R, W, M1 to M2, CO to C15, PREF can be selected.
:
:
START      #0
NOP        XB<0  YB<0  TP<0
ST0: JN11  ST0 W  XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
ST1: JN11  ST1 R*2 XB<XB+1  YB<YB+1^BX  X<XB  Y<YB  FP1
JZD  ST0
STPS
END

```

^{*1} MWS: Multiwindow strobe

^{*2} The output from the MUT is logically compared with the expected data in the cycle specified by CPE_n in the REGISTER statement.

! Important

- "R" is selected when no specification has been made by CPE1, CPE2, CPE3, or CPE4.
- Logical comparisons cannot be performed in STPS and STFL cycles.

2. 6. 6 Specifying to Turn Driver ON or OFF in I/O Operation

The control bit turns the driver on or off to test the DUT whose DIN and DOUT are in the I/O structure. The I/O mode for pins is specified by the pin condition statement of the test plan program. The driver operates in the cycle during which the control bit specified by the DRE1, DRE2 is entered in the pattern program, and it is placed in the High-Z state during other cycles.

```
DREL1 = 100NS
DRET1 = 200NS
      :
      :
P33-40=IN1, OUT1, IOC*1, XOR, ACLK1 BCLK1,STRB1,(DRE1),(DRECLK1),DRERZ,<D0-7>
      :
      :
```

**1 Specifying I/O*

```
MPAT SAMPLE
REGISTER
      :
      :
DRE=W ; FIXL, FIXH, R, W, M1 to M2, C0 to C23 can be selected.
      :
      :
START      #0
NOP
ST0: JN11 ST0 W*1 XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
ST1: JN11 ST1 R XB<XB+1 YB<YB+1^BX X<XB Y<YB FP1
      JZD ST0
      STPS
END
```

**1 The driver is enabled only in the cycle during which "W" specified by DRE is described.*

Important

- When selecting a DRE pattern using an I/O pin pin condition statement, make the specification using DRE1 (DRE1 pattern) or DRE2 (DRE2 pattern). When no specification is made, the default is DRE1.
- When selecting a DREL/T pattern using an I/O pin pin condition statement, make the specification using DRECLKn (DREL/Tn). When no specification is made, the default is DRECLK1. (n=1 to 128)

2. 6. 7 Specifying Real Time Inversion of the Driver

The driver pattern from ALPG is output inverted at a cycle describing the control bits (R, W, M1 to 2 and C0 to 23), specified by RINV in the pattern program. Specify RINV by the pin conditional statement to designate pins for real time inversion.

PD1 = IN1, NRZA, ACLK1, RINV, <D0>

RINV specifies real-time inversion.

```
MPAT SAMPLE
REGISTER
:
RINV=C0 ; FIXL, FIXH, R, W, M1 to 2, C0 to 23 can be specified.
:
START #0
NOP
NOP C0*1
NOP
:
```

**1 Driver pattern from ALPG is output inverted at only a cycle describing "C0" designated by RINV.*

Driver pattern chosen by PDS	0	1	1	0	1	1	0	0
RINV signal from ALPG	0	0	1	1	1	0	1	0
Pattern when PDn = IN1, RINV ~	0	1	0	1	0	1	1	0
Pattern when PDn = IN1 ~	0	1	1	0	1	1	0	0

Numbers surrounded by the circle indicate a pattern inverted in real time.

2. 6. 8 Timing Lot Switching and Address Multiplex Control

The control bits R, W, M1, and M2 provide the following controls.

- When MODE MUX is specified and SDM (Swap data mode) is specified by the pin conditional statement, pin data are controlled as follows:

PDn = IN1,, SDM, <A, B>

Use such modes to change the waveform mode of the driver in real time and to inhibit the output of the driver waveform.

The following signals can be chosen for the pin data.

R, W, M1 to 2, C0 to 11, FL, FH, PG, PGC, PGI, PGCC11, PM1

FL

Fixed low level

FH

Fixed high level

PM1

Signal 1-cycle before describing M1

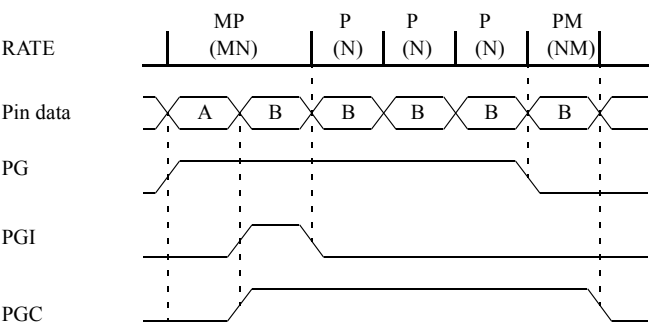
PG, PGI, PGC

Signal to be output when MODE PAGE or NIBBLE is specified.

PGCC11

Specifying by logical product of PGC and C11.

PG, PGI, and PGC are output at the following cycles.



MP: Page in cycle

MN: Nibble in cycle

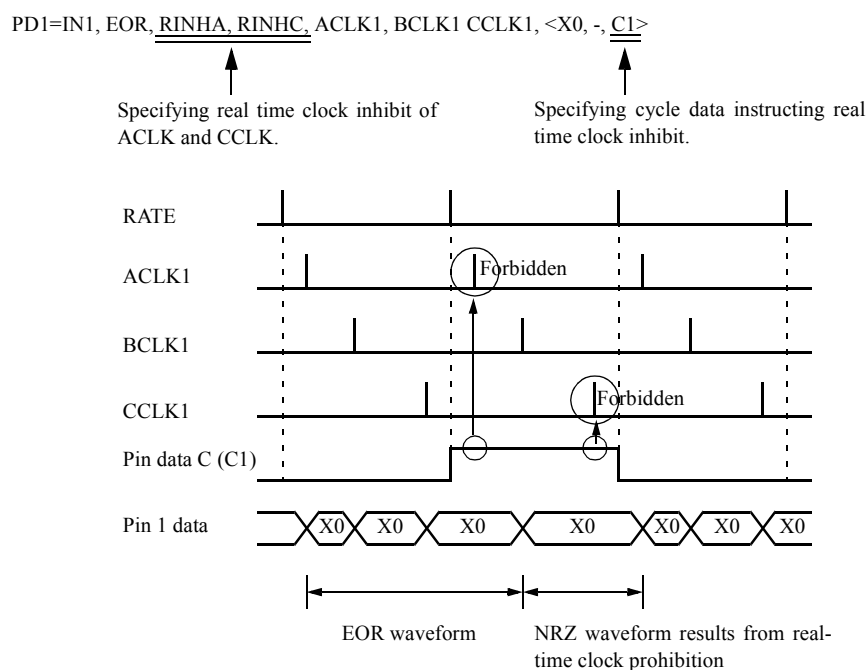
P: Page cycle

N: Nibble cycle

PM: Page out cycle

NM: Nibble out cycle

See the examples below.



2. 6.11 DBMAP Field

When PM is used in DBM mode, DBMAP field is used to generate the address which accesses PM.

DBMAP<DBMAP

Hold DBMAP.

DBMAP<DBMAP+1

Increment DBMAP.

DBMAP is a 24-bit (16-bit for T5371, T5336 and T5311) address pointer.

2. 6.12 Address Control Field

XT

Output the XT register value to the X address temporally.

YT

Output the XT register value to the Y address temporally.

Address control field is used to output logical data temporally for the address line on the DUT measurement which executes the command setting from the address line.

2. 6.13 SCRAM Control Field

SCROFF

Descramble of the address is temporally off.

When converting the address generated in the ALPG with the address descramble memory, SCROFF is used to stop the next conversion temporally.

It is used to execute the command setting from the address line, together with [2. 6. 12 "Address Control Field" on page 2-171](#).

```
MPAT filenameA

REGISTER
  XT=n
  YT=n

SCRAM filenameB
:
:
NOP  X<XB Y<YB
NOP  X<XB Y<YB      XT YT SCROFF  *1
NOP  X<XB Y<YB
:
:
```

**1 In this cycle, the XT register value is output to the X address, and the YT register value is output to the Y address without being descrambled.*

2. 6.14 ARIRAM Control Field

ARIOFF

To reverse the data with the area inversion memory is temporally off.

It is used to stop the area inversion by ARIRAM temporally when measuring the device which executes the command and mode setting from the data line.

2. 6.15 PM Control Field

PM control field is used for the ROM test with the byte/word converted function.

2. 6.17 Cycle Palette Field

CYPn

Specifies cycle pallet.

Specify with range of n=1 to 16.

When nothing is described, CYP1 is allocated.

Output data from the ALPG and pin data in real time can be switched per cycle with cycle palette.

Pin data can be switched per cycle by specifying CYPn in the pattern program. (The specification of CYPn is valid in specified cycle.)

CYPn corresponds with pin data by using data selector in the pin statement.



Tip

This instruction is available on the T5377, T5376, T5375 and T5371.

```
PRO file name
:
P33=...,CYP1,<D1>,<D2>,<D3>,<D4> ; Specify CYP1=D1, CYP2=D2, CYP3=D3 and CYP4=D4
:
END
```

```
MPAT file name
:
NOP CYP1      ; D1 is output to P33.
NOP CYP2      ; D2 is output to P33.
NOP CYP3      ; D3 is output to P33.
NOP CYP4      ; D4 is output to P33.
:
END
```



Important

If the pin data of cycle palette is not specified in the pin statement, the previously specified data is retained.

➔ For details, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

2. 6.18 X Address Save Field

Memory devices, which contain two or more banks, can control the banks separately (multi bank operation).

Interleaving each bank enables such operations as applying row addresses to bank A and applying row addresses to bank B, then returning to bank A to apply the column addresses and read data.

When testing this kind of operation, row addresses, which are applied to the FM and which are used for the inversion conditions of the FP function and the ARIRAM, are in a different bank from the one used to read the data.

The X Address Save Field instruction is used to solve the problem described above.

For example, by saving the row addresses for each bank and by loading the row addresses saved in the cycle in which data is read, the row addresses in the bank which reads data can be used in the FM, the FP function, and the ARIRAM.

XASV

Saves the row addresses in the cycle in which the XASV is specified (the X addresses from the ALPG).

Enter an active command to the bank, and specify the XASV in the cycle which applies the row addresses.

By specifying the bank address on the row addresses in the BANK SAVE ADDRESS statement, the row addresses for each bank can be saved (up to 16 row addresses can be saved).

XALD

Loads the row addresses saved in the XASV (the X addresses from the ALPG) by the cycle in which the XALD is specified.

Applies the column addresses to the bank and specify the XALD in all cycles from the ones in which the read command is entered to the ones in which data is read.

By specifying the bank address on the column addresses in the BANK LOAD ADDRESS statement, the row addresses for each bank can be saved.

Tip

- *X Address Save Field is available in the T5377.*
- *The XASV and the XALD can be described in the same step. In this case, the XASV is executed first, and then the XALD is executed.*
- *The X addresses that are loaded by the XALD instruction are output according to the specification in the SELECT PRESCRAM ADDRESS statement.*

◆ Example

These examples show how to generate the addresses for the FM, the FP function, and the ARIRAM in the read operation of the multi bank operation under the following conditions.

Wrap type = Sequential

Burst length = 4

CAS latency (CL) = 2

MPAT SAMPLE

SDEF ACT = C0 ; Activate command
SDEF READ = C1 ; Read command

REGISTER

NH=0

BH=0

ADDRESS DEFINE

X13-14 = B0-1 ; Row Bank address

Y13-14 = N0-1 ; Column Bank address

XY=ZN

BANK SAVE ADDRESS X13,X14 ; Row Bank address selection

BANK LOAD ADDRESS Y13,Y14 ; Column Bank address selection

START #0

NOP ; Command Row address

NOP B<BH ;

NOP ACT XASV ; [BANK A] Activate, Save

NOP B<B+1 ; NOP

NOP ACT XASV N<NH ; [BANK B] Activate, Save

NOP READ XALD R B<B+1 ; [BANK A] Read, Load

NOP ACT XASV XALD R N<N+1 ; [BANK C] Activate, Save, [BANK A] Load

NOP READ XALD R B<B+1 ; [BANK B] Read, Load

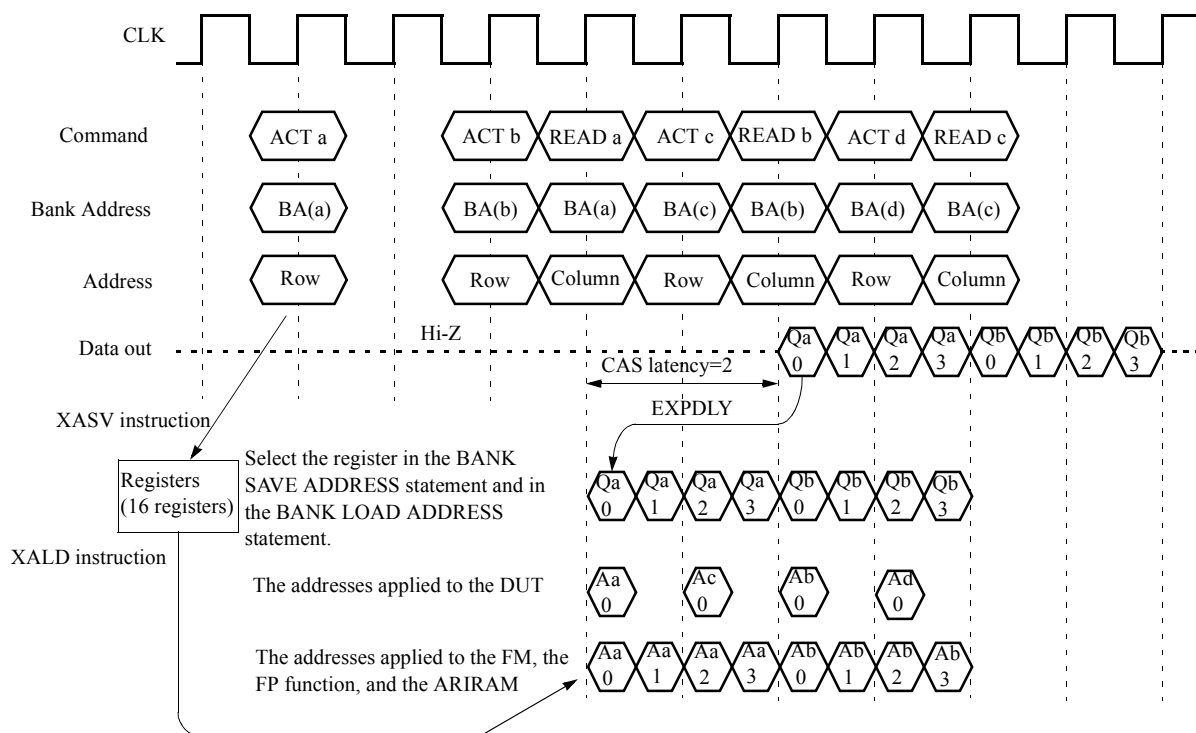
NOP ACT XASV XALD R N<N+1 ; [BANK D] Activate, Save, [BANK B] Load

NOP READ XALD R ; [BANK C] Read, Load

NOP

STPS

END

Figure 2-14 Read Operation of Multi Bank Operation

The a, b, c, and d indicates bank A, bank B, bank C, and bank D, respectively.

2. 6.19 Y Address Save Field

The SDRAM provides the stop function of device operations, which stops the clock transmission into the device during the burst transfer. While the clock is stopped, updating the column addresses, which are applied to the FM, the FP function, and the ARIRAM, must also be stopped. Using the Y Address Save Field instruction generates these column addresses.

YASV

The column addresses in the cycle in which the YASV instruction is specified (the Y addresses from the PRESCRAM) are saved in the FIFO memory. (At the time of saving, the write address pointers of the FIFO memory are automatically incremented.)

Specifies the YASV in all cycles from the cycle in which the read command is entered to the read cycle, excluding the cycle that stops the device clock.

YALD

The read address pointers of the FIFO memory are incremented in the cycle in which the YALD instruction is specified.

Specifies the YASV in all cycles from the cycle after the CAS latency minutes of the cycles entered the read command to the read cycle, excluding the cycle which stops the device clock.

YASL

The cycles, in which the YASL instruction is specified, read out the column addresses saved in the FIFO memory by using the YASV instruction from the read address pointers specified by the YALD instruction, and output them to the FM, the FP function, and the ARIRAM.

Specify the YASL in all cycles from the cycle after the CAS latency minutes of the cycles entered the read command to the read cycle.

Tip

- *Y Address Save Field is available in the T5377.*
- *The YASV and the YALD can be described in the same step. In this case, the YASV is executed first, and then the YALD is executed.*
- *The Y addresses from the PRESCRAM section which are saved by the YASV instruction are the Y addresses after being controlled by the following statement.*
 - *SEND PRESCRAM statement*
 - *SELECT PRESCRAM statement*
 - *SELECT PRESCRAM MODE statement*
 - *SELECT PRESCRAM ADDRESS statement*
 - *SET WRAP ADDRESS statement*
- *The Y addresses which are loaded by the YASL instruction are output on the FM and the FP regardless of the specification in the SELECT PRESCRAM ADDRESS statement.*

◆ Example

These examples show how to generate the addresses for the FM with the internal clock suspended, the FP function, and the ARIRAM under the following conditions.

Wrap type = Sequential

Burst length = 4

CAS latency (CL) =2

MPAT SAMPLE

SDEF CS = C0
SDEF RAS = C1
SDEF CAS = C2
SDEF CKE = C3

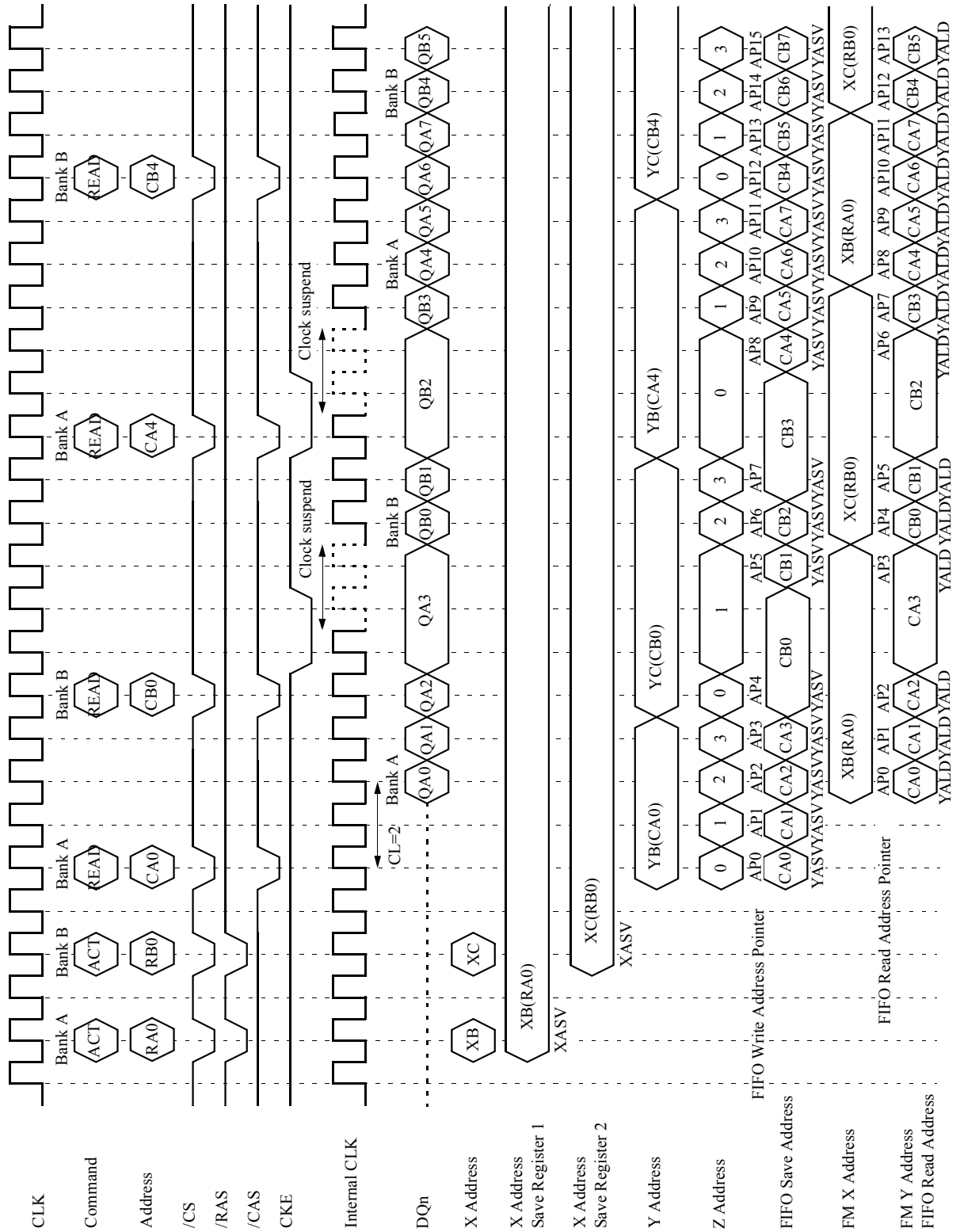
REGISTER

D1A=#100 ; XC initial Data
D2A=#200 ; YC initial Data
D2B=#4 ; Burst length=4

```
START #0
NOP XB<0 XC<D1 YB<0 YC<D2A Z<0          CKE          ; ALPG Add.  FM Add.
NOP          X<XB Y<YB      CS RAS CKE XASV          ; X   Y Z   X   Y
NOP          X<XB Y<YB          CKE          ; #0   -   -   -   -
NOP          X<XC Y<YC      CS RAS CKE XASV          ; #100 -   -   -   -
NOP          X<XC Y<YC          CKE          ; #100 -   -   -   -
NOP          Z<Z+1 X<XB Y<YB      CS CAS CKE          YASV ; -   #0 0   -   -
NOP          Z<Z+1 X<XB Y<YB          CKE          YASV ; -   #0 1   -   -
NOP          Z<Z+1 X<XB Y<YB R          CKE XALD YASV YALD YASL ; -   #0 2   #0  #0
NOP YB<YB+D2B Z<Z+1 X<XB Y<YB R          CKE XALD YASV YALD YASL ; -   #0 3   #0  #1
NOP          Z<Z+1 X<XC Y<YC R CS CAS CKE XALD YASV YALD YASL ; - #200 0   #0  #2
NOP          X<XC Y<YC R          XALD          YASL ; - #200 1   #0  #3
NOP          X<XC Y<YC R          XALD          YASL ; - #200 1   #0  #3
NOP          Z<Z+1 X<XC Y<YC R          CKE XALD YASV YALD YASL ; - #200 1   #0  #3
NOP          Z<Z+1 X<XC Y<YC R          CKE XALD YASV YALD YASL ; - #200 2 #100 #200
NOP YC<YC+D2B Z<Z+1 X<XC Y<YC R          CKE XALD YASV YALD YASL ; - #200 3 #100 #201
NOP          X<XB Y<YB R CS CAS          XALD          YASL ; -   #4 0 #100 #202
NOP          X<XB Y<YB R          XALD          YASL ; -   #4 0 #100 #202
NOP          Z<Z+1 X<XB Y<YB R          CKE XALD YASV YALD YASL ; -   #4 0 #100 #202
NOP          Z<Z+1 X<XB Y<YB R          CKE XALD YASV YALD YASL ; -   #4 1 #100 #203
NOP          Z<Z+1 X<XB Y<YB R          CKE XALD YASV YALD YASL ; -   #4 2   #0  #4
NOP YB<YB+D2B Z<Z+1 X<XB Y<YB R          CKE XALD YASV YALD YASL ; -   #4 3   #0  #5
NOP          Z<Z+1 X<XC Y<YC R CS RAS CKE XALD YASV YALD YASL ; - #204 0   #0  #6
NOP          Z<Z+1 X<XC Y<YC R          CKE XALD YASV YALD YASL ; - #204 1   #0  #7
NOP          Z<Z+1 X<XC Y<YC R          CKE XALD YASV YALD YASL ; - #204 2 #100 #204
NOP          Z<Z+1 X<XC Y<YC R          CKE XALD YASV YALD YASL ; - #204 3 #100 #205
:
STPS
```

END

Figure 2-15 Address Generation under Clock Suspension



2. 6.20 Burst Length Field

BLn

This instruction is used to convert the burst length of the burst operation into real time.

Specify a burst length of 2, 4, 8, or 16 for n.

The burst length is specified by the effective bits of the Y address output from the pre-scrambler according to the SET WRAP ADDRESS statement. This specification can be converted into real time by describing the BLn instruction in the pattern program. The setting is valid from the cycle after the cycle in which the BLn instruction is specified, to the pattern end or to the cycle in which a different BLn instruction is described.

If no BLn instruction is described, follow the specification in the SET WRAP ADDRESS statement. The burst length specifications (n=2,4,8,16) of the BLn instruction correspond to the Y address specifications of the SET WRAP ADDRESS statement as follows:

- BL2 : Y0
- BL4 : Y0-1
- BL8 : Y0-2
- BL16 : Y0-3

Tip

- The BLn instruction can be executed for the T5377.
- Group A registers in the SET instruction and the DSET instruction cannot be specified in the same cycle with the BLn instruction.
 - ➔ For more information on the group A registers, refer to [2. 3. 3 "Operation Register Setting Instruction \(SET reg. data\)" on page 2-65](#).
- The specification of the SET WRAP ADDRESS statement is changed by the BLn instruction, however, it is recovered by the next function test.

◆ Example

These examples show how to convert the burst length and the wrap type into real time as follows:

Burst length	4 → 8
Wrap type	Sequential type → Interleave type

MPAT SAMPLE

SDEF CS = C0
SDEF RAS = C1
SDEF CAS = C2
SDEF WE = C3

REGISTER

D1=3 ; Burst Start Address
D2A=4 ; Burst length=4
D2B=8 ; Burst length=8

START #0

NOP XC<0 YC<D1 Z<0 XT<#2

NOP SCROFF PSCROFF XT CS RAS CAS WE BL4 WPTSQ; Set Mode Register

NOP *1 *2 ;

NOP X<XC Y<YC CS RAS ; Active ; Addresssing

NOP X<XC Y<YC ; ; Y Z Burst

NOP Z<Z+1 X<XC Y<YC R CS CAS ; Read ; 3 0 3

NOP Z<Z+1 X<XC Y<YC ; ; 3 1 0

NOP Z<Z+1 X<XC Y<YC ; ; 3 2 1

NOP YC<YC+D2A Z<Z+1 X<XC Y<YC R ; ; 3 3 2

NOP Z<Z+1 X<XC Y<YC R CS CAS ; Read ; 7 0 7

NOP Z<Z+1 X<XC Y<YC ; ; 7 1 4

NOP Z<Z+1 X<XC Y<YC ; ; 7 2 5

NOP YC<YC+D2A Z<Z+1 X<XC Y<YC R ; ; 7 3 6

NOP X<XC Y<YC CS RAS WE ; Precharge

NOP XT<#B ;

NOP SCROFF PSCROFF XT CS RAS CAS WE BL8 WPTIL; Set Mode Register

NOP *3 *4 ;

NOP X<XC Y<YC CS RAS ; Active ; Addresssing

NOP X<XC Y<YC ; ; Y Z Burst

NOP Z<Z+1 X<XC Y<YC R CS CAS ; Read ; B 0 B

NOP Z<Z+1 X<XC Y<YC ; ; B 1 A

NOP Z<Z+1 X<XC Y<YC ; ; B 2 9

NOP Z<Z+1 X<XC Y<YC ; ; B 3 8

NOP Z<Z+1 X<XC Y<YC ; ; B 4 F

NOP Z<Z+1 X<XC Y<YC ; ; B 5 E

NOP Z<Z+1 X<XC Y<YC ; ; B 6 D

NOP YC<YC+D2B Z<Z+1 X<XC Y<YC R ; ; B 7 C

NOP Z<Z+1 X<XC Y<YC ; ; 13 0 13

:

STPS

END

*1 Burst length = 4

*2 *Wrap type = Sequential type*

*3 *Burst length = 8*

*4 *Wrap type = Interleave type*

2. 6.21 Wrap Type Field

This instruction is used to convert the wrap type of the burst operation into real time.

WPTSQ

The wrap type is converted into the sequential type.

WPTIL

The wrap type is converted into the interleave type.

The wrap type is specified by the ADDZ1 to 4 (sequential type) and the EORZ1 to 4 (interleave type) in the SELECT PRESCRAM MODE statement. This specification can be converted into real time by describing the WPTSQ and the WPTIL instruction in the pattern program. The setting is valid from the cycles after the cycle in which the WPTSQ and the WPTIL instructions are specified to the pattern end or to the cycle in which different WPTSQ and the WPTIL instructions are described.

If the WPTSQ and the WPTIL instructions are not described, the specification in the SELECT PRESCRAM MODE statement is used.

The specification of the SELECT PRESCRAM ADDRESS statement is changed as follows by the WPTSQ and the WPTIL instructions.

- WPTSQ (sequential mode)
 - NORMAL → ADDZ1
 - EORZ1 → ADDZ1
 - EORZ2 → ADDZ2
 - EORZ3 → ADDZ3
 - EORZ4 → ADDZ4
 - ADDZ1 to 4 → ADDZ1 to 4 (the mode is not changed)
- WPTIL (interleave mode)
 - NORMAL → EORZ1
 - ADDZ1 → EORZ1
 - ADDZ2 → EORZ2
 - ADDZ3 → EORZ3
 - ADDZ4 → EORZ4
 - EORZ1 to 4 → EORZ1 to 4 (the mode is not changed)

Tip

- The WPTSQ and the WPTIL instructions can be executed in the T5377.

- *The group B registers in the SET instruction and the DSET instruction cannot be specified in the same cycle with the WPTSQ and the WPTIL instructions.*
 - ➔ For more information on the group B registers, refer to [2. 3. 3 "Operation Register Setting Instruction \(SET reg. data\)" on page 2-65](#).
 - *The specification of the SELECT PRESCRAM MODE statement is changed by the WPTSQ and the WPTIL instructions, however, it is recovered by the next function test.*
-

◆ **Example**

- ➔ For more information on how to use the instruction, refer to [2. 6. 20 "Burst Length Field" on page 2-181](#).

2. 7 ROM Test Pattern Data Program Section

This section describes the format of the ROM test pattern data program section. The ROM test pattern data program section begins with AFM PATTERN statement and ends with an END statement, a data declaration statement, or a test pattern program.

2. 7. 1 Programming Format

Describe a ROM test pattern in the following format.

```
MPAT program-name
:
AFM PATTERN                ---- (1)
  BITSIZE = m,n            ---- (2)
  AMAX   = x1,y1          ---- (3)
  AMIN   = x2,y2          ---- (4)
  FMA    = x3,y3          ---- (5)
  XX
  XXXX                ---- (6)
  XX!a
  XXXX!b
  XX__XXXX, XX!c__XXXX!d
    <tab>      <space>
:
END
```

(2) to (6) must be described after the AFM PATTERN statement.

2. 7. 1. 1 AFM PATTERN Statement

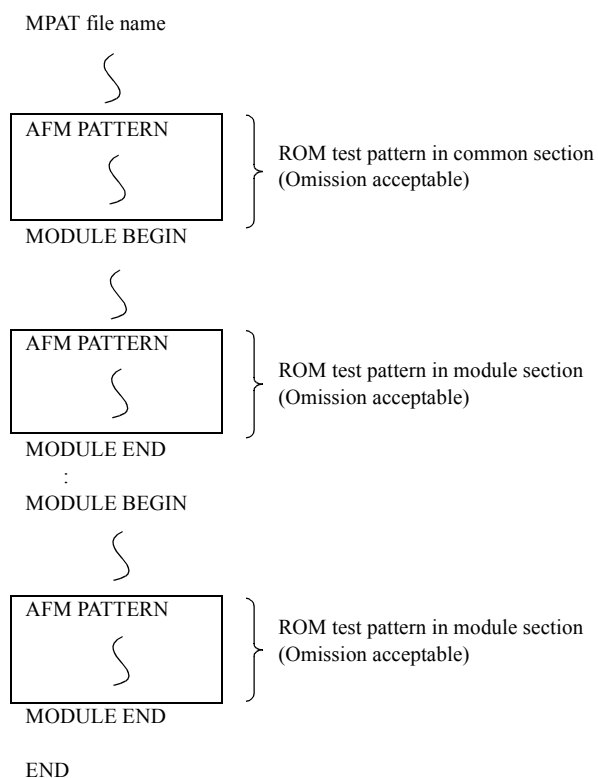
Function: This is a ROM test pattern start declaration statement; it can be entered in the common section and each module section.

Syntax

```
AFM PATTERN
```

Example

(1) Example of Describing a ROM Test Pattern



Tip

Describe the ROM test pattern after the MODE and REGISTER statements.

2. 7. 1. 2 BITSIZE Statement

Function: This statement is a compilation control instruction. It defines the transfer mode for ROM test pattern data.

This statement can be defined only once before ROM test pattern data.

If BITSIZE statement is omitted, results are m=8 and n=1.

m and n are compiled with the current radix.

Syntax

```
BITSIZE = m[,n]
```


Argument: m

Specifies the configuration of a ROM test pattern in the memory.

m=1	1bit
m=4	4bit
m=8	8bit
m=9	9bit
m=16	16bit
m=18	18bit
m=19 to 36	36bit

n

Specifies the number of patterns in a single piece of data.

m=1	$1 \leq n \leq 32$
m=4	$1 \leq n \leq 8$
m=8	$1 \leq n \leq 4$
m=9	$1 \leq n \leq 4$
m=16	$1 \leq n \leq 2$
m=18	$1 \leq n \leq 2$
m=19 to 36	$1 \leq n \leq 1$

Results if omitted

n=1

2. 7. 1. 3 AMAX Statement

Function: This statement is a compilation control instruction. It can define the maximum values of the X and Y addresses.

This statement can be entered only once before ROM test pattern data.

Syntax

$AMAX = x_1[, y_1]$

Description

Restrictions for x_1 and y_1 are as follows:

$$0 \leq x_1 \leq 262143$$

$$0 \leq y_1 \leq 262143$$

X and Y must not exceed the maximum X and Y determined by the AFM ADDRESS statement of a test plan program.

If the AMAX statement is not specified, operation is performed at the maximum X and Y determined by the AFM ADDRESS statement in a test plan program. Any one of X and Y can be specified by the following method:

AMAX = x_1 ; Specify X only

AMAX = 0, y_1 ; Specify Y only

2. 7. 1. 4 AMIN Statement

Function: This statement is a compilation control instruction. It can define the minimum values of the X and Y addresses.

This statement can be entered only once before ROM test pattern data.

Syntax

```
AMIN =  $x_2$  [,  $y_2$ ]
```

Description

Restrictions for x_2 and y_2 are as follows:

$$0 \leq x_2 \leq 262143 \text{ and } x_2 \leq x_1$$

$$0 \leq y_2 \leq 262143 \text{ and } y_2 \leq y_1$$

If omitted, the results are $x_2=0$ and $y_2=0$.

2. 7. 1. 5 FMA Statement

Function: This statement is a compilation control instruction. Data coming after this statement is compiled as ROM test pattern data until another compilation control instruction appears. This statement also specifies the start address of PM which transfers ROM test pattern data.

If this statement is not specified, it assumes the same value as that specified by the AMIN statement.

Syntax

```
FMA =  $x_3$  [,  $y_3$ ] or ROMPAT =  $x_3$  [,  $y_3$ ]
```

Argument: x_3

Specifies an X start address value

y_3

Specifies an Y start address value

Description

Up to 500 FMA statements can be used in a single program.

x_3 and y_3 are compiled using the current radix.

x_3 and y_3 must fall within the range specified by AMAX and AMIN.

Example

(1) Example of Describing an FMA Statement

```
AFM PATTERN
:
:
FMA =  $x_1$ ,  $y_1$  ----- (one)
#4
#6
:
FMA =  $x_2$ ,  $y_2$  ----- (two)
#7
#8
:
:
(MAX 500)
```

2. 7. 1. 6 ROM Test Pattern Data

Function: The same ROM test pattern data can be repeated.

Syntax

```
 $p_1$ [! $n_1$ ][, $p_2$ [! $n_2$ ]]-----
```

Argument: p_1 , p_2

Describe ROM test pattern data.

n_1 and n_2

Specify the number of repetitions for p_1 , p_2 .

If omitted or $n_1=0$.

It is assumed as $n_1=1$

Description

p_1, p_2, n_1, n_2 are compiled using the current radix.

They can assume the following characters:

- Numerals from 0 to 9
- Alphabetic characters from A to F
- # (Compilation in hexadecimal regardless of the current radix)
- \$ (Compilation in octal regardless of the current radix)

However, although the current radix is 16, even if the starting character is an alphabetic character, an error will result. In this case, begin with 0 or #. The maximum number of characters per line is 132.

Entries of ROM test patterns are explained below in conjunction with PM.

Pattern data delimiter

Tabs, commas (,) and spaces can be used as delimiters to separate pattern data and to handle each as a unit data.

For X, Y address only

```
MPAT SAMPLE1

AFM PATTERN
AMAX      = #F, #F
FMA       = #0, #0

2
#AA
#55!2
#11, #22 #33!2 #44

END
```

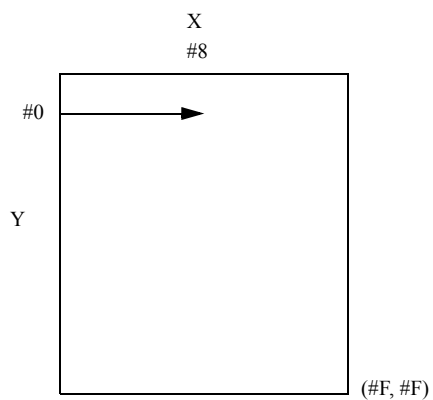
The following pattern is obtained.

Address		Data	
X	Y	BIT0	BIT7
		↓	↓
#0	#0	00000010	(2)
#1	#0	10101010	(#AA)
#2	#0	01010101	(#55)
#3	#0	01010101	(#55)
#4	#0	00010001	(#11)
#5	#0	00100010	(#22)
#6	#0	00110011	(#33)
#7	#0	00110011	(#33)
#8	#0	01000100	(#44)
⋮	⋮	↕ No data	
#F	#F		

["!2" causes the same data to be repeated twice.)

["!2" causes the same data to be repeated twice.)

Correspondence with PM is as follows:



2. 7. 2 Example of Programming

A program to transfer a 64 K ROM tester pattern to PM is exemplified below.

```

PRO file name
:
:
RESET AFM ALL
AFM ADDRESS=X0-7, Y0-7
AFM RATE=HS
AFM BITSIZE=8
AFM ACTION=PM>EXP
SEND FM SAMPLE1
MEAS MPAT SAMPLE1
:
:
:
END

MPAT SAMPLE1

REGISTER
  XMAX=#FF
  YMAX=#FF
  IDX1=#FFFE
  TPH =#0
START      #0
  NOP      XB<0  YB<0  TP<0
  :
  :
  :
AFM PATTERN
  BITSIZE=8
  AMAX=#FF,#FF
  AMIN=#0,#0
  FMA =#0,#0
  #AA!#10000

END

```

The following pattern is obtained.

Address		Data	
X	Y	BIT0	BIT7
#0	#0	↓	↓
#1	#0	10101010	10101010
#2	#0	10101010	
#3	#0		
⋮	⋮		
#FF	#0		
#0	#1		
#1	#1		
⋮	⋮		
#FF	#FF	10101010	

Pattern 10101010 is transferred to PM. This pattern can be used as a driver pattern or expectation pattern.

Tip

There are two kinds of expectation patterns. One is generated by ALPG, and the other, by PM.

When specifying D.XOR.FDA as the PDS data (using the PDS DATA statement under the DATA DEFINE declaration) and specifying Dn as the expected value pattern for the pin statement data selector, the expected value pattern is given as the exclusive-OR between the ALPG and PM.

To conduct a test with an expectation pattern from PM only, an expectation pattern from ALPG must be always set to "0". Specifically, always make test pattern register (TP) #0 where perform logical comparison in an MPAT program.

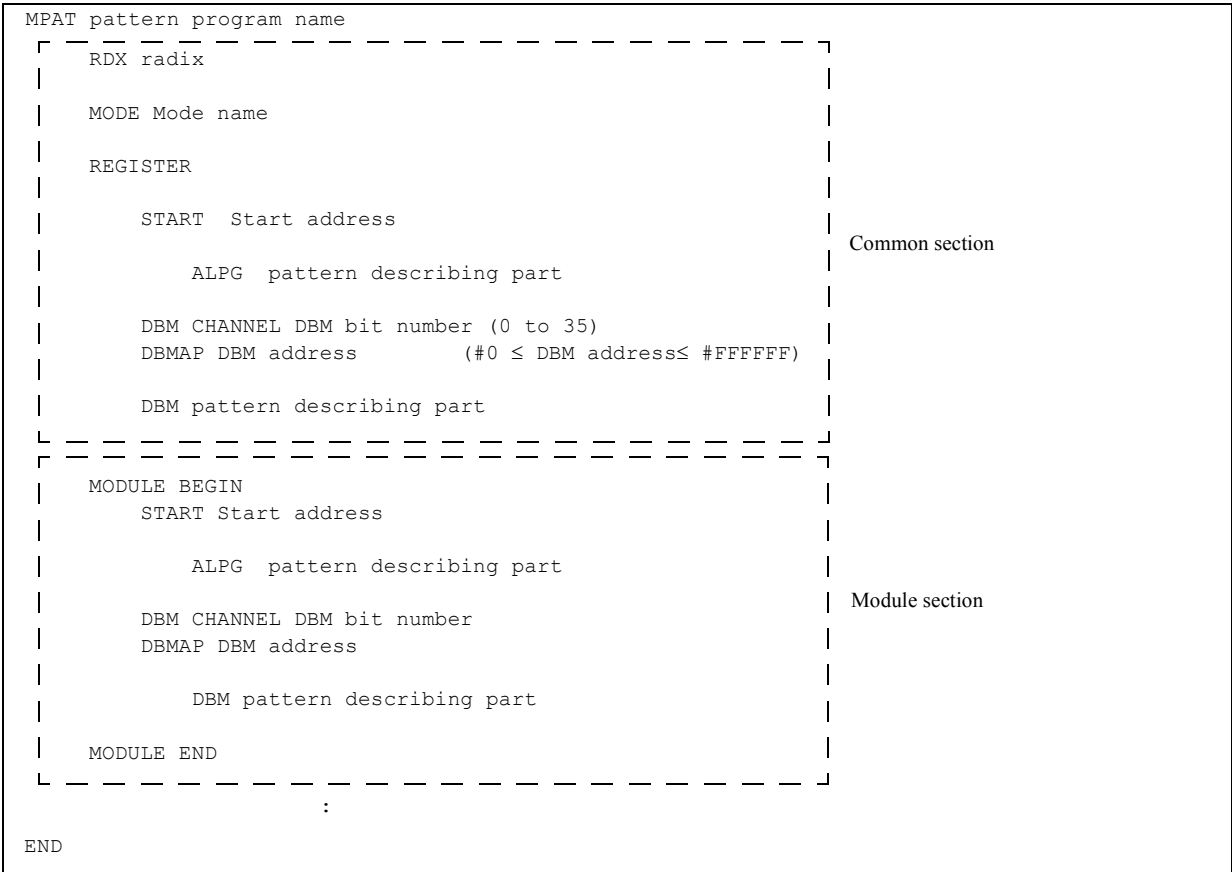
When specifying FDA (FDB) as the PDS data and specifying Dn (SDn) as the expected value pattern for the pin statement data selector, it is not necessary to set the test pattern register to #0.

2. 8 DBM Pattern

It is possible to generate random data using Data Buffer Memory (DBM).
Transfer random data that have been programmed by the pattern program to DBM, using the SEND DBM statement to control DBMAP (DBM Address Pointer) from ALPG. This enables any test pattern to be generated. (Mainly, this is used for generating logic patterns.)
Settings to generate an expected value from DBM using the AFM ACTION statement are required.

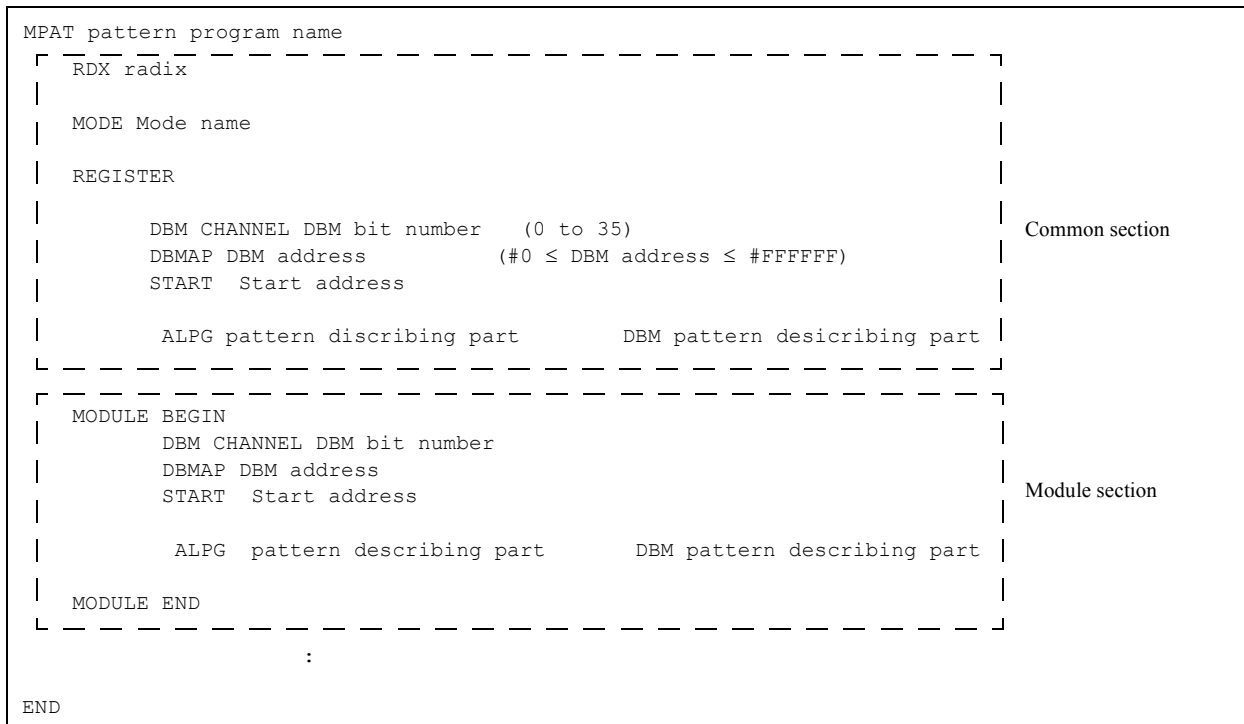
2. 8. 1 Programming Format

Syntax 1



Although Syntax 1 shows that a single pattern program can have both ALPG and DBM patterns at the same time, only a DBM pattern can be written.
A DBM pattern can be written anywhere in the module (or in the common section) only if it follows the DBMAP statement. Thus, it can also be written on the right side of an ALPG pattern, as shown below in Syntax 2.

Syntax 2



While this means that a single pattern program can have both ALPG and DBM patterns at the same time, only a DBM pattern can be written.

 Tip

- For the purpose of explanation, a distinction is made between Syntaxs 1 and 2; in practice, they are syntactically identical.
- When writing a DBM pattern on the same line as an ALPG pattern, be sure to write it to the right of the ALPG pattern.

2. 8. 2 DBM CHANNEL Statement

Function: This statement specifies a DBM data bit number.

Syntax

```
DBM CHANNEL i
DBM CHANNEL i,j
DBM CHANNEL i-j
```

Description

This statement specifies a DBM data bit number. Specify it only once before the DBMAP statement in the module. (Cannot be omitted when a DBM pattern is described. Also, multiple descriptions cannot be made within a single module.)

Ascending and descending continuous specification and random specification can be used together in the range from 0 to 35.

The channel number is treated as a decimal number regardless of the radix currently in use.

Example

(1) DBM CHANNEL Specification

```
DBM CHANNEL 0-35
DBM CHANNEL 0,18-35
DBM CHANNEL 0-17
DBM CHANNEL 0,1,2,3,8-11
```

(2) Entry for Each Module

```
MODULE BEGIN
  DBM CHANNEL 0-7
  DBMAP #0
    ?00001100
    ?11111111
    :
  START      #0
  NOP
  :
  STPS
MODULE END
MODULE BEGIN
  DBM CHANNEL 8-15
  DBMAP #100
    ?11110000
    ?00000000
    :
  START      #100
  NOP
  :
  STPS
MODULE END
```

2. 8. 3 DBMAP Statement

Function: This statement specifies a DBM starting address from which DBM pattern data is to be transferred.

Multiple starting addresses can be entered in the module (common section, also). However, an address larger than the starting address of a previously entered DBMAP statement should be specified. A DBM starting address should not be identical to the DBM data address that has already been entered.

Be sure to specify this statement before entering the DBM pattern data. (This step cannot be omitted.)

Syntax

```
DBMAP n
```

Argument: n

Starting address

Block size	Range
64 KW	#0 ≤ n ≤ #FFFF
16 MW	#0 ≤ n ≤ #FFFFFF

2. 8. 4 DBM Data Field

The DBM pattern data (logic pattern) is entered. The DBM CHANNEL statement and DBMAP statement are required before the DBM pattern data is entered.

DBM pattern data

A test pattern is entered for the DBM data bit selected by the DBM CHANNEL statement. A pattern must begin with "?".

The following two patterns can be generated in the DBM:

Data	Added pattern
0	0
1	1

! Important

The driver enable cannot be controlled in units of pins. However, it is possible to control it through signals selected by DRE1 and DRE2.

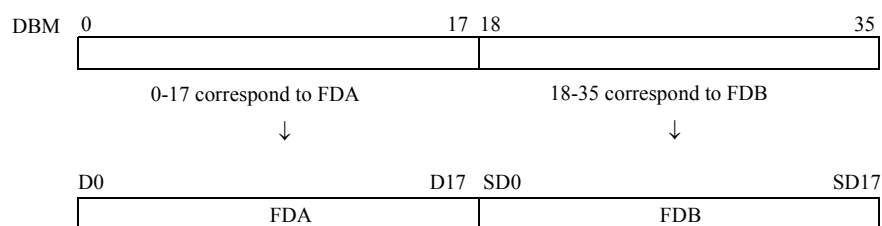
Data allocation

Data is entered from left to right, according to the order specified by the DBM CHANNEL statement. An address is assigned to a single line.

```
DBM CHANNEL 0,2,5,8
DBMAP #0 ;0 2 5 8 ← bit number
?0101 ;0 1 0 1 ← Address #0 data
?1111 ;1 1 1 1 ← Address #1 data
?1111 ;1 1 1 1 ← Address #2 data

DBM CHANNEL 0-7.8
DBMAP #10 ;0 1 2 3 4 5 6 7 8 ← bit number
?111100001 ;1 1 1 1 0 0 0 0 1 ← Address #10 data
?110011001 ;1 1 0 0 1 1 0 0 1 ← Address #11 data
?100000001 ;1 0 0 0 0 0 0 0 1 ← Address #12 data
```

The DBM 0-35 bits correspond to FDA and FDB for PDS DATA as follows:



Whether to output FDA or FDB data to PDS is determined by SELECT PDS DATA. Also, a pin statement determines the pin to which to output the data selected for PDS.

Tip

PDS data can also be selected by the PDS DATA statement in the pattern program. However, a statement in the test plan program is given higher priority.

◆ Settings in the Test Program

```

PRO SAM3
:
SELECT PDS DATA D.XOR.FDA/FDB ..... Selects data to be output to PDS pin.
:
PD1-12 = IN1,NRZA,ACLK1,<D0-11> ..... Selects FDA0-11 for PD1-12.
PC33-40 = OUT1,<SD0-7> ..... Select FDB0-7 for PC33-40.
:
AFM ACTION = PM>DBM(AP) ..... Selects data generation by DBM.
:
SEND DBM PATDB ..... Transfers the DBM pattern.
SEND MPAT PATM ..... Transfers the ALPG pattern.
MEAS MPAT ..... Performs measurement.

```

Mode PM>DBM(AP) uses DBMAP as a DBM address pointer. In addition to this mode, DBM(PC) mode, in which PC is used for the DBM address pointer, is available.

Data omission

Entry of only "?" is processed under the assumption that data was omitted. At that time, data becomes "0". When the number of bits of the DBM pattern is smaller than that selected by the DBM CHANNEL statement, data is assumed to be omitted and "0" is assigned.

◆ Data Omission

```

DBM CHANNEL 0-35
DBMAP #0
?1111 1111 1111 ← Bits 0 to 11 data, bits 12 to 35 are omitted.
?1111 1111      ← Bits 0 to 7 data, bits 8 to 35 are omitted.
?1111 1111 1111 ← Bits 0 to 11 data, bits 12 to 35 are omitted.
?              ← All omitted.
?0000         ← Bits 0 to 3 data, bits 4 to 35 are omitted.

```

Tip

When field division is used, data cannot be omitted for a part of the field.

➔ For information on field division, refer to "Field division".

Field division

A pattern can be field-divided by using delimiters.

Field division enables the pattern field immediately after the DBMAP statement to be effective for the later patterns. (This field on the first line is called base field.)

Specifying (-) sets the same data as that at the preceding address in the same field.

◆ Setting the Same Data as the Preceding One

```
DBM CHANNEL 0-35
DBMAP #0
?00000000 00000000 00 00000000 00000000 00
?11111111 -      11 11111111 -      -
?00000000 11111111 - -      -      11
?11111111 -      - 00000000 -      -
```

Data is set as follows:

```
00000000 00000000 00 00000000 00000000 00
11111111 00000000 11 11111111 00000000 00
00000000 11111111 11 11111111 00000000 11
11111111 11111111 11 00000000 00000000 11
```



Tip

The start line of the DBM data field serves as a base for determining later data format, so (-) cannot be specified at that position. And, if data is omitted from the start line, all the omitted bits are processed as a single field.

◆ When Data is Omitted in the Start Line

```
DBM CHANNEL 0-35
DBMAP #0
?00000000 00000000 00 00000000
?11111111 -      11 11111111 0000000000
?00000000 11111111 - -      0000000011
?11111111 -      - 00000000 -
```

The data is identical to that in ["Setting the Same Data as the Preceding One" on page 2-200](#).

2. 8. 4. 1 REPEAT Statement

Function: This is a compilation control statement for generating the same pattern repeatedly.

Syntax

```
REPEAT n,m
    DBM pattern
```

Argument: n

Number of repetitions

$1 \leq n \leq 16777216(\#1000000)$

Results if omitted

n=1

m

The number of data items repeated (cannot be omitted)

$1 \leq m \leq 16777216(\#1000000)$

Description

The above n and m are processed as decimal numbers regardless of the current radix, unless they are marked with # to indicate use of hexadecimal.

Example

(1) When Using REPEAT

```
DBM CHANNEL 0-7
DBMAP #0
?00000000      ← Address #0 data
REPEAT 2,3
?11111111      ← Repeat this data twice.
?01011100      ← Repeat this data twice.
?11000011      ← Repeat this data twice.
?11111111      ← Address #7 data
```

In the DBM pattern specified by the REPEAT statement, field division and data omission function can be used in the same way as for regular DBM patterns.

2. 8. 5 Control of Driver Enable/Comparator Enable

Driver enable control

Control is performed by the ALPG DRE1 and DRE2. (Driver enable cannot be controlled from the DBM, so this is not suitable for data generation of I/O pin.)

Comparator-enable control

Control is performed by the ALPG CPE1 and CPE2. (Comparator enable cannot be controlled from the DBM.)

2. 8. 6 ALPG Pattern and DBM Pattern

DBM pattern entry method

As explained in [2. 8. 1 "Programming Format" on page 2-194](#), the DBM pattern can be entered in the line that contains the ALPG pattern. DBM data can be entered for each PC in the operation mode DBM(PC), in which the ALPG program counter (PC) is used as a DBM address counter.

The following three program examples all yield identical examples:

◆ DBM Data Entry after the ALPG Pattern

```
MPAT xxx
REGISTER
  IDX1 = #3FE
  :
  DBM CHANNEL 0-17
  DBMAP #10
  START #10
  NOP      /T1  ?00000000 00000000 00
  JN11 . W  /T2  ?11111111 00000000 11
  JN11 . R  /T2  ?11111111 00000000 11
  STPS
END
```

◆ Individual Entry of the ALPG Pattern and DBM Pattern

```
MPAT xxx
REGISTER
  IDX1 = #3FE

  DBM CHANNEL 0-17
  DBMAP #10
  ?00000000 00000000 00
  ?11111111 00000000 11
  ?11111111 00000000 11

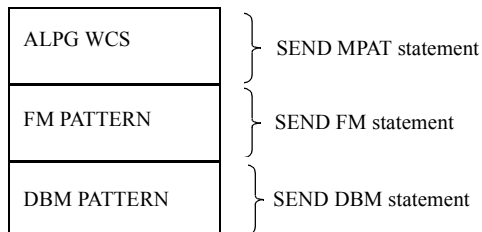
  START #10
  NOP      /T1
  JN11 . W  /T2
  JN11 . R  /T2
  STPS
END
```


◆ **Alternate Entry of the ALPG Pattern and DBM Pattern**

```
MPAT xxx
REGISTER
  IDX1 = #3FE
  :
  DBM CHANNEL 0-17
  START #10
  DBMAP #10
  NOP      /T1
  ?00000000 00000000 00
  JN11 . W  /T2
  ?11111111 00000000 11
  JN11 . R  /T2
  ?11111111 00000000 11
  STPS
END
```

Pattern transfer

ALPG, AFM, and DBM pattern data are all included in the pattern program object file.
Data is forwarded by the SEND MPAT, SEND FM, and SEND DBM statements.



Actually, all of these need not be entered together in a single pattern program. Separate pattern programs can be created for each of them.

```
PRO SAMPLE1
:
SEND MPAT PAT1
SEND FM ROMPAT1
MEAS MPAT
:
```

```
PRO SAMPLE2
:
SEND DBM DBMPAT1
SEND MPAT PAT2
MEAS MPAT
:
```

```
MPAT ROMPAT1
AFM PATTERN
BITSIZE = 8
FMA = #FF,#FF
#00
#FF!10
:
```

```
MPAT DBMPAT1

DBM CHANNEL 0-17
DBMAP #0
?000111011101111010
?001111111111111111
:
```

```
MPAT PAT1

START #0
NOP XB<0 YB<0 TP0
JN11 . XB<XB+1 W R
:
```

```
MPAT PAT2

START #0
NOP DBMAP<0
JN11 . DBMAP<DBMAP+1
:
```

The AFM/DBM pattern can be entered in both common section and module section.

The module transferred by the SEND FM statement or SEND DBM statement depends on certain conditions.

➔ For details, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

3. Test Plan Program Instructions Related to Pattern Generation

This chapter explains test plan program statements related to pattern generation.

3. 1 Instruction List

Test plan program statements related to pattern file transfer and execution appear below:

```
MEAS MPAT filename
MEAS MPAT(3) filename
MEAS MPAT(4, block-name) filename
MEAS MPAT(5) filename
REG MPAT register-name=data
RESET REG
RESET REG ALL
RESET REG MPAT
```

```
SEND MPAT filename
```

```
START MPAT
START MPAT*
START MPAT (1)
START MPAT (1)*
START MPAT (3)
START MPAT (3)*
START MPAT (4,block-name)
START MPAT (5)
STOP MPAT
RESET MPAT
```

```
SEND SCRDATA name
SEND SCRDATA name1 name2
SELECT SCRDATA ENABLE
SELECT SCRDATA DISABLE
SELECT SCRDATA NORMAL
SELECT SCRAM MODE NORMAL
SELECT SCRAM MODE DIVIDE
SELECT SCRAM MODE XDIVIDE
SELECT SCRAM MODE YDIVIDE
SELECT SCRAM MODE SEPARATE
RESET SCRDATA
```

```

SEND PRESCRAM filename
SELECT PRESCRAM NORMAL
SELECT PRESCRAM ENABLE
SELECT PRESCRAM DISABLE
SELECT PRESCRAM MODE NORMAL
SELECT PRESCRAM MODE EORZ
SELECT PRESCRAM MODE ADDZ
SELECT PRESCRAM MODE EORZ2
SELECT PRESCRAM MODE ADDZ2
SELECT PRESCRAM MODE EORZ3
SELECT PRESCRAM MODE ADDZ3
SELECT PRESCRAM MODE EORZ4
SELECT PRESCRAM MODE ADDZ4
SELECT PRESCRAM ADDRESS DISABLE
SELECT PRESCRAM ADDRESS DUT
SELECT PRESCRAM ADDRESS FM
SELECT PRESCRAM ADDRESS DUT,FM
SELECT PRESCRAM ADDRESS FP
SELECT PRESCRAM ADDRESS DUT,FP
SELECT PRESCRAM ADDRESS FM,FP
SELECT PRESCRAM ADDRESS DUT,FM,FP
SELECT PRESCRAM ADDRESS PAGE
SELECT PRESCRAM ADDRESS DUT,PAGE
SELECT PRESCRAM ADDRESS FM,PAGE
SELECT PRESCRAM ADDRESS DUT,FM,PAGE
SELECT PRESCRAM ADDRESS FP,PAGE
SELECT PRESCRAM ADDRESS DUT,FP,PAGE
SELECT PRESCRAM ADDRESS FM,FP,PAGE
SELECT PRESCRAM ADDRESS DUT,FM,FP,PAGE
SET WRAP ADDRESS = Xi-j,Ym-n
RESET PRESCRAM

SELECT ARIRAM ENABLE
SELECT ARIRAM DISABLE
SELECT ARIRAM NORMAL

SELECT PDS DATA pds data

SET PATDELAY EXP  n
SET PATDELAY X    n
SET PATDELAY Y    n
SET PATDELAY DATA n
RESET PATDELAY EXP
RESET PATDELAY X
RESET PATDELAY Y
RESET PATDELAY DATA

```

```
TIME LIMIT t:label
WAIT MPAT t:label

SELECT MPAT NORMAL
SELECT MPAT INHIBIT
SELECT MPAT REPEAT
SELECT MPAT INHIBIT, REPEAT

BURST BLOCK BEGIN(block-name)
condition
BURST BLOCK END
SET BURST(v,n)

WINDOW MODE=DISABLE
WINDOW MODE=X
WINDOW MODE=Y
WINDOW MODE=X,Y
WINDOW XADRS=a
WINDOW YADRS=a

SELECT REFRESH NORMAL
SELECT REFRESH A
SELECT REFRESH B
SELECT REFRESH DISABLE

SELECT PAUSE NORMAL
SELECT PAUSE DISABLE

REG MPAT register-name=data
```

3. 2 Functional Test Execution

The functional test is executed with a MEAS statement.

```
      :  
MEAS MPAT object filename  
      :
```

A pattern program normally consists of multiple tester patterns as shown in ["Example of Pattern Program when not Using MODULE" on page 3-6](#) and ["Example of Pattern Program when Using MODULE" on page 3-7](#). Selection of individual test patterns is executed by specifying PC using the test plan program REG MPAT statement.

```
      :  
REG MPAT PC = #10  
MEAS MPAT object filename  
      :
```

If the pattern program is divided by the MODULE statement (["Example of Pattern Program when Using MODULE" on page 3-7](#)), the value specified by the START statement described at the beginning of each module is set for PC.(["Selects the Test Pattern Program File" on page 3-8](#))

If no value is specified in the REG MPAT PC statement, the test pattern specified by the REGISTER statement in the pattern program is executed.(["Selects the Test Pattern Program File" on page 3-8](#))

The REG MPAT statement specification is valid only for the first test, as shown in ["Valid Range for the REG MPAT Statement" on page 3-9](#) (a).

To repeat execution or a test pattern for such as shmoo plotting, generate the test program shown in ["Valid Range for the REG MPAT Statement" on page 3-9](#) (b) and (c).

◆ Example of Pattern Program when not Using MODULE

```

MPAT M256K                ;sample program for 256K dynamic RAMs.

MODE MUX
REGISTER
    XMAX=#FF
    YMAX=#3FF
    IDX1=#3FFFE
SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX    ;increment base

START #0      ;checker board
IDXI2 #6      INIT
ST0:  JNII1 ST0 W      INC      FP1
ST1:  JNII1 ST1 R      INC      FP1
      JZD ST0
      STPS

START #10     ;marching
IDXI1 #6      INIT
ST2:  JNII1 ST2 W      INC
ST3:  NOP          R
      JNII1 ST3 W      INC          /D
ST4:  NOP          R          /X /Y /D
      JNII1 ST4 W      INC          /X /Y
      JZD ST2
      STPS
END

```


◆ Example of Pattern Program when Using MODULE

```

MPAT M256K                      ;sample program for 256K dynamic RAMs.

MODE MUX
REGISTER
    XMAX=#FF
    YMAX=#3FF
    IDX1=#3FFFE
SDEF INIT=XB<0 YB<0 TP<0          ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX      ;increment base

MODULE BEGIN                      ;checker board
START      #0
    IDXI1 #6      INIT
ST0:  JN11 ST0 W      INC      FP1
ST1:  JN11 ST1 R      INC      FP1
    JZD ST0
    STPS
MODULE END

MODULE BEGIN                      ;marching
START      #10
    IDXI2 #6      INIT
ST0:  JN11 ST0 W      INC
ST1:  NOP      R
    JN11 ST1 W      INC      /D
ST2:  NOP      R      /X /Y /D
    JN11 ST2 W      INC      /X /Y
    JZD ST0
    STPS
MODULE END

END

```

◆ Selects the Test Pattern Program File

```

PRO program-name
:
:
MEAS MPAT M256K
:
:
REG MPAT PC=#10
MEAS MPAT M256K
:
:
END

MPAT M256K           ;sample program for 256K dynamic RAMs.

MODE MUX
REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE
SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX   ;increment base

MODULE BEGIN              ;checker board
START #0
  IDX12 #6      INIT
ST0:  JN11 ST0 W      INC      FP1
ST1:  JN11 ST1 R      INC      FP1
      JZD ST0
      STPS
MODULE END

MODULE BEGIN              ;marching
START #10
  IDX12 #6      INIT
ST0:  JN11 ST0 W      INC
ST1:  NOP      R
      JN11 ST1 W      INC      /D
ST2:  NOP      R      /X /Y
      JN11 ST2 W      INC      /X /Y
      JZD ST0
      STPS
MODULE END

END

```

◆ Valid Range for the REG MPAT Statement

```

PRO filename

    MEAS MPAT filename
        :
        :
    REG MPAT PC=#10      ---- (1)
    MEAS MPAT filename
        :
        :
    MEAS MPAT filename    ; (1) REG MPAT statement is invalid

END

(a)

PRO filename
:
REG MPAT PC=#10      ---- (2)
DO LP01 I=3V, 5V, 0.2V
    VS1=I
    MEAS MPAT(3) filename
        :
        :
LP01:    CONT
        :
        :
    MEAS MPAT filename ;REG MPAT statement set in (2) is valid.
        :
        :
END

(b) MEAS MPAT(3)

PRO filename
:
DO LP01 I=3V, 5V, 0.2V
    VS1=I
    REG MPAT PC=#10    ;Code REG MPAT statement in the loop.
    MEAS MPAT filename
        :
        :
LP01:    CONT
:
:
END

(c) Shmoo

```

3. 3 Register Setting and Modification

When the same pattern program is used as a test pattern for a memory with a different capacity, or when the test is performed in a different test order, the registers for the test are modified with the test plan program REG MPAT statement.

```
      :  
REG MPAT register-name = data  
MEAS MPAT filename  
      :
```

If a value is specified in the REG MPAT statement, the value specified by the REGISTER statement in the pattern program becomes invalid.(["Valid Range for the REG MPAT Statement" on page 3-9](#))

The data specified in the REG MPAT statement is valid for only the first MEAS statement; the value specified in the pattern program is valid for subsequent MEAS statements.(["Valid Range for the REG MPAT Statement" on page 3-9](#))

To repeat execution or a test pattern for such as shmoo plotting, generate the test program shown in ["Valid Range for the REG MPAT Statement" on page 3-9](#) (b) and (c).

3. 4 Failure Analysis Memory (not Available on the T5336 and T5311)

To reduce the normal test time, test pattern generation by the ALPG is terminated if a failure is detected.

To execute the test pattern to the end and store the failures in the AFM, inhibit test pattern generation termination by using the following statement.

```
      :  
SELECT MPAT INHIBIT  
MEAS MPAT filename  
      :
```

The INHIBIT mode specified by this statement is effective until it is disabled by the following statement.

```
SELECT MPAT NORMAL
```

Determining whether GO or NOGO is valid by using the following statement in this INHIBIT mode.

```
IF PASS ~  
IF FAIL ~
```

◆ Example of SELECT MPAT INHIBIT

```

PRO filename

TEST    1          ; functional test
        :
        MEAS MPAT filename
        :
TEST    2          ;fail data display

        RESET AFM ALL
        AFM MULTIMODE=SINGLE
        AFM DUTSEL=1
        AFM ADDRESS=X0-7, Y0-9
        AFM RATE=LS
        AFM BITSIZE=1
        AFM ACTION=FT>FMX

        SELECT MPAT INHIBIT ; Inhibits termination of pattern generation, even if
                               fail occurs.

        MEAS MPAT filename
        IF FAIL THEN ENTRY
            CALL FBMAP(...)
        EXIT

TEST    3          ; functional test
        :
        SELECT MPAT NORMAL ; The mode is disabled.
        MEAS MPAT filename
        :
        :

END

```

3. 5 Shmoo Test

The T5300 series memory test system obtains a shmoo plot by either of the following two methods:

- Specify MANUAL and HOLD in the MEAS MPAT statement for obtaining a shmoo plot, and call the subroutine (SHM2 or SHM3) for shmoo plotting
- Include a CALL statement that calls the subroutine for shmoo plotting in the test plan program

This section explains how to include the CALL statement that calls the subroutine for shmoo plotting in the test plan program.

Sample programs are shown in ["Example of Shmoo Test Program" on page 3-14](#).

The subroutines for shmoo plotting include subroutines for two- and three-dimensional shmoo plotting.

The subroutines for shmoo plotting are called by the following statements:

CALL SHM2 ("XXX, n") : Two-dimensional shmoo plot

CALL SHM3 ("XXX, n") : Three-dimensional shmoo plot

XXX indicates the name of the file in which the conditions for the X, Y, and Z axes for shmoo plotting are stored. This file must be created with the controller commands before using this statement, as follows:

```
## TEST.┐
:
/SHM2.┐ (/SHM3.┐) ; Shmoo plot execution
:
Enter conditions for X, Y, and Z axes with the CHANGE command.
:
Write the set conditions into file XXX with the WRITE command.
:
```

n specifies the testing mode.

➔ For details, refer to [T5500 Series Tester Utility Program Reference Manual\(No. : 8256295\)](#).

The test pattern transferred before calling shmoo plot is executed during shmoo plotting.

If conditions are set by the pattern program REG MPAT statement, the REG MPAT statement must be written before the CALL statement.

◆ Example of Shmoo Test Program

```
PRO Sample

TIME1=1MS :VS1
TIME2=1MS :IN

IN1=2.4V, 0.8V
OUT1=2.4V, 0.4V

RATE=270NS
ACLK1=0NS
:
:

PD1-8=IN1, XOR, ACLK1, BCLK1, CCLK1, SDM,<X0-7, Y0-7>
:
:
PC33=OUT1, STRB1, <D0>

SEND MPAT filename
REG MPAT PC=#10

CALL SHM2("xxxx, 1")
:
:
END
```


3. 6 Scramble Program Transfer

The T5300 series memory test system has an address descrambler to compensate for DUT address topology and decoder topology.

The address data for X and Y generated by the ALPG is converted by the address descrambler and applied to the DUT.

The conversion table stored in the address descrambler is generated by the scrambler program.

➔ For details, refer to [ATL-51 Scramble Program Reference Manual\(No. : 8256294\)](#).

The following instructions transfer the data in the conversion table generated by the scramble program to the address descrambler.

SEND SCRDATA statement

Specify this statement in the test plan program to transfer the data specified by the file name to the address descrambler.

SCRAM statement

Specify this statement in the pattern program to transfer the data specified by the file name to the address descrambler.

! Important

If file names are specified in both the test plan program and pattern program, only the file name specified in the test plan program is valid.

Independently of the above data transfer, the following statements determine whether address conversion is performed by the address descrambler.

SELECT SCRDATA NORMAL

The following statements perform address conversion using the data transferred to the address descrambler:

SEND SCRDATA statement (test plan program)

SCRAM statement (pattern program)

SELECT SCRDATA DISABLE

Address conversion by the address descrambler is unconditionally disabled.

SELECT SCRDATA ENABLE

The address conversion disabled by SELECT SCRDATA DISABLE is enabled.

If the above statements are not specified, SELECT SCRDATA NORMAL is the default.

The following example shows how to specify a scramble program in a pattern program:

```
MPAT M256K

    MODE MUX
    REGISTER
        XMAX=#FF

    MODULE BEGIN
    START    #0
    IDXI2    #6
        :
    STPS
    MODULE END

    MODULE BEGIN
    START    #10
    IDXI2    #6
        :
    STPS
    MODULE END

    MODULE BEGIN
    SCRAM S256K2      Scramble program name specification
    START    #20
    IDXI2    #2
        :
    STPS
    MODULE END

END
```

If different conversion data is stored in the address descrambler in each pattern program, a file name can be specified by the statement shown in the example.

SCRAM statement

The following example shows how to specify a scramble program in a test plan program:

```

PRO P256K                      ;Sample program for 256K dynamic RAMs.

    TIME=1MS:VS1
    :
    VS1=5V
    :
    PD1-8=IN1, XOR, ACLK1, BCLK1, CCLK1, SDM,<X0-7, Y0-7> ;A0-7
    :
    PC33=OUT1, STRB1,<D0>                      ;DOUT
    :
    RATE=260NS
    :

TEST 1
    REG MPAT PC=#0
    MEAS MPAT M256K

TEST 2
    SEND SCRDATA S256K1*1
    REG MPAT PC=#10
    MEAS MPAT M256K

TEST 3
    SELECT SCRDATA DISABLE*2
    REG MPAT PC=#10
    MEAS MPAT M256K

TEST 4
    SELECT SCRDATA ENABLE*3
    REG MPAT PC=#10
    MEAS MPAT M256K

TEST 5
    REG MPAT PC=#20
    MEAS MPAT M256K

END

```

**1 No SCRAM statement is specified in the pattern program; the test is performed by transferring the address descrambler data with the SEND SCRDATA statement.*

**2 To disable address conversion with the address descrambler after transferring the address descrambler data with SEND SCRDATA statement, use the following statement:*

```
SELECT SCRDATA DISABLE
```

**3 To release the address conversion disabled by SELECT SCRDATA DISABLE, use the following statement:*

```
SELECT SCRDATA ENABLE
```

3. 7 Window Address Specification

The T5300 series memory test system has a window function which performs logical comparison in a specific address range.

The following statement specifies an address range:

WINDOW XADRS = a
WINDOW YADRS = b

Address specification
a, b = #0 - #3FFFF (T5377, T5376 and T5375)
#0 - #FFFF (T5371, T5336 and T5311)

WINDOW MODE = X
WINDOW MODE = Y
WINDOW MODE = X, Y

Operation mode specification

WINDOW MODE = DISABLE →Operation mode release

Data output from the DUT and the expected value pattern are logically compared in the cycle during which the control bits set in CPE1 to CPE2 are specified.

If the window mode is specified, the above logical comparison is performed only in the following cycle:

WINDOW MODE	Condition for valid logical comparison	
	T5377, T5376 and T5375	T5371, T5336 and T5311
X	X0 -17 = XA0 -17	X0 -15 = XA0 -15
Y	Y0 -17 = YA0 -17	Y0 -15 = YA0 -15
X, Y	(X0-17 = XA0-17) .AND. (Y0-17 = YA0-17)	(X0-15 = XA0-15) .AND. (Y0-15 = YA0-15)

XA0-17 and YA0-17 (XA0-15, YA0-15) are addresses which are set by the following statements:

WINDOW XADRS = x
WINDOW YADRS = y

➔ For details on the WINDOW statement, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

A test plan program using the window function appears below:

```

PRO P16K           ;Sample program for 16K static RAMs.

    TIME1=1MS :VS1                ;time sequence
    TIME2=1MS :IN

    VS1=5V                    ;power
    IN1=2.2V, 0.8V            ;driver level
    OUT1=2.4V, 0.4V           ;reference level

    PD1-7=IN1, NRZB, BCLK1, <X0-6>      ;A0-6
    PD8-14=IN1, NRZB, BCLK1, <Y0-6>     ;A7-13
    PD15=IN1, /NRZA, ACLK1, <C0>        ;/CS
    PD16=IN1, /RZO, BCLK2, CCLK2, <WT>   ;/WE
    PD34=IN1, XOR, ACLK3, BCLK3, CCLK3, <D0>;Din
    PC33=OUT1, STRB1, <D0>              ;Dout

    RATE=70NS
    ACLK1=0NS                    ;/CS
    BCLK1=0NS                    ;An
    BCLK2=10NS                   ;/WE leading
    CCLK2=55NS                   ;/WE trailing
    ACLK3=0NS                    ;Din
    BCLK3=10NS                   ;Din leading
    CCLK3=55NS                   ;Din trailing
    STRB1=70NS                   ;Dout

    CALL CALB("C16K", NORMAL)        ;calibration

TEST  1

    REG MPAT PC=#0
    MEAS MPAT M16K

TEST  2

    WINDOW MODE=X *1
    WINDOW XADRS=#1F *2

    REG MPAT PC=#10
    MEAS MPAT M16K

    WINDOW MODE=DISABLE *3

END

```

**1 Operation mode setting*

In the example above, logical comparison is made only when address X applied to the DUT matches the address (#1F) set by WINDOW XADRS.

**2 Address specification*

**3 Operation mode release*

The operation mode is valid until this instruction is executed.

4. Programming I

This chapter explains programming methods for power supply current measurement, voltage measurement, DRAM testing, bump testing, and partial testing.

4. 1 Power Supply Current Measurement

The current measurement function in the bias power supply measures the average power supply current. The current is measured by executing the pattern program as shown in ["Example of a Power Supply Current Measurement Program" on page 4-2](#), and by using the MEAS statement after making the device under test operational.

Pattern program

Use an endlessly looping program for the pattern program.

Since pattern generation is stopped if the logical comparison result is a failure, inhibit pattern generation termination during power supply current measurement, as follows: (["Example of a Power Supply Current Measurement Program" on page 4-2](#))

- Mask logical comparison by disabling CPE1 and CPE2.
Select "FIXL" (write FL in the test plan program) for CPE1 and CPE2 in the pattern program or test plan program.
- Inhibit termination if a failure occurs in the test plan program.
Inhibit termination if a failure occurs, by specifying the following statement in the test plan program:

```
SELECT MPAT INHIBIT
```

Once this statement is specified, it inhibits termination until the following statement is executed. Therefore, if a functional test or other tests are to be performed next, clear the mode by specifying the following statement:

```
SELECT MPAT NORMAL
```

◆ Example of a Power Supply Current Measurement Program

```

MPAT filename

    MODE MUX
    REGISTER
        CPE1=FIXL *1
        CPE2=FIXL *1

        :
        :

    MODULE BEGIN
    REGISTER
        CPE1=FIXL
        CPE2=FIXL
        :
        : Power supply current measurement ;Creates Endless Loop

    MODULE END
        :
        :
END

PRO filename
    :
    :

    SEND MPAT filename
    REG MPAT PC=#10
    SELECT MPAT INHIBIT
    SRON
    START MPAT*
    WAIT TIME t
    MEAS VS1 ;IDD
    STOP MPAT
        :
        :
    SELECT MPAT NORMAL
END

```

**1 If specified by the common section, these statements are valid for all modules.*

Test plan program

The power supply current is measured by the current measurement function in the bias power supply after pattern generation is started, by performing reference-on for the bias power supply and driver voltage supply as shown below.

```

PRO filename

    IN1=2.4V, 0.8V
    :
    RATE=260NS
    ACLK1=0NS
    :
    :
    VS1=5V, R5V, M800MA, 100MA, -100MA *1
    SEND MPAT filename *2
    TIME1=t1:VS1
    TIME2=t2:IN
    SELECT MPAT INHIBIT
    REG MPAT PC=#10
    SRON *3
    START MPAT * *4
    WAIT TIME t3 *5
    MEAS VS1 *6
    STOP MPAT *7

END

```

*1 Bias power supply condition setting

*2 SEND MPAT filename

Transfer the pattern program to the ALPG.

If a pattern program with the same file name has already been transferred to the ALPG, the pattern program is not transferred.

*3 Reference-on

Carry out reference-on for the bias power and driver voltage supply.

The order for turning on the powers is specified by the TIME statement.

May be omitted. If the SRON statement is omitted, the next START MPAT* statement performs reference-on.

4 START MPAT

Start pattern generation and proceed to the next statement.

When pattern generation is performed with this instruction, it must be terminated by specifying the following statement before starting the next pattern program transfer and pattern generation:

STOP MPAT

The register data in pattern program is modified with the REG statement.

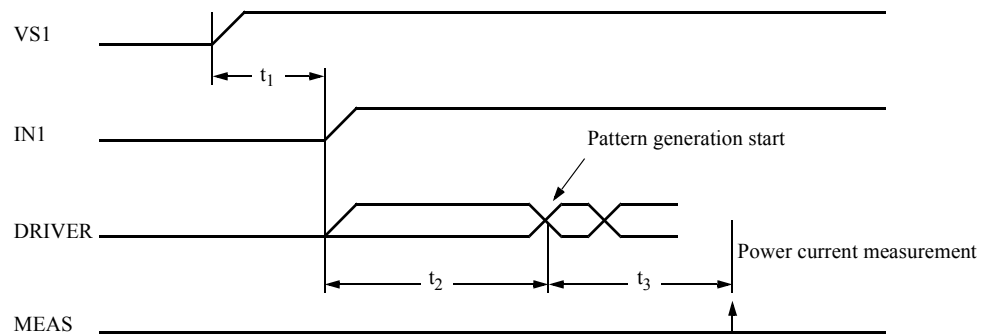
The data specified by the REG statement is valid only for the first START MPAT* after the specification.

4. 1 Power Supply Current Measurement

***5 WAIT TIME t**

This statement adjusts the time from application of the pattern to the DUT until the power supply current measurement.

The times mentioned above appear below:

***6 MEAS VSI**

Current is measured by the current measurement function in the bias power supply.

***7 STOP MPAT**

Terminates pattern generation.

Clears the REG MPAT statement specification.

Use this statement with START MPAT.*

4. 2 Output Voltage Measurement (When Output Voltage is Maintained)

When the output voltage from the DUT is maintained, as in a static memory, the voltage is measured using the procedure below. Measurement is performed by the DC parametric test unit ISVM function after pattern generation for DUT initialization.

Pattern program

The pattern program is created so that the read data is held after data (0 or 1) to be measured is written into the DUT.

This program is created by the following three methods:

1. Form a loop in the read cycle.

If clock pulses are required to maintain the output voltage, the pattern program is created so that the read cycle forms an endless loop as shown in ["Example of VO Measurement Pattern Program \(Endless Loop\)" on page 4-6](#).

2. Stop pattern generation with the STPS instruction.

Stop pattern generation with the STPS instruction.

➔ Refer to ["Example of VO Measurement Pattern Program \(Stop by STPS\)" on page 4-6](#).

Create the program so that the STPS instruction cycle is a read cycle.

When pattern generation is stopped by the STPS instruction, two read cycles are applied to the DUT.

3. Stop pattern generation with the WAIT instruction.

Stop pattern generation with the WAIT instruction.

➔ Refer to ["Example of VO Measurement Pattern Program Using the WAIT Instruction" on page 4-7](#).

Create the program so that the WAIT instruction cycle is a read cycle.

The WAIT instruction temporarily stops pattern generation. It can be restarted by specifying the following instruction:

```

:
START MPAT (1)
:
```

4. 2 Output Voltage Measurement (When Output Voltage is Maintained)

◆ Example of VO Measurement Pattern Program (Endless Loop)

```

MPAT  filename

REGISTER
      XMAX=#7F
      XMAX=#F

START  #0
NOP                    XB<0 YB<0  TP<0
NOP                    W      X<XB Y<YB
ST0:   JMP            ST0      X<XB Y<YB

END

```

◆ Example of VO Measurement Pattern Program (Stop by STPS)

```

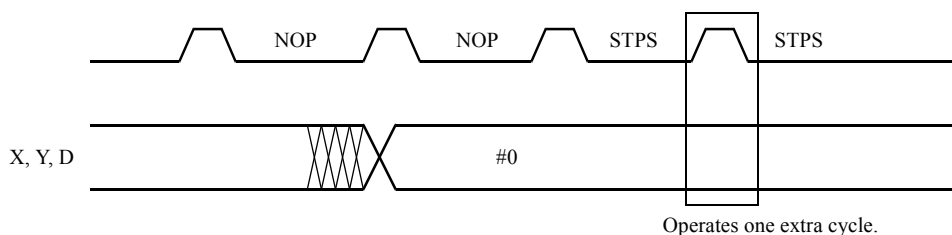
MPAT  filename

REGISTER
      XMAX=#7F
      XMAX=#F

START  #0
NOP                    XB<0 YB<0  TP<0
NOP                    W      X<XB Y<YB
STPS                                ;Data is applied to DUT during two cycles.

END

```



4. 2 Output Voltage Measurement (When Output Voltage is Maintained)

◆ Example of VO Measurement Pattern Program Using the WAIT Instruction

```

MPAT  filename

    REGISTER
        XMAX=#7F
        XMAX=#F

    START  #0
    NOP                    XB<0 YB<0  TP<0
    NOP      W            X<XB Y<YB          ;write 0
    WAIT     X<XB Y<YB          ;read 0
    NOP      W            X<XB Y<YB  /D      ;write 1
    WAIT     X<XB Y<YB          ;read 1
    STPS

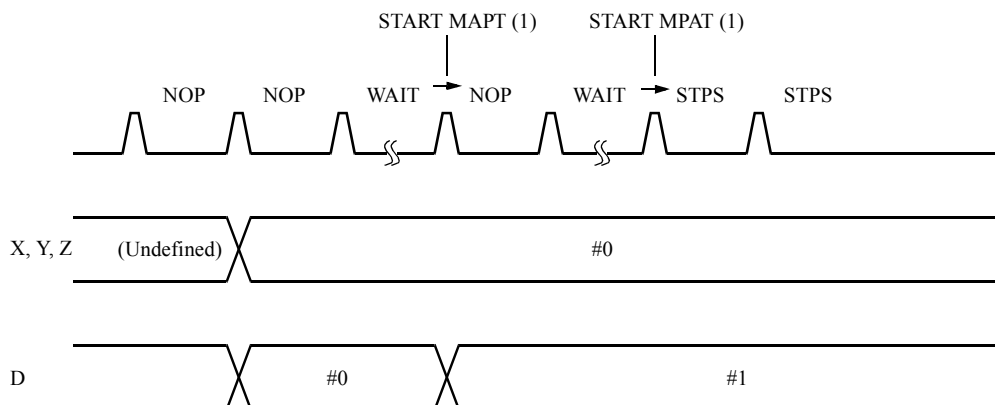
END

PRO filename

    SEND MPAT filename
    :
    :
    START MPAT
    :
    VOL measurement
    :
    START MPAT (1)
    :
    VOH measurement
    :
    STOP MPAT

END

```



4. 2 Output Voltage Measurement (When Output Voltage is Maintained)

Test plan program

"Example of VO Measurement Test Plan Program" on page 4-8 shows an example of a test plan program. Create a program as follows:

To measure output voltage, use DC ISVM after performing reference-on for the bias power supply and driver voltage supply and starting pattern generation.

◆ Example of VO Measurement Test Plan Program

```

PRO program-name          ; 8 bits I/O

    TIME1=1MS:VS1          *1
    TIME2=1MS:IN, DC
    SELECT DCOFF

    IN1=2.4V, 0.8V
    VS1=5V
    :
    RATE=200NS:16
    :
    P1-7=IN1, XOR, ACLK1, BCLK1, <X0-6>
    P8-11=IN1, XOR, ACLK1, BCLK1, <Y0-3>
    :
    P33-40=IN1, OUT1, IOC,.....,<D0-7>
    :
    SEND MPAT filename
    SELECT MPAT INHIBIT
    :
    ISVM=4MA, R8MA, M8V, 6V, -1V
    SETTLING TIME=1MS, 1MS
    SRON

    REG MPAT PC=#0
    START MPAT *
    WAIT TIME 1MS

    MEAS DC(PC33-40)

    STOP MPAT

    ISVM=-1MA, R8MA, M8V, 6V, -1V
    SRON

    REG MPAT PC=#10
    START MPAT *
    WAIT TIME 1MS

    MEAS DC (PC33-40)

    STOP MPAT
    :
    :
END

```

*1 Set reference-on time sequence.

➔ Refer to [Figure 4-1](#).

4. 2 Output Voltage Measurement (When Output Voltage is Maintained)

**2 Set test conditions.*

Set conditions for writing data into the DUT. These conditions must be the same as those for the functional test. During the output voltage measurement, inhibit pattern generation termination when a failure is detected, without performing comparison with the comparator.

**3 Set the ISVM mode.*

To measure V_{OL} , set I_{OL} to 4 mA. (This value depends on the DUT used.)

**4 Set the time from application of current to the DUT until measurement.*

This depends on the current value and range.

➔ Refer to [Figure 4-1](#).

**5 Perform reference-on.*

➔ Refer to [Figure 4-1](#).

May be omitted. (If the SRON statement is omitted, reference-on is performed by the next START MPAT statement.)*

**6 Start the ALPG and wait until V_{OH} or V_{OL} has been valid.*

This depends on the ALPG pattern program type.

- *When forming a looping pattern in the read cycle:*

```
REG MPAT PC = #0
START MPAT *           ; PG start
WAIT TIME 1MS          ; Wait until  $V_{OH}$  and  $V_{OL}$  are certain.
MEAS DC(PC33 - 40)     ; Measurement
STOP MPAT              ; Stop PG
```

- *For a pattern to stop STPS*

```
REG MPAT PC = #0
START MPAT
MEAS DC(PC33 - 40)
```

- *For a pattern to stop WAIT*

```
REG MPAT PC = #0
START MPAT
MEAS DC(PC33 - 40)
STOP MPAT              ; Necessary when restart does not occur
:
START MPAT(1)          ; When restart occurs
MEAS DC(PC33 - 40)
STOP MPAT
```

To measure V_{OH} , V_{OL} continuously, stop pattern generation with WAIT, and restart it with START MPAT(1).

4. 3 DRAM Test

4. 3. 1 Address Multiplex

"[Example of an Address Multiplex Pattern Program](#)" on page 4-12 shows an example of pattern program used to test a DUT with multiplex address inputs. "[Example of an Address Multiplex Test Plan Program](#)" on page 4-15 shows an example of a test plan program.

Pattern program

◆ Example of an Address Multiplex Pattern Program

```

MPAT M256K

MODE MUX          ;sample program for 256K dynamic RAMs.  ... (1)

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

  SDEF INIT=XB<0 YB<0 TP<0          ;clear
  SDEF INC=XB<XB+1 YB<YB+1^BX      ;increment base

  MODULE BEGIN                      ;checker board
  START      #0

  IDX12 #6      INIT
ST0:  JN11 ST0 W      INC      FP1
ST1:  JN11 ST1 R      INC      FP1
      JZD ST0
      STPS

  MODULE END

  MODULE BEGIN                      ;marching
  START      #10

  IDX12 #6      INIT
ST0:  JN11 ST0 W      INC
ST1:  NOP      R
      JN11 ST1 W      INC      /D
ST2:  NOP      R      /X /Y /D
      JN11 ST2 W      INC      /X /Y
      JZD ST0
      STPS

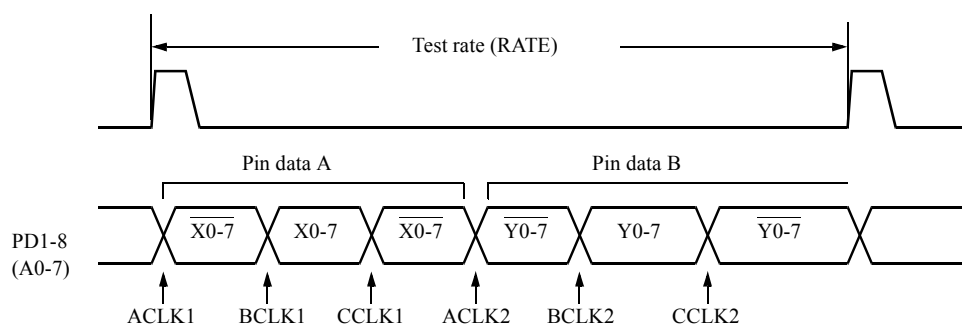
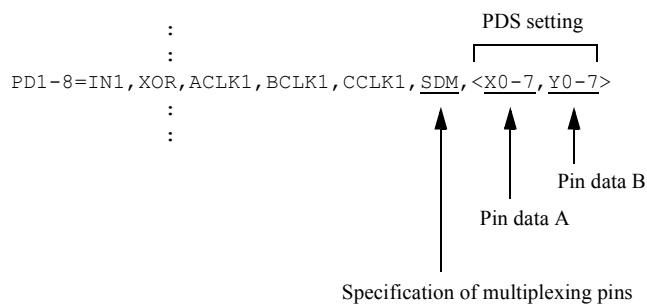
  MODULE END

END

```

When the DUT which address inputs are multiplex type is tested, the multiplex mode is specified in the pattern program. ((1), above)

When the multiplex mode is specified, the following data is applied to the pin specified in the test plan program as follows:



When the multiplex mode is specified in the pattern program, the cycle in which control bits M1 and M2 are specified operates as follows.

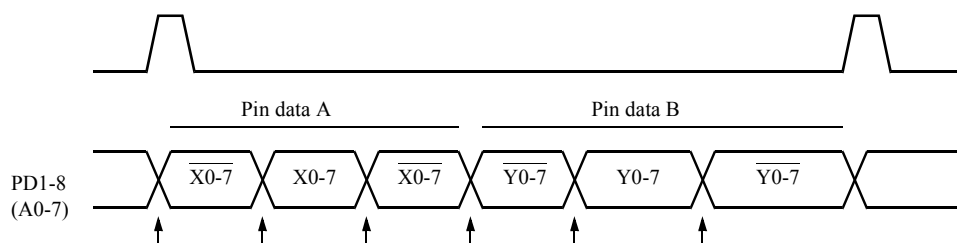
This mode is used for the refresh test and page mode test to be explained later:

```

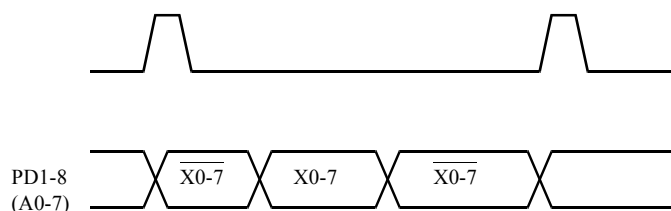
:
PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7>
:

```

- When neither M1 nor M2 is specified



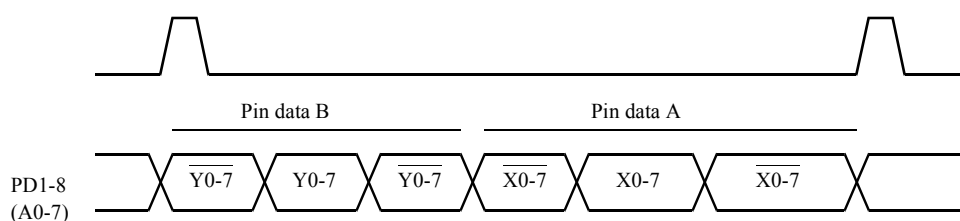
- M1 (for RAS only refresh)



Tip

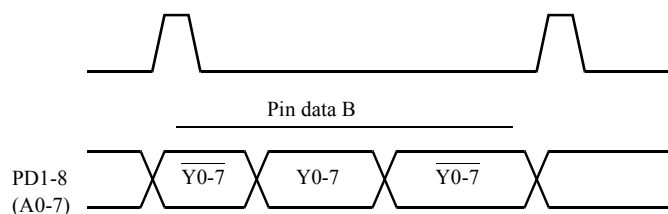
Prohibits output of $ACLK2$, $BCLK2$, and $CCLK2$ clocks, which are used for outputting pin data B(Y0-7). Switch the timing set and use clock OPEN for prohibiting output.

- M2



Pin data A and pin data B are switched with each other.

- M1, M2 (for page cycle generation)



Tip

Prohibits output of $ACLK1$, $BCLK1$, and $CCLK1$ clocks, which are used for outputting pin data A(Y0-7) in this cycle. Switch the timing set and use clock OPEN for prohibiting output.

Test plan program

◆ Example of an Address Multiplex Test Plan Program

```

PRO P256K ;Sample program from 256K dynamic RAMs.
TIME1=1MS :VS1 *1
TIME2=1MS :IN ]

VS1=5V
IN1=2.4V,0.8V *2
OUT1=2.4V,0.4V ]

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 *4 *5 ;/RAS *3
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK1,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

TR=1NS & TF=1NS ;Timing set
RATE=260NS
ACLK1=0NS
BCLK1=10NS
CCLK1=10NS+15NS+TF+TR
ACLK2=35NS
BCLK2=45NS
CCLK2=45NS+25NS+TF+TR
ACLK3=0NS
BCLK3=10NS
CCLK3=10NS+TF+150NS
BCLK4=45NS
CCLK4=45NS+TF+75NS
BCLK5=45NS
CCLK5=45NS+TF+25NS
BCLK6=45NS
CCLK6=45NS+TF+25NS+TR & STRB1=45NS+TF+75NS *6

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10(RESET) @
TCHANGE PD11(RESET)? *7
CALL CALB("C256K",TCAL)

TEST 1

REG MPAT PC=#0 *8
MEAS MPAT M256K *9

TEST 2

REG MPAT PC=#10
MEAS MPAT M256K

END

```

*1 Power-up sequence setting

*2 Power supply (VS1), driver level (IN1), and comparison voltage (OUT1) settings

*3 Pin data setting

*4 Multiplex pin setting

- *5 Pin data setting (PDS)
- *6 Timing setting
- *7 Timing calibration
- *8 Select a test pattern with the REG statement.
- *9 Test

4. 3. 2 Pause Test

A pattern program for the pause test is shown below:

◆ Example of a Pause Test Pattern Program

```
MPAT M256K                ;sample program for 256K dynamic RAMs.

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX    ;increment base

MODULE BEGIN                  ;PAUSE

MODE PAUSE *1

REGISTER
  TIMER=4MS *2

START      #20

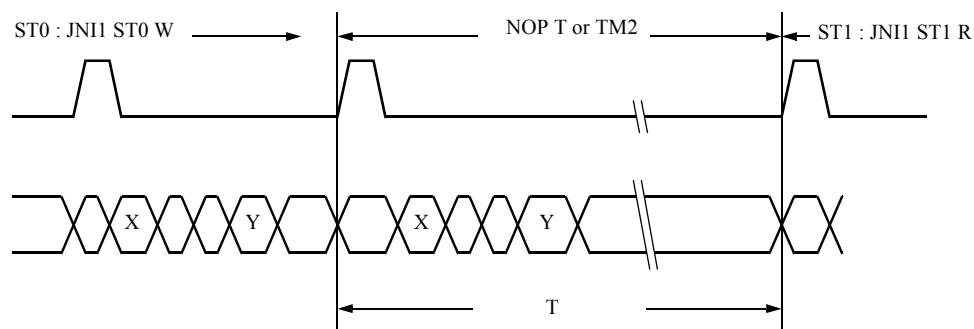
  IDXI2 #6      INIT
ST0:  JN11 ST0 W      INC
  NOP      T *3      ;PAUSE
ST1:  JN11 ST1 R      INC
  JZD      ST0
  STPS

MODULE END

END
```

*1 Set the pause mode.

*2 Timer data setting

 $TIMER = 100\text{ ns to }409.5\text{ s}$
 $TPAUSE = 100\text{ ns to }409.5\text{ s}$
*3 Pattern generation is stopped during the following period in the cycle in which control bit *T* or *TM2* is specified.

$$T = \text{setting value} + 2 * \text{RATE} + \text{error}$$

See the table below.

Period of the cycle in which *T* or *TM2* is specified.

Stop time errors for pattern generation appear in the following table:

Test system	AFM ACTION statement	AFM RATE statement	Stop time errors for pattern generation
T5377	No specification of the mask mode	HS	0.9 μs to 1.2 μs
		MS	1.1 μs to 1.4 μs
		LS	1.5 μs to 1.8 μs
	With the specification of the mask mode	HS	1.3 μs to 1.6 μs
		MS	2.1 μs to 2.4 μs
		LS	3.6 μs to 3.9 μs
T5376 and T5375	---	HS, MS	0.9 μs to 1.2 μs
		LS	1.1 μs to 1.4 μs
T5371	---	---	0.8 μs to 1.2 μs
T5336 and T5311	---	---	0.6 μs to 0.9 μs

--- in the table shows that the relevant test systems are independent of the specification by the AFM ACTION statement and the AFM RATE statement.

4. 3. 3 Refresh Test

◆ Auto Refresh Test Mode Specification

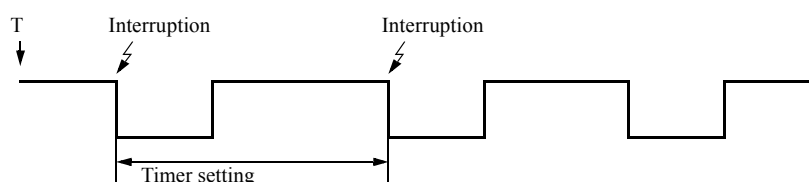
The refresh test can be performed with the internal timer and interrupt mechanism.

In the pattern program in which the refresh mode is specified, the internal timer is started in the cycle during which control bit T or TM1 is specified.

There are two timer operation modes:

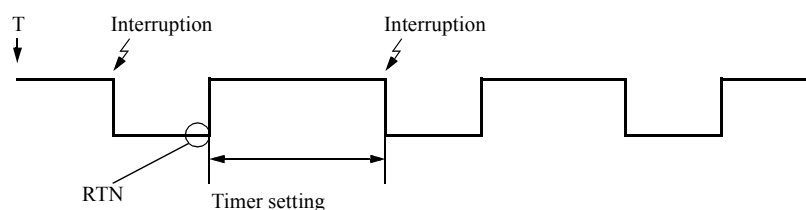
- REFRESHA n

When the timer operation is started with "T" or "TM1", an internal interrupt occurs during the time interval specified in the internal timer.



- REFRESHB n

When the timer operation is started with "T" or "TM1", an internal interrupt occurs after the time specified in the internal timer, and the internal timer stops. The internal timer is restarted by specifying RTN in the interrupt processing program.



When generating an internal interruption, ALPG performs the operations shown in [Figure 4-2](#) and [Figure 4-3](#). Refresh address generation for refresh testing is performed by the RF (refresh counter).

The refresh counter is incremented by the following instruction:

```
RF < RF + 1
```

The refresh counter is cleared at the beginning of the test.

The refresh address stored in the refresh counter is output to X0 to X17 (X0 to X15 for the T5371, T5336 and T5311) with the X < RF instruction:

```
MODE REFRESHA n
:
X<RF
:
```

n specifies the bit which outputs the contents of the refresh counter to address X.

Table 4-1 Refresh Counter Output Bits (T5377, T5376 and T5375)

n	X0 to X17	Y0 to Y17
64	(RF0-5)→ X0-5, (X6-17)t-1→ X6-17	(Y0-17)t-1
128	(RF0-6)→ X0-6, (X7-17)t-1→ X7-17	(Y0-17)t-1
256	(RF0-7)→ X0-7, (X8-17)t-1→ X8-17	(Y0-17)t-1
512	(RF0-8)→ X0-8, (X9-17)t-1→ X9-17	(Y0-17)t-1
1024	(RF0-9)→ X0-9, (X10-17)t-1→ X10-17	(Y0-17)t-1
2048	(RF0-10)→ X0-10, (X11-17)t-1→ X11-17	(Y0-17)t-1
4096	(RF0-11)→ X0-11, (X12-17)t-1→ X12-17	(Y0-17)t-1
8192	(RF0-12)→ X0-12, (X13-17)t-1→ X13-17	(Y0-17)t-1
16384	(RF0-13)→ X0-13, (X14-17)t-1→ X14-17	(Y0-17)t-1
32768	(RF0-14)→ X0-14, (X15-17)t-1→ X15-17	(Y0-17)t-1
65536	(RF0-17)→ X0-17, (X16-17)t-1→ X16-17	(Y0-17)t-1
131072	(RF0-17)→ X0-17, (X17)t-1→ X17	(Y0-17)t-1
262144	(RF0-17)→ X0-17	(Y0-17)t-1

(Xn-17) t-1 and (Y0-17) t-1 are address data output to X and Y in the previous cycle.

Table 4-2 Refresh Counter Output Bits (T5371, T5336 and T5311)

n	X0 to X15	Y0 to Y15
64	(RF0-5)→ X0-5, (X6-15)t-1→ X6-15	(Y0-15)t-1
128	(RF0-6)→ X0-6, (X7-15)t-1→ X7-15	(Y0-15)t-1
256	(RF0-7)→ X0-7, (X8-15)t-1→ X8-15	(Y0-15)t-1
512	(RF0-8)→ X0-8, (X9-15)t-1→ X9-15	(Y0-15)t-1
1024	(RF0-9)→ X0-9, (X10-15)t-1→ X10-15	(Y0-15)t-1
2048	(RF0-10)→ X0-10, (X11-15)t-1→ X11-15	(Y0-15)t-1
4096	(RF0-11)→ X0-11, (X12-15)t-1→ X12-15	(Y0-15)t-1
8192	(RF0-12)→ X0-12, (X13-15)t-1→ X13-15	(Y0-15)t-1
16384	(RF0-13)→ X0-13, (X14-15)t-1→ X14-15	(Y0-15)t-1
32768	(RF0-14)→ X0-14, (X15)t-1→ X15	(Y0-15)t-1
65536	(RF0-15)→ X0-15	(Y0-15)t-1

(Xn-15) t-1 and (Y0-15) t-1 are address data output to X and Y in the previous cycle.

Tip

- To use the REFRESHB mode and burst by the OUT instruction at the same time, the timer restart instruction (NOP T) is required after the OUT instruction.
- To use the REFRESHB and PAUSE modes at the same time, the REFRESH timer restart instruction (NOP TM1) is required after the PAUSE timer is started.
- To use the REFRESHA or REFRESHB mode on the T537I, set the operation period (RATE) to 21 ns or more in every cycle.
- Refresh counter n can also be specified using the RLMAX register.
The following two specifications have the same results:

◆ Example

```
MPAT SAMPLE1
  MODE REFRESHA n
  :
END
```

```
MPAT SAMPLE2
  MODE REFRESHA
  REGISTER
  RLMAX=n-1
  :
END
```

Interrupt mask by the control bit "I"

• MODE REFRESHA n

When an interrupt occurs in a cycle where control bit "I" is set, the interrupt is processed (PC<ISP) in the next cycle in which control bits are not set. ((2) in [Figure 4-2](#))

The timer restarts at the same time as the interrupt whether or not the bit "I" was set. ((4) in [Figure 4-2](#))

As a result, intervals between interrupts depends on how many control bits are set.

The time from interrupt processing (1) to (2) in [Figure 4-2](#) is longer than t_2 (timer setting value + setting error) and the time from interrupt processing (2) to (3) is shorter than t_2 (timer setting value + setting error). Calculate the error caused by setting the control bit as shown below.

The maximum time from interruption processing to interruption processing =
 $\{\text{maximum time of } t_2\} + \{(\text{maximum execution number of "I"}) * (\text{maximum RATE})\}$

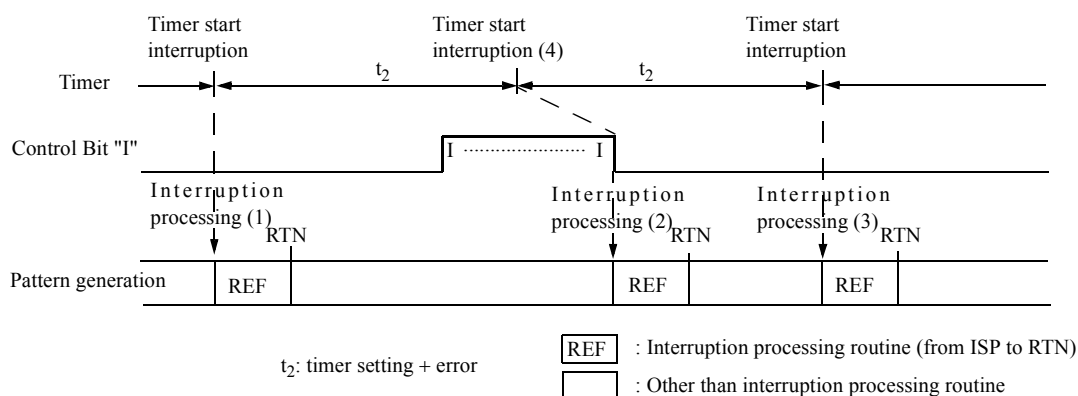
The minimum time from interruption processing to interruption processing =
 $\{\text{minimum time of } t_2\} + \{(\text{maximum execution number of "I"}) * (\text{maximum RATE})\}$

"Maximum RATE" is the longest RATE used for one test. For information on t_2 maximum time/minimum time, see t_2 in Figure 4-4. The time from control bit "T (TM1)" to interruption processing is similar.

The interrupt processing routine cannot be executed correctly if an interrupt occurs in this routine. You can avoid this by doing the following:

1. Set the timer and RATE so that the minimum time between an interrupt to an interrupt is $> (\text{the number of patterns in an interrupt processing routine}) * (\text{maximum RATE})$.
2. If (1) cannot be satisfied, set the control bit "I" for all interrupt processing routines.

Figure 4-2 MODE REFRESH A n by Control Bit "I"



• MODE REFRESH B n

When an interrupt occurs in a cycle where control bit "I" is set, the interrupt is processed (PC<ISP) in the next cycle in which control bits are not set. ((6) in Figure 4-3)

As a result, intervals from RTN to interruptions depends on how many control bits are set.

The time from RTN (5) to interrupt processing (6) shown in Figure 4-3 is longer than t_3 (timer setting value + setting error).

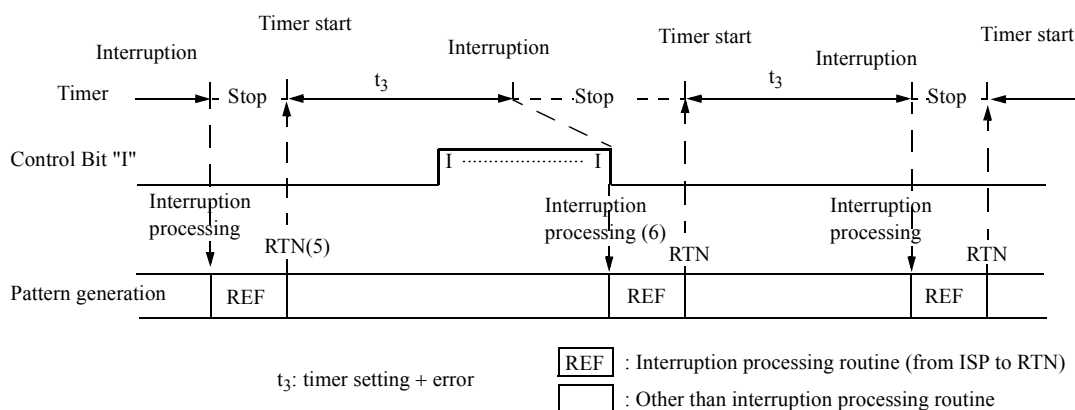
Calculate the error caused by setting the control bit as shown below.

The maximum time from RTN to interruption processing =

{maximum time of t_3 } + {(maximum execution number of "I") * (maximum RATE)}

The minimum time from RTN to interruption processing = {the minimum time for t_3 }

"Maximum RATE" is the longest RATE used for one test. For information on t_3 maximum time/minimum time, see t_3 in Figure 4-4. The time from control bit "T (TM1)" to interruption processing is similar.

Figure 4-3 MODE REFRESHB n by Control Bit "I"

◆ Distributed Refresh and Burst Refresh

Examples of distributed refresh and burst refresh programs appear below:

◆ Pattern Program for Distributed Refresh and Burst Refresh (1/2)

```

MPAT REFRESH      ;Sample program for refresh test.

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  LMAX=#3FF
  HMAX=#FF
  IDX1=#3FFFE

SDEF  INIT=XB<0 YB<0 TP<0
SDEF  YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB
SDEF  REF =RF<RF+1 X<RF
           *5          *6

MODULE BEGIN      ;distributed refresh

MODE REFRESHA 256 *1

REGISTER
TIMER=16US      *2
ISP=STI         *3

START  #0
NOP    T*7 INIT
IDX12  #5
ST0:   JN11 ST0   W      YINC   FP1
ST1:   JN11 ST1   R      YINC   FP1
      JZD  ST0
      STPS
STI:   RTN                      REF *4

MODULE END

```

◆ Pattern Program for Distributed Refresh and Burst Refresh (2/2)

```

MODULE BEGIN      ;burst refresh

MODE REFRESHA 256      *1

REGISTER
    TIMER=4MS          *2
    ISP=STI2            *3

START #10
NOP          T*7      INIT
IDX12 #5

ST2: JN11 ST2 W YINC FP1
ST3: JN11 ST3 R YINC FP1
JZD ST3
STPS

STI2: NOP REF ] *4
      IDX12 #FC REF
      RTN REF

MODULE END

END

```

*1 Refresh mode specification

- REFRESHA *n* (available specifications are *n* = 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144)
The interval during which interrupts occur is shown in [Table 4-3](#).
- REFRESHB *n* (available specifications are *n* = 64, 128, 256, 512, 1024, 2048, 4096, 8192, 16384, 32768, 65536, 131072, 262144)
The interval during which interrupts occur is shown in [Table 4-3](#).

*2 Timer data setting

*3 Branch destination specification if an interrupt occurs.

*4 Interrupt processing cycle description

*5 Instruction that increments the refresh counter (RF)

*6 Instruction that outputs the refresh address stored in the refresh counter

*7 Timer start instruction ("T" or "TMI")

Table 4-3 Refresh Period

Mode	Test system	Time/ number of times	t_1	t_2	t_3
REFRESH A	T5377 (No specification of the mask mode in the AFM ACTION state- ment)	Time	Set value +(65 to 66 cycles) +(300 ns to 400 ns)	Set value + (4 to 5 cycles)	--
		Number of times	Set value + (63 cycles)	Set value	--
	T5377 (With the speci- fication of the mask mode in the AFM ACTION state- ment)	Time	Set value +(141 to 142 cycles) +(300 ns to 400 ns)	Set value + (4 to 5 cycles)	--
		Number of times	Set value + (139 cycles)	Set value	--
	T5376 T5375	Time	Set value +(60 to 61 cycles) +(300 ns to 400 ns)	Set value + (4 to 5 cycles)	--
		Number of times	Set value + (58 cycles)	Set value	--
	T5371	Time	Set value +(28 to 29 cycles) +(300 ns to 400 ns)	Set value + (1 to 2 cycles)	--
		Number of times	Set value + (29 cycles)	Set value	--
	T5336 T5311	Time	Set value +(26 to 27 cycles) +(400 ns to 500 ns)	Set value + (1 to 2 cycles)	--
		Number of times	Set value + (27 cycles)	Set value	--

Mode	Test system	Time/ number of times	t ₁	t ₂	t ₃
REFRESH B	T5377 (No specification of the mask mode in the AFM ACTION state- ment)	Time	Set value +(65 to 66 cycles) +(300 ns to 400 ns)	--	Set value +(8 to 9 cycles) +(300 ns to 400 ns)
		Number of times	Set value + (63 cycles)	--	Set value + (6 cycles)
	T5377 (With the speci- fication of the mask mode in the AFM ACTION state- ment)	Time	Set value +(141 to 142 cycles) +(300 ns to 400 ns)	--	Set value +(8 to 9 cycles) +(300 ns to 400 ns)
		Number of times	Set value + (139 cycles)	--	Set value + (6 cycles)
	T5376 T5375	Time	Set value +(60 to 61 cycles) +(300 ns to 400 ns)	--	Set value +(8 to 9 cycles) +(300 ns to 400 ns)
		Number of times	Set value + (58 cycles)	--	Set value + (6 cycles)
	T5371	Time	Set value +(28 to 29 cycles) +(300 ns to 400 ns)	--	Set value +(3 to 4 cycles) +(300 ns to 400 ns)
		Number of times	Set value + (29 cycles)	--	Set value + (5 cycles)
	T5336 T5311	Time	Set value +(26 to 27 cycles) +(400 ns to 500 ns)	--	Set value +(6 to 7 cycles) +(400 ns to 500 ns)
		Number of times	Set value + (27 cycles)	--	Set value + (7 cycles)

TIMER set value must be within the following range:

TIMER=(time) 200 ns to 409.5 s, or

TREFRESH (number of times) 2 to 4095

Notes on setting TIMER in time notation for the RTTC test that changes RATE values during the test

- Minimum and maximum cycle errors of t₁, t₂, t₃ minimum cycle error:

Minimum value of the t₁ cycle error: minimum RATE for a pattern other than refresh routine.

Maximum value of the t₁ cycle error: maximum RATE for a pattern other than refresh routine.

Minimum value of the t₂ and t₃ cycle errors: minimum RATE for all patterns including refresh routine.

Maximum value of the t₂ and t₃ cycle errors: maximum RATE for all patterns including refresh routine.

- t₂, t₃ include the following errors in addition to cycle error.

(Rate error)=±{(maximum rate of all patterns) – (minimum rate of all patterns)}*N

Here, patterns to be used for the refresh routine are also included with all patterns.

N=21 for the T5336 and T5311, N=25 for the T5371, and N=54 for the T5376 and T5375. N=59 for the T5377 if the mask mode is not specified in the AFM ACTION statement, and N=135 for the T5377 if the mask mode is specified in the AFM ACTION statement.

In other words, the result is $t_2, t_3 = (\text{setting value}) + (\text{cycle error}) \pm (\text{RATE error})$.

When the rate error is larger than the value of $\{(\text{set value}) + (\text{minimum cycle error})\}$, set the number of cycles for the timer.

```

MPAT SAMPLE
MODE MUX REFRESHB 256
REGISTER
  XMAX=#FF
  YMAX=#3FF
  TIMER=1MS
  ISP=STIS
  IDX1=#3FFFE

  START #0
  NOP      XB<0      YB<0      TP<0 /T1
ST0: JN11 ST0 XB<XB+1 YB<YB+1^BX W FP1 /T1
ST1: JN11 ST1 XB<XB+1 YB<YB+1^BX R FP1 /T2
      JZD ST0 /T1
      STPS

STIS:NOP      RF<RF+1 X<RF /T3
      IDXI2 #FC RF<RF+1 X<RF /T4
      RTN      RF<RF+1 X<RF /T3
END

```

The example shown in [Figure 4-4](#) shows calculation of t_1 and t_3 when running the pattern program shown above with RATE=50NS, 100NS, 150NS, and 1US. (The description below is an example for the T5376 and T5375.)

Here, the maximum and minimum pattern rates produced by the pattern program are listed below.

	Maximum RATE	Minimum RATE
Other than refresh routine	100 ns (TS2)	50 ns (TS1)
All patterns	1 μ s(TS4)	50 ns (TS1)

The pattern to be used for the fresh routine is created by the STIS label to RTN instruction in the above pattern program.

As [Table 4-3](#) shows, the t_1 cycle error is 60 to 61 cycles and the t_3 cycle error is 8 to 9 cycles.

The t_1 cycle error is calculated with the rate of patterns which are not used for the refresh routine below.

t_1 cycle error = 60×50 ns (minimum RATE) to 61×100 ns (maximum RATE) = 3.0 μ s to 6.1 μ s

The t_3 cycle error is calculated using RATE for all patterns.

t_3 cycle error = 8×50 ns (minimum RATE) to 9×1 μ s (maximum RATE) = 0.4 μ s to 9.0 μ s

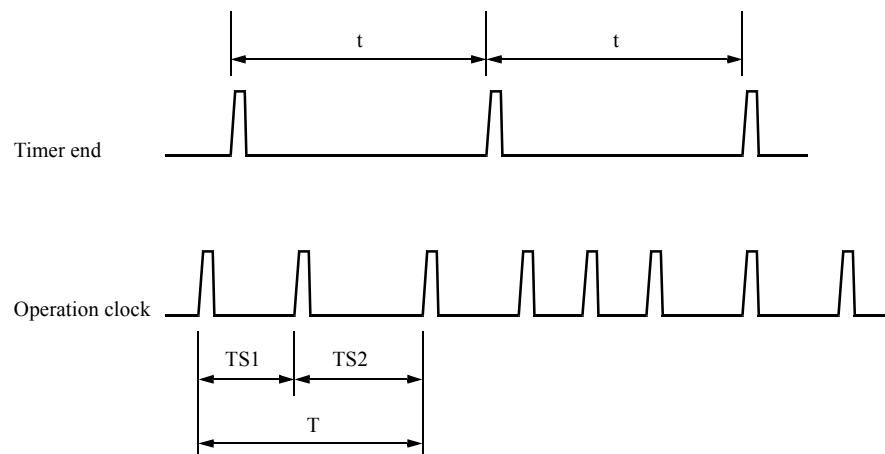
Then, the t_3 rate error is calculated using the equation of (RATE error) = $\pm \{(\text{maximum RATE of all patterns}) - (\text{minimum RATE of all patterns})\} \cdot N$ below. $N=54$ is set for the T5376 and T5375. t_3

RATE error = $\pm \{1$ μ s (maximum RATE) - 50 ns (minimum RATE) $\} \times 54 = \pm 51.3$ μ s

The TIMER setting for the pattern programs shown above is 1 ms. Therefore, considering only cycle error for t_1 , $t_1 = (1$ ms + 3.0 μ s) to $(1$ ms + 6.1 μ s) = 1.0030 ms to 1.0061 ms. Considering cycle error and also RATE error for t_3 , $t_3 = (1$ ms + 0.4 μ s to 51.3 μ s) to $(1$ ms + 9.0 μ s + 51.3 μ s) = 0.9491 ms to 1.0603 ms.

In the REFRESHA mode, calculation proceeds in similar fashion, but substituting t_2 for t_3 .

Limit of TIMER set value when REFRESHA MODE is set



$$t + t \times 50 \times 10^{-6} > T$$

t

Timer settings

$$50 \times 10^{-6}$$

Timer clock error

T

Sum of arbitrary continuous 2 cycle rates (TS1 + TS2 in example)

When a total of optional continuous 2 cycle rates in the pattern program is more than the timer set value, no timer interrupts. Therefore, no refresh is executed.

Also, when (refresh routine execution time) $> t_2$, an interruption occurs during the refresh, and incorrect operation results.

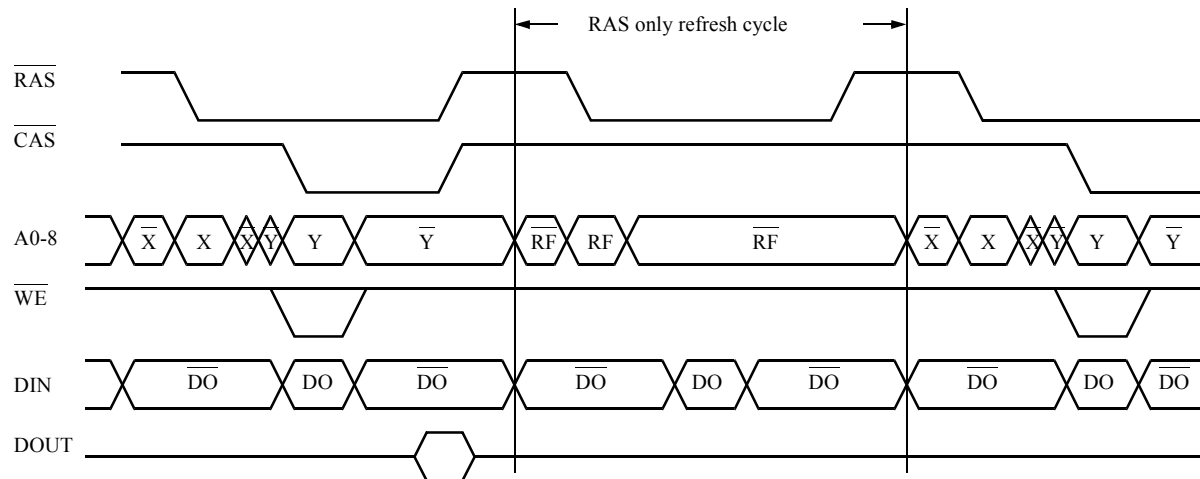
t_2 is the time shown in [Figure 4-4](#) and [Table 4-3](#).

t_2 is a value that takes into consideration cycle and RATE errors.

◆ RAS Only Refresh

Figure 4-5 shows the RAS-only-refresh cycle waveforms.

Figure 4-5 RAS Only Refresh Cycle



Pattern program

An example of a RAS-only refresh testing pattern program appears below.

```
MPAT M256K ;Sample program for refresh test.
;RAS only refresh
MODE MUX

REGISTER
XMAX=#FF
YMAX=#3FF
LMAX=#3FF
HMAX=#FF
IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0
SDEF YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB
SDEF REF =RF<RF+1 X<RF
           *5 *6

MODULE BEGIN ;distributed refresh

MODE REFRESHA 256 *1

REGISTER
TIMER=16US *2
ISP=STI *3
CPE1=R
CPE2=C0 *11
START #0
NOP T INIT
IDX12 #5 *7

ST0: JN11 ST0 W YINC FP1
ST1: JN11 ST1 R YINC FP1
JZD ST0

STI: RTN C0 M1 TS2 REF *4
      *10 *8 *9

MODULE END

END
```

**1 Set refresh mode.*

REFRESHA 256
 |
 Refresh mode

Specification that outputs RF0-7 to X0-7

**2 Timer data setting*

TIMER = 200NS to 409.5S (time)
(TRAS) 2 to 4095 (number of times)

**3 Branch destination specification if an interrupt occurs.*

**4 Generate refresh addresses.*

**5 Instruction that increments the refresh counter*

**6 Instruction that outputs the refresh counter contents to address X*

**7 Internal timer start ("T" or "TM1")*

**8 Multiplex mode release by control bit "M1."*

Since only the refresh address is applied in the RAS-only-refresh cycle, release the multiplex mode.

**9 Timing set switching*

The multiplex mode is released by control bit "M1" and only the refresh address (row address) is applied. To inhibit the clock output column address, switch the timing set.

Inhibit the clock pulses with the OPEN instruction as shown in in the RAS-only refresh testing test plan program example.

**10 Disable $\overline{\text{CAS}}$ using control bit C0.*

Disable $\overline{\text{CAS}}$ in the refresh cycle by selecting C0 as the pin data for the $\overline{\text{CAS}}$ pin.

**11 DOUT is High-Z during the RAS only refresh. High-Z is checked by STRB2.*

For only the RAS only refresh, set CPE2 to check it by STRB2.

Test plan program

An example of a RAS only refresh test plan program appears below.

```

PRO P256K ;Sample program for 256K dynamic RAMs.

TIME1=1MS :VS1 ] *1
TIME2=1MS :IN ]

VS1=5V
IN1=2.4V, 0.8V ] *2
OUT1=2.4V, 0.4V ]

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZZ,BCLK4,CCLK4,<C0> *4 ;/CAS *3
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,STRB2,HZ2,<D0> ;DOUT ]

TR=1NS & TF=1NS ;Timing set
RATE=260NS:2
ACLK1=0NS:2
BCLK1=10NS:2
CCLK1=10NS+15NS+TF+TR:2
ACLK2=35NS,OPEN *8
BCLK2=45NS,OPEN *8
CCLK2=45NS+25NS+TF+TR,OPEN *8
ACLK3=0NS:2
BCLK3=10NS:2
CCLK3=10NS+TF+150NS:2
BCLK4=45NS:2
CCLK4=45NS+TF+75NS:2
BCLK5=45NS:2
CCLK5=45NS+TF+25NS:2
BCLK6=45NS:2
CCLK6=45NS+TF+25NS+TR:2
STRB1=45NS+TF+75NS:2
STRB2=45NS+TF

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10(RESET)@ *9
TCHANGE PD11(RESET) ?
CALL CALB("C256K", TCAL)

TEST 1

REG MPAT PC=#0 *10
MEAS MPAT M256K *11

END

```

*1 Power-up sequence specification

*2 Power supply (VS1), driver level (IN1), and comparison level (OUT1) setting

*3 Pin data setting

*4 Select control bit C0 for pin data to disable CAS in the RAS-only-refresh cycle.

**5 Set the detection mode to High-Z, using STRB2.*

Because it is impossible to specify HZ2 in pin statements on the T5336 and T5311, select SELECT HZSENSE ENABLE.

**6 Select STRB2 to determine the DOUT state in the RAS-only-refresh cycle.*

**7 Timing setting*

**8 Only the row address is applied in the RAS only refresh cycle. To inhibit the column address clock, open (inhibit) the clock.*

**9 Timing calibration*

**10 Select a test pattern with the REG statement.*

**11 Test*

◆ CAS-before-RAS-refresh

The CAS-before-RAS-refresh test is executed with the ALPG timing switching function (TSn) by changing the phases of the clock pulses that generate CAS in the refresh cycle only.

Pattern program

An example of a CAS-before-RAS refresh pattern program appears below.

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  LMAX=#3FF
  HMAX=#FF
  IDX1=#3FFFE

SDEF  INIT=XB<0 YB<0 TP<0
SDEF  YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

MODULE BEGIN

MODE REFRESHA 256 *1

REGISTER
  TIMER=16US *2
  ISP=STI *3

START  #0
NOP                                T *4                INIT
IDX12  #5
ST0:   JN11  ST0    W                YINC                FP1
ST1:   JN11  ST1    R                YINC                FP1
JZD    ST0
STPS
STI:   RTN                TS2 *5

MODULE END

END

```

*1 Set refresh mode.

*2 Timer data setting

*3 Branch destination specification if an interrupt occurs.

*4 Internal timer operation start

*5 Change the timing set in the refresh cycle only.

Test plan program

An example of a CAS-before-RAS refresh test test plan program appears below.

```

PRO P256K

TIME1=1MS :VS1
TIME2=1MS :IN

VS1=5V
IN1=2.5V, 0.8V
OUT1=2.4V, 0.4V

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK1,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

TR=1NS & TF=1NS ;Timing set
RATE=260NS:2
ACLK1=0NS:2
BCLK1=40NS:2
CCLK1=40NS+15NS+TF+TR:2
ACLK2=65NS:2
BCLK2=75NS:2
CCLK2=75NS+25NS+TF+TR:2
ACLK3=0NS:2
BCLK3=40NS:2
CCLK3=40NS+TF+15NS:2
BCLK4=75NS,40NS-30NS-TF *1
CCLK4=75NS+TF+75NS,40NS+TF+30NS *1
BCLK5=75NS:2
CCLK5=75NS+TF+25NS:2
BCLK6=75NS:2
CCLK6=75NS+TF+25NS+TR:2
STRB1=75NS+TF+76NS:2

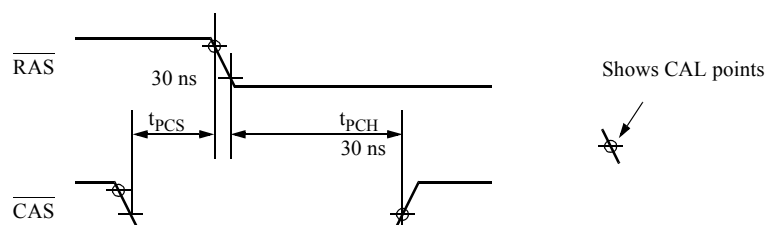
STRING TCAL(200) ;Calibration
TCAL=?DRREF PF1-12,33=2V,1.2V,TCHANGE PD10(RESET)@
TCHANGE PD11(RESET)?
CALL CALB("C256K",TCAL)

TEST 1
REG MPAT PC=#0
MEAS MPAT M256K

END

```

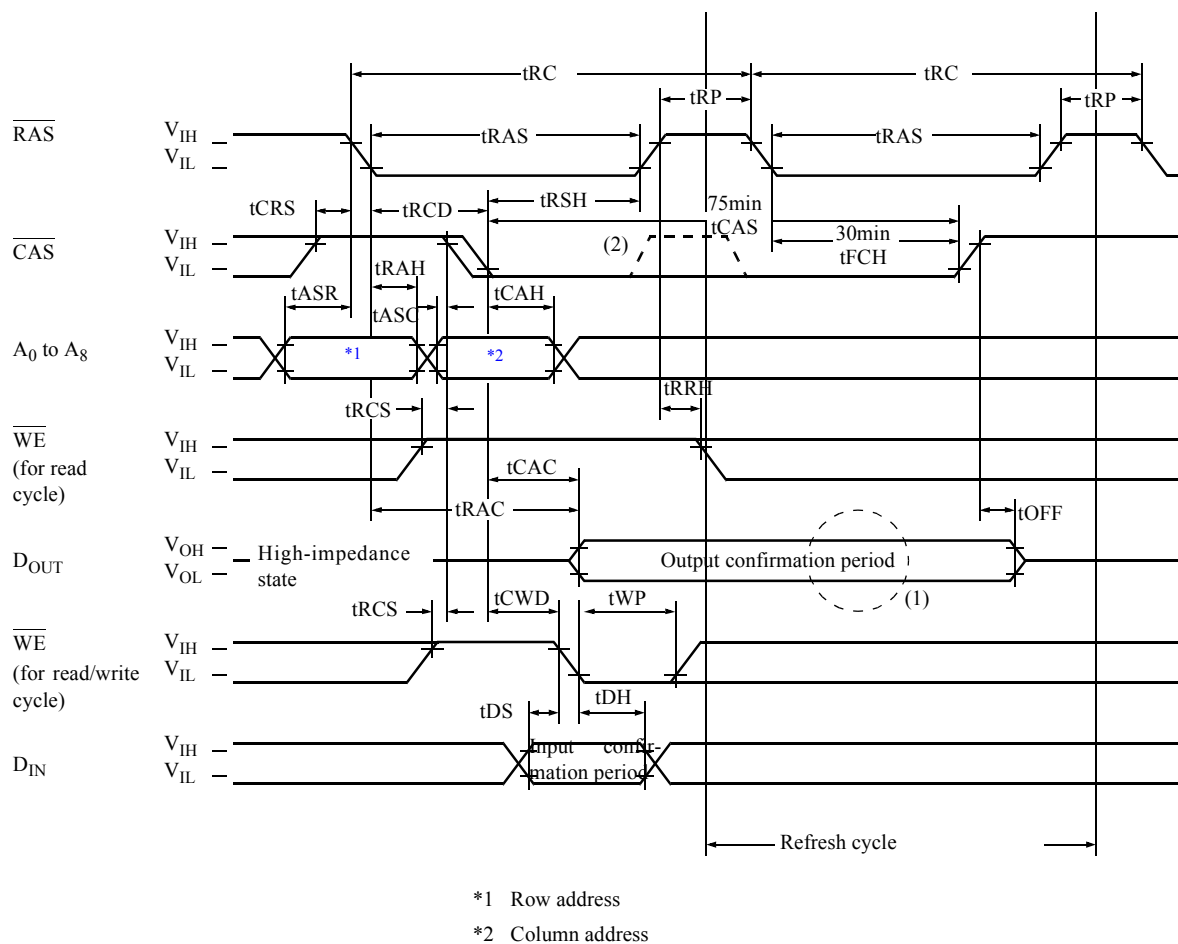
*1 Change the setting of the clock pulses that generates the CAS clock in the refresh cycle.



◆ Hidden Refresh (CAS before RAS)

Figure 4-6 shows a hidden refresh timing chart.

Figure 4-6 Hidden Refresh Timing Chart



Pattern program

An example of a hidden refresh testing pattern program appears below.

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  LMAX=#3FF
  HMAX=#FF
  IDX1=#3FFFE

SDEF  INIT=XB<0 YB<0 TP<0
SDEF  YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

MODULE BEGIN

MODE REFRESHA 256 *1

REGISTER
  TIMER=16US *2
  ISP=STI *3
  CPE1=PREF *4

START  #0
NOP          T *5
IDX12  #5
ST0:  JN11  ST0  W          YINC  FP1
ST1:  JN11  ST1  R          YINC  FP1
JZD    ST0
STPS
STI:   RTN          M1  TS2  PD
          *6      *7      *8

MODULE END

END

```

*1 Refresh mode specification

*2 Timer data setting

*3 Branch destination setting if an internal interrupt occurs.

**4 In the hidden refresh cycle, when a memory read cycle precedes a refresh cycle, the read data is held (since the CAS clock does not go high). ((1) in [Figure 4-6](#))*

Set PREF at CPE to check the data shown above.

When PREF is set at CPE, logical comparison of the DUT output is performed in the following cycles:

- Cycle in which control bit "R" is specified*
- Cycle in which control bit "M1" is specified and control bit "R" has been specified in the preceding cycle*

**5 Begin internal timer operation.*

**6 To check the read data in the hidden refresh cycle, specify control bit M1.*

*➔ Refer to *4*

**7 Switch the timing set.*

**8 When "PD" is written, the expected value is the same as the one for the preceding cycle.*

Test plan program

An example of a hidden refresh testing test plan program appears below.

```

PRO P256K

TIME1=1MS :VS1
TIME2=1MS :IN

VS1=5V
IN1=2.4V, 0.8V
OUT1=2.4V, 0.4V

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK1,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4,RINHC,<-,-,PM1> *1 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

TR=1NS & TF=1NS
RATE=260NS:2
ACLK1=1NS:2
BCLK1=40NS:2
CCLK1=40NS+15NS+TF+TR:2
ACLK2=65NS,OPEN *2
BCLK2=75NS,OPEN *2
CCLK2=75NS+25NS+TF+TR,OPEN *2
ACLK3=0NS:2
BCLK3=40NS:2
CCLK3=40NS+TF+150NS:2
BCLK4=75NS,40NS-30NS-TF
CCLK4=75NS+TF+75NS,40NS+TF+30NS
BCLK5=75NS:2
CCLK5=75NS+TF+25NS:2
BCLK6=75NS:2
CCLK6=75NS+TF+25NS+TR:2
STRB1=75NS+TF+75NS:2

STRING TCAL(200)
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10(RESET) @
TCHANGE PD11(RESET)?
CALL CALB("256K2",TCAL)

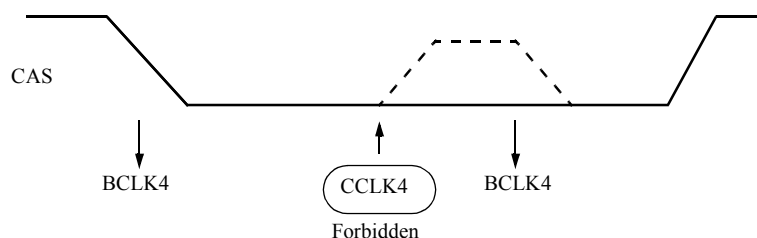
TEST 1
REG MPAT PC=#0
MEAS MPAT M256K

END

```

*1 Inhibit the clock pulse corresponding to the CAS clock rising edge in the cycle preceding the refresh cycle. ((2) in [Figure 4-6](#))

Write RINHC in the PIN statement to inhibit the clock pulse (Cannot be described on the T5377, T5376, T5375 and T5371). Select PM1 as the pin data for Pin Data C. For PM1, pattern data 1 is output from the ALPG to the cycle preceding the cycle in which control bit M1 is specified in the pattern program. If RINHC is written in the PIN statement, and PM1 selected for pin data C is 1, data generation by CCLK4 is disabled.



*2 In the cycle in which the multiplex mode (MODE MUX) is specified and M1 is specified in the pattern program, the address generation clock pulse is disabled since the multiplex mode is released.



Tip

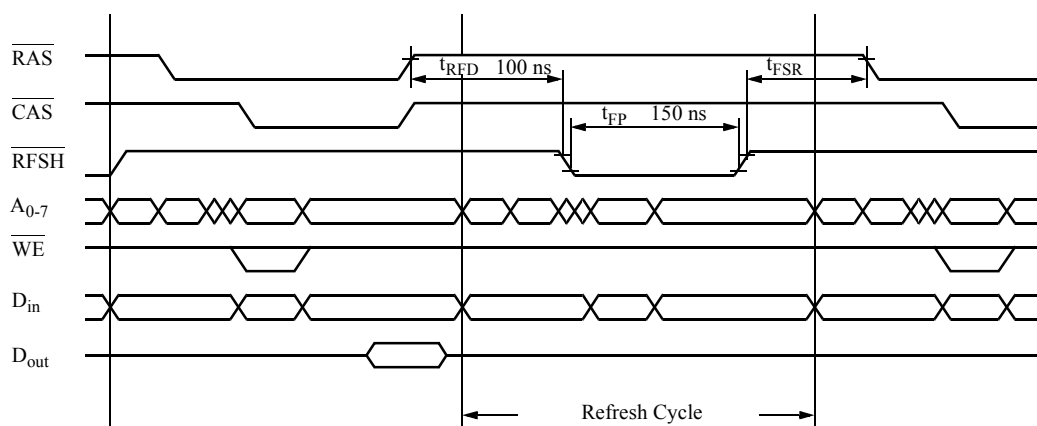
If the waveform mode in *1 is RZX, clock pulse CCLK4 is disabled with the OPEN instruction.

◆ Memory with Refresh Control Terminal

[Figure 4-7](#) shows the refresh cycle for the memory with a refresh control terminal. This figure shows a distributed refresh cycle. Clock pulses RAS and CAS in this cycle are disabled by the control bits in the pattern program.

The clock pulse applied to terminal RFSH is controlled by the control bit in the pattern program.

Figure 4-7 Refresh Cycle for Memory with Refresh Control Terminal



Pattern program

An example of a memory pattern program with refresh terminal appears below.

```

MPAT M64K

MODE MUX

REGISTER
  XMAX=#7F
  YMAX=#1FF
  LMAX=#1FF
  HMAX=#7F
  IDX1=#FFFE

SDEF   INIT=XB<0 YB<0 TP<0
SDEF   YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

MODULE BEGIN

MODE REFRESHA 128 *1

REGISTER
  TIMER=16US *2
  ISP=STI *3

START   #0
NOP                      T *4          INIT
IDX12   #5
ST0:    JN11   ST0    W          YINC   FP1
ST1:    JN11   ST1    R          YINC   FP1
        JZD     ST0
        STPS
STI:    RTN                      C0 *5

MODULE END

END

```

*1 Refresh mode specification

*2 Timer data setting

*3 Branch destination setting if an internal interrupt occurs.

*4 Begin timer operation.

*5 Specify the control bit indicating the refresh cycle.

When creating the program, note that if M1 and M2 are described, the multiplex mode operation changes.

➔ For details, refer to [4. 3. 1 "Address Multiplex" on page 4-11](#).

Test plan program

An example of a memory test plan program with refresh terminal appears below.

```

PRO P64K
    TIME1=1MS :VS1
    TIME2=1MS :IN
    VS1=5V
    IN1=2.4V, 0.8V
    OUT1=2.4V, 0.4V

    PD1-7=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-6,Y0-6> ;A0-6
    PD8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y7,Y8> ;A7
    PD9=IN1,/RZO,BCLK7,CCLK7,<C0> *1 ;/RFSH
    PD10=IN1,/RZZ,BCLK3,CCLK3,<C0> *2 ;/RAS
    PD11=IN1,/RZZ,BCLK4,CCLK4,<C0> *2 ;/CAS
    PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
    PD34=IN1,XOR,ACLK1,BCLK6,CCLK6,<D0> ;DIN
    PC33=OUT1,STRB1,<D0> ;DOUT

    TR=1NS & TF=1NS
    RATE=270NS
    ACLK1=0NS
    BCLK1=40NS
    CCLK1=40NS+15NS+TF+TR
    ACLK2=65NS
    BCLK2=75NS
    CCLK2=75NS+45NS+TF+TR
    BCLK3=40NS
    CCLK3=40NS+TF+150NS
    BCLK4=75NS
    CCLK4=75NS+TF+100NS
    BCLK5=75NS
    CCLK5=75NS+TF+45NS
    BCLK6=75NS
    CCLK6=75NS+TG+45NS+TR
    BCLK7=20NS+TR+TF
    CCLK7=20NS+TR+TF+150NS
    STRB1=40NS+TF+150NS

    STRING TCAL(200)
    TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10(RESET) @
    TCHANGE PD11(RESET) @
    TCHANGE PD9(RESET) ?
    CALL CALB("C64K",TCAL)

TEST 1
    REG MPAT PC=#0
    MEAS MPAT M64K

END

```


*1 Apply clock pulses to terminal $\overline{RFSH}$ in the refresh cycle only.

*2 Disable the $\overline{RAS}$ and $\overline{CAS}$ clocks in the refresh cycle.

4. 3. 4 Area Inversion Test (ARIRAM)

In many dynamic RAMs, I/O data and data stored in internal cells are inverted at specific addresses, as shown in [Table 4-4](#).

Accordingly, when performing testing emphasizing cell data, it is necessary to invert the expected values for input and output data applied to the DUT at specific addresses. The memory test system has a area inversion function to facilitate this test.

Table 4-4 Example of DRAM Data Topology

R7 ^{*1}	R0 ^{*1}	Input data	Cell data ^{*2}	Output data
0	0	0	0	0
0	0	1	1	1
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	0
1	1	1	1	1

*1 Rn indicates row address.

*2 Cell data 0 : discharging status
1 : charging status

Area inversion mechanism

DA0 to DA17 and DB0 to DB17 used as expected value data for input data applied to the DUT and output data from the DUT are inverted under the inversion conditions set in the area inversion circuit.

Write the inversion conditions in the ARIRAM DEFINE statement used in the pattern program.

➔ For details on the ARIRAM DEFINE statement, refer to [2. 2. 5 "ARIRAM DEFINE Statement" on page 2-29](#).

If the double speed mode which uses the pre-scrambler is specified for the T5377, each Y addresses for the first half and for the second half of the double speed mode can be used as the inversion conditions for the TP1 and TP2 data.

Y address used for the TP1 data inversion condition : The Y addresses for the first half of the double speed mode

Y address used for the TP2 data inversion condition : The Y addresses for the second half of the double speed mode

Programming

A pattern program using the area inversion mechanism appears below:

```
MPAT M64K

MODE MUX

REGISTER
  XMAX=#7F
  YMAX=#1FF
  IDX1=#FFFE

ARIRAM DEFINE
  X=X0
  Y=Y0
  INV=X"Y
] *2

SDEF INIT=XB<0 YB<0 TP<0 ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX ;increment base

MODULE BEGIN ;checker board

ARIRAM DEFINE
  X=X0
  Y=Y0
  INV=X"Y
] *1

START #10

IDX12 #6 INIT
ST0: JN11 ST0 W INC FP1
ST1: JN11 ST1 R INC FP1
JZD ST0
STPS

MODULE END

END
```

**1 Inversion condition setting*

DA0-17 and DB0-1 applied to DUT are inverted for output at the address satisfying the following condition:

$X0 \text{ " } Y0 = 1$

(") indicates exclusive OR.

**2 When inversion conditions are specified in the common section of the pattern program, the conditions are valid for all the pattern programs specified in the pattern program.*

4. 4 Bump Test (Burst block)

4. 4. 1 How to Specify Conditions in the Test Plan Program

When conditions are specified in the test plan program, the power supply voltage is changed during the bump test when pattern generation is stopped by the PAUSE instruction in the pattern program. The voltage to be changed is specified in the test plan program.

The following section explains how to create a pattern program and a test plan program for the bump test shown in [Figure 4-8](#).

Pattern program

A pattern program for the bump test appears below.

```
MPAT BUMP

MODE MUX PAUSE *1

REGISTER
  XMAX=#FF
  YMAX=#3FF
  TIMER=t *2
  IDX2=#3FFFE
  IDX3=#0

START #0
  IDX11 #6      XB<0 YB<0 TP<0
ST0:  JN12  ST0  W      XB<XB+1 YB<YB+1^BX *4
      NOP    T *3      ; PAUSE
ST1:  JN12  ST1  R      XB<XB+1 YB<YB+1^BX *4
      JN13  ST0
      JZD   ST0
      STPS

END
```

*1 Set the pause mode.

*2 Set the timer.

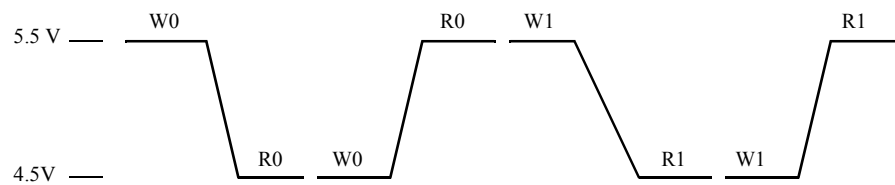
Set the following value for the timer:

$t = \text{data transfer time} + \text{settling time}$

➔ For details, refer to [ATL-51 Test Plan Program Reference Manual\(No. : 8431498\)](#).

*3 Pause instruction

*4 Create a pattern program so that the minimum time from the pattern start to the first pause instruction is 5 μs or more and between pause instructions (NOPT) are 25 μs or more.

Figure 4-8 VS Change During Bump Test

W0 : Write "0" to all cells

R0 : Read "0" from all cells

W1 : Write "1" to all cells

R1 : Read "1" from all cells

Test plan program

A test plan program for bump testing appears below.

```

PRO BUMP

DIM(1) A(200)=0 *1

TIME1=1MS :VS1
TIME2=1MS :IN
VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V
RATE=260NS
ACLK1=0NS
:
:
STRB1=200NS
PD1-8=IN1,XOR,ACLK1,BCLK1,SDM,<X0-7,Y0-7>
PD9=IN1
:
:
PC33=OUT1,STRB1,<D0>

BURST BLOCK BEGIN(A) *2

VS1=5.0V & BURST WAIT TIME 1MS *3
VS1=4.0V *4
VS1=5.0V *5
VS1=4.0V *6
VS1=5.0V *7

BURST BLOCK END *2
:
:
MEAS MPAT(4,A) BUMP *8
:
:

END

```

*1 Secure the variable area which stores data (data specified in BURST BLOCK) transferred during the bump test.

*2 Define the data whose conditions are changed during the bump test.

*3 Specify the voltage set before starting pattern generation and the time to start pattern generation in the MEAS statement.

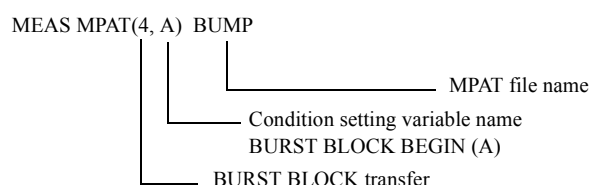
*4 Voltage set for each pause during pattern generation

*5 Voltage set for each pause during pattern generation

*6 Voltage set for each pause during pattern generation

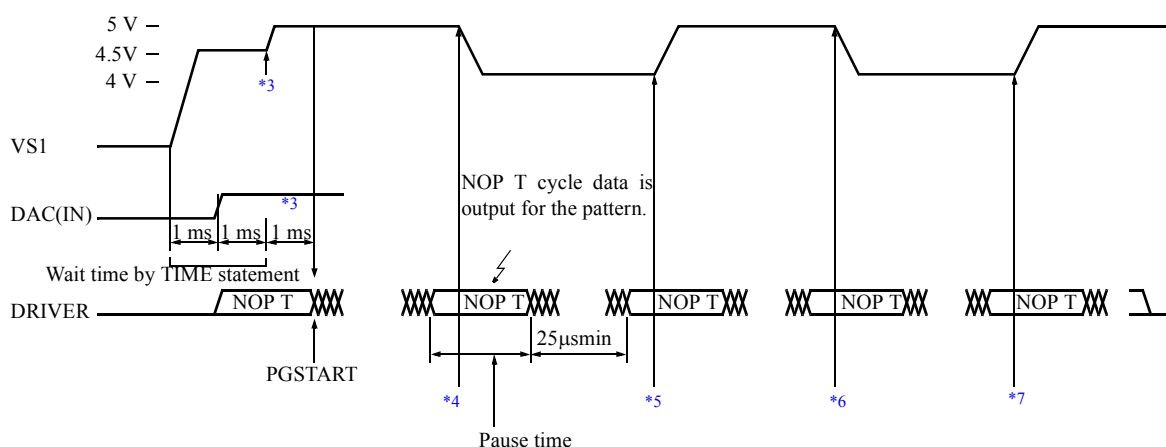
*7 Voltage set for each pause during pattern generation

*8 Test Execution



The power supply voltage changes in the time sequence shown in [Figure 4-9](#).

Figure 4-9 Burst Data Transfer and VS Change



4. 4. 2 How to Specify Conditions in the Pattern Program

When conditions are specified in the pattern program, the power supply voltage is changed by the OUT instruction in the pattern program during the bump test. The voltage to be changed is specified in the pattern program.

The following section explains how to create a pattern program and a test plan program for the bump test shown in [Figure 4-8](#).

Pattern program

A pattern program for the bump test appears below.

```

MPAT BUMP

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX2=#3FFFE

  BURST_BLOCK_BEGIN(A) *1
    VS1=4.5V *2
    WAIT TIME 1MS *3
  BURST_BLOCK_END (3)

  BURST_BLOCK_BEGIN(B)
    VS1=5.0V
    WAIT TIME 1MS
  BURST_BLOCK_END (4)

START #0
IDX11 #6 XB<0 YB<0 TP<0
ST0: JN12 ST0 W XB<XB+1 YB<YB+1^BX
      OUT A (1) *4
ST1: JN12 ST1 R XB<XB+1 YB<YB+1^BX
ST2: JN12 ST2 W XB<XB+1 YB<YB+1^BX
      OUT B (2) *4
ST3: JN12 ST3 R XB<XB+1 YB<YB+1^BX
      JZD ST0
      STPS

END

```

The data set which is set by the OUT instruction is described.

**1 Specify a variable (Burst block name) in parentheses. When the burst block name is specified as the OUT instruction, data from BURST BLOCK BEGIN to BURST BLOCK END is set with the OUT instruction.*

**2 Set the power supply voltage.*

**3 Waiting time after setting the power supply*

**4 Changing the voltage with the OUT instruction*

Set the data set ((3) and (4) in the example) specified by the OUT instruction operands ((1) and (2) in the example).

While the above data is set, data with the cycle specified by the OUT instruction is applied to the MUT.

When the OUT instruction is executed, the timing generator stops with the pattern applied to the MUT by the OUT instruction, and stops clock pulse generation.

When the data set selected by the OUT instruction has been transferred, pattern generation restarts.


Tip

- Set the voltage range so that the bias power supply source range to be applied to the DUT for setting the power supply voltage during the bump test is not changed.
- On the T5377, T5376 and T5375, be careful to not change the polarity of the power supply voltage applied to the DUT during the bump test.

Test plan program

A test plan program for bump testing appears below.

```

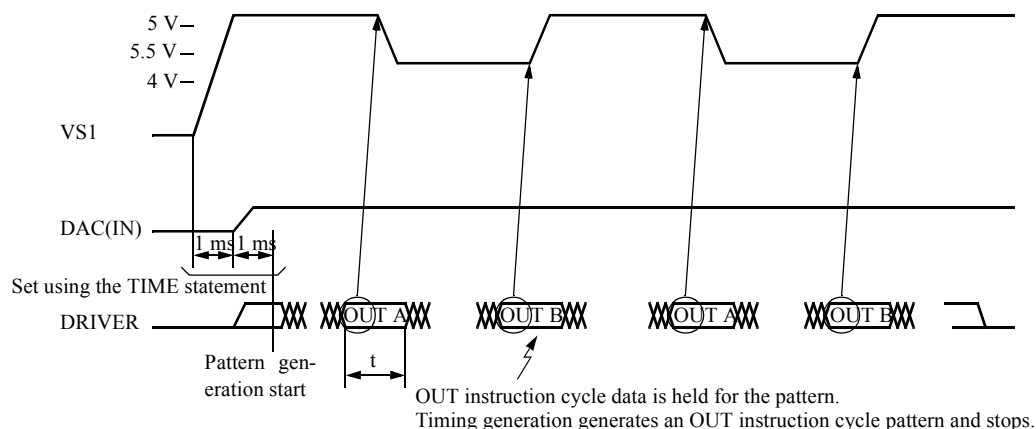
PRO BUMP

TIME1=1MS:VS1      *1
TIME2=1MS:IN ]

VS1=5.0V           *2
IN1=2.4V,0.8V
OUT1=2.4V,0.4V

RATE=260NS
ACLK1=0NS
:
:
STRB1=200NS
PD1-8=IN1,XOR,ACLK1,BCLK1,SDM,<X0-7,Y0-7>
PD9=IN1
:
:
PC33=OUT1,STRB1,<D0>
:
:
MEAS MPAT BUMP     *4
:
:
END
  
```

- *1 Reference-on time sequence
- *2 Set the power supply voltage for the initial state.
In this example, the power supply voltage for starting the test is set to 5.5 V because the power supply voltage is 5.5 V.
- *3 Set the test conditions.
- *4 Test
Figure 4-10 shows the time sequence for power supply voltage changes. Table 4-5 shows execution time (t) for the OUT instruction. Also, Table 4-6 lists transfer data and its transfer time.

Figure 4-10 OUT Instruction and VS Changes**Table 4-5 OUT Instruction Transfer Time (t)**

Test system	AFM ACTION statement	AFM RATE statement	OUT instruction transfer time (t)
T5377	No specification of the mask mode	HS	$t = 2 * \text{RATE} + (1.3 \mu\text{s to } 1.5 \mu\text{s}) + \text{data transfer time}$
		MS	$t = 2 * \text{RATE} + (1.5 \mu\text{s to } 1.7 \mu\text{s}) + \text{data transfer time}$
		LS	$t = 2 * \text{RATE} + (2.0 \mu\text{s to } 2.2 \mu\text{s}) + \text{data transfer time}$
	With the specification of the mask mode	HS	$t = 2 * \text{RATE} + (1.8 \mu\text{s to } 2.0 \mu\text{s}) + \text{data transfer time}$
		MS	$t = 2 * \text{RATE} + (2.6 \mu\text{s to } 2.8 \mu\text{s}) + \text{data transfer time}$
		LS	$t = 2 * \text{RATE} + (4.1 \mu\text{s to } 4.3 \mu\text{s}) + \text{data transfer time}$
T5376 and T5375	---	HS, MS	$t = 2 * \text{RATE} + (1.3 \mu\text{s to } 1.5 \mu\text{s}) + \text{data transfer time}$
		LS	$t = 2 * \text{RATE} + (1.5 \mu\text{s to } 1.7 \mu\text{s}) + \text{data transfer time}$
T5371, T5336 and T5311	---	---	$t = 2 * \text{RATE} + (1.8 \mu\text{s to } 1.9 \mu\text{s}) + \text{data transfer time}$

--- in the table shows that the relevant test systems are independent of the specification by the AFM ACTION statement and the AFM RATE statement.

Execution time for the entire cycle described by the OUT instruction = TS2 RATE + TS3 RATE + t
 RATE indicates the operation cycle of the cycle specified by the OUT instruction.

Transfer time is the sum of transfer times for each data transferred by the OUT instruction.

Table 4-6 lists transfer times for each data.

Table 4-6 OUT Instruction Transfer Time

Data to be transferred	Transfer time
VS _n = vs, R (vr)	4 μs
VS _n = vs	

Data to be transferred	Transfer time
WAIT TIME t	$1.6 \mu\text{s} + t$

4. 5 Partial Test (Z address)

4. 5. 1 Partial Test Using Z Address

The pattern program for the partial test can be performed easily with the Z address. When the test is performed by dividing the DUT into memory blocks, Z0 to Z7 are used as block addresses. Z0 to Z7 are applied to the DUT as follows:

ADDRESS DEFINE

Output Z0 to Z7 to X4 to X17(15) and Y4 to Y17(15) with the ADDRESS DEFINE statement specified in the pattern program.

Addresses are converted by the address descrambler after the Z address has been multiplexed.

```
ADDRESS DEFINE
  Xl - m = Zn - o      4 ≤ l, m ≤ 17(15)      1 ≤ m
  Yl - m = Zn - o      0 ≤ n, o ≤ 7           n ≤ o
```

The numbers inside the parentheses apply to the T5371, T5336 and T5311.

◆ When Dividing 256K DRAMs into 16 Parts

```
MPAT PART
:
ADDRESS DEFINE
  Y6 - 9 = Z0 - 3
:
END
```

4. 5. 2 Programming

An example of a partial test pattern program appears below:

◆ Example of Partial Test Pattern Program (1/2)

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 Z<0 TP<0
SDEF BINC=XB<XB+1 YB<YB+1^BX
SDEF PINC=XB<XB+1 YB<YB+1^BX Z<Z+1^BX *3
SDEF CINC=XC<XC+1 YC<YC+1^CY
SDEF CLAD=XC<XB+1 YC<YB+1^BX
SDEF ZINC=Z<Z+1
SDEF BASE=X<XB Y<YB
SDEF DIST=X<XC Y<YC

MODULE BEGIN ;checker board
START #0

  IDXI2 #6 INIT
ST0: JN11 ST0 W FP1 BINC
ST1: JN11 ST1 R FP1 BINC
JZD ST0
STPS

MODULE END

```

◆ Example of Partial Test Pattern Program (2/2)

```

MODULE BEGIN                                ;partial

REGISTER
  XMAX=#FF
  YMAX=#3F *2
  ZMAX=#F
  IDX1=#3FFFE
  IDX2=#3FFE
  IDX3=#3FFD
  IDX4=#E

ADDRESS DEFINE
  Y6-9=Z0-3 *1
  XY=ZN *2

START      #10
IDXI5      #6
ST0:  JN11  ST0      W      BASE      INIT
ST1:  NOP                    W      BASE      /D      PINC
ST2:  NOP                    R      DIST      CLAD
      NOP                    R      BASE      /D
      JN13  ST2      R      DIST      CINC
      JN12  ST1      W      BASE      BINC
      JN14  ST1                        ZINC *4
      JZD    ST0
      STPS

MODULE END

END

```

*1 Output Z0 to Z3 to Y6 to Y9 with the ADDRESS DEFINE statement.

*2 To output Z0 to Z3 to Y6 to Y9, after ORing them with other signals, mask Y6 to Y9 with YMAX.
If they are not masked, the high-order bits (YB6-9) of YB and YC are output.
Or, specify XY=ZN in the ADDRESS DEFINE statement.

*3 To scan all cells, use the link instruction.

*4 Increment the Z address.

5. *Programming II*

This chapter explains programming methods for the following types of tests: page mode, nibble mode, multiple-bit memory, read-modify-write, and EPROM.

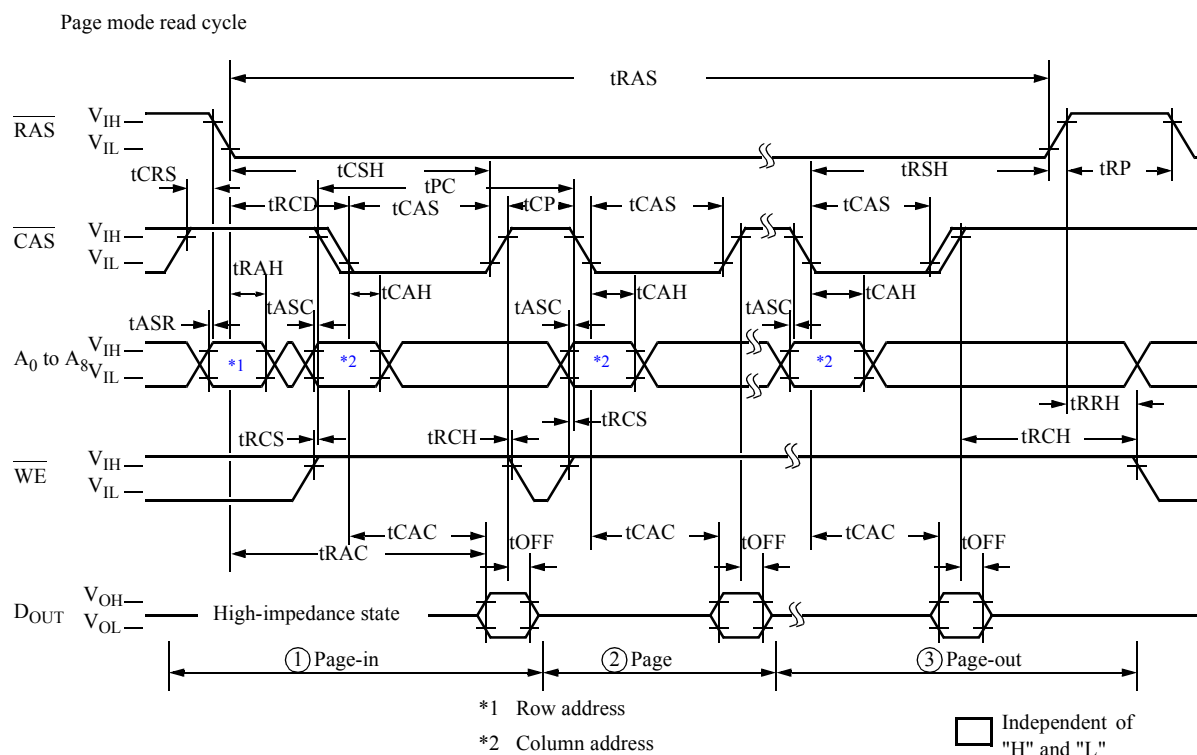
5. 1 Page Mode Test

5. 1. 1 Page Mode Test

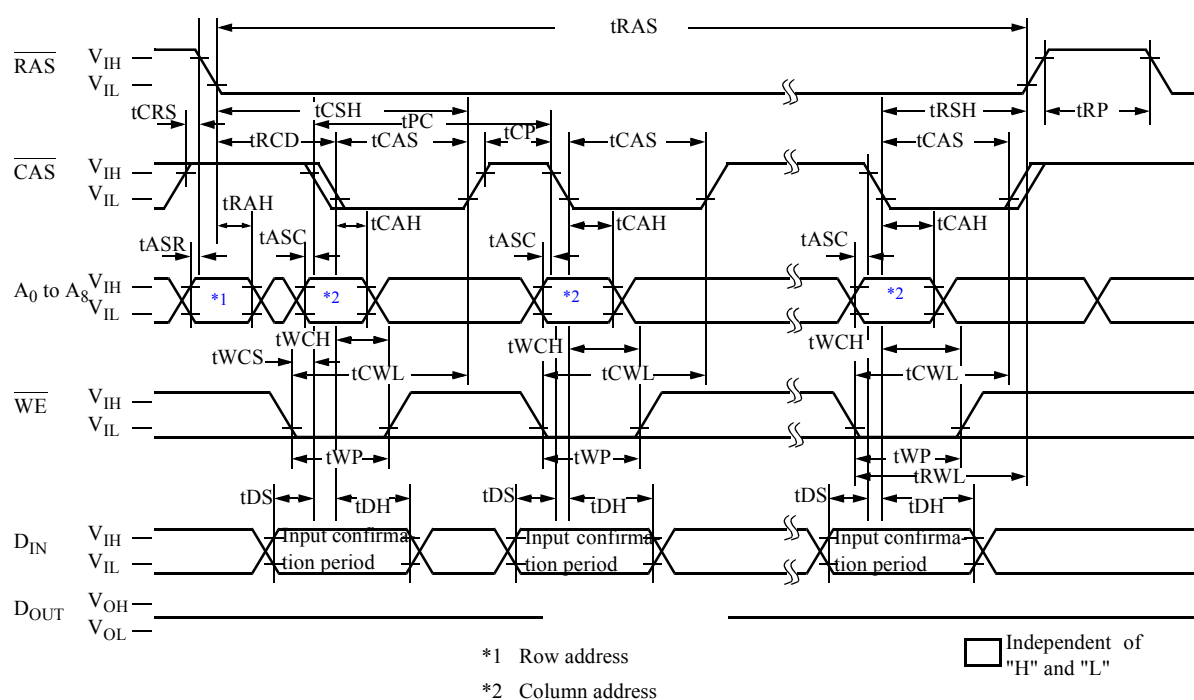
Most DRAMs include an access mode called the page mode to shorten cycle time.

[Figure 5-1](#) shows page mode storage: a row address is latched internally by an $\overline{\text{RAS}}$ clock pulse, and a $\overline{\text{CAS}}$ clock pulse is applied to change a column address with $\overline{\text{RAS}}$ held low. A page mode test program is created by either of the following:

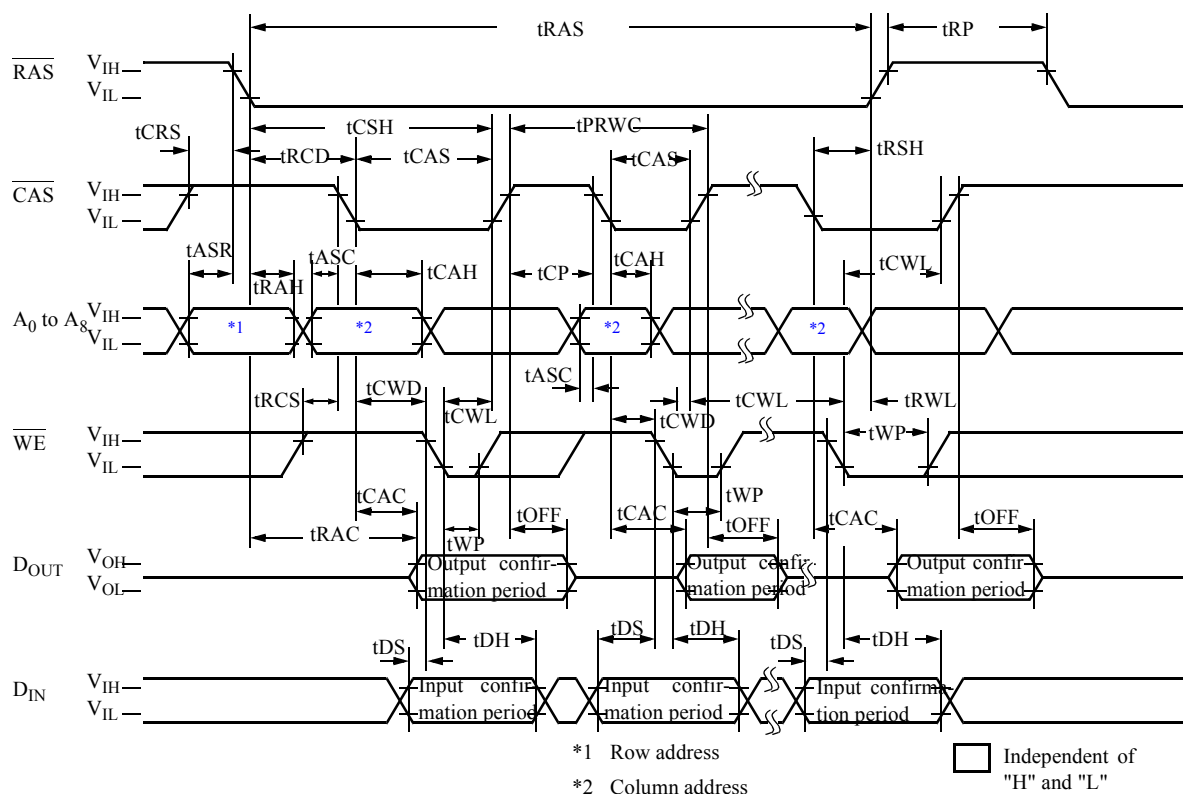
- Controlling address multiplexing and $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ clock pulses with control bits coded in the pattern program.
- Auto page mode test
Controlling address multiplexing and $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ clock pulses with address records applied to DUT.

Figure 5-1 Page mode Time Chart

Page mode write cycle (early write)



Page mode read write cycle



5. 1. 2 How to Control the Waveform by Control Bit

An example of a pattern program appears in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#), while an example of a test plan program appears in ["Test Plan Program for Page Mode Test Using Control Bits" on page 5-6](#). Create a program as follows:

Address and data pattern generation

In the program for generating addresses and data patterns, address multiplexing and $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ clock pulses are controlled by control bits coded in the pattern program. Therefore, steps for generating page-in, page, and page-out addresses must be coded separately. ((a) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

The page-in cycle is the cycle in which page mode operation begins. ((a) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

The page cycle indicates the cycle in which page mode operation is executed. ((a) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

The page-out cycle is the cycle in which page mode operation ends. ((a) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

The operation period, address data to be applied to DUT, and waveforms of $\overline{\text{RAS}}$ and $\overline{\text{CAS}}$ clock pulses differ for each cycle.

Operation period

Page-in, page, and page-out cycles for page mode operation are specified in TSn instructions in the pattern program.((b) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

Set operation periods corresponding to the TSn instructions in the test plan program.((d) in ["Test Plan Program for Page Mode Test Using Control Bits" on page 5-6](#))

Inhibiting $\overline{\text{RAS}}$ clock pulses

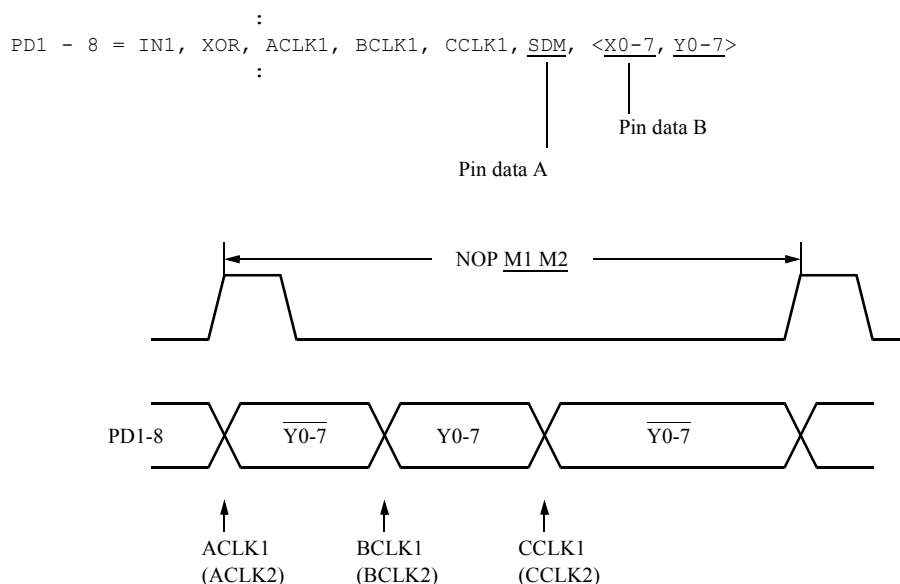
Inhibit RAS clock pulses in each cycle, by specifying the OPEN instruction for the clock pulses that generate RAS clock edges for each timing set.((e) in ["Test Plan Program for Page Mode Test Using Control Bits" on page 5-6](#))

Address multiplexing control

Address multiplexing must be inhibited to apply only column addresses to DUT in page and page-out cycles.

Address multiplexing can be inhibited by coding M1 and M2 in the same step in the pattern program when multiplex mode (MODE MUX) is specified.((c) in ["Pattern Program for Page Mode Test Using Control Bits" on page 5-5](#))

When M1 and M2 are coded in the pattern program, only pin data B is output as pattern data to pins for which SDM is specified.



When multiplex mode is inhibited by M1 and M2, clock pulses generating row addresses must also be inhibited.

Use the OPEN instruction to inhibit clock pulses.((f) in ["Test Plan Program for Page Mode Test Using Control Bits" on page 5-6](#))

In "Test Plan Program for Page Mode Test Using Control Bits" on page 5-6, clock pulses ACLK2, BCLK2, and CCLK2 are inhibited and ACLK1, BCLK1, and CCLK1 are used to generate column address.

◆ Pattern Program for Page Mode Test Using Control Bits

```

MPAT M256K

    MODE MUX

    REGISTER
        XMAX=#FF
        YMAX=#3FF
        IDX1=#3FFFE

    SDEF INIT=XB<0 YB<0 TP<0 ;clear
    SDEF INC=XB<XB+1 YB<YB+1^BX ;increment base
    SDEF YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

    MODULE BEGIN ;checker board
    START #0

        IDXI2 #6 INIT
    ST0: JN11 ST0 W INC FP1
    ST1: JN11 ST1 R INC FP1
        JZD ST0
        STPS

    MODULE END

    MODULE BEGIN ;page mode

    REGISTER
        LMAX=#3FF
        HMAX=#FF
        IDX1=#1FC
        IDX3=#1FE

    START #10

        IDXI2 #6
    ST0: NOP W TS2 YINC FP1 ;page-in
    ST1: JN11 ST1 W M1 M2 TS3 (b) YINC FP1 ;page
        JN13 ST0 W M1 M2 TS4 YINC FP1 ;page-out
    ST2: NOP R TS2 YINC FP1 ;page-in
    ST3: JN11 ST3 R M1 M2 TS3 YINC FP1 ;page
        JN13 ST2 R M1 M2 TS4 YINC FP1 ;page-out
        JZD ST0
        STPS

    MODULE END

END

```

◆ Test Plan Program for Page Mode Test Using Control Bits

```

PRO P256K      ;Sample program for 256K dynamic RAMs.
               ;page mode

TIME1=1MS :VS1
TIME2=1MS :IN
VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V
PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y9,Y8> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

;Timing set.
TR=2NS & TF=2NS
;      TS1          TS2          TS3          TS4
;      normal      page-in      page      page-out
RATE=  260NS      229NS      150NS      179NS      (d)
ACLK1=  0NS:4
BCLK1=  10NS:4
CCLK1=  10NS+TF+15NS+TR:2      10NS+TF+25NS+TR:2
ACLK2=  57NS:2      (f)      OPEN      OPEN
BCLK2=  10NS+TF+75NS-TR:2      OPEN      OPEN
CCLK2=  10NS+TF+75NS-TR+TF+25NS+TR:2      OPEN      OPEN
ACLK3=  0NS:4
RAS [ BCLK3=  10NS:2      OPEN      OPEN
Clock [ CCLK3=  10NS+TF+150NS      OPEN      (e)      OPEN      10NS+TF+75NS
      BCLK4=  10NS+TF+75NS-TR:2      10NS:2
      CCLK4=  10NS+TF+75NS-TR+TF+75NS:2      10NS+TF+75NS:2
      BCLK5=  10NS+TF+75NS-TR:2      10NS:2
      CCLK5=  10NS+TF+75NS-TR+TF+25NS:2      10NS+TF+25NS:2
      BCLK6=  20NS+TF+75NS-TR:2      10NS:2
      CCLK6=  10NS+TF+75NS-TR+TF+25NS:2      10NS+TF+25NS+TF:2
      STRB1=  10NS+TF+75NS-TR+TF+75NS:2      10NS+TF+75NS:2

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10-11(RESET)?
CALL CALB("C256K",TCAL)

TEST 1
REG MPAT PC=#10
MEAS MPAT M256K

END

```

5. 1. 3 Auto Page Mode Test

Examples ["Auto Page Mode Test Pattern Program" on page 5-10](#) and ["Auto Page Mode Test Plan Program \(1/2\)" on page 5-11](#) show a pattern program and a test plan program, respectively, for the auto page mode test.

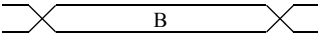
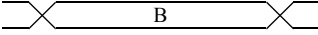
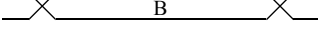
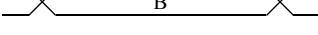
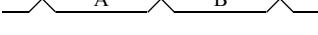
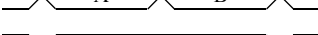
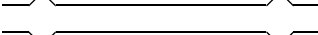
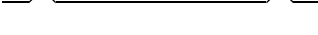
Function

Specify mode as PAGE in the pattern program for the auto page mode test. (*1 in ["Auto Page Mode Test Pattern Program" on page 5-10](#))

ALPG automatically switches timing sets (TSn) and controls address multiplexing with the records of address data (*2 in ["Auto Page Mode Test Pattern Program" on page 5-10](#)) specified in PAGE in the pattern program and control bits R, W, and M1 in auto page mode.

TSn assignment for auto page mode test appears below.

Timing set for auto page mode

TSn	Cycle	Control bit	Pin data
1	MM - N	-	
2	not used	-	
3	MM - REF	M1	
4	not used	-	
5	MM - R	R	
6	MP - R	R	
7	PM - R	R	
8	P - R	R	
9	MM - W	W	
10	MP - W	W	
11	PM - W	W	
12	P - W	W	
13	MM - RW	RW	
14	MP - RW	RW	
15	PM - RW	RW	
16	P - RW	RW	

MM : Multiplex cycle
 MP : Page-in cycle
 PM : Page-out cycle
 P : Page cycle
 REF : Refresh cycle

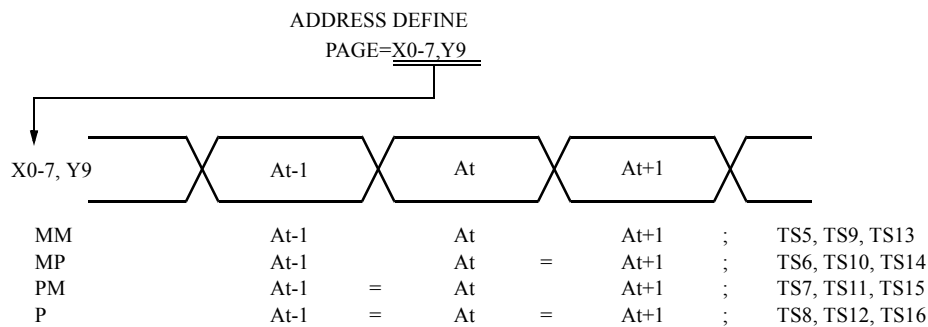
N: Normal cycle (Dummy)
 R: Read cycle
 W: Write cycle
 RW: Read / Write cycle

The correspondence between control bits and TSn is as follows:

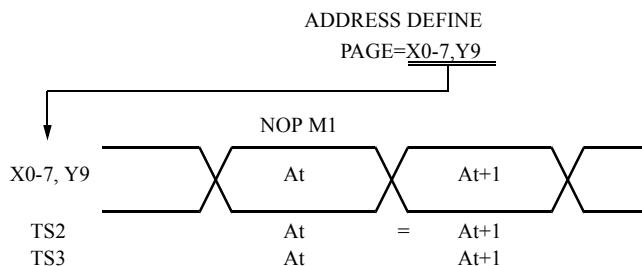
NOP	;TS1	^{*1}	
NOP R	;TS5	TS6 TS7 TS8	*2
NOP W	;TS9	TS10 TS11 TS12	
NOP R W	;TS13	TS14 TS15 TS16	
NOP M1	;TS2	TS3 ^{*3}	
Control bits			

^{*1} TS1 is unconditionally assigned to the cycle for which R, W, or M1 is not specified.

*2 TS_n is automatically switched to the cycle for which either or both R and W is specified, according to the records of addresses specified in $PAGE$ in the pattern program.



*3 $M1$ is specified for the refresh cycle to which $TS2$ and $TS3$ are assigned on the following conditions:



Since row addresses are required, the refresh cycle must be an MM or MP cycle.
The cycle before the refresh cycle is a PM or an MM cycle.

Pattern program

An example of a pattern program for the auto page mode test appears below:

◆ Auto Page Mode Test Pattern Program

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0          ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX      ;increment base
SDEF YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB

MODULE BEGIN                      ;checker board
START      #0

  IDXI2 #6      INIT
ST0:  JN11 ST0 W      INC      FP1
ST1:  JN11 ST1 R      INC      FP1
      JZD ST0
      STPS

MODULE END

MODULE BEGIN                      ;page mode

MODE PAGE      *1

ADDRESS DEFINE
  PAGE=X0-7,Y9      *2

REGISTER
  LMAX=#3FF
  HMAX=#FF

START      #10

  IDXI2 #6      INIT
ST0:  JN11 ST0 W      YINC      FP1
ST1:  JN11 ST1 R      YINC      FP1
      JZD ST0
      STPS

MODULE END

END

```

**1 Specify auto page mode.*

*2 Specify an address for detecting the page cycle in the ADDRESS DEFINE statement. The following data can be specified as the address:

ADDRESS DEFINE

PAGE = $Xl - m, Yn - o$

$0 \leq l, m, n, o \leq 17$ (for T5371, T5336 and T5311, $o \leq 15$)

$l \leq m$

$n \leq o$

Addresses for detecting page cycles are not normally used via a prescrambler. When specifying PAGE using the SELECT PRESCRAM ADDRESS statement, however (available on the T5377, T5376, T5375 and T5371), addresses are used via a prescrambler.

Test plan program

An example of a test plan program for the auto page mode test appears below:

◆ Auto Page Mode Test Plan Program (1/2)

```
PRO P256K ;Sample program for 256K dynamic RAMs.
          ;page mode

TIME1=1MS :VS1
TIME2=1MS :IN

VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y9,Y8> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZ0,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT
```


◆ Auto Page Mode Test Plan Program (2/2)

```

*1      ;timing set
      ;TS1      TS2      TS3      TS4      TS5      TS6      TS7      TS8
      ;MM-N      MP-REF      MM-REF      -      MM-R      MP-R      PM-R      P-R
RATE= 260NS      260NS      260NS      260NS      260NS      229NS      179NS      150NS
ACLK1= 0NS      0NS      0NS      0NS      0NS      0NS      0NS      0NS
BCLK1= 10NS      10NS      10NS      10NS      10NS      10NS      10NS      10NS
CCLK1= 29NS      29NS      29NS      29NS      29NS      29NS      39NS      39NS
ACLK2= 57NS      57NS      57NS      57NS      57NS      57NS      OPEN      OPEN
BCLK2= 85NS      85NS      85NS      85NS      85NS      85NS      OPEN      OPEN
CCLK2= 114NS      114NS      114NS      114NS      114NS      114NS      OPEN      OPEN
ACLK3= 0NS:16
BCLK3= 10NS      10NS      10NS      10NS      10NS      10NS      OPEN      OPEN
CCLK3= 162NS      OPEN      162NS      162NS      162NS      OPEN      87NS      OPEN
BCLK4= 85NS      85NS      85NS      85NS      85NS      85NS      10NS      10NS
CCLK4= 162NS      162NS      162NS      162NS      162NS      162NS      87NS      87NS
BCLK5= 85NS      85NS      85NS      85NS      85NS      85NS      10NS      10NS
CCLK5= 112NS      112NS      112NS      112NS      112NS      112NS      37NS      37NS
BCLK6= 85NS      85NS      85NS      85NS      85NS      85NS      10NS      10NS
CCLK6= 112NS      112NS      112NS      112NS      112NS      112NS      39NS      39NS
STRB1= 162NS      162NS      162NS      162NS      162NS      162NS      87NS      87NS

      ;TS9      TS10      TS11      TS12      TS13      TS14      TS15      TS16
      ;MM-W      MP-W      PM-W      P-W      MM-RW      MP-RW      PM-RW      P-RW
RATE= <9>260NS      229NS      179NS      150NS      260NS      229NS      179NS      150NS
ACLK1=<9>0NS      0NS      0NS      0NS      0NS      0NS      0NS      0NS
BCLK1=<9>10NS      10NS      10NS      10NS      10NS      10NS      10NS      10NS
CCLK1=<9>29NS      29NS      39NS      39NS      29NS      29NS      39NS      39NS
ACLK2=<9>57NS      57NS      OPEN      OPEN      57NS      57NS      OPEN      OPEN
BCLK2=<9>85NS      85NS      OPEN      OPEN      85NS      85NS      OPEN      OPEN
CCLK2=<9>114NS      114NS      OPEN      OPEN      114NS      114NS      OPEN      OPEN
BCLK3=<9>10NS      10NS      OPEN      OPEN      10NS      10NS      OPEN      OPEN
CCLK3=<9>162NS      OPEN      87NS      OPEN      162NS      OPEN      87NS      OPEN
BCLK4=<9>85NS      85NS      10NS      10NS      85NS      85NS      10NS      10NS
CCLK4=<9>162NS      162NS      87NS      87NS      162NS      162NS      87NS      87NS
BCLK5=<9>85NS      85NS      10NS      10NS      85NS      85NS      10NS      10NS
CCLK5=<9>112NS      112NS      37NS      37NS      112NS      112NS      37NS      37NS
BCLK6=<9>85NS      85NS      10NS      10NS      85NS      85NS      10NS      10NS
CCLK6=<9>112NS      112NS      39NS      39NS      112NS      112NS      39NS      39NS
STRB1=<9>162NS      162NS      87NS      87NS      162NS      162NS      87NS      87NS

      STRING TCAL(200) ;Calibration
      TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10-11(RESET)?
      CALL CALB("C256K",TCAL)

TEST      1

      REG MPAT PC=#10
      MEAS MPAT M256K

END

```

*1 Set timing sets TS1 to TS16.

*2 Control the RAS clock for the MP, P, and PM using OPEN.(Clock output is inhibited using OPEN.)
An OPEN instruction can be used only when the waveform mode is independent of pattern data.
(RZX is used in the example.)

*3 As shown in the timing sets for auto page mode, multiplex mode is inhibited and only pin data B is output to pins in the P and PM cycles. Since pin data A is automatically inhibited, inhibit clock pulses ACLK, BCLK, and CCLK, which output pin data A. Inhibit clock pulses by opening the clock using OPEN. In the example, ACLK2, BCLK2, and CCLK2 are inhibited.

5. 1. 4 Auto Page Mode Refresh Test

An example of a pattern program for the auto page mode refresh test appears below:

◆ Pattern Program for Auto Page Mode Refresh Test

```
MPAT M256K
REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0          ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX        ;increment base
SDEF YINC=XB<XB+1 YB<YB+1^BX X<YB Y<XB
SDEF REF=RF<RF+1 X<RF              *8

MODE BEGIN                          ;page mode
                                   ;distributed refresh

MODE PAGE REFRESHA 256
    *1      *2

ADDRESS DEFINE      *3
    PAGE=X0-7,Y9      *3

REGISTER
  LMAX=#3FF
  HMAX=#FF
  TIMER=16US          *4
  ISP=STI              *5

START #0
NOP          T          INIT
            *6
IDX12 #5

ST0:  JN11    ST0    W    YINC    FP1
ST1:  JN11    ST1    R    YINC    FP1
      JZD     ST0
      STPS
STI:  RTN          M1    REF
            *7      *8

MODULE END

END
```

*1 Set auto page mode.

*2 Set refresh mode.

```

      :
REFRESHA 256
      :
      |
      | RF0 to 7 are output to X0 to 7 for the X<RF instruction.
      |
Refresh mode

```

*3 Specify an address for detecting the page cycle in the ADDRESS DEFINE statement.

*4 Timer data setting

*5 Specify the address to which the program branches if an internal interrupt occurs.

*6 Begin internal timer operation.

*7 Specify a refresh cycle by coding control bit M1.

*8 Generate refresh addresses.



Tip

In the pattern program not using the multiplex cycle, timing set of TS5, TS9, or TS13 may be selected depending on the interruption timing by using the internal timer.

5. 1. 5 Auto Page Mode RAS Only Refresh Test

Function

Specify mode as PAGE in the pattern program for the RAS-only refresh test.

```

      :
MODE PAGE RASREF REFRESHA 256
      :
      |
      | RAS-only refresh mode

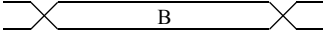
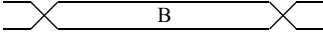
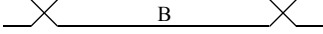
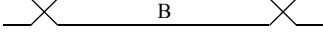
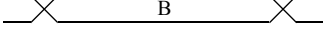
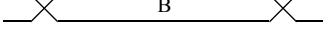
```

The timing set for auto page mode lists TS_n assignments for RAS-only-refresh testing.

TS3 is assigned to the RAS-only-refresh cycle. Multiplex mode is inhibited in the RAS-only-refresh cycle, and only pin data A is output to pins.

➔ Refer to (1) in ["Timing Set for Auto Page Mode" on page 5-15](#)

◆ Timing Set for Auto Page Mode

TSn	Cycle	Control bit	Pin data	
1	MM - N	-		
2	not used	-		
3	MM - REF	M1		(1)
4	not used	-		
5	MM - R	R		
6	MP - R	R		
7	PM - R	R		
8	P - R	R		
9	MM - W	W		
10	MP - W	W		
11	PM - W	W		
12	P - W	W		
13	MM - RW	RW		
14	MP - RW	RW		
15	PM - RW	RW		
16	P - RW	RW		

MM : Multiplex cycle
 MP : Page-in cycle
 PM : Page-out cycle
 P : Page cycle
 REF : Refresh cycle
 RAS : RAS only refresh cycle

N: Normal cycle (Dummy)
 R: Read cycle
 W: Write cycle
 RW: Read / Write cycle

Pattern program

Set the following mode to the MODE in ["Pattern Program for Auto Page Mode Refresh Test" on page 5-13](#).

```
MODE PAGE RASREF REFRESHA 256
```

Test plan program

An example of a test plan program for the auto page mode RAS-only refresh test appears below:

◆ Test Plan Program for an Auto Page Mode RAS Only Refresh Test (1/2)

```

PRO P256K ;Sample program for 256K dynamic RAMs.
          ;page mode+RAS only refresh

TIME1=1MS :VS1
TIME2=1MS :IN

VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y9,Y8> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;DIN
PC33=OUT1,STRB1,<D0> ;DOUT

```

◆ Test Plan Program for an Auto Page Mode RAS Only Refresh Test (2/2)

```

;timing set
;TS1      TS2      TS3      TS4      TS5      TS6      TS7      TS8
;MM-N      -      RAS-REF  -      MM-R      MP-R      PM-R      P-R
RATE=      260NS    260NS    260NS    260NS    260NS    229NS    179NS    150NS
ACLK1=      0NS     0NS     0NS     0NS     0NS     0NS     0NS     0NS
BCLK1=      10NS    10NS    10NS    10NS    10NS    10NS    10NS    10NS
CCLK1=      29NS    29NS    29NS    29NS    29NS    29NS    39NS    39NS
ACLK2=      57NS    57NS    *1 OPEN    57NS    57NS    57NS    OPEN    OPEN
BCLK2=      85NS    85NS    OPEN    85NS    85NS    85NS    OPEN    OPEN
CCLK2=      114NS   114NS   OPEN    114NS   114NS   114NS   OPEN    OPEN
ACLK3=      0NS:16
BCLK3=      10NS    10NS    10NS    10NS    10NS    10NS    OPEN    OPEN
CCLK3=      162NS   OPEN    162NS   162NS   162NS   OPEN    87NS    OPEN
BCLK4=      85NS    85NS    *2 OPEN    85NS    85NS    85NS    10NS    10NS
CCLK4=      162NS   162NS   OPEN    162NS   162NS   162NS    87NS    87NS
BCLK5=      85NS    85NS    85NS    85NS    85NS    85NS    10NS    10NS
CCLK5=      112NS   112NS   112NS   112NS   112NS   112NS    37NS    37NS
BCLK6=      85NS    85NS    85NS    85NS    85NS    85NS    10NS    10NS
CCLK6=      112NS   112NS   112NS   112NS   112NS   112NS    39NS    39NS
STRB1=      162NS   162NS   162NS   162NS   162NS   162NS    87NS    87NS
;TS9      TS10     TS11     TS12     TS13     TS14     TS15     TS16
;MM-W      MP-W     PM-W     P-W      MM-RW     MP-RW     PM-RW     P-RW
RATE= <9>260NS 229NS 179NS 150NS 260NS 229NS 179NS 150NS
ACLK1=<9>0NS 0NS 0NS 0NS 0NS 0NS 0NS 0NS
BCLK1=<9>10NS 10NS 10NS 10NS 10NS 10NS 10NS 10NS
CCLK1=<9>29NS 29NS 39NS 39NS 29NS 29NS 39NS 39NS
ACLK2=<9>57NS 57NS OPEN OPEN 57NS 57NS OPEN OPEN
BCLK2=<9>85NS 85NS OPEN OPEN 85NS 85NS OPEN OPEN
CCLK2=<9>114NS 114NS OPEN OPEN 114NS 114NS OPEN OPEN
BCLK3=<9>10NS 10NS OPEN OPEN 10NS 10NS OPEN OPEN
CCLK3=<9>162NS OPEN 87NS OPEN 162NS OPEN 87NS OPEN
BCLK4=<9>85NS 85NS 10NS 10NS 85NS 85NS 10NS 10NS
CCLK4=<9>162NS 162NS 87NS 87NS 162NS 162NS 87NS 87NS
BCLK5=<9>85NS 85NS 10NS 10NS 85NS 85NS 10NS 10NS
CCLK5=<9>112NS 112NS 37NS 37NS 112NS 112NS 37NS 37NS
BCLK6=<9>85NS 85NS 10NS 10NS 85NS 85NS 10NS 10NS
CCLK6=<9>112NS 112NS 39NS 39NS 112NS 112NS 39NS 39NS
STRB1=<9>162NS 162NS 87NS 87NS 162NS 162NS 87NS 87NS

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12, 33=2V, 1.2V, TCHANGE PD10-11 (RESET) ?
CALL CALB ("C256K", TCAL)

TEST 1
REG MPAT PC=#0
MEAS MPAT M256K

END

```

*1 Since only row addresses are applied to the DUT in the RAS-only-refresh cycle, inhibit clock pulses for generation column addresses.

*2 Inhibit clock pulses (BCLK4 and CCLK4) to set the $\overline{\text{CAS}}$ clock to high in the RAS-only-refresh cycle.

5. 1. 6 Auto Page Mode TRAS Test

The T5300 series memory test system has TRAS timer to define $\overline{\text{RAS}}$ clock pulse width. The TRAS timer enables creation of a pattern program which generates address data which is independent of the $\overline{\text{RAS}}$ pulse which for the page mode (or auto page mode) test.

Function

To use the TRAS timer, set the mode in the pattern program as follows:

MODE PAGE TRAS

Specifying this mode starts TRAS timer operation in the page-in (MP) cycle.

The following data can be specified in TIMER:

TIMER = #8 to #FFF : Specification as number of cycles

TIMER = 800NS to 409.5S : Specification as time

If this mode is specified and the page (P) cycle follows the page-in (MP) cycle, the page cycles are replaced by a page-out (PM) cycle and page-in cycle at timer end.

If a cycle is switched to the page-in cycle during TRAS timer operation, the TRAS timer is restarted with the page-in (MP) cycle.

Figure 5-2 shows timer values and corresponding page, page-in, and page-out cycles.

Pattern program

A program using the TRAS timer appears below:

◆ Pattern Program for TRAS Testing

```

MPAT M26K

REGISTER
  XMAX = #FF
  YMAX = #3FF
  IDX1 = #3FFFE

SDEF INIT = XB<0  YB<0  TP<0      ;clear
SDEF YINC = XB<XB+1  YB<YB+1^BX  X<YB  Y<XB

MODULE BEGIN                                ;page mode
                                           ;TRAS
MODE  PAGE  TRAS
           *1

ADDRESS DEFINE
  PAGE = X0-7,Y9

REGISTER
  LMAX = #3FF
  HMAX = #FF
  TIMER = 100US
           *2

START      #10

  IDXI2 #6          INIT
ST0:  JN11  ST0      W      YINC  FP1
ST1:  JN11  ST1      R      YINC  FP1
      JZD    ST0
      STPS

MODULE END

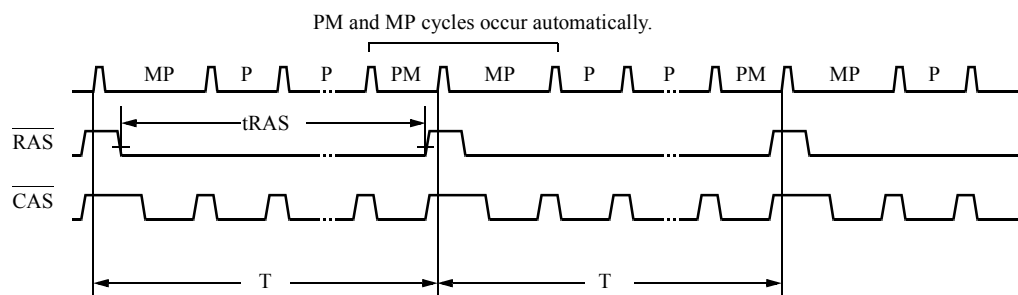
END

```

**1 Set the mode.*

**2 Timer data setting*

The internal timer used for the pause test and refresh test is used as the TRAS timer. (Specification of a T timer start instruction is not necessary.)

Figure 5-2 TRAS Timer Time Chart

```

MODE PAGE TRAS
REGISTER
  TIMER = n
  :

```

Test system	n	T
T5377, T5376 and T5375	#8 to #FFF	n +6 cycles
	8*RATE to 409.5 s	n + (8 cycles to 9 cycles) + (300 ns to 400 ns)
T5371	#8 to #FFF	n +6 cycles
	8*RATE to 409.5 s	n + (4 cycles to 5 cycles) + (300 ns to 400 ns)
T5336 and T5311	#8 to #FFF	n + 9 cycles
	8*RATE to 409.5 s	n + (6 cycles to 8 cycles) + (400 ns to 500 ns)

5. 2 Nibble Mode Test

5. 2. 1 Nibble Mode Test

Most DRAMs include an access mode called the nibble mode to shorten cycle time. In the nibble mode, as shown in [Figure 5-3](#), the storage sequence is: a row address is latched by RAS, a column address is latched by CAS, then the CAS clock pulse is applied with the RAS held low.

The T5336 and T5311 provides the following functions for a nibble mode test:

Register N

The T5336 and T5311 has register N to facilitate nibble address generation for a nibble test.

Register N, a 4-bit counter, enables the following calculations:

$N < N$: Hold

$N < NH$: The contents of register NH are loaded into register N.

$N < N+1$: Increment

Addresses generated in register N can be output to any address from X4 to X15 and from Y4 to Y15 by the following statement in the pattern program:

```
ADDRESS DEFINE
```

```
Xl - m = Nn - o 4 ≤ l, m ≤ 15, 1 ≤ m
```

```
Yl - m = Nn - o 0 ≤ n, o ≤ 3, n ≤ o
```

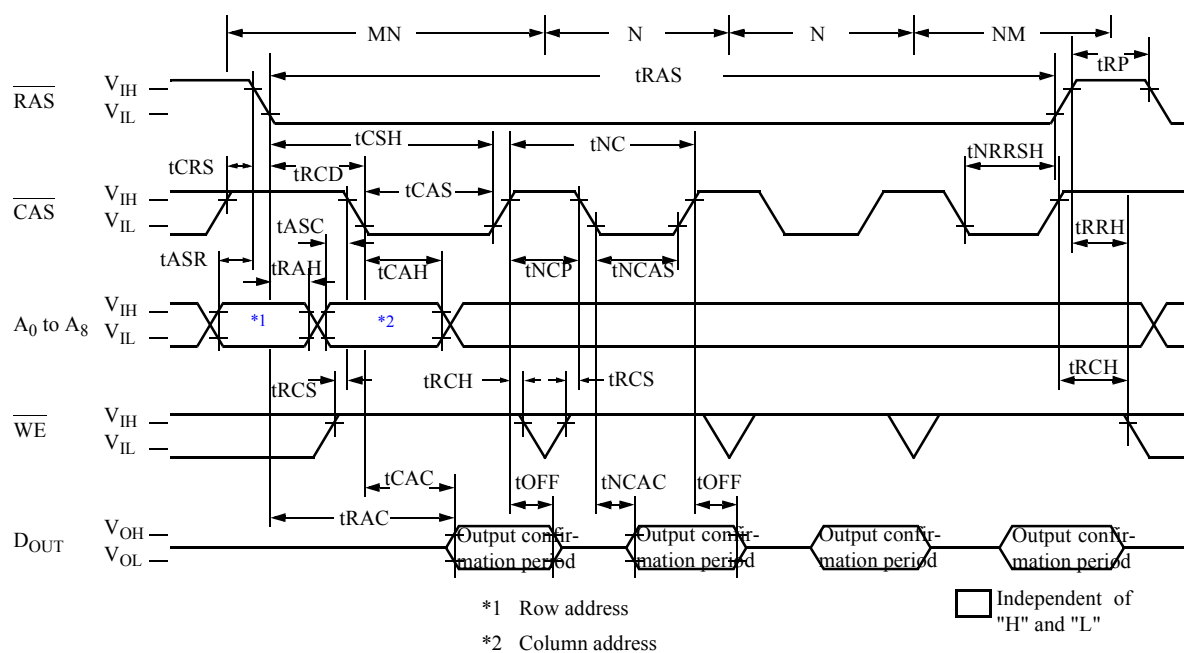
♦ Example

```
ADDRESS DEFINE
```

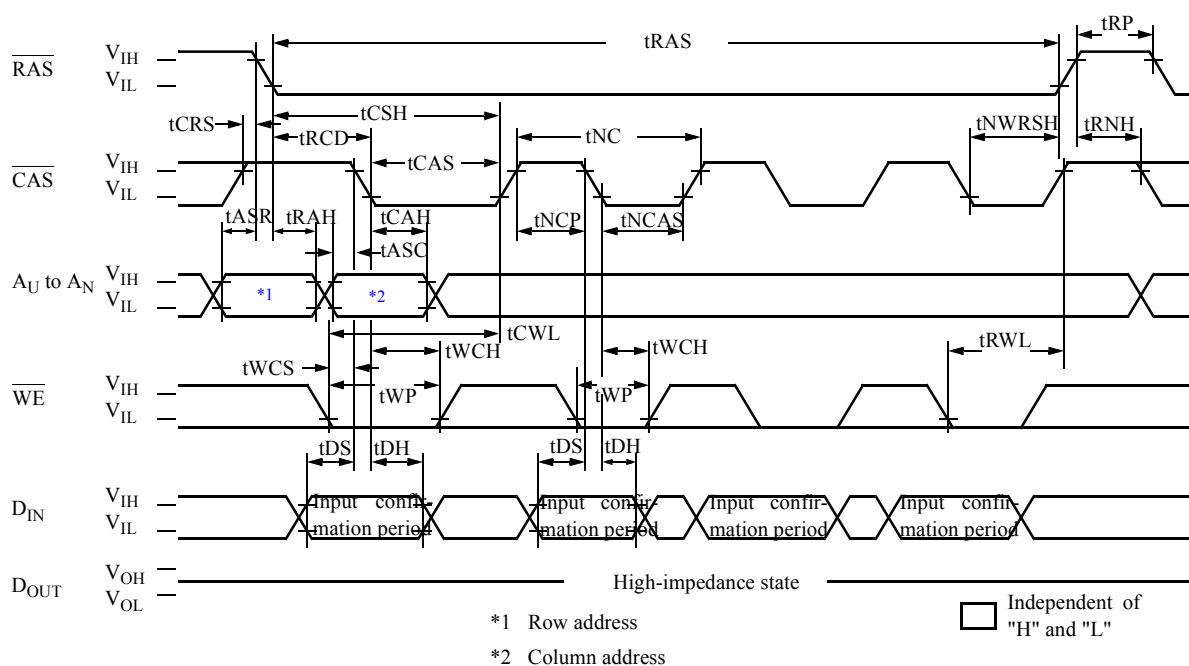
```
Y8 - 9 = N0 - 1
```

Figure 5-3 Nibble Mode Time Chart

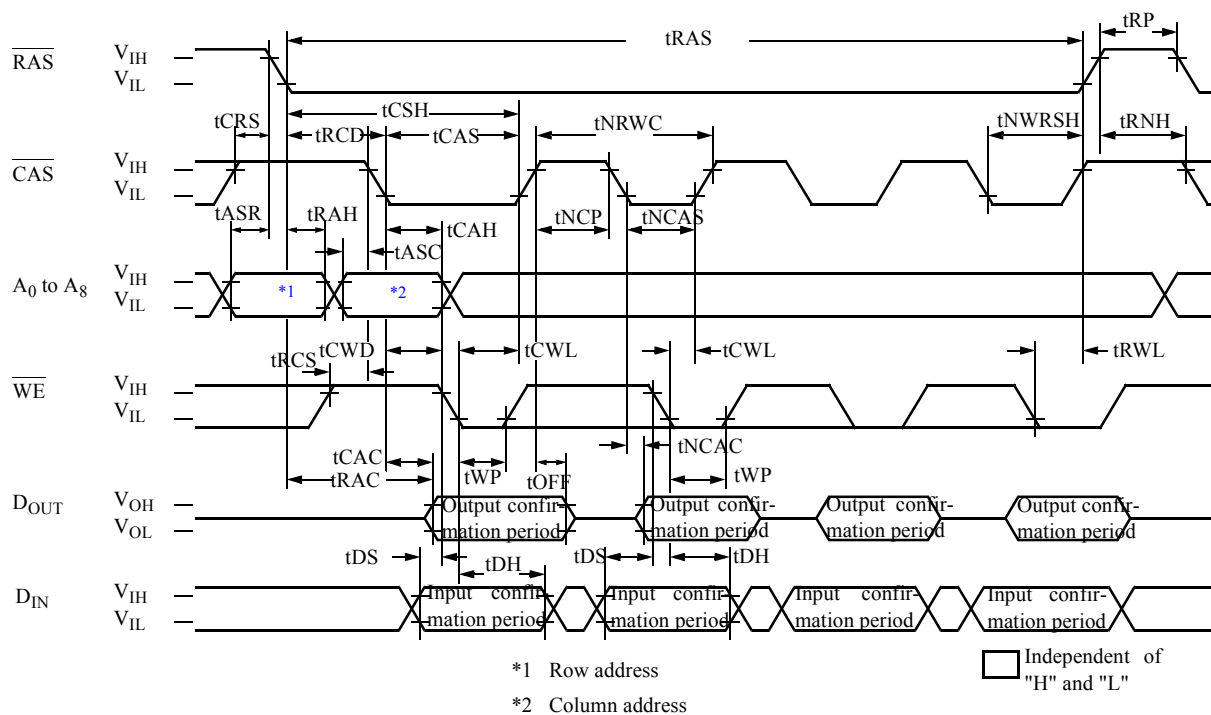
Nibble mode read cycle



Nibble mode write cycle (early write)



Nibble mode read write cycle



Nibble mode

The T5336 and T5311 has an operation mode called the nibble mode to simplify program creation for the nibble mode test.

Specify NIBBLE in MODE in the pattern program for setting nibble mode.

MODE NIBBLE

Nibble mode setting

If nibble mode is specified, the following operation starts:

- In the step in the pattern program in which the control bit N is coded, operation is repeated as many times as is specified in RC (Repeat Counter) in the REGISTER statement.

```

MPAT P256KN
MODE  NIBBLE
REGISTER
  RC = n (n = #1 to #FFF)
  :
  NOP N  XB<XB+1  N<N+1
  :
END

```

Operation instructions except one for N register are inhibited.

```

  NOP  N<N+1
  NOP  N<N+1
  :
  NOP  N<N+1
  NOP  XB<XB+1  N<N+1

```

← n + 1 times

If N is coded in steps, the calculation instruction is repeated $n + 1$ times only for register N, and other operation instructions are executed in the last cycle.

If a conditional branch instruction and control bit N are coded in the same step, the conditional branch instruction is executed in the last cycle.

```

REGISTER
  RC = #3
  :
ST0:  JN11  ST0  N  XB<XB+1  N<N+1
  :
ST0:  NOP          N<N+1
      NOP          N<N+1
      NOP          N<N+1
      JN11  ST0    XB<XB+1  N<N+1

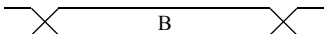
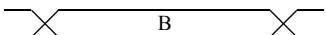
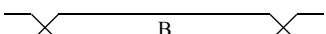
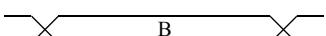
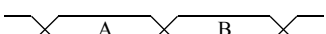
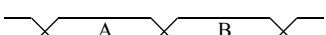
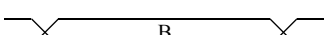
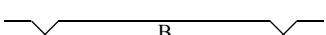
```

Executed in the last cycle.

- Timing set (TSn) is automatically switched by control bits N, I, R, W, and M1 coded in the pattern program.

➔ Refer to ["Timing Set for Auto Nibble Mode" on page 5-25](#).

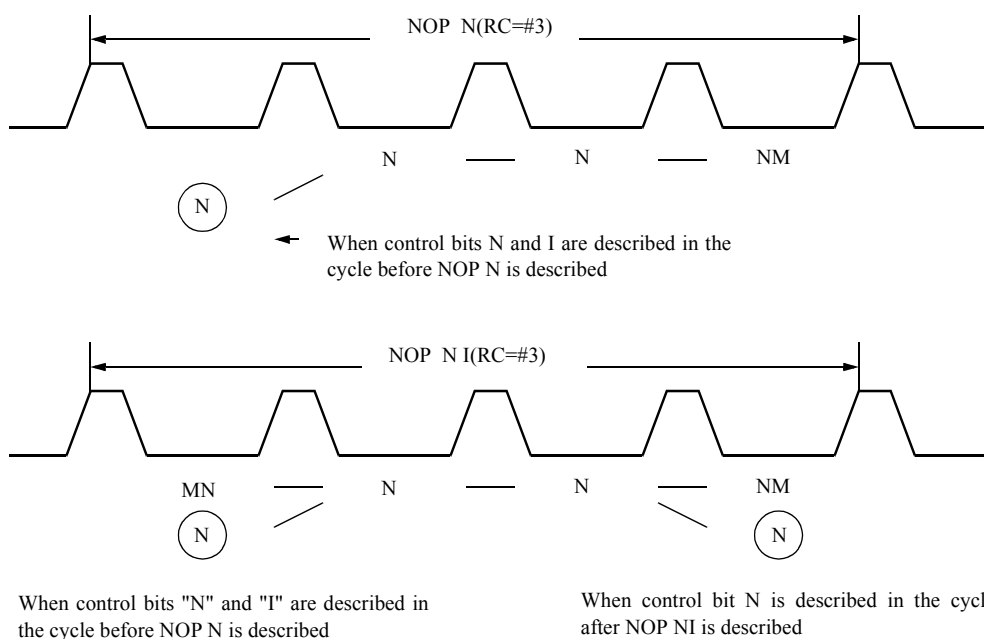
◆ Timing Set for Auto Nibble Mode

TSn	Cycle	Control bit	Pin data
1	MM - N	-	
2	not used	-	
3	MM - REF	M1	
4	not used	-	
5	MM - R	R	
6	MN - R	N, (I), R	
7	MN - R	N, (I), R	
8	N - R	N, (I), R	
9	MM - W	W	
10	MN - W	N, (I), W	
11	NM - W	N, (I), W	
12	N - W	N, (I), W	
13	MM - RW	R, W	
14	MN - RW	N, (I), R, W	
15	NM - RW	N, (I), R, W	
16	N - RW	N, (I), R, W	

MM: Multiplex cycle
 MN: Nibble-in cycle
 NM: Nibble-out cycle
 N: Nibble cycle
 REF: Refresh cycle

N: Dummy cycle
 R: Read cycle
 W: Write cycle
 RW: Read / Write cycle

MN, NM, and N are cycles in the nibble mode which are automatically switched by coding control bit N or I.



TS1 is a dummy cycle, indicating the cycle in which control bits N, I, R, W, or M1 are not coded.

TS3 is a refresh cycle, indicating the cycle in which only control bit M1 is coded.

5. 2. 2 Nibble Mode Test Program

An example of a nibble mode test pattern program appears below:

◆ Pattern Program for a Nibble Mode Test

```

MPAT M256K

REGISTER
  XMAX=#FF
  YMAX=#3FF
  NH=#0*7
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 N<NH*7 TP<0      ;clear
SDEF INC =XB<XB+1 YB<YB+1^BX      ;increment base
SDEF NINC=XB<XB+1 YB<YB+1^BX N<N+1*8

MODULE BEGIN                                ;checker board

MODE MUX

START   #0

  IDXI2 #6      INIT
ST0:    JN11 ST0 W      INC      FP1
ST1:    JN11 ST1 R      INC      FP1
  JZD ST0
  STPS

MODULE END

MODULE BEGIN                                ;nibble mode
                                           ;checker board
MODE NIBBLE*1

ADDRESS DEFINE
  Y8-9=N0-1*2

REGISTER
  YMAX=#FF*6
  RC=#3*3
  IDX3=#FFFE

START   #0

  IDXI2 #6      INIT
ST0:    JN13 ST0 

|   |                 |
|---|-----------------|
| N | W <sup>*4</sup> |
| N | R               |



|                    |
|--------------------|
| NINC <sup>*5</sup> |
| NINC               |

      FP1
ST1:    JN13 ST1 

|   |   |
|---|---|
| N | R |
| N | R |



|                    |
|--------------------|
| NINC <sup>*5</sup> |
| NINC               |

      FP1
  JZD ST0
  STPS

MODULE END

END

```

*1 Set the mode for nibble mode test.

*2 Set address bits to which the contents of register N are output.

*3 Data settings for the repeat counter

$RC = n \quad n = \#1 \text{ to } \#FFF$

**4 Write control bits.*

The cycle for which N is specified is repeated n + 1 times (n: number of times set in RC).

```

ST0:  JN13ST0    N  W  XB<XB+1  YB<YB+1^BX  N<N+1
      ↓
ST0:  NOP        N  W  XB and YB operations are inhibited.  N+1
      NOP        N  W
      NOP        N  W
      JN13 ST0    N  W  XB<XB+1  YB<YB+1^BX  N<N+1

```

Branch instruction is executed in the last cycle.

TSns listed in "Timing Set for Auto Nibble Mode" on page 5-25 are assigned to the cycles for which N, R, or W is coded.

**5 Nibble address generation is performed by the N register.*

Addresses generated by the N register are logically summed and output to any bits from X4 to X7 and Y4 to Y7 specified in the ADDRESS DEFINE statement.

**6 Since the contents of register N are ORed with other signals and output to any bits from X4 to X7 and Y4 to Y7 specified in the ADDRESS DEFINE statement, address bits to which the contents register N are output must be masked by XMAX and YMAX.*

Or, specify XY=ZN in the ADDRESS DEFINE statement.

ADDRESS DEFINE

Y8-9=N0-1

XY=ZN

:

ST0 : JN13 ST0 N W XB<XB+1 YB<YB+1^BX N<N+1 (a)

ST1 : JN13 ST1 N R XB<XB+1 YB<YB+1^BX N<N+1 (b)

:

#100 is contained in register YB when program (a) terminates.

*Since #100 is contained in register YB when step *1 terminates in program (b), 1 is output to Y8.*

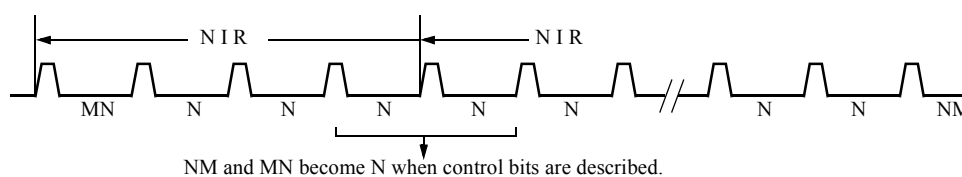
Invalid data output to Y8 by the YB operation is masked by YMAX in the pattern program.

**7 Initialize register N.*

**8 Operation instructions in register N*

To repeatedly access only nibble addresses, code the following instruction in the pattern program:

```
RC=#3
:
IDXI4  #F  N I R  N<N+1
:
```



The method for changing the RC value is also used.

An example of a nibble mode test pattern program appears below:

◆ **Nibble Mode Test Plan Program (1/2)**

```
PRO P256K ;Sample program for 256K dynamic RAMs.
; nibble mode

TIME1=1MS :VS1
TIME2=1MS :IN

VS1=5V
IN1=2.4V,0.8V
OUT1=2.4V,0.4V

PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7> ;A0-7
PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9> ;A8
PD10=IN1,/RZX,BCLK3,CCLK3 ;/RAS
PD11=IN1,/RZX,BCLK4,CCLK4 ;/CAS
PD12=IN1,/RZO,BCLK5,CCLK5,<WT> ;/WE
PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,<D0> ;/DIN
PC33=OUT1,STRB1,<D0> ;DOUT
```

◆ Nibble Mode Test Plan Program (2/2)

```

;timing set
;TS1      TS2      TS3      TS4      TS5      TS6      TS7      TS8
;MM-N      -      MM-REF    -      MM-R      MN-R      NM-R      N-R
RATE=      260NS    260NS    260NS    260NS    260NS    184NS    164NS    80NS
ACLK1=      0NS     0NS     0NS     0NS     0NS     0NS     0NS     0NS
BCLK1=      10NS    10NS    10NS    10NS    10NS    10NS    10NS    10NS
CCLK1=      29NS    29NS    29NS    29NS    29NS    29NS    39NS    39NS
ACLK2=      57NS    57NS    57NS    57NS    57NS    57NS    OPEN    OPEN *3
BCLK2=      85NS    85NS    85NS    85NS    85NS    85NS    OPEN    OPEN
CCLK2=      114NS   114NS   114NS   114NS   114NS   114NS   OPEN    OPEN
ACLK3=      0NS:16
BCLK3=      10NS    10NS    10NS    10NS    10NS    10NS    OPEN    OPEN *2
CCLK3=      162NS   162NS   162NS   162NS   162NS   162NS   72NS    OPEN
BCLK4=      85NS    85NS    85NS    85NS    85NS    85NS    10NS    10NS
CCLK4=      162NS   162NS   162NS   162NS   162NS   162NS   52NS    52NS
BCLK5=      85NS    85NS    85NS    85NS    85NS    85NS    10NS    10NS
CCLK5=      112NS   112NS   112NS   112NS   112NS   112NS   37NS    37NS
BCLK6=      85NS    85NS    85NS    85NS    85NS    85NS    10NS    10NS
CCLK6=      112NS   112NS   112NS   112NS   112NS   112NS   39NS    39NS
STRB1=      162NS   162NS   162NS   162NS   162NS   162NS   52NS    52NS
;TS9      TS10     TS11     TS12     TS13     TS14     TS15     TS16
;MM-W      MN-W     NM-W     N-W      MM-RW    MN-RW    NM-RW    N-RW
RATE= <9>260NS 184NS 164NS 80NS 260NS 184NS 164NS 80NS
ACLK1=<9>0NS 0NS 0NS 0NS 0NS 0NS 0NS 0NS
BCLK1=<9>10NS 10NS 10NS 10NS 10NS 10NS 10NS 10NS
CCLK1=<9>29NS 29NS 39NS 39NS 29NS 29NS 39NS 39NS
ACLK2=<9>57NS 57NS OPEN OPEN 57NS 57NS OPEN OPEN
BCLK2=<9>85NS 85NS OPEN OPEN 85NS 85NS OPEN OPEN
CCLK2=<9>114NS 114NS OPEN OPEN 114NS 114NS OPEN OPEN
BCLK3=<9>10NS 10NS OPEN OPEN 10NS 10NS OPEN OPEN
CCLK3=<9>162NS OPEN 72NS OPEN 162NS OPEN 72NS OPEN
BCLK4=<9>85NS 85NS 10NS 10NS 85NS 85NS 10NS 10NS
CCLK4=<9>162NS 162NS 52NS 52NS 162NS 162NS 52NS 52NS
BCLK5=<9>85NS 85NS 10NS 10NS 85NS 85NS 10NS 10NS
CCLK5=<9>112NS 112NS 37NS 37NS 112NS 112NS 37NS 37NS
BCLK6=<9>85NS 85NS 10NS 10NS 85NS 85NS 10NS 10NS
CCLK6=<9>112NS 112NS 39NS 39NS 112NS 112NS 39NS 39NS
STRB1=<9>162NS 162NS 52NS 52NS 162NS 162NS 52NS 52NS
]

STRING TCAL(200) ;Calibration
TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10-11(RESET)?
CALL CALB("C256K",TCAL)

TEST 1
REG MPAT PC=#0
MEAS MPAT M256K

END

```

*1 Set timing.

Write a timing set for each nibble mode cycle.

*2 Inhibit the RAS clock pulse in the MN, N, and NM cycles by using the OPEN instruction.

*3 Since multiplex mode is inhibited in the N and NM cycled, inhibit the clock pulsed for address waveform generation. Address data is not usually required in the N and NM cycles, but column addresses are used in this example.

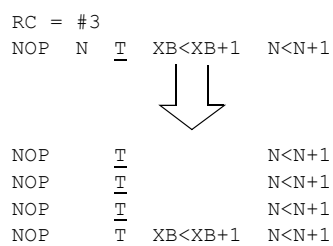
MN : Nibble in cycle.

N : Nibble cycle.

NM : Nibble out cycle.

! Important

- Control bits T and O are validated in the following cycle:



T is validated in all cycles.

- The RC value is from #1 to #FFF.
- Control bit N cannot be specified in the STPS and STFL instructions.
- The repeat counter (RC) data set in the STRC instruction is validated in the following cycle:

STRC	m
NOP	
NOP	N

 ; m is validated in this cycle.

5. 2. 3 Nibble Mode Refresh Test Program

An example of a pattern program for a refresh test in nibble mode appears below.

◆ Pattern Program for a Nibble Mode Refresh Test

```

MPAT M256K                      ;sample program for 256K dynamic RAMs.

REGISTER
  XMAX=#FF
  YMAX=#3FF
  NH=#0
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 N<NH TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX        ;increment base
SDEF REF=X<RF RF<RF+1
SDEF NINC=XB<XB+1 YB<YB+1^BX N<N+1

MODULE BEGIN                      ;nibble mode
                                ;checker board

MODE NIBBLE  REFRESHA 256*1

ADDRESS DEFINE
  Y8-9=N0-1

REGISTER
  YMAX=#FF
  RC=#3
  IDX3=#FFFE
  TIMER=16US*2
  ISP=STI*3

START  #10

NOP          T*4          INIT
IDX12  #5

ST0:  JN13    ST0    N    W    NINC    FP1
ST1:  JN13    ST1    N    R    NINC    FP1
      JZD      ST0
      STPS
STI:  RTN          M1*5    REF*6

MODULE END

END

```

*1 Set refresh mode.

REFRESHA 256



Specification of the bit width of the RF register output to X
during execution of $X < RF$ instruction.

$X0-7 < RF0-7$ is output.

Additionally, 64, 128, 512, 1024, 2048, 4096, 8192, 16384, 32768, and 65536 can also be described.

*2 Set the refresh interval by setting timer data.

TIMER = 200NS to 409.5S

In this example, $4\text{ ms}/256 = 16\text{ }\mu\text{s}$ is set, because it is a distributed refresh test.

*3 Specify the branch destination in the program for an internal interrupt.

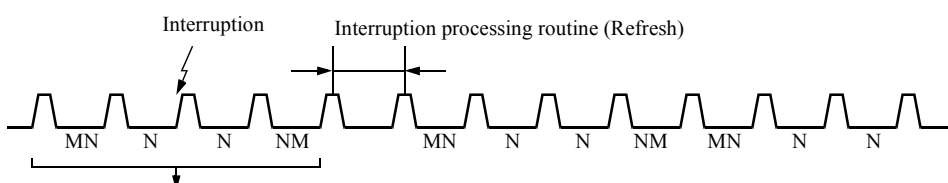
*4 Begin timer operation.

*5 Write code control bit M1 for refresh cycle.

TS3 is assigned as the refresh cycle timing set.

*6 Generate refresh addresses.

If an internal interrupt (timer end) occurs in the MN or N cycle, the interrupt processing routine is executed when a series of nibble cycles are terminated.



Interruptions generated during this time are not executed until the NM cycle terminates.

IDXIn command is not available in steps where both timer interruption processing (e.g. refresh mode) and control bit "N" are used.

Use JNIn or other command.

5. 3 Test for Multiple-Bit Memory Devices

5. 3. 1 Data Generation

Data can be generated with register TP and the inversion function.

Pattern programs for multiple-bit configuration memory testing appears below:

◆ Pattern Program for Multiple-Bit Memory Testing

```
MPAT M16KS

REGISTER
  XMAX=#7F
  YMAX=#F
  TPH=#55 *2
  IDX1=#7FE

SDEF INIT=XB<0 YB<0 TP<TPH *1 ;initialize reg.
SDEF INC=XB<XB+1 YB<YB+1^BX ;increment base

MODULE BEGIN ;checker board
START #0

NOP          INIT
ST0: JN11 ST0 W INC FP1 ] *3
ST1: JN11 ST1 R INC FP1 ]
      JZD ST0
      STPS

MODULE END

MODULE BEGIN ;marching
START #10

NOP          INIT
ST0: JN11 ST0 W INC
ST1: NOP      R
      JN11 ST1 W INC /D /D *4
ST2: NOP      R /X /Y /D
      JN11 ST2 W INC /X /Y
      JZD ST0
      STPS

MODULE END

END
```

*1 Register TP initialization

Initialize register TP with the following instructions:

TP < 0 : Clear

TP < TPH : Loads the contents of register TPH

TP < D5 : Loads the contents of register D5

**2 Register TRH initialization for register TP initialization*

Register TPH can also be initialized with the following instruction in the test plan program:

```

      :
REG MPAT TPH = #55
      :

```

**3 Data inversion with the address function*

Invert the contents of register TP according to the condition indicated by the FPN address function to output it to DA0 to DA17 (DB0 to DB17).

**4 Data inversion with the /D instruction*

Invert the contents of register TP with the /D instruction to output it to DA0 to DA17 (DB0 to DB17). Bits to be inverted can be specified by the DCMR.

Data for testing multiple-bit memory chips can be generated by combining operation instructions in register TP (below) and the above inversion function.

TP<TP+1

TP<TP-1

TP</TP

*TP<TP*2*

TP < TP'D5

TP < TP&D5

TP < TP"D5

TP<TP+D5

TP<TP-D5

5. 3. 2 I/O Control

In memory consisting of multiple-bit chips, both data input and output use the same pins. Example below shows a test plan program for I/O device testing.

◆ Multiple-Bit Configuration Memory Test Plan Program

```

PRO P16KS                      ;sample program for 2K*8 static RAMs.
    TIME=1MS:VS1
    TIME=1MS:IN

    VS1=5V
    IN1=2.4V,0.6V
    OUT1=2.2V,0.8V

    PD1-4=IN1,NRZA,ACLK1,<Y0-3>      ;A0-3
    PD5-11=IN1,NRZA,ACLK1,<X0-6>     ;A4-11
    PD12=IN1,/RZO,BCLK1,CCLK1,<WT>   ;/WE
    PD13=IN1,/NRZA,ACLK1,<FH>        ;/OE
    PD14=IN1,/NRZA,ACLK1,<FH>        ;/CS
    P33-40=IN1,OUT1,IOC,XOR,ACLK2,BCLK2,DRERZ,(DRE1,DRECLK1)STRB1,<D0-7>; I/O *1
                                           *2

    RATE=200NS
    ACLK1=0NS
    BCLK1=20NS
    CCLK1=160NS
    ACLK2=0NS
    BCLK2=100NS
    CCLK2=164NS
    DREL1=90NS      *2
    DRET1=170NS     *2
    STRB1=200NS

    STRING TCAL(100)
    TCAL=?DRREF PD1-14,33-40=2.2V,0.8V TSELECT PD1-11(A-SET,A-RESET)?
    CALL CALB("C16KS1",TCAL)

TEST 1
    REG MPAT PC=#0
    MEAS MPAT M16KS

END

```

*1 Specify I/O pins.

*2 Specify timing in which write data is input to the device.

The procedure for cycle specification is as follows:

- Use the REG statement in the pattern program.

```
MPAT filename
REGISTER
:
DRE=n      (n = FIXH, FIXL, R, W, M1-2, C0-23)
:
```

n is a control bit.

FIXH, *FIXL*, *R* and *W* can also be written with *FH*, *FL*, *RD* and *WT*, respectively.

Write data is input to the chip in the cycle for which a control bit is specified as *n*.

When no specified, the default is *DRE = W*.

- Use the REG statement in the test plan program.

```
:
REG MPAT DRE = n      (n = FL, FH, RD, WT, M1-2, C0-23)
:
```

The function has a 2-way I/O control of *DRE1* and *DRE2*.

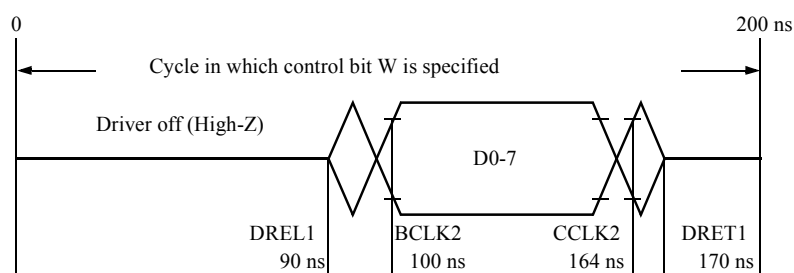
Specify *DRE1* or *DRE2* as a register name. For default, *DRE1* is selected.



Tip

DRE and *DRE1* have a same meaning.

In this example, the following data is input to the DUT in the cycle for which control bit *W* is specified in the pattern program.



5. 3. 3 Generation of Parity Data

The memory of the X9 configuration is convenient for the use of the parity function. It can be tested with the parity generating function of the tester.

Pattern program

An example of a pattern program for multiple-bit memory testing using the parity generating function appears below:

◆ Multi-Bit Configuration Memory Test Plan Program Using Parity Generating Function

```
MPAT      P72K      ; SAMPLE PROGRAM FOR 8K*9 SRAM

MODE PARITY *1
REGISTER
  XMAX=#FF
  YMAX=#1F
  TPH=#FF
  IDX1=#1FFE
  IDX2=#7
  DRE1=W
  CPE1=R

SDEF ACLR=XB<0 YB<0
SDEF AINC=X<XB Y<YB XB<XB+1 YB<YB+1^BX
SDEF AHLD=X<XB Y<YB

START #0
NOP      ACLR  TP<TPH
ST0:     JN11  ST0  W  AINC
ST1:     NOP   R   AHLD
          JN11  .   W  AINC  /D
          JN12  ST1  ACLR  TP<TP*2 *2
STPS

END
```

*1 Set a mode for generation parity.

Parity of D0 to D7 interrupts D16.

Parity of D8 to D15 interrupts D17.

The parity referred to is odd parity. Data for D16 (or D17) is generated such that the sum of bits D0 to D7 and D16 (or D8 to D15, D17) is odd.

*2 In this example, testing is performed by changing D0 to 7 to #FF → #7F → #3F → #1F → #F → #7 → #3 → #1 and matching patterns.

Test plan program

An example of a test plan program for multiple-bit memory testing using the parity generating function appears below:

◆ Multi-Bit Configuration Memory Test Plan Program Using Parity Generating Function

```

PRO      M72K      ;   SAMPLE PROGRAM FOR 8K*9 SRAM

      TIME1=1MS :VS1
      TIME2=1MS :IN

      VS1=5V
      IN1=2.2V,0.8V
      OUT1=2.4V,0.4V

      PD1-8 =IN1,XOR,ACLK1,BCLK1,CCLK1,<X0-7>           ;ADDRESS
      PD9-13=IN1,XOR,ACLK1,BCLK1,CCLK1,<Y0-4>           ;ADDRESS
      PD14  =IN1,/RZO,BCLK2,CCLK2,<WT>                 ;/WE
      PD15  =IN1,/NRZA,ACLK2,<RD>                       ;/OE
      PD16  =IN1,FIXL                                   ;/CS1
      PD17  =IN1,FIXH                                   ; CS2
      P33-40=IN1,XOR,ACLK3,BCLK3,CCLK3,OUT1,STRB1,IOC,DRERZ,<D0-7>
      P41   =IN1,XOR,ACLK3,BCLK3,CCLK3,OUT1,STRB1,IOC,DRERZ,<D16> *1

      RATE =65NS
      ACLK1= 0NS & BCLK1=20NS & CCLK1=55NS             ;ADDRESS
      ACLK2= 0NS                                       ;OE
      BCLK2=32NS & CCLK2=52NS                         ;/WE
      ACLK3= 0NS & BCLK3=35NS & CCLK3=52NS             ;IO1-9
      STRB1=55NS
      DREL1=25NS
      DRET1=62NS

      CALL CALB("C72K",NORMAL)

TEST    1

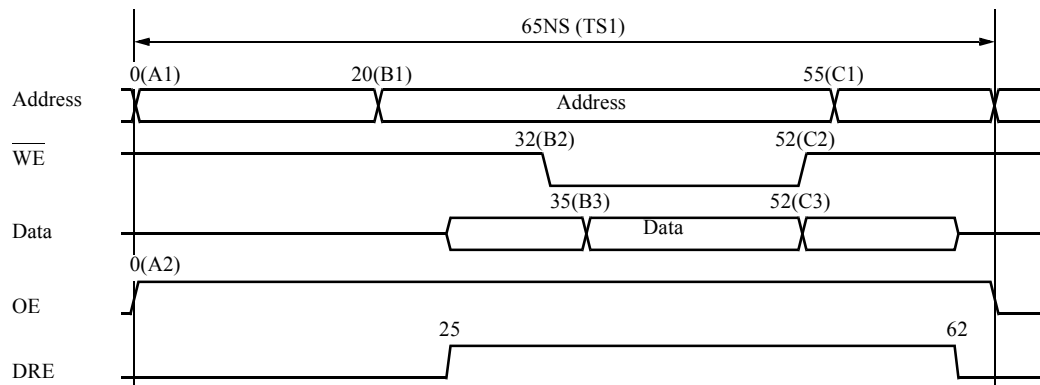
      REG MPAT PC=#0
      MEAS MPAT P72K

      END

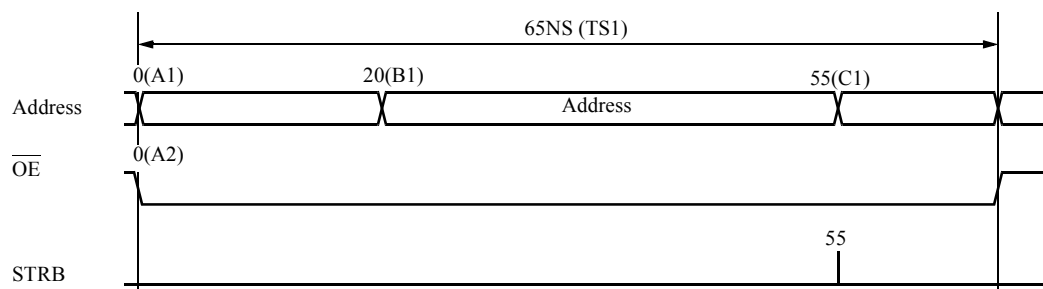
```

**1 Allocate parity bit D16 to the data bit of the device.*

Write cycle



Read cycle



5. 4 Read Modify Write Test

5. 4. 1 Read/Write Data Inversion

Since both a read and a write operation must be done in one memory cycle in a read modify write test, write data and expected data must be generated separately. The write data and expected data can be inverted, with the following data inversion function:

RINV

Pattern data of a driver pin for which RINV is specified in the PIN statement is inverted in the cycle for which the control bit, selected by RINV in the pattern program, is specified.

```
MPAT  filename
REGISTER
    RINV = C0      ; Select a control bit.
    :              FIXH, FIXL, R, W, M1, M2 and C0-23 can be specified.
    NOP  C0        ; D0 of PD32 is inverted at a cycle describing the control bits,
    :              specified by RINV.
    :              D0 of PD34 is inverted.
```

```
    :
    PD34 = IN1, XOR, ACLK1, BCLK2, RINV,< D0 >
    :
```

5. 4. 2 Programming

In following examples, read modify write test is performed with marching pattern.

◆ Read Modify Write Test Pattern Program

```

MPAT M256K

MODE MUX

REGISTER
  XMAX=#FF
  YMAX=#3FF
  IDX1=#3FFFE

SDEF INIT=XB<0 YB<0 TP<0      ;clear
SDEF INC=XB<XB+1 YB<YB+1^BX   ;increment base

MODULE BEGIN      ;RMW march.

REGISTER
  RINV=C0 *1

START      #0

  IDXI2 #6      INIT
ST0:  JN11 ST0 W      INC
ST1:  JN11 ST1 R W C0 *1 INC
ST2:  JN11 ST2 R W C0  INC      /X /Y /D
      JZD ST0
      STPS

MODULE END

END

```

*1 Specify a cycle in which a driver pattern is inverted.

In example above, the driver pattern is inverted in the cycle during which control bit C0 is specified.

◆ Read Modify Write Test Plan Program

```

PRO P256K                ;Sample program for 256K dynamic RAMs.
  TIME1=1MS :VS1
  TIME2=1MS :IN

  VS1=5V
  IN1=2.4V,0.8V
  OUT1=2.4V,0.4V

  PD1-8=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<X0-7,Y0-7>      ;A0-7
  PD9=IN1,XOR,ACLK1,BCLK1,CCLK1,SDM,<Y8,Y9>            ;A8
  PD10=IN1,/RZX,BCLK3,CCLK3                            ;/RAS
  PD11=IN1,/RZX,BCLK4,CCLK4                            ;/CAS
  PD12=IN1,/RZO,BCLK5,CCLK5,<WT>                      ;/WE
  PD34=IN1,XOR,ACLK3,BCLK6,CCLK6,RINV*1,<D0>          ;DIN
  PC33=OUT1,STRB1,<D0>                                ;DOUT

  TR=2NS & TF=2NS                                     ;Timing set
  RATE=260NS
  ACLK1=0NS
  BCLK1=10N
  CCLK1=10NS+15NS+TF+TR
  ACLK2=35NS
  BCLK2=45NS
  CCLK2=45NS+25NS+TF+TR
  ACLK3=0NS
  BCLK3=10NS
  CCLK3=10NS+TF+150NS
  BCLK4=45NS
  CCLK4=45NS+TF+85NS
  BCLK5=45NS+TF+25NS                                  *2
  CCLK5=45NS+TF+25NS+TF+25NS                          *2
  BCLK6=45NS+TF+25NS                                  *2
  CCLK6=45NS+TF+25NS+TF+25NS+TR                        *2
  STRB1=45NS+TF+75NS

  STRING TCAL(200)                                     ;Calibration
  TCAL=?DRREF PD1-12,33=2V,1.2V,TCHANGE PD10-12(RESET)?
  CALL CALB("C256K",TCAL)

TEST 1
  REG MPAT PC=#0
  MEAS MPAT M256K
END

```

**1 Specify RINV in the pin statement.*

**2 Set timing for read modify write cycle.*

5. 5 EPROM Test

5. 5. 1 Using the Match Flag Sense Instruction

The T5300 series memory test system provides flag sense instructions which change the program sequence according to the results of logical comparison (Pass or Fail), to facilitate EPROM testing.

[Table 5-2](#) shows flag sense instructions.

FLG1 indicates the result of logical comparison. It is set under the following conditions:

```

:
PC33-40=OUT1, STRB1, MATCH, <D0 - 7>
:

```

The output data is compared at the STRB1 pulse, in the cycle in which a match flag sense instruction is specified. If the result for a pin for which MATCH is specified in the PIN statement is pass, FLG1 = 1.

If MATCH is specified for several pins, FLG = 1 if the results for all the applicable pins are pass.

Operation period (RATE) for a match flag sense instruction is as follows:

Table 5-1 RATE Setting Conditions for the Cycle in which the Match Flag Sense Instruction is Written

Test system	AFM ACTION statement	AFM RATE statement	RATE setting condition
T5377	No specification of the mask mode	HS	RATE ≥ STRB1 set value + 1.0 μs
		MS	RATE ≥ STRB1 set value + 1.4 μs
		LS	RATE ≥ STRB1 set value + 1.7 μs
	With the specification of the mask mode	HS	RATE ≥ STRB1 set value + 1.6 μs
		MS	RATE ≥ STRB1 set value + 2.3 μs
		LS	RATE ≥ STRB1 set value + 3.8 μs
T5376 and T5375	---	HS, MS	RATE ≥ STRB1 set value + 1.0 μs
		LS	RATE ≥ STRB1 set value + 1.4 μs
T5371	---	---	RATE ≥ STRB1 set value + 1.0 μs
T5336 and T5311	---	---	RATE ≥ STRB1 set value + 1.4 μs

--- in the table shows that the relevant test systems are independent of the specification by the AFM ACTION statement and the AFM RATE statement.

In Example of ["EPROM Test Pattern Program" on page 5-46](#), an INCn instruction is not a match flag sense instruction; it counts the number of write operations in EPROM quick programming. This instruction generates that number of additional write pulses by counted number.

Table 5-2 Match Flag Sense Instruction

Mnemonic	Action
INCn (n=1 to 8)	(PC) + 1 → PC (IDXRn) + 1 → IDXRn
FLGW1	If FLG1 = 0 then (PC) → PC else (PC) + 1 → PC
FLGJ1 m	If FLG1 = 0 then (PC) → PC else m → PC
FLGS1 m	If FLG1 = 0 then (PC) + 1 → PC else m → PC
FLGL1 m	If FLG1 = 0 then m → PC else (PC) + 1 → PC
FLGLI1 m	1st (IDXR8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC 2nd If (IDXW8) = 0 then (BAR) → PC else (IDXW8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC
FLGLIZ m (Available on the T5377, T5376, T5375 and T5371)	1st (IDXR8) → IDX (IDX) - 1 → IDXW8 If FLG1 = 0 then m → PC else (PC) + 1 → PC 2nd If FLG1 = 0 then If (IDXW8) = 0 then (BAR) → PC else (IDX) - 1 → IDXW8 m → PC else (PC) + 1 → PC

5. 5. 2 Programming

An example of a pattern program for a EPROM test appears below:

◆ EPROM Test Pattern Program

MPAT ME256K

```

REGISTER
  XMAX=#FF
  YMAX=#7F
  TPH=#55

SDEF      INIT=XB<0 YB<0 TP<TPH
SDEF      INC=XB<XB+1 YB<YB+1^BX
SDEF      WRITE=/T2 W M1*1
SDEF      VERIFY=/T3 R M1 M2*2

MODULE BEGIN

REGISTER
  IDX2=#7FFE
  CPE1=R*13
  DRE=W*12

START      #0

STI8      #12*3      INIT*4
STBAR      ST2*5
ST0:      STI1      #0*6
ST1:      NOP
          FLGLI1*7 STI          WRITE          FP1          ;write
          VERIFY          FP1          ;verify
ST2:      JNII1      ST3
ST3:      *8 [ NOP*14
          JNII1      ST3          WRITE          FP1          ;add pulse(write)
          JNII2      ST0          INC*11
          STI8      #7FFE
          STBAR      STP
ST4:      *9 [ FLGLI1      STF          VERIFY INC          FP1          ;verify
          JMP      ST4
*10 [ STP:      STPS
      [ STF:      STFL
STI:      INC1*7
          JMP      ST1

MODULE END

END

```

**1 WRITE*

/T2: Changes the operation period.

W: Control of I/O

M1: Control of CE

*2 *VERIFY*

/T3 : Changes the operation period.

The operation period for a flag sense instruction is as follows:

$$RATE \geq STRB1 \text{ delay} + 3.2\mu s$$

R: Control of CPE

M1: Control of CE

M2: Control of OE

*3 *Set the number of write operations.*

*4 *Initialize the register.*

*5 *Set a branch address for when write operation fails.*

*6 *Clear IDX1 so that the number of write operations is counted by the INC1 instruction.*

*7 *Generate the write and verification sequence shown in [Figure 5-5 \(1\)](#) with flag sense and INC instructions.*

*8 *Enter write pulses for additional writing to the DUT as shown in [Figure 5-5 \(2\)](#), using the number counted by the INC instruction.*

In this example, programming is done so that if $IDX_n = 0$, then the loop is executed twice by using the JNIn instruction.

*9 *Use a flag sense instruction for verification ([Figure 5-5 \(3\)](#)).*

*10 *Use the STPS or STFL instruction to determine Pass or Fail.*

*11 *The address is incremented and execution goes to the next step.*

*12 *Specify W in DRE.*

➔ *For details, refer to ["WRITE" on page 5-46](#).*

*13 *Specify R in CPE1.*

➔ *For details, refer to ["VERIFY" on page 5-47](#).*

*14 *NOP instruction (["EPROM Test Pattern Program" on page 5-46](#)) is necessary.*

An example of a test plan program for EPROM test appears below:

◆ EPROM Test Plan Program

```

PRO 27C256      ;EPROM

TIME1=1MS:VS1      ;Vcc
TIME2=1MS:IN      *1
TIME3=3MS:VS2      ;Vcc

VS1=6V             ;Vcc
VS2=21V            ;Vpp
IN1=2V,0.8V
OUT1=2.4V,0.45V

PD1-7=IN1,NRZA,ACLK1,<Y0-6>      ;A0-6
PD8-15=IN1,NRZA,ACLK1,<X0-7>     ;A7-14
PD16=IN1,/RZO,BCLK1,<M1>        ;/CE
PD17=IN1,/NRZA,ACLK1,<M2>       ;/OE
P33-40=IN1,OUT1,IOC,NRZA,ACLK1,DRE1,DRECLK1@
DRENrz,STRB1,MATCH,<D0-7>      ;D0-7
                                *2

;      TS1    TS2    TS3
;      dummy write verify
RATE=    32NS    1MS    6.2US
ACLK1=    0NS    0NS    0NS
BCLK1=    0NS    2US    2US
CCLK1=    0NS    952US  4.2US      ;tpw=0.8ms
DREL1=    0NS    0NS    0NS
DRET1=    0NS    0NS    0NS
STRB1=    0NS    0NS    3US

CALL CALB("E256",NORMAL)

TEST      1

REG MPAT PC=#0
MEAS MPATME256K

END

```

*1 Specify the power-up sequence.
The VPP is referenced ON at last.

*2 Describe STRB1 and MATCH.

*3 The operation period and timing for WRITE cycle and VERIFY cycle are switched with TSn.

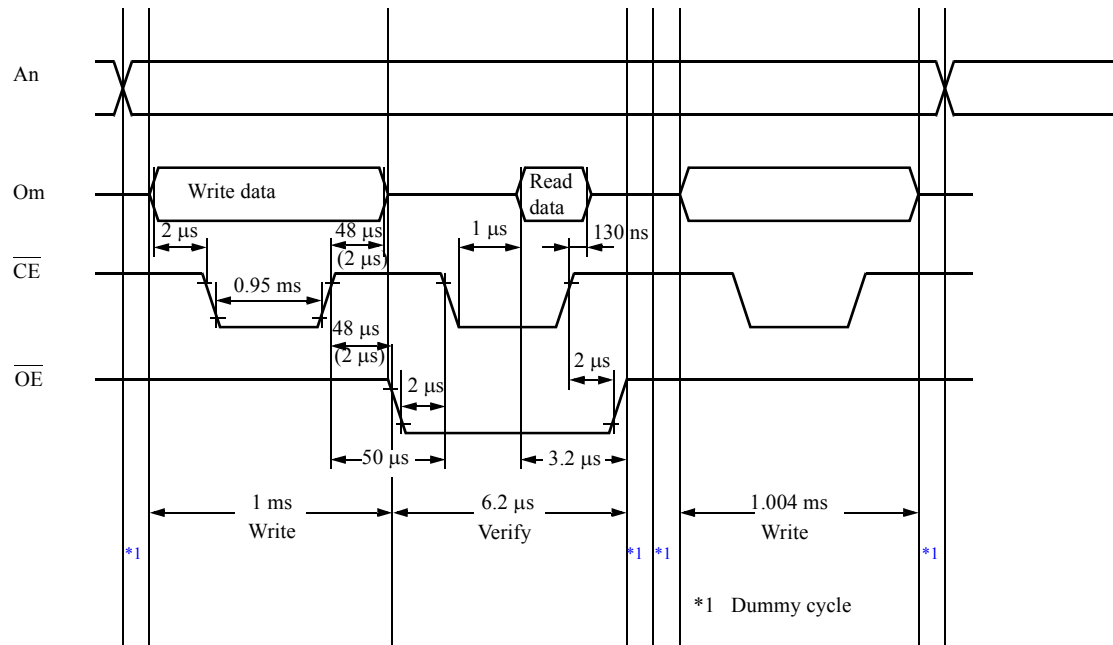
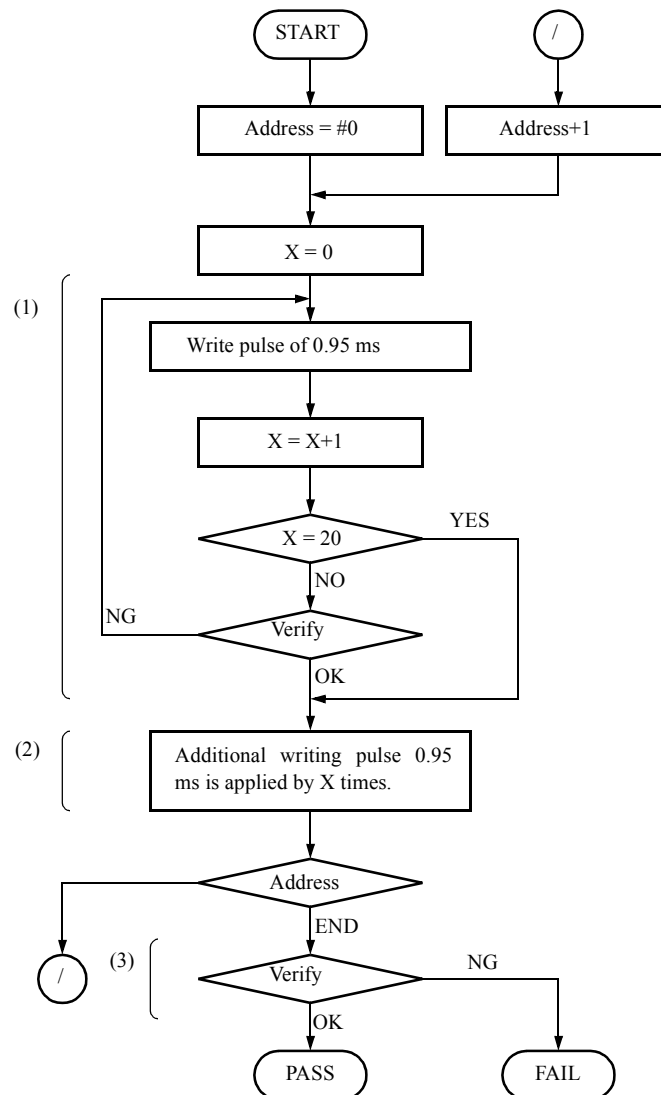
Figure 5-4 WRITE and VERIFY Timing of EPROM

Figure 5-5 EPROM Quick Program Flow Chart



Appendix 1. Pattern Program Instruction List

This chapter explains instructions to be used in pattern program.

file-format

MPAT name			
MODE a1,a2,....	_____	Operation mode specification	Common section
RDX n	_____	Radix specification	
REGISTER	_____	Initial setting of register	
ARIRAM DEFINE	_____	ARIRAM (area inversion) specification	
ADDRESS DEFINE	_____	X, Y and PAGE addresses specification	
BURST BLOCK BEGIN(block-name)	_____	Burst block definition	
BURST BLOCK END	_____		
SDEF name=ALPG instruction	_____	Define statement (main or sub) in common	
SCRAM name ...	_____	Address descramble file define statement	
PRESCRAM name ...	_____	Prescramble file define statement	
DATA DEFINE	_____	PDS DATA selection define statement	
AFM PATTERN	_____	Start declaration statement for ROM test pattern	
DBM PATTERN	_____	Start declaration statement for DBM test pattern	
BANK SAVE ADDRESS	_____	The specification of the row bank addresses which are used by the XASV instruction.	
BANK LOAD ADDRESS	_____		The specification of the column bank addresses which are specified by the XALD instruction.
START 1	Common section pattern program		
MODULE BEGIN			Module block (MAX29)
MODE a1,a2,....	_____	Operation mode specification	
RDX n	_____	Radix specification	
REGISTER	_____	Initial setting of register	
ARIRAM DEFINE	_____	ARIRAM (area inversion) specification	
ADDRESS DEFINE	_____	X, Y and PAGE addresses specification	
BURST BLOCK BEGIN(block-name)	_____	Burst block definition	
BURST BLOCK END	_____		
SDEF name=ALPG instruction	_____	Define statement	
SCRAM name ...	_____	Address descramble file define statement	
PRESCRAM name ...	_____	Prescramble file define statement	
DATA DEFINE	_____	PDS DATA selection define statement	
AFM PATTERN	_____	Start declaration statement for ROM test pattern	
DBM PATTERN	_____	Start declaration statement for DBM test pattern	
BANK SAVE ADDRESS	_____	The specification of the row bank addresses which are used by the XASV instruction.	
BANK LOAD ADDRESS	_____		
START 1	Module section pattern program		ALPG pattern program
MODULE END			
:			
MODULE BEGIN			
:			
MODULE END			
:			
END			

Pattern program format

---A--- ---B---

Tip

Field B may be omitted. When omitted, it is filled with a default value.

field A

PC Operation code, Operand & N (specified on the T5336 and T5311), I, T, Instruction (e.g. JMP n, NOP,)

field B

ALPG Instruction (e.g. : XB<XB+1, X<XB, /X)

MODE (Setting ALPG operation mode)

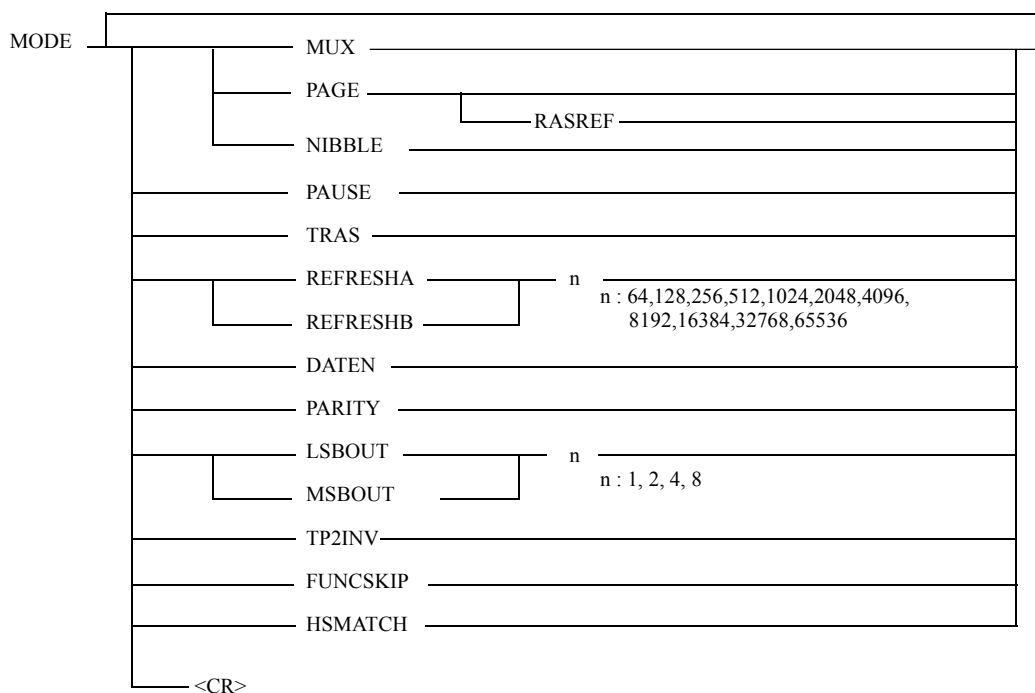
For the T5377

MODE	MUX	
	PAGE	RASREF
	PAUSE	
	TRAS	
	REFRESHA	n
	REFRESHB	n : 64,128,256,512,1024,2048,4096, 8192,16384,32768,65536,131072,262144
	DATEN	
	PARITY	
	LSBOUT	n
	MSBOUT	n : 1, 2, 4, 8
	TP2INV	
	ZSET1	
	SRDOUT n (n=1,2,4,8)	
	DINVn (n=1,2,3)	
	CSEL	
	TPM	n
		n : 18, 36
	<CR>	

For T5371

MODE			
	MUX		
	PAGE		
		RASREF	
	PAUSE		
	TRAS		
	REFRESHA		
	REFRESHB	n	n : 64,128,256,512,1024,2048,4096, 8192,16384,32768,65536
	DATEN		
	PARITY		
	LSBOUT	n	
	MSBOUT		n : 1, 2, 4, 8
	TP2INV		
	ZSET1		
	SRDOUT	n	n : 1, 2, 4, 8
	<CR>		

For the T5336 and T5311



RDX n (Setting a cardinal number)

n=2, 8, 10, 16

default=16

REGISTER (Initializing the register)

Table A 1-1 ALPG Register List

Register	Maximum value				Default value	Remarks
	T5377	T5376 T5375	T5371	T5336 T5311		
TIMER	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
TREFRESH	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
TRAS	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
TPAUSE	#FFFF	#FFFF	#FFFF	#FFFF	#100F	Time can be specified (100 ns to 409.5 s)
PC	#7FF	#7FF	#3FF	#1FF	#0	--
ISP	#7FF	#7FF	#3FF	#1FF	#0	--

Register	Maximum value				Default value	Remarks
	T5377	T5376 T5375	T5371	T5336 T5311		
BAR	#7FF	#7FF	#3FF	#1FF	#0	--
IDX1 to 8	#FFFFFFF	#FFFFFFF	#FFFFFFF	#FFFFFFF	#0	--
IDX9 to 16	#FFFFFFF	#FFFFFFF	--	--	#0	--
CFLG	#FFFF	#FFFF	#FF	#FF	#0	--
RC	--	--	--	#FFF	#0	#1 to #FFF
CPE1 to 4	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	R	--
DRE(DRE1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	W	Can be described as DRE1.
DRE2	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	W	--
RINV	Signal name (*1)	Signal name (*1)	Signal name (*1)	Signal name (*1)	R	--
LMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	XMAX	--
HMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	YMAX	--
XMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFF	--
YMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFF	--
XH	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
YH	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
ZH	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
NH	#FF	#FF	#FF	#F	#0	--
D1(D1A)	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	Can be described as D1A.
D1B to D1H	#3FFFF	--	--	--	#0	--
D2(D2A)	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	Can be described as D2A.
D2B to D2D	#3FFFF	--	--	--	#0	--
D3B, D4B	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
XMASK	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFD	--
YMASK	#3FFFF	#3FFFF	#FFFF	#FFFF	#FFD	--
RCD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
CCD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--

Register	Maximum value				Default value	Remarks
	T5377	T5376 T5375	T5371	T5336 T5311		
DSD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
XOS	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
YOS	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
XT	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
YT	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
TPH(TPH1, TPH1A)	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	Can be described as TPH1 or TPH1A.
TPH1B to 1D	#FFFFFFFF	--	--	--	#0	--
TPH2 (TPH2A)	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	Can be described as TPH2A.
TPH2B to 2D	#FFFFFFFF	--	--	--	#0	--
DCMR (DCMR1)	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	Can be described as DCMR1.
DCMR2	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	--
D5	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	--
D6	#FFFFFFFF	#FFFFFFFF	#3FFFF	#3FFFF	#0	--
MR1	#FF	#FF	#FF	#FF	#0	--
RLMAX	#3FFFF	#3FFFF	#FFFF	#FFFF	#FF	#3F, #7F, #FF, #1FF, #3FF, #7FF, #FFF, #1FFF, #3FFF, #7FFF, #FFFF, #1FFFF or #3FFFF can be speci- fied.
ZD	#3FFFF	#3FFFF	#FFFF	#FFFF	#0	--
ND(ND1)	#FF	#FF	--	--	#0	Can be described as ND1.
ND2 to 7	#FF	--	--	--	#0	--
BD(BD1)	#FF	#FF	--	--	#0	Can be described as BD1.
BD2 to 7	#FF	--	--	--	#0	--
BH	#FF	#FF	--	--	#0	--
FPSEL	Address func- tion name (*2)	Address func- tion name (*2)	--	--	DISABLE	--

-- means the register cannot be specified.

**1 Specify the following signal names in the CPE1 to 4, DRE, DRE2, and RINV registers.*

T5377, T5376 and T5375
FIXL(FL), FIXH(FH), R(RD), W(WT), M1 to 2, C0 to 23, PREF

T5371
FIXL(FL), FIXH(FH), R(RD), W(WT), M1 to 2, C0 to 15, PREF

T5336 and T5311
FIXL(FL), FIXH(FH), R(RD), W(WT), M1 to 2, C0 to 11, PREF

Note that PREF can only be specified for the CPE1 to 4 registers.

**2 Specify the following address function names or DISABLE in the FPSEL register.*
FP0, FP1, FP3, FP4, FP5, FP6, FP7, FP8, FP9 or FP10

ARIRAM DEFINE (Setting of area inversion memory)

ARIRAM DEFINE
X=Xk₁ O₁ Xk₂
Y=Yl₁ O₂ Yl₂
INV=X O₃ Y

0 ≤ k_n ≤ 11 On:&=.AND.
0 ≤ l_n ≤ 11 "=".XOR.
 '=.OR.
 %=.NOT.

Dn=Xk₁ O₁ Yl₁

T5377, T5376 and T5375	T5371, T5336 and T5311
0 ≤ n ≤ 17	0 ≤ n ≤ 17
0 ≤ k _n ≤ 17	0 ≤ k _n ≤ 15
0 ≤ l _n ≤ 17	0 ≤ l _n ≤ 15

On:&=.AND.
"=".XOR.
'=.OR.
%=.NOT.

ARlj=Xk₁ O₁ Yl₁

$D_n = ARI_j$

T5377, T5376 and T5375	T5371, T5336 and T5311
$0 \leq j \leq 7$	$0 \leq j \leq 7$
$0 \leq n \leq 17$	$0 \leq n \leq 17$
$0 \leq k_n \leq 17$	$0 \leq k_n \leq 15$
$0 \leq l_n \leq 17$	$0 \leq l_n \leq 15$

On:&=.AND.

"= .XOR.

'= .OR.

%= .NOT.

ADDRESS DEFINE

ADDRESS DEFINE

X-m=Zp-q, Nr-s

Y-m=Zp-q, Nr-s

Xn-o=Bt-u

Yn-o=Bt-u

XY=ZN(ZNB) or XY.OR.ZN(ZNB)

	T5377, T5376 and T5375	T5371	T5336 and T5311
l, m	4 to 17	4 to 17	4 to 17
p, q	0 to 7	0 to 7	0 to 7
r, s	0 to 7	0 to 7	0 to 3
n, o	0 to 17	--	--
t, u	0 to 7	--	--

PAGE=Xl-m, Yn-o

	T5377, T5376 and T5375	T5371, T5336 and T5311
l, m, n, o	0 to 17	0 to 15

X=Z0 or X=Z0-n

Y=Z0 or Y=Z0-n

	T5377, T5376 and T5375	T5371, T5336 and T5311
n	1 to 17	1 to 15

BURST BLOCK

BURST BLOCK BEGIN (Block-name)

VS_n=vs, r (vr) n=1 to 12VS_n=vs n=1 to 12

WAIT TIME n n=100ns to 409.5s

BURST BLOCK END

SDEF

SDEF name=ALPG instruction

SCRAM

SCRAM name

SCRAM name1 name2

PRESCRAM

PRESCRAM name

DATA DEFINE

PDS DATA = D.XOR.FDA/D2

PDS DATA = D.XOR.FDA/FDB

PDS DATA = D/D2

PDS DATA = FDA/D2

PDS DATA = FDA/FDB

PDS DATA = D.XOR.FDA/D2.XOR.FDB *1

PDS DATA = D/D2.XOR.FDB *1

PDS DATA = FDA/D2.XOR.FDB *1

1 Can be executed for the T5377, T5376 and T5375.*BANK SAVE ADDRESS**

BANK SAVE ADDRESS a0 ,a1, a2, a3

a0 to a3 = X0 to 17 or Y0 to 17 or 0 (Continuous specification is possible.)

Can be executed for the T5377.

BANK LOAD ADDRESS

BANK LOAD ADDRESS a0 ,a1, a2, a3

a0 to a3 = X0 to 17 or Y0 to 17 or 0 (Continuous specification is possible.)

Can be executed for the T5377.

MODULE (Defining module)

MODULE BEGIN

(Setting the modes in the module)

(pattern in the module)

MODULE END

field A

Table A 1-2 Field A Instruction List

	T5377, T5376 and T5375	T5371	T5336 and T5311
NOP	--	--	--
JMP 1	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
JSR 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
STIn m	n:1 to 8 m:#0 to #FFFFFFF		
INCn *	n:1 to 8		
IDXIn m *	n:1 to 8 m:#0 to #FFFFFFF		
LDIn m	n:1 to 8 m:9 to 16 or m=n	Not available	
JSIn 1 *	n:1 to 8 l:#0 to #7FF	n:1 to 8 l:#0 to #3FF	n:1 to 8 l:#0 to #1FF
JNIn 1 *	n:1 to 8 l:#0 to #7FF	n:1 to 8 l:#0 to #3FF	n:1 to 8 l:#0 to #1FF
JNInI 1 *	n:1 to 8 l:#0 to #7FF	n:1 to 8 l:#0 to #3FF	n:1 to 8 l:#0 to #1FF
JNInD 1 *	n:1 to 8 l:#0 to #7FF	n:1 to 8 l:#0 to #3FF	n:1 to 8 l:#0 to #1FF
JNCn 1 *	n:1 to 8 l:#0 to #7FF	n:1 to 8 l:#0 to #3FF	n:1 to 8 l:#0 to #1FF
FLGLI1 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
FLGLIZ 1 *	l:#0 to #7FF	l:#0 to #3FF	Not available
FLGS1 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
FLGL1 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
FLGJ1 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
HFLGLI1 *	Not available	Not available	l:#0 to #1FF
FJN18 1 *	Not available	Not available	l:#0 to #1FF
JAF 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
FLGW1 *	--	--	--
STTM m	m:100NS to 409.5S, #1 to #FFF		

	T5377, T5376 and T5375	T5371	T5336 and T5311
STTM1 m	m:100NS to 409.5S, #1 to #FFF		
STTM2 m	m:100NS to 409.5S		
STRC m	Not available	Not available	l:#0 to #7FF
RTN *	--	--	--
STBAR m	l:#0 to #7FF	l:#0 to #7FF	l:#0 to #3FF
STISP m	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
JZD 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
JET 1 *	l:#0 to #7FF	l:#0 to #3FF	l:#0 to #1FF
WAIT *	--	--	--
OUT block-name *	--	--	--
STPS *	--	--	--
SET XH dt1 YH dt1 ZH dt1 NH dt2 Z dt1 ZD dt1 XOS dt1 YOS dt1 XT dt1 YT dt1 D1(D1A) dt1 D2(D2A) dt1 D3B dt1 D4B dt1 XMASK dt1 YMASK dt1 DSD dt1 CCD dt1 RCD dt1 TPH(TPH1,TPH1A) dt3 TPH2(TPH2A) dt3 TP(TP1) dt3 TP2 dt3 DCMR(DCMR1) dt3 DCMR2 dt3 D5 dt3 D6 dt3 DBMAP dt4 ND(ND1) dt2 BD(BD1) dt2 BH dt2	dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt2:#0 to #FF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt1:#0 to #3FFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt3:#0 to #FFFFFFF dt4:#0 to #FFFFF dt2:#0 to #FF dt2:#0 to #FF dt2:#0 to #FF	dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt2:#0 to #FF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt4:#0 to #FFFF Not available Not available Not available	dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt2:#0 to #F dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt1:#0 to #FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt3:#0 to #3FFFF dt4:#0 to #FFFF Not available Not available Not available
N	Not available	--	--

	T5377, T5376 and T5375	T5371	T5336 and T5311
I	--	--	--
T	--	--	--
TM1	--	--	--
TM2	--	--	--

Each instruction indicated with the * symbol cannot be included with TM2 or T (in the pause mode).

Tip

- The flag sense instruction cannot be described simultaneously with TM2 or T (in pause mode).
- N, I, T, TM1 and TM2 control bits, address data operation instructions, and MUT signals cannot be specified for a cycle in which STPS or STFL is used.
- Control bits T, TM1, and TM2 cannot be specified for a cycle in which a WAIT instruction is used.

field B (ALPG)

• XB & YB

XB<XB	A<D1A	*1
XB<0	A<D1B	*1
XB<XH	A<D1C	*1
XB<XB+1	A<D1D	*1
XB<XB-1	A<D1E	*1
XB<XB+2	A<D1F	*1
XB<XB+2	A<D1G	*1
XB<A	A<D1H	*1
XB<XB+A	A<D2A	*1
XB<XB-A	A<D2B	*1
	A<D2C	*1
	A<D2D	*1
YB<YB	A<D1A	*1
YB<YB+1	A<D1B	*1
YB<YB-1	A<D1C	*1
YB<YH	A<D1D	*1
YB<YB+1^BX	A<D1E	*1
YB<YB-1^BX	A<D1F	*1
YB<YB+1+BX	A<D1G	*1
YB<YB-1-BX	A<D1H	*1
YB<0	A<D2A	*1
YB<A	A<D2B	*1
YB<YB+A	A<D2C	*1
YB<YB-A	A<D2D	*1
YB<YB+A^BX		*1
YB<YB-A^BX		*1
YB<YB+A+BX		*1
YB<YB-A-BX		*1

**1 Can be executed for the T5377.*

• XC & XS & XK

XC<XC	XS<XS	XK<XK	*1	F<A	A<XB	B<D1A
XC<F	XS<XS	XK<XK	*1	F<A+1	A<XC	B<D1B
XC<XS	XS<XS	XK<XK	*1	F<A-1	A<XS	B<D1C
XC<XK	XS<XS	XK<XK	*1	F<A*2	A<XK	B<D1D
XC<XC	XS<F	XK<XK	*1	F</A		B<D1E
XC<XC	XS<XC	XK<XK	*1	F<B		B<D1F
XC<XC	XS<XS	XK<F	*1	F<A+B		B<D1G
XC<XC	XS<XS	XK<XC	*1	F<A-B		B<D1H
XCS<F		XK<XK	*1	F<A&B		B<D2A
XCK<F	XS<XS			F<A"B		B<D2B
XCSK<F				F<A'B		B<D2C
XC<XS<F		XK<XK	*1	F<0		B<D2D
XS<XC<F		XK<XK	*1			B<D3
XC<XK<F	XS<XS					B<D4
XK<XC<F	XS<XS					
XC<>XS		XK<XK	*1			
XC<>XK	XS<XS					

**1 Can be executed for the T5377.*

• YC & YS & YK

YC<YC	YS<YS	YK<YK	*1	F<A	A<YB	B<D1A
YC<F	YS<YS	YK<YK	*1	F<A+1	A<YC	B<D1B
YC<YS	YS<YS	YK<YK	*1	F<A-1	A<YS	B<D1C
YC<YK	YS<YS	YK<YK	*1	F<A*2	A<YK	B<D1D
YC<YC	YS<F	YK<YK	*1	F</A		B<D1E
YC<YC	YS<YC	YK<YK	*1	F<B		B<D1F
YC<YC	YS<YS	YK<F	*1	F<A+B		B<D1G
YC<YC	YS<YS	YK<YC	*1	F<A-B		B<D1H
YCS<F		YK<YK	*1	F<A&B		B<D2A
YCK<F	YS<YS			F<A"B		B<D2B
YCSK<F				F<A'B		B<D2C
YC<YS<F		YK<YK	*1	F<0		B<D2D
YS<YC<F		YK<YK	*1	F<A+1+CY		B<D3
YC<YK<F	YS<YS			F<A-1-CY		B<D4
YK<YC<F	YS<YS			F<A+B+CY		
YC<>YS		YK<YK	*1	F<A-B-CY		
YC<>YK	YS<YS			F<A+1+BX		
				F<A-1-BX		
				F<A+B+BX		
				F<A-B-BX		
				F<A+1^CY		
				F<A-1^CY		
				F<A+B^CY		
				F<A-B^CY		
				F<A+1^BX		
				F<A-1^BX		
				F<A+B^BX		
				F<A-B^BX		

**1 Can be executed for the T5377.*

- Z

Z<Z	Z<ZD
Z<0	Z<Z'ZD(OR)
Z<ZH	Z<Z&ZD(AND)
Z<Z+1	Z<Z"ZD(XOR)
Z<Z-1	Z<Z+ZD
Z<Z*2	Z<Z-ZD
Z<Z/2	Z</ZD
Z</Z	
Z<Z+1^BX	
Z<Z-1^BX	
Z<Z+1^CY	
Z<Z-1^CY	
Z<0^BX	
Z<0^CY	(Z<H : Z<#FFF)
Z<H	

- N

N<N	A<ND1	*1
N<NH	A<ND2	*2
N<N+1	A<ND3	*2
N<N-1	A<ND4	*2
N<N+A	A<ND5	*2
N<N-A	A<ND6	*2
N<A	A<ND7	*2

*1 Can be executed for the T5377, T5376 and T5375.

*2 Can be executed for the T5377.

- B

B<B	*1	A<BD1	*1
B<BH	*1	A<BD2	*2
B<B+1	*1	A<BD3	*2
B<B-1	*1	A<BD4	*2
B<B+A	*1	A<BD5	*2
B<B-A	*1	A<BD6	*2
B<A	*2	A<BD7	*2

*1 Can be executed for the T5377, T5376 and T5375.

*2 Can be executed for the T5377.

- D3 & D4

D3<D3	D4<D4
D3<D3B	D4<D4B
D3<D3*2	D4<D4*2
D3<D3/2	D4<D4/2
D3<D3*2L	D4<D4*2L
D3<D3+1	
D3<D3-1	
D3<0	

- RF

RF<RF
RF<RF+1

- X & Y

X<XB	Y<YB
X<XC	Y<YC
X<XS	Y<YS
X<XK *1	Y<YK *1
X<YB	Y<XB
X<YC	Y<XC
X<YS	Y<XS
X<YK *1	Y<XK *1
X<XB'Z	Y<XB'Z
X<XC'Z	Y<XC'Z
X<XS'Z	Y<XS'Z
X<XK'Z *1	Y<XK'Z *1
X<Z	Y<Z
X<RF	

**1 Can be executed for the T5377.*

- /X & /Y & /Z

/X
/Y
/Z

- SWAP

LMAX<HMAX

- TP

TP<TP	TP2<TP2
TP<TP*2	TP2<TP2*2
TP<0	TP2<0
TP<TP/2	TP2<TP2/2
TP</TP	TP2</TP2
TP<TP+1	TP2<TP2+1
TP<TP-1	TP2<TP2-1
TP<TPH	TP2<TPH2
TP<TPH1B *1	TP2<TPH2B *1
TP<TPH1C *1	TP2<TPH2C *1
TP<TPH1D *1	TP2<TPH2D *1
TP<TPH	TP2<TPH2
TP<TPH1B *1	TP2<TPH2B *1
TP<TPH1C *1	TP2<TPH2C *1
TP<TPH1D *1	TP2<TPH2D *1
TP<D5	TP2<D6
TP<TP'D5 (OR)	TP2<TP2'D6 (OR)
TP<TP&D5 (AND)	TP2<TP2&D6 (AND)
TP<TP"D5 (XOR)	TP2<TP2"D6 (XOR)
TP<TP+D5	TP2<TP2+D6
TP<TP-D5	TP2<TP2-D6
TP</D5	TP2</D6

**1 Can be executed for the T5377.*

TP can be described as TP1, TPH can be described as TPH1 or TPH1A, and TPH2 can be described as TPH2A.

- Data select

D<TP
D<TP2

Can be executed for the T5377, T5376 and T5375.

- DSD & RCD & CCD

DSD<DCD+1
RCD<RCD+1
CCD<CCD+1
RCD<RCD+1 CCD<CCD+1

- /D & /D2

/D
/D2

- PD

PD

- Data shift

DSR
DLOAD

Can be executed for the T5377, T5376, T5375 and T5371.

- Data swap

DS

- Address shift

XYSL
XYSR
XYLOAD

- XT & YT

XT
YT

- SCROFF & ARIOFF & PSCROFF

SCROFF
ARIOFF
PSCROFF *1

**1 Can be executed for the T5377, T5376, T5375 and T5371.*

- PM/2 & PM/4

PM/2
PM/4

- Data SET

XH<m
YH<m
D1<m
D2<m
D3B<m
D4B<m
NH<l
ZH<m
Z<m
XOS<m
YOS<m
XT<m
YT<m
TP1<n
TPH1<n
TP2<n
TPH2<n
D5<n
D6<n
DCMR1<n
DCMR2<n
XMASK<m
YMASK<m
RCD<m
CCD<m
DSD<m
DBMAP<p
ZD<m
ND<l *1
BD<l *1
BH<l *1

**1 Can be executed for the T5377, T5376 and T5375.*

	T5377, T5376 and T5375	T5371	T5336 and T5311
l	#0 to #FF		#0 to #F
m	#0 to #3FFFF	#0 to #FFFF	
n	#0 to #FFFFFFFF	#0 to #3FFFF	
p	#0 to #FFFFFF	#0 to #FFFF	

- FP

- FP0 (FIX0)
 - FP1 (Checker board)
 - FP3 (Diagonal)
 - FP4 (Inversion diagonal)
 - FP5 (Diagonal 2) ^{*1}
 - FP6 (Inversion diagonal 2) ^{*1}
 - FP7 (Mask parity)
 - FP8 (Row bar)
 - FP9 (Column bar)
 - FP10 (Row column bar)
 - FPX (Universal FP) ^{*1}

**1 Can be executed for the T5377, T5376 and T5375.*

- Control bit

For the T5377, T5376 and T5375

MMA, MMB, R, W, M1, M2, C0 to 23, /T1 to 16 (TS1 to 16)

For T5371

MMA, MMB, R, W, M1, M2, C0 to 15, /T1 to 16 (TS1 to 16)

For the T5336 and T5311

R, W, M1, M2, C0 to 11, /T1 to 16 (TS1 to 16)

- Timing set control

TSn	Normal	Page mode	Nibble mode
TS1	TS1	MM-NN	MM-NN
TS2	TS2	not used	not used
TS3	TS3	MM-REF	MM-REF
TS4	TS4	not used	not used
TS5	TS5	MM-R	MM-R
TS6	TS6	MP-R	MN-R
TS7	TS7	PM-R	NM-R
TS8	TS8	P-R	N-R
TS9	TS9	MM-W	MM-W
TS10	TS10	MP-W	MN-W
TS11	TS11	PM-W	NM-W
TS12	TS12	P-W	N-W
TS13	TS13	MM-RW	MM-RW
TS14	TS14	MP-RW	MN-RW
TS15	TS15	PM-RW	NM-RW
TS16	TS16	P-RW	N-RW

MM: Multiplex cycle

MP: Page in

PM: Page out

P: Page

MN: Nibble in

NM: Nibble out

N: Nibble

R: Read cycle

W: Write cycle

RW: Read/write cycle

REF: Refresh cycle

NN: Normal cycle (dummy)

- Cycle palette

CYPn [n=1 to 16]

Can be executed for the T5377, T5376, T5375 and T5371.

- X Address save

XASV

XALD

Can be executed for the T5377.

- Y Address save

YASV

YALD

YASL

Can be executed for the T5377.

- Burst Length

BLn [n=2, 4, 8, 16]

Can be executed for the T5377.

- Wrap Type

WPTSQ
WPTIL

Can be executed for the T5377.

- Expected data inversion

DINV

Can be executed for the T5377.

List of Figures

Figure 1-1	ALPG Function Block Diagram 1 (T5377, T5376, T5375 and T5371)	1-3
Figure 1-2	ALPG Function Block Diagram 2 (T5336 and T5311)	1-4
Figure 2-1	Specifiable Operating Modes (on the T5377)	2-7
Figure 2-2	Specifiable Operating Modes (on the T5376 and T5375)	2-8
Figure 2-3	Specifiable Operating Modes (on the T5371)	2-9
Figure 2-4	Specifiable Operating Modes (on the T5336 and T5311)	2-10
Figure 2-5	D, D2, FDA and FDB Data Flow for the T5377, T5376 and T5375	2-41
Figure 2-6	Function Block Diagram of Sequence Control Section-1	2-53
Figure 2-7	Function Block Diagram of Sequence Control Section-2	2-54
Figure 2-8	Function Block Diagram of Address Generator Section (T5377) ...	2-80
Figure 2-9	Function Block Diagram of Address Generator Section (T5376 and T5375)	2-81
Figure 2-10	Function Block Diagram of Address Generator Section (T5371, T5336 and T5311)	2-82
Figure 2-11	Function Block Diagram of Data Generator Section (T5377)	2-138
Figure 2-12	Function Block Diagram of Data Generator Section (T5376 and T5375)	2-139
Figure 2-13	Function Block Diagram of Data Generator Section (T5371,T5336 and T5311)	2-140
Figure 2-14	Read Operation of Multi Bank Operation	2-177
Figure 2-15	Address Generation under Clock Suspension	2-180
Figure 4-1	VO Measurement Timing	4-10
Figure 4-2	MODE REFRESHA n by Control Bit "I"	4-21
Figure 4-3	MODE REFRESHB n by Control Bit "I"	4-22
Figure 4-4	Refresh Period	4-24
Figure 4-5	RAS Only Refresh Cycle	4-29
Figure 4-6	Hidden Refresh Timing Chart	4-36
Figure 4-7	Refresh Cycle for Memory with Refresh Control Terminal	4-40
Figure 4-8	VS Change During Bump Test	4-46
Figure 4-9	Burst Data Transfer and VS Change	4-48
Figure 4-10	OUT Instruction and VS Changes	4-51
Figure 5-1	Page mode Time Chart	5-2
Figure 5-2	TRAS Timer Time Chart	5-20
Figure 5-3	Nibble Mode Time Chart	5-22
Figure 5-4	WRITE and VERIFY Timing of EPROM	5-49

Figure 5-5	EPROM Quick Program Flow Chart	5-50
------------	--------------------------------------	------

List of Tables

Table 2-1	Register List	2-21
Table 2-2	Burst Data Word Count	2-37
Table 2-3	Index Reference Instructions for the Sequence Controller	2-57
Table 2-4	Subroutine Call Instructions	2-64
Table 2-5	List of Registers that can be Set by the SET Instruction	2-65
Table 2-6	Condition Flag Reference Branch Instruction	2-69
Table 2-7	Data Flag Reference Branch Instruction	2-69
Table 2-8	Timer Flag Reference Branch Instruction	2-70
Table 2-9	Loop Execution Time of the JET Instruction	2-71
Table 2-10	Match Flag Sense Instruction	2-71
Table 2-11	RATE Setting Conditions for the Cycle in which the Match Flag Sense Instruction is Written	2-73
Table 2-12	Fail Flag Reference Branch Instruction	2-77
Table 2-13	Data Output to DA0 to DA17 and DB0 to DB17 when D and D2 are Specified for the Test Data	2-159
Table 4-1	Refresh Counter Output Bits (T5377, T5376 and T5375)	4-19
Table 4-2	Refresh Counter Output Bits (T5371, T5336 and T5311)	4-19
Table 4-3	Refresh Period	4-25
Table 4-4	Example of DRAM Data Topology	4-43
Table 4-5	OUT Instruction Transfer Time (t)	4-51
Table 4-6	OUT Instruction Transfer Time	4-51
Table 5-1	RATE Setting Conditions for the Cycle in which the Match Flag Sense Instruction is Written	5-44
Table 5-2	Match Flag Sense Instruction	5-45
Table A 1-1	ALPG Register List	A1-6
Table A 1-2	Field A Instruction List	A1-12

Index

[Symbols]

%ENDC	2-47
%IFE	2-47
%IFN	2-47
%INSERT	2-50
%REMARK	2-46
/D	2-156
/D2	2-156

[A]

ADDRESS DEFINE	2-32
AFM PATTERN	2-42, 2-185
AMAX	2-187
AMIN	2-188
ARIOFF(control bit)	2-172
ARIRAM	2-142
ARIRAM DEFINE	2-29

[B]

B	2-84
BANK LOAD ADDRESS	2-44
BANK SAVE ADDRESS	2-42
BAR(REGISTER)	2-24
BD	2-84
BD(REGISTER)	2-28
BD(SET)	2-66
BD1 to BD7	2-84
BD1 to BD7(REGISTER)	2-28
BH	2-84
BH(REGISTER)	2-28
BH(SET)	2-66
BITSIZE	2-186
BLn	2-181
BURST BLOCK	2-36
BURST BLOCK BEGIN	2-36
BURST BLOCK END	2-36

[C]

CCD	2-141, 2-155
CCD(REGISTER)	2-27

CCD(SET)	2-66
CFLG(REGISTER)	2-25
Cn(control bit)	2-163
CPE1 to 4(REGISTER)	2-25
CSEL(MODE)	2-18
CYPn(control bit)	2-174

[D]

D1	2-83
D1(REGISTER)	2-26
D1(SET)	2-66
D1A to D1H	2-83
D1A to D1H(REGISTER)	2-26
D2	2-83
D2(REGISTER)	2-26
D2(SET)	2-66
D2A to D2D	2-83
D2A to D2D(REGISTER)	2-26
D3	2-83
D3B	2-83
D3B(REGISTER)	2-26
D3B(SET)	2-66
D4	2-83
D4B	2-83
D4B(REGISTER)	2-26
D4B(SET)	2-66
D5	2-141
D5(SET)	2-66
D6	2-141
D6(SET)	2-66
DATA DEFINE	2-40
DATEN(MODE)	2-14
DBM CHANNEL	2-195
DBM PATTERN	2-42
DBMAP	2-197
DBMAP(control bit)	2-171
DBMAP(SET)	2-66
DCMR	2-141
DCMR(REGISTER)	2-26
DCMR1(SET)	2-66

DCMR2	2-141
DCMR2(SET)	2-66
DFLG	2-141, 2-156
DINV	2-159
DINVn	2-19
DLOAD	2-157
DRE(REGISTER)	2-25
DS	2-157
DSD	2-141, 2-155
DSD(REGISTER)	2-27
DSD(SET)	2-66
DSR	2-157

[E]

END	2-46
-----------	------

[F]

FJNl8	2-73
FLGJ1	2-72
FLGL1	2-72
FLGLI1	2-72
FLGLIZ	2-72
FLGS1	2-72
FLGW1	2-71
FMA	2-188
FP	2-141
FP0	2-147
FP1	2-147
FP10	2-154
FP3	2-147
FP4	2-148
FP5	2-149
FP6	2-151
FP7	2-152
FP8	2-153
FP9	2-153
FPSEL(REGISTER)	2-29
FPX	2-154
FUNCSKIP(MODE)	2-16

[H]

HFLGLI	2-73
HMAX	2-83

HMAX(REGISTER)	2-26
HSMATCH(MODE)	2-16

[I]

I(control bit)	2-76
IDX1 to 8(REGISTER)	2-24
IDX9 to 16(REGISTER)	2-25
IDXIn	2-59
INCn	2-59
INSERT	2-50
ISP(REGISTER)	2-24

[J]

JAF	2-77
JET	2-70
JNCn	2-68
JNIn	2-59
JNInD	2-59
JNInI	2-59
JSIn	2-65
JSR	2-64
JZD	2-69

[L]

LDIn	2-59
LMAX	2-83
LMAX(REGISTER)	2-26
LSBOUTn(MODE)	2-15

[M]

MEAS MPAT	3-5
MMA(control bit)	2-163
MMB(control bit)	2-163
Mn(control bit)	2-163
MODE	2-5
MODULE BEGIN	2-42
MODULE END	2-42
MPAT	2-5
MR1(REGISTER)	2-27
MSBOUTn(MODE)	2-16
MUX(MODE)	2-10

[N]		ROM.....	2-189
N	2-84	RTN	2-65
N(control bit)	2-76	[S]	
ND	2-84	SCRAM	2-38
ND(REGISTER)	2-28	SCROFF(control bit)	2-172
ND(SET)	2-66	SDEF	2-37
ND1 to ND7	2-84	SELECT MPAT	3-11
ND1 to ND7(REGISTER)	2-28	SELECT SCRDATA	3-15
NH	2-84	SELECT SCRDATA DISABLE	3-15
NH(REGISTER)	2-26	SELECT SCRDATA ENABLE	3-15
NH(SET)	2-65	SEND SCRDATA	3-15
NIBBLE(MODE)	2-11	SET	2-65
[O]		SRDOUT(MODE)	2-17
OUT	2-74	START	2-39
[P]		STBAR	2-74
PAGE(MODE)	2-11	STFL	2-76
PARITY(MODE)	2-15	STIn	2-59
PAUSE(MODE)	2-11	STISP	2-75
PC(REGISTER)	2-24	STPS	2-76
PD	2-157	STRC	2-75
PDS DATA	2-40	STTM	2-75
PM(control bit)	2-172	STTM1	2-75
PRESCRAM	2-38	STTM2	2-75
PSCROFF(control bit)	2-173	[T]	
[R]		T(control bit)	2-77
R(control bit)	2-163	TIMER(REGISTER)	2-24
RASREF(MODE)	2-11	TM1(control bit)	2-77
RC(REGISTER)	2-25	TM2(control bit)	2-77
RCD	2-141, 2-155	TP	2-140, 2-144
RCD(REGISTER)	2-27	TP1(SET)	2-66
RCD(SET)	2-66	TP2	2-140
RDX	2-5	TP2(SET)	2-66
REFRESHA(MODE)	2-11	TP2INV(MODE)	2-16
REFRESHB(MODE)	2-11	TPAUSE(REGISTER)	2-24
REG MPAT	3-10	TPH	2-140
REGISTER	2-21	TPH(REGISTER)	2-26
REPEAT	2-200	TPH1(SET)	2-66
RF	2-84	TPH1A to TPH1D	2-140
RINV(REGISTER)	2-26	TPH1A to TPH1D(REGISTER)	2-26
RLMAX(REGISTER)	2-28	TPH2	2-140
		TPH2(SET)	2-66
		TPH2A to TPH2D	2-140

TPH2A to TPH2D(REGISTER)	2-26
TPM(MODE)	2-18
TRAS(MODE)	2-11
TRAS(REGISTER)	2-24
TREFRESH(REGISTER)	2-24
TSn(control bit)	2-162

[W]

W(control bit)	2-163
WAIT	2-75
WINDOW MODE	3-18
WINDOW XADRS	3-18
WINDOW YADRS	3-18
WPTIL	2-183
WPTSQ	2-183

[X]

XALD	2-175
XASV	2-175
XB	2-83
XC	2-83
XH	2-83
XH(REGISTER)	2-26
XH(SET)	2-65
XK	2-83
XMASK	2-141
XMASK(REGISTER)	2-26
XMASK(SET)	2-66
XMAX	2-84
XMAX(REGISTER)	2-26
XOS	2-84
XOS(SET)	2-66
XS	2-83
XT	2-84
XT(control bit)	2-171
XT(SET)	2-66
XYLOAD	2-134
XYSL	2-133
XYSR	2-133

[Y]

YALD	2-177
YASL	2-178

YASV	2-177
YB	2-83
YC	2-83
YH	2-83
YH(REGISTER)	2-26
YH(SET)	2-65
YK	2-83
YMASK	2-141
YMASK(REGISTER)	2-26
YMASK(SET)	2-66
YMAX	2-84
YMAX(REGISTER)	2-26
YOS	2-84
YOS(SET)	2-66
YS	2-83
YT	2-84
YT(control bit)	2-171
YT(SET)	2-66

[Z]

Z	2-83
Z(SET)	2-66
ZD	2-84
ZD(SET)	2-66
ZH	2-83
ZH(REGISTER)	2-26
ZH(SET)	2-65
ZSET1(MODE)	2-17